

C 软件开发说明书

本手册中所提及的其它软硬件产品的商标与名称，都属于相应公司所有。

本手册的版权属于中国大恒(集团)有限公司北京图像视觉技术分公司所有。未得到本公司的正式许可，任何组织或个人均不得以任何手段和形式对本手册内容进行复制或传播。

本手册的内容若有任何修改，恕不另行通知。

© 2025 中国大恒（集团）有限公司北京图像视觉技术分公司版权所有

网 站：www.daheng-imaging.com

公 司 总 机：010-82828878

客户服务热线：400-999-7595

销 售 信 箱：sales@daheng-imaging.com

支 持 信 箱：support@daheng-imaging.com

目录

- 1. 相机工作流程 1
 - 1.1. 整体工作流程1
 - 1.2. 相机控制流程2
 - 1.3. DQBuf 采集流程（零拷贝）3
 - 1.4. DQAllBufs 采集流程（零拷贝，仅 Linux 支持）4
 - 1.5. 回调采集流程5
 - 1.6. 掉线事件获取流程6
 - 1.7. 远端设备事件获取流程7
- 2. 编程指引 8
 - 2.1. 搭建编程环境8
 - 2.1.1. Windows 8
 - 2.1.2. Linux11
 - 2.1.3. macOS 14
 - 2.2. 快速上手17
 - 2.2.1. 初始化和反初始化 GxI API 运行时库 17
 - 2.2.2. 枚举相机并获取信息 18
 - 2.2.3. 获取 Interface 信息 19
 - 2.2.4. 获取 Interface 对象 19
 - 2.2.5. 配置相机 IP 地址 19
 - 2.2.6. 打开关闭相机 20
 - 2.2.7. 控制相机功能 21
 - 2.2.8. DQBuf 图像采集 24
 - 2.2.9. 回调图像采集 26
 - 2.3. 使用千兆网相机调试程序28
- 3. 相机功能属性说明 29
 - 3.1. 设备控制29
 - 3.1.1. 设备信息 29
 - 3.1.2. 设备控制 30
 - 3.2. 图像格式31
 - 3.2.1. 感兴趣区域 31
 - 3.2.2. 图像分辨率 33

3.2.3. 数据格式.....	35
3.2.4. 测试图	37
3.2.5. 帧信息控制	38
3.2.6. Sensor 曝光模式	39
3.3. 采集控制	39
3.3.1. 采集	39
3.3.2. 触发	44
3.3.3. 曝光	48
3.3.4. 传输控制	50
3.3.5. 突发采集模式	51
3.3.6. 帧存机制	52
3.3.7. 帧率控制	53
3.3.8. 曝光模式	53
3.3.9. 交叠曝光时间最大值	54
3.3.10. 多帧灰度控制模式	55
3.4. 数字 I/O	56
3.4.1. 引脚控制	56
3.4.2. USB2.0 相机 I/O 控制	57
3.5. 计数器和计时器控制	59
3.5.1. 计时器	59
3.5.2. 计数器	60
3.6. 模拟控制	61
3.6.1. 增益	61
3.6.2. 黑电平	63
3.6.3. 白平衡	64
3.7. 传输层控制	67
3.7.1. PayloadSize	67
3.7.2. IP 配置方式	67
3.7.3. 预估带宽	68
3.7.4. 设备心跳超时时间	68
3.7.5. 包长	68
3.7.6. 包间隔	69
3.7.7. 链接速度	69

3.8. 带宽控制	70
3.9. 事件控制	70
3.10. 查找表控制.....	72
3.11. 用户配置	72
3.12. 设置相机 IP	73
3.12.1. 配置静态 IP 地址	73
3.12.2. Force IP	74
3.13. 其他	75
3.13.1. 自动曝光自动增益相关功能.....	75
3.13.2. 坏点校正.....	78
3.13.3. ADC Level (DigitalShift)	79
3.13.4. 消隐控制.....	81
3.13.5. 用户加密区	81
3.13.6. 用户数据区	82
3.13.7. 取消参数范围限制	83
3.13.8. 平场校正.....	84
4. 本地设备控制	85
4.1. 相关参数	85
4.2. 示例代码	85
4.3. 注意事项	85
5. 流层控制	86
5.1. 统计参数	86
5.1.1. 相关参数.....	86
5.1.2. 注意事项.....	86
5.1.3. 代码样例.....	86
5.2. 控制参数	86
5.2.1. 相关属性.....	86
5.2.2. 注意事项.....	88
5.2.3. 代码样例.....	88
6. 受相机型号影响的功能	89
7. GxI API 库说明	90

7.1. 类型	90
7.1.1. 数据类型.....	90
7.1.2. 句柄类型.....	90
7.1.3. 回调函数类型	90
7.2. 常量	91
7.2.1. GX_STATUS_LIST	91
7.2.2. GX_FRAME_STATUS	92
7.2.3. GX_ACCESS_MODE.....	92
7.2.4. GX_OPEN_MODE	92
7.2.5. GX_IP_CONFIGURE_MODE_LIST.....	93
7.2.6. GX_RESET_DEVICE_MODE	93
7.2.7. GX_TL_TYPE_LIST	93
7.2.8. GX_NODE_ACCESS_MODE	94
7.2.9. GX_DEVICE_CLASS	94
7.2.10. GX_LOG_TYPE_LIST	95
7.3. 结构体.....	95
7.3.1. GX_OPEN_PARAM.....	95
7.3.2. GX_FRAME_CALLBACK_PARAM	96
7.3.3. GX_FRAME_DATA.....	96
7.3.4. GX_FRAME_BUFFER	97
7.3.5. GX_REGISTER_STACK_ENTRY	98
7.3.6. GX_INTERFACE_INFO.....	99
7.3.7. GX_DEVICE_INFO	101
7.3.8. GX_INT_VALUE	104
7.3.9. GX_FLOAT_VALUE.....	104
7.3.10. GX_STRING_VALUE	105
7.3.11. GX_ENUM_VALUE.....	105
7.3.12. GX_GIGE_ACTION_COMMAND_RESULT	106
7.4. 接口	107
7.4.1. 获取 SDK 版本信息	107
7.4.2. 初始化及销毁	107
7.4.3. 设备参数设置	120
7.4.4. 设备功能接口	136
7.4.5. 设备采集接口	147
7.4.6. 设备事件接口	169
7.5. 不推荐使用定义	176
7.5.1. 类型.....	176
7.5.2. 常量.....	176
7.5.3. 结构体	178

7.5.4. 接口	181
8. 图像处理接口说明	213
8.1. 类型	213
8.1.1. 数据类型	213
8.2. 常量	213
8.2.1. 状态码	213
8.2.2. 像素 Bayer 格式	214
8.2.3. Bayer 转换类型	214
8.2.4. 有效数据位	215
8.2.5. 图像实际位数	215
8.2.6. 图像镜像翻转类型	216
8.2.7. 彩色图像 RGB 顺序	216
8.2.8. 图像格式转换句柄	216
8.2.9. 系统参数	216
8.3. 结构体	217
8.3.1. 黑白图像处理功能设置结构体	217
8.3.2. 彩色图像处理功能设置结构体	217
8.3.3. 平场校正处理功能设置结构体	218
8.3.4. 颜色校正系数用户设置结构体	219
8.3.5. 静态坏点校正功能参数结构体	219
8.4. 接口	220
8.4.1. DxRaw12PackedToRaw16	220
8.4.2. DxRaw10PackedToRaw16	221
8.4.3. DxRaw8toRGB24	222
8.4.4. DxRaw8toRGB24Ex	224
8.4.5. DxRotate90CW8B	225
8.4.6. DxRotate90CCW8B	226
8.4.7. DxRotate90CW16B	227
8.4.8. DxRotate90CCW16B	229
8.4.9. DxBrightness	230
8.4.10. DxContrast	232
8.4.11. DxSharpen24B	233
8.4.12. DxSharpenMono8	234
8.4.13. DxSaturation	235
8.4.14. DxGetWhiteBalanceRatio	236
8.4.15. DxAutoRawDefectivePixelCorrect	239
8.4.16. DxRaw16toRaw8	240

8.4.17. DxRGB48toRGB24.....	241
8.4.18. DxRaw16toRGB48	243
8.4.19. DxRaw8toARGB32	244
8.4.20. DxGetContrastLut.....	246
8.4.21. DxGetGammatLut.....	247
8.4.22. DxImageImprovment	249
8.4.23. DxARGBImageImprovment.....	253
8.4.24. DxImageImprovmentEx	256
8.4.25. DxImageMirror	260
8.4.26. DxImageMirror16B	262
8.4.27. DxGetLut.....	263
8.4.28. DxCalcCCParam	264
8.4.29. DxRaw8ImgProcess	265
8.4.30. DxMono8ImgProcess	268
8.4.31. DxGetFFCCoefficients.....	270
8.4.32. DxFlatFieldCorrection	272
8.4.33. DxCalcUserSetCCParam	273
8.4.34. DxImageFormatConvert	275
8.4.35. DxImageFormatConvertCreate	279
8.4.36. DxImageFormatConvertDestroy.....	279
8.4.37. DxImageFormatConvertSetOutputPixelFormat	280
8.4.38. DxImageFormatConvertSetAlphaValue	281
8.4.39. DxImageFormatConvertSetInterpolationType.....	282
8.4.40. DxImageFormatConvertGetBufferSizeForConversion.....	283
8.4.41. DxImageFormatConvertGetOutputPixelFormat.....	284
8.4.42. DxImageFormatConvertSetValidBits.....	285
8.4.43. DxImageFormatConvertSet3DCalibParam	286
8.4.44. DxImageFormatConvertSetYStep.....	289
8.4.45. DxStaticDefectCorrection	289
8.4.46. DxCalcCameraLutBuffer.....	290
8.4.47. DxReadLutFile	292
8.4.48. DxSetSystem	293
8.4.49. DxGetSystem.....	294
8.5. 功能	294
8.5.1. 图像质量提升功能.....	294
9. 版本说明	303

1. 相机工作流程

1.1. 整体工作流程

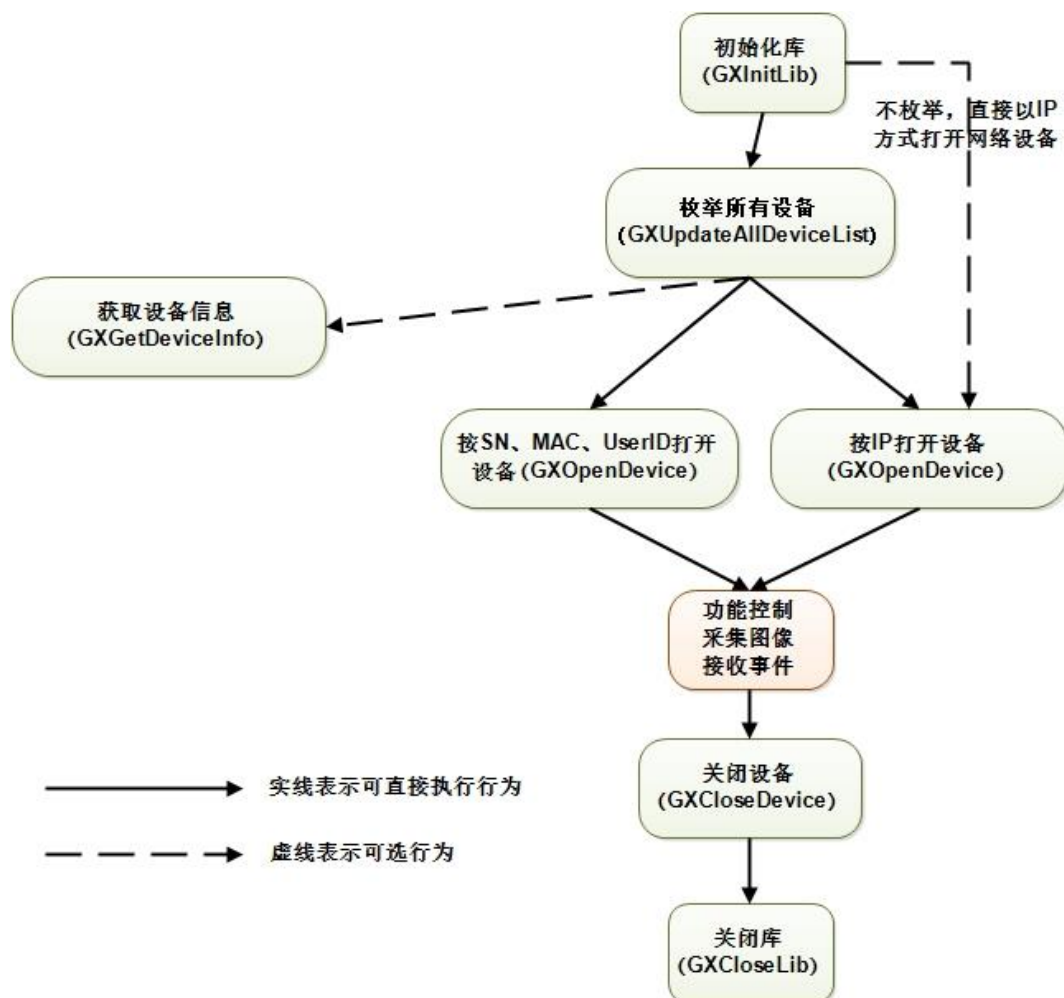


图 1-1 整体工作流程

1.2. 相机控制流程

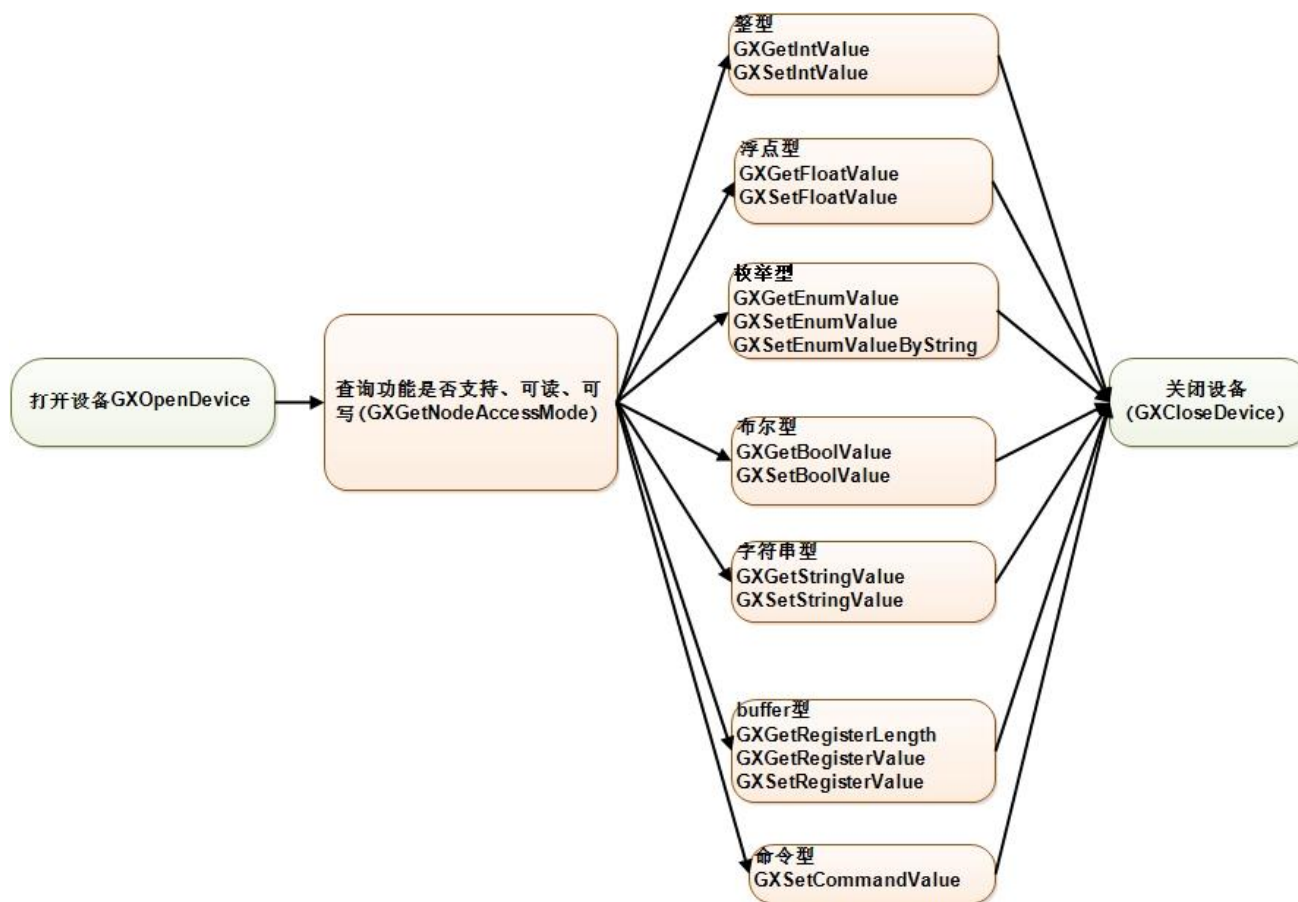


图 1-2 相机控制流程

1.3. DQBuf 采集流程（零拷贝）

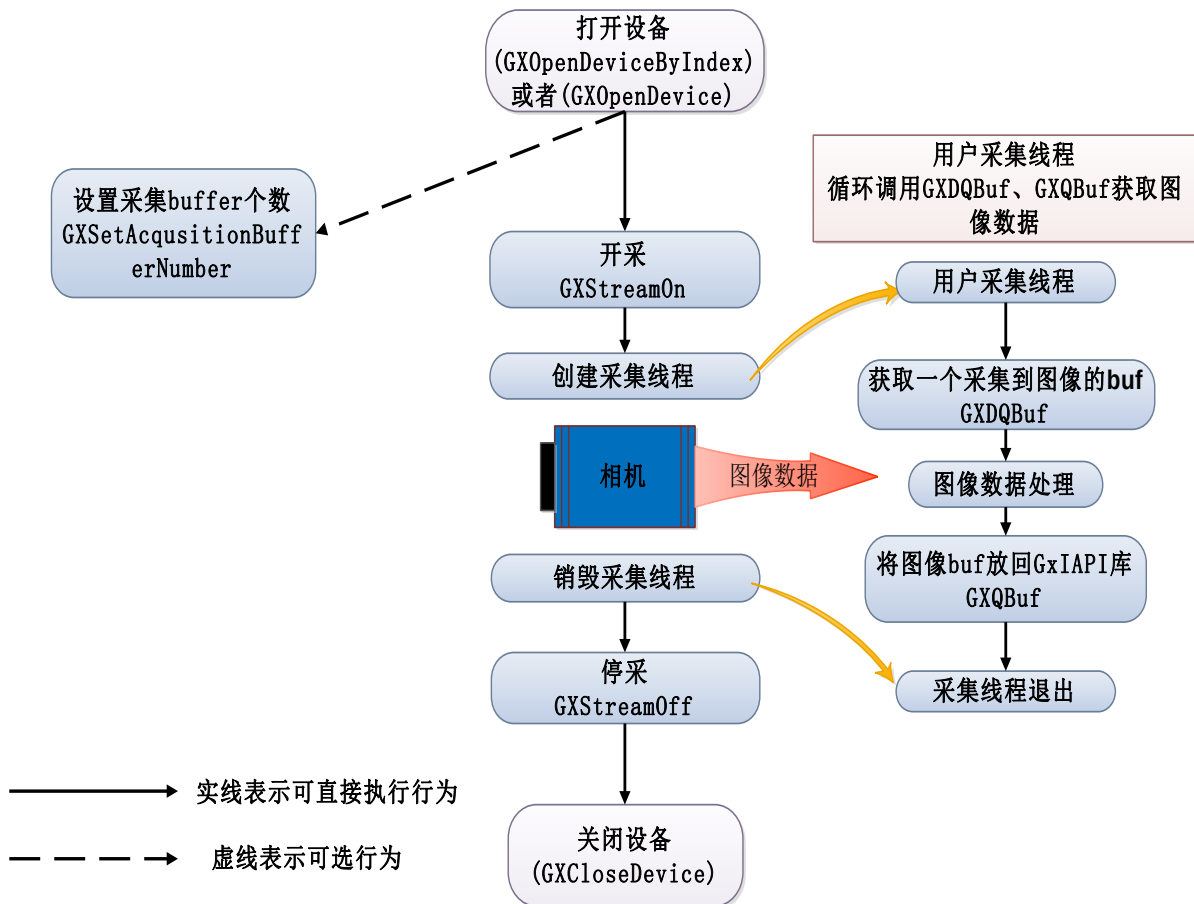


图 1-3 DQBuf 采集流程

注意事项：

- 1) 用户采集线程必须在开采之后启动，在停采之前销毁；
- 2) Windows 当前不支持 GXStreamOn()/GXStreamOff()接口，使用 GXSetCommandValue(hDevice, "AcquisitionStart")/GXSetCommandValue(hDevice, "AcquisitionStop")替代；
- 3) 调用 GXQBuf()接口将图像 buf 放回 GxIAPI 库后，不能再访问该图像 buf 指针；
- 4) GXStreamOff()停采接口会将所有通过 GXDQBuf 获取到的图像 buf 放回 GxIAPI 库，之后不能再访问这些图像 buf 指针；
- 5) 上述流程演示了启动线程的方式进行图像采集，也可以不启动线程直接在开采后调用 GXDQBuf()采集图像。

1.4. DQAllBufs 采集流程（零拷贝，仅 Linux 支持）

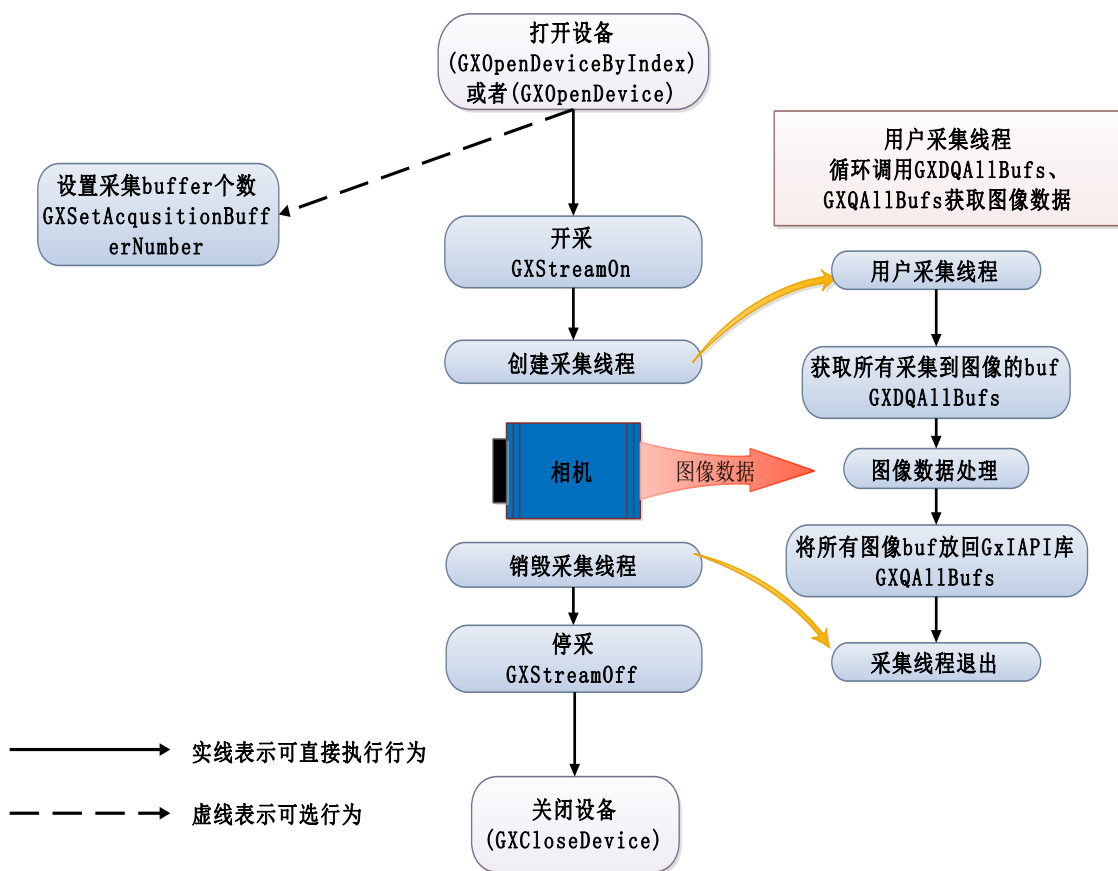


图 1-4 DQAllBufs 采集流程

注意事项：

- 1) 用户采集线程必须在开采之后启动，在停采之前销毁；
- 2) GXStreamOff()停采接口或 GXQAllBufs()接口会将所有通过 GX DQAllBufs()获取到的图像 buf 放回 GxI API 库，之后不能再访问这些图像 buf 指针；
- 3) 上述流程演示了启动线程的方式进行图像采集，也可以不启动线程直接在开采后调用 GX DQAllBufs()采集图像。

推荐使用场景：

- 1) 图像处理或显示出现延迟的情况下，可采用此种采集方案只处理或显示数组中的最后一幅图像来缓解延迟。

1.5. 回调采集流程

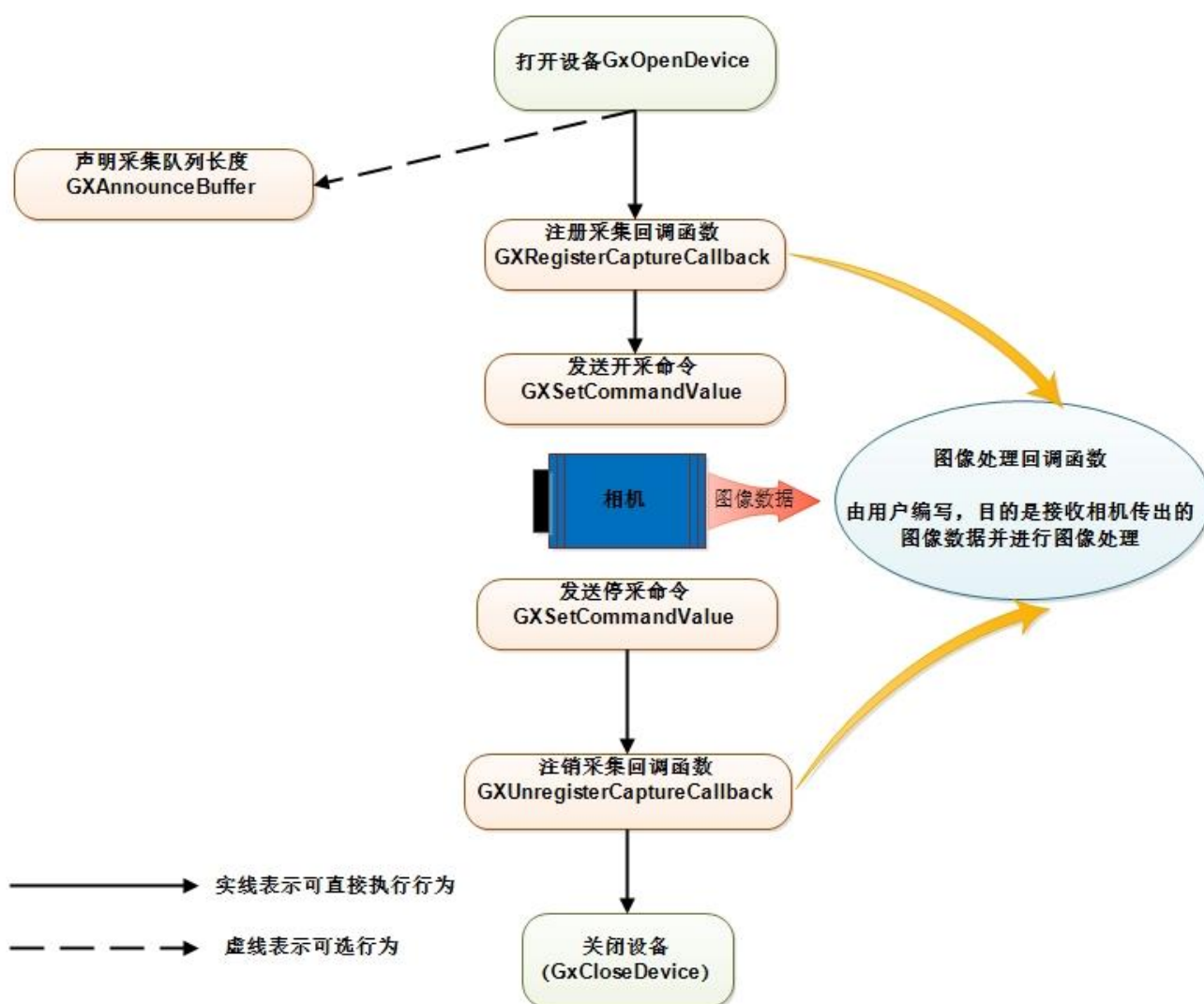


图 1-5 回调采集流程

1.6. 掉线事件获取流程

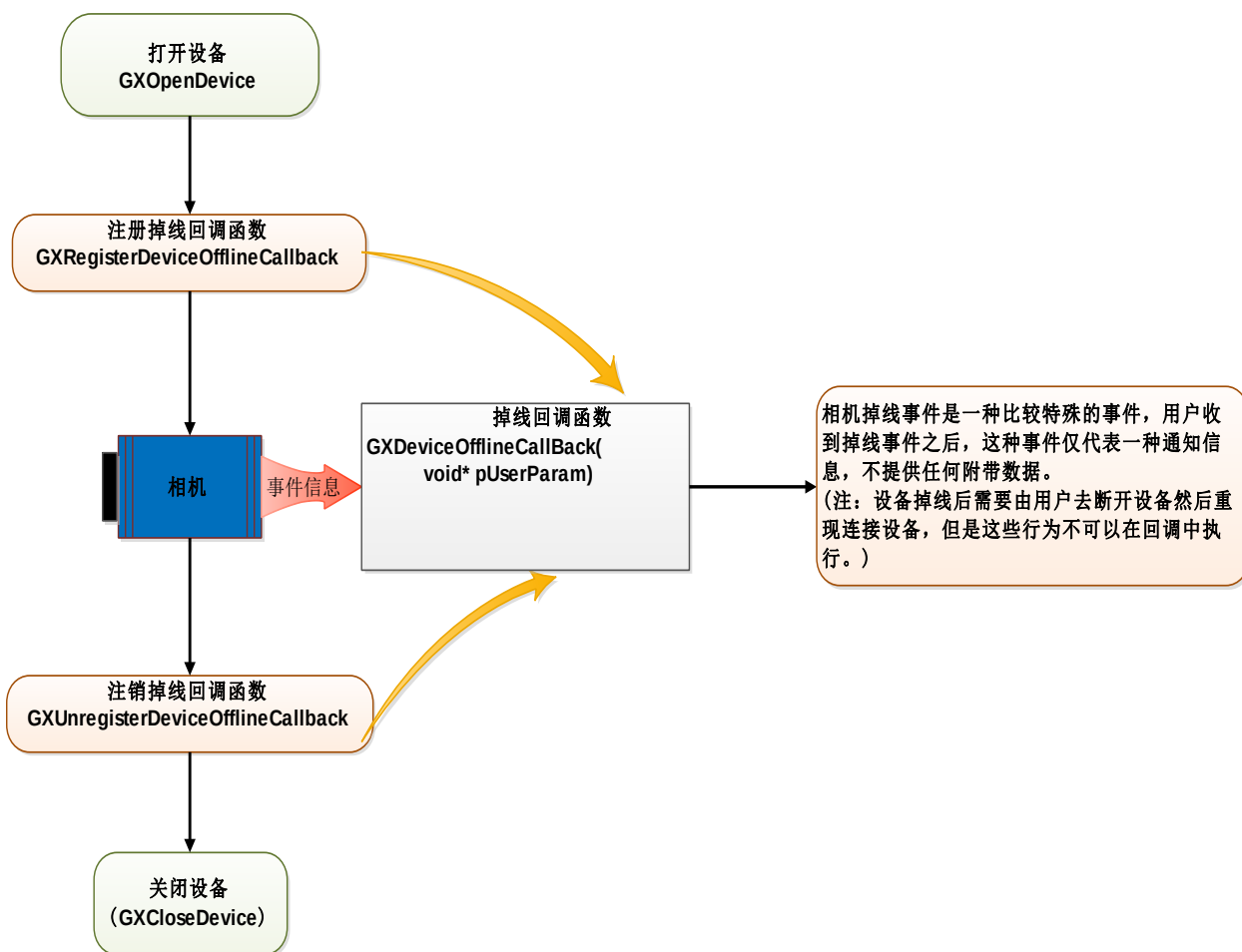


图 1-6 掉线事件获取流程

1.7. 远端设备事件获取流程

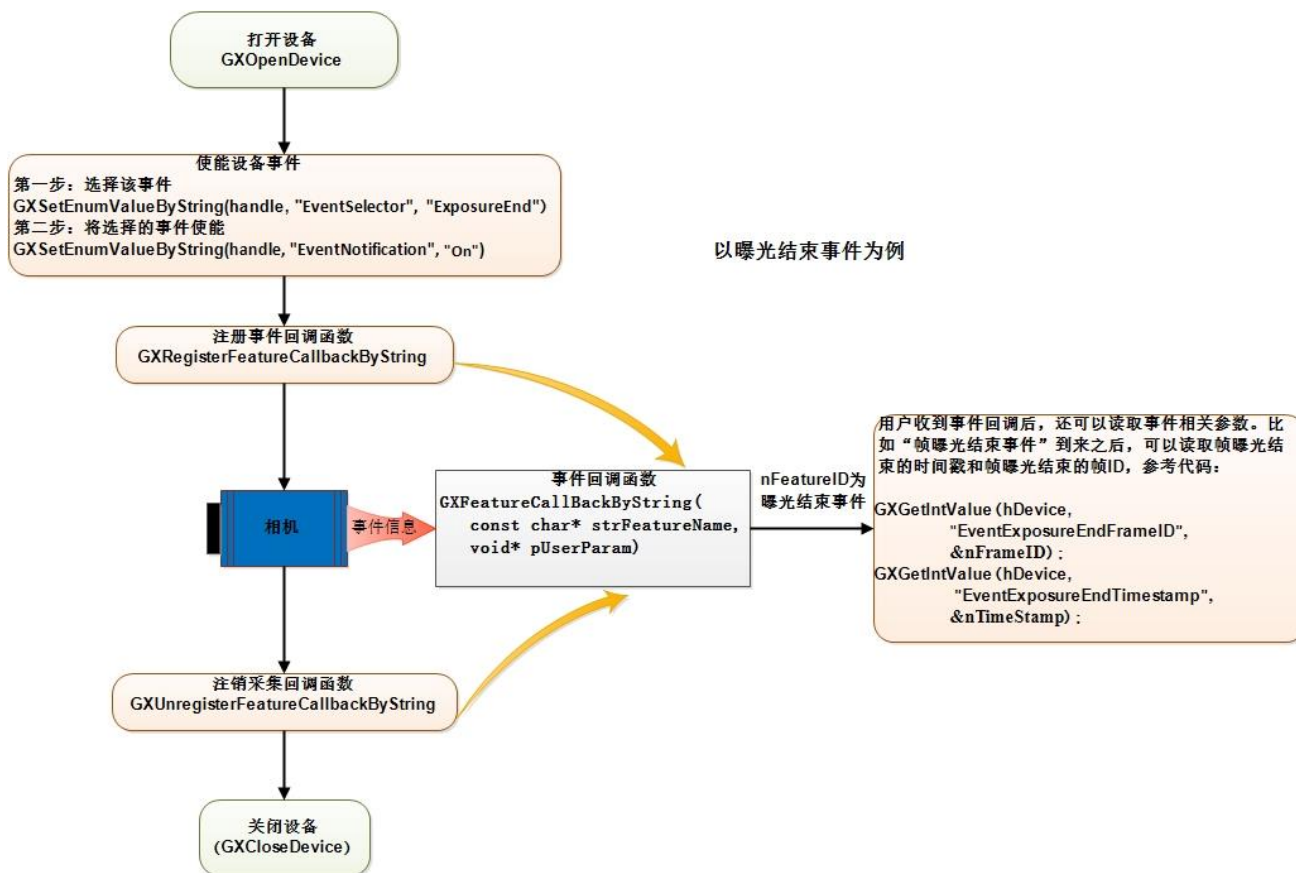


图 1-7 远端设备事件获取流程

2. 编程指引

2.1. 搭建编程环境

2.1.1. Windows

2.1.1.1. C

GxI API 和 DxImageProc 是大恒图像软件部出品的通用 C 编程接口和图像处理算法接口，适用于本公司所产的任何支持 GenICam 协议的相机。在安装包的安装目录下./ Development/VC SDK 文件夹内包含 GxI API.h ,DxImageProc.h 头文件和 GxI API.lib ,DxImageProc.lib 静态链接库文件及一些简单的示例程序。

- GxI API.dll、DxImageProc.dll 动态链接库文件
- GxI API.lib、DxImageProc.lib 编译程序时静态链接
- GxI API.h、DxImageProc.h 接口及宏声明头文件

安装包在安装的时候会将.dll 文件所在的安装路径添加到系统环境变量 path 中，所以用户编写的工程会自动从环境变量中搜索.dll 文件，进行加载。

用户使用 GxI API 或 DxImageProc 创建工程的时候需要包含.h 头文件和.lib 静态链接库，才能正常调用、编译。

➤ VC6 下配置环境

点击菜单中的 Project->Settings 弹出 project settings 窗口，点击 C/C++页，设置 Category 为 Preprocessor，然后在 Additional include directories 中填写.h 文件所在目录路径地址（依用户安装目录为准），如图 2-1 所示：

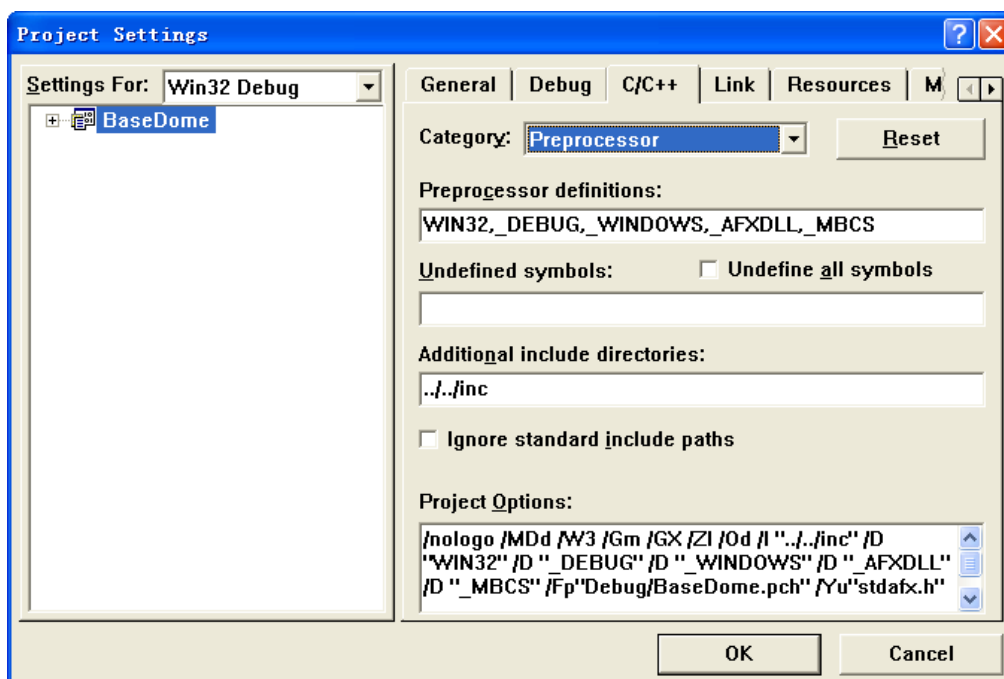


图 2-1

然后切换到 Link 页，设置 Category 为 General，然后在 Object/library modules 中填写 GxIAPi.lib 和 DxImageProc.lib，如图 2-2 所示：

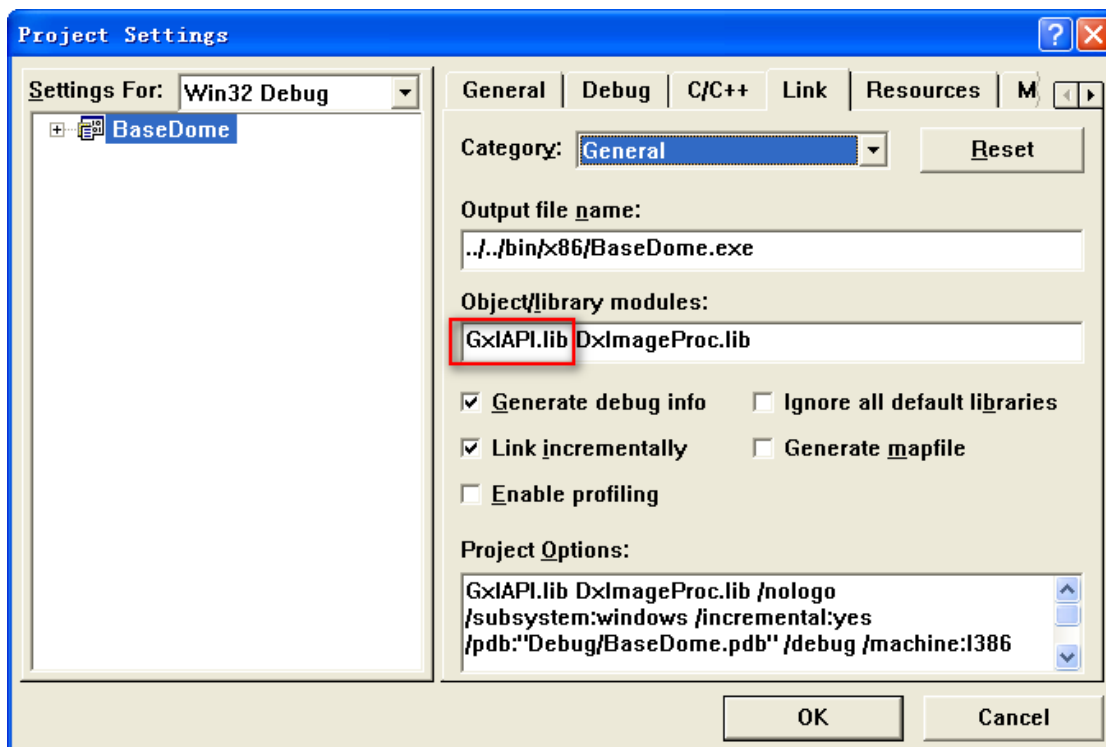


图 2-2

然后仍然在 Link 页，将 Category 设置为 Input，然后在 Additional library path 中填写 GxIAPi.lib 和 DxImageProc.lib 所在目录路径地址（依用户安装目录为准），如图 2-3 所示：

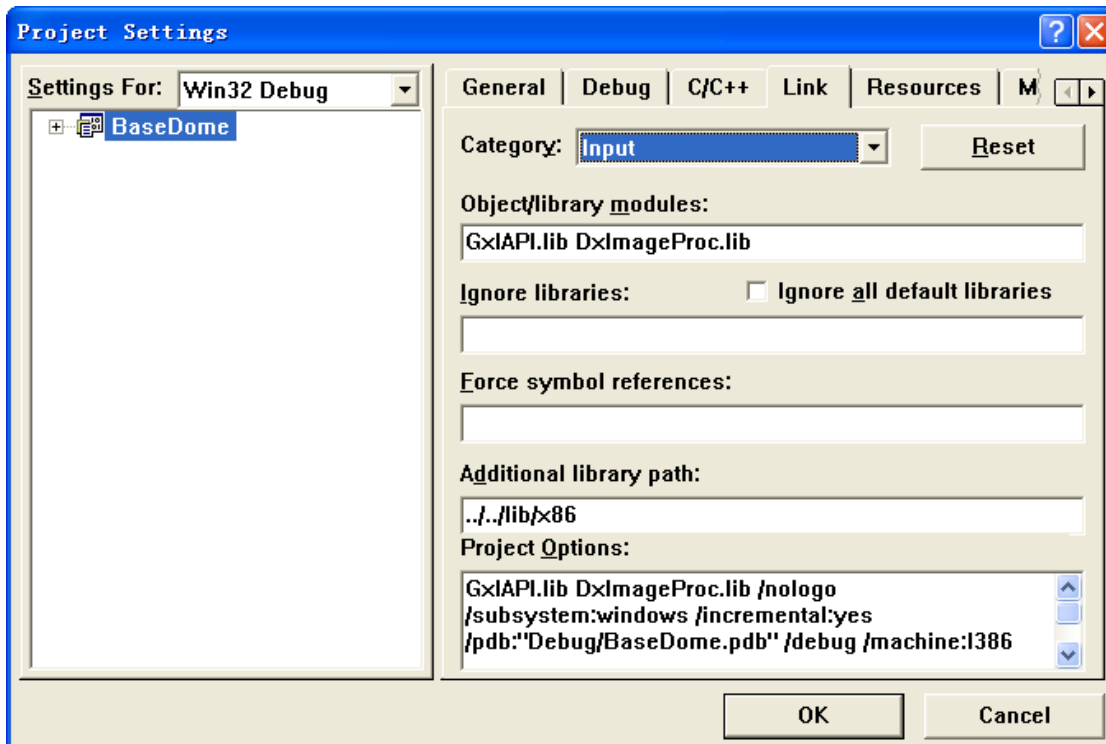


图 2-3

➤ VS2005 下配置环境

在解决方案资源管理窗口中选中用户创建的工程,然后点击菜单中的 project->properties 弹出 Property page 窗口。选择 Configuration Properties->C/C++->General 在 Additional Include Directories 中填写 GxIAP.h 和 DxImageProc.h 所在目录路径地址(依用户安装目录为准),如图 2-4 所示:

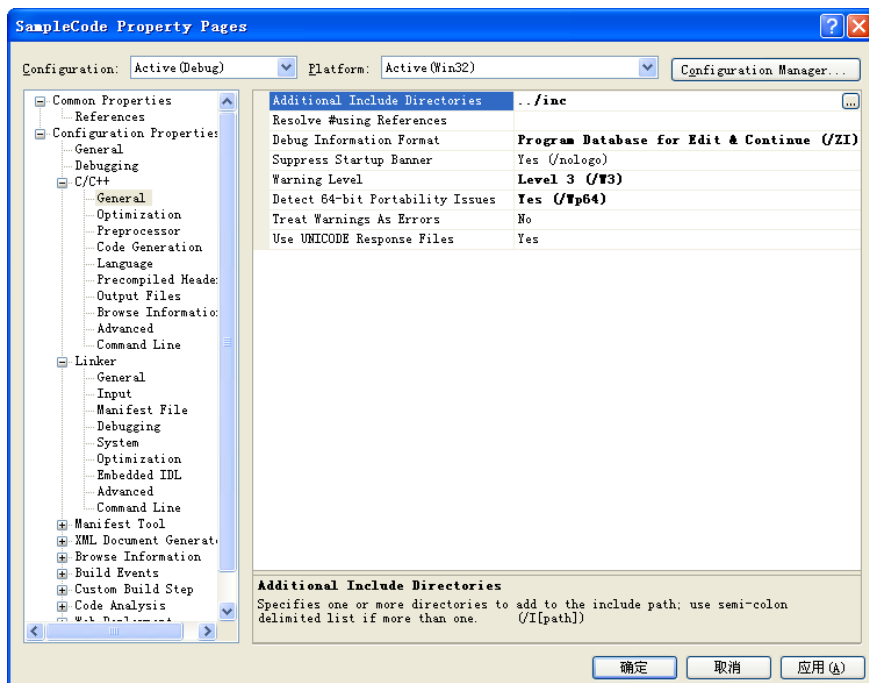


图 2-4

然后选择 Configuration Properties->Linker->General 在 Additional Library Directories 中填写 GxIAP.lib 和 DximageProc.lib 所在目录路径地址(依用户安装目录为准),如图 2-5 所示:

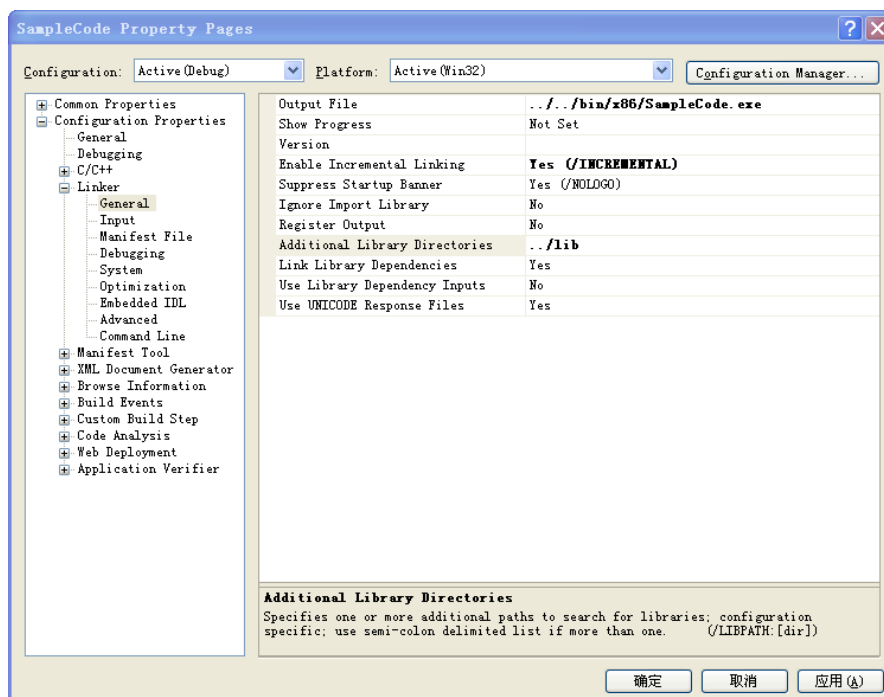


图 2-5

然后选择 Configuration Properties->Linker->Input 在 Additional Dependencies 中填写 GxIAPL.lib 和 DxImageProc.lib 如图 2-6 所示：

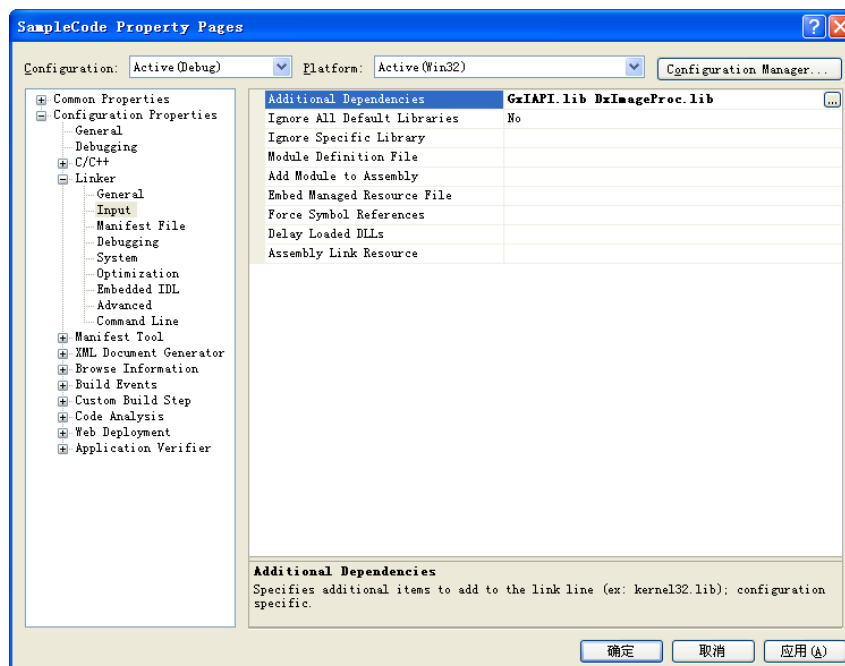


图 2-6

2.1.2. Linux

2.1.2.1. 推荐配置

- 1) 支持 Linux 内核版本：>=3.5.0(32 和 64 位)
g++版本：>= 4.8
glibc 版本：>= 2.17
- 2) 推荐使用的系统发布版本：Ubuntu 16.04 以上、CentOS7、CentOS8。

2.1.2.2. 控制台程序

➤ Makefile

1) 在编译程序之前，需要将 SDK 库 inc 目录中 GxIAPL.h、DxImageProc.h 两个头文件拷贝到工程目录下，或者通过“-I”编译参数，将 SDK 库的 inc 目录添加到搜索路径。

2) 应用程序在链接的时候，必须要链接到 libgxiapi.so (-lgxiapi)。

安装包中有一系列的示例程序，下面的内容是示例程序的通用的 Makefile 文件内容。

红色标记：**samplename** 是编译输出的程序的名称。

```
# Makefile for sample program
.PHONY : all clean

# the program to build
NAME    := samplename

# Build tools and flags
CXX     := g++
```

```
LD      := g++
SRCS    := $(wildcard *.cpp)
OBJS    := $(patsubst %cpp, %o, $(SRCS))
CPPFLAGS := -w -I./

LDLFLAGS := -lgxiapi -lpthread

all      : $(NAME)

$(NAME)  : $(OBJS)
$(LD) -o $@ $^ $(CPPFLAGS) $(LDLFLAGS)

%.o      : %.cpp
$(CXX) $(CPPFLAGS) -c -o $@ $<
clean    :
$(RM) *.o $(NAME)
```

2.1.2.3. 基于 qt 的示例程序

下面介绍 qtcreator 集成开发环境的安装方法及配置步骤。

➤ 安装 qtcreator

1) 安装

sudo apt-get install qtcreator

2) 运行

终端运行 qtcreator 即可

➤ qtcreator 下配置环境 (以 Qt Creator 3.5.1 为例)

步骤一：运行程序后，显示界面如下图，选择“Open Project”，在弹出对话框中选择要打开的工程的文件。

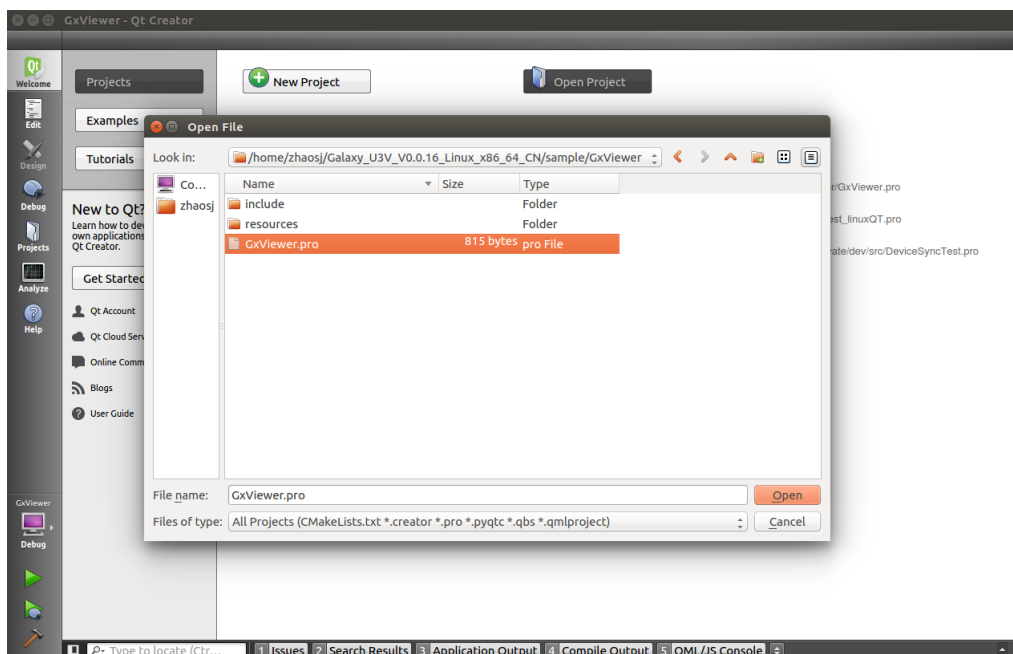


图 2-7

步骤二：打开工程时，若工程无法打开，提示 Kit 相关内容，则需检查 qtcreator 的配置。选择菜单 Tool -> Options -> Build & Run -> Qt Versions，查看是否检测到 qmake，若没有自动检测到，则需手动加载，如下图。若工程能够正常打开，则可忽略该步骤。

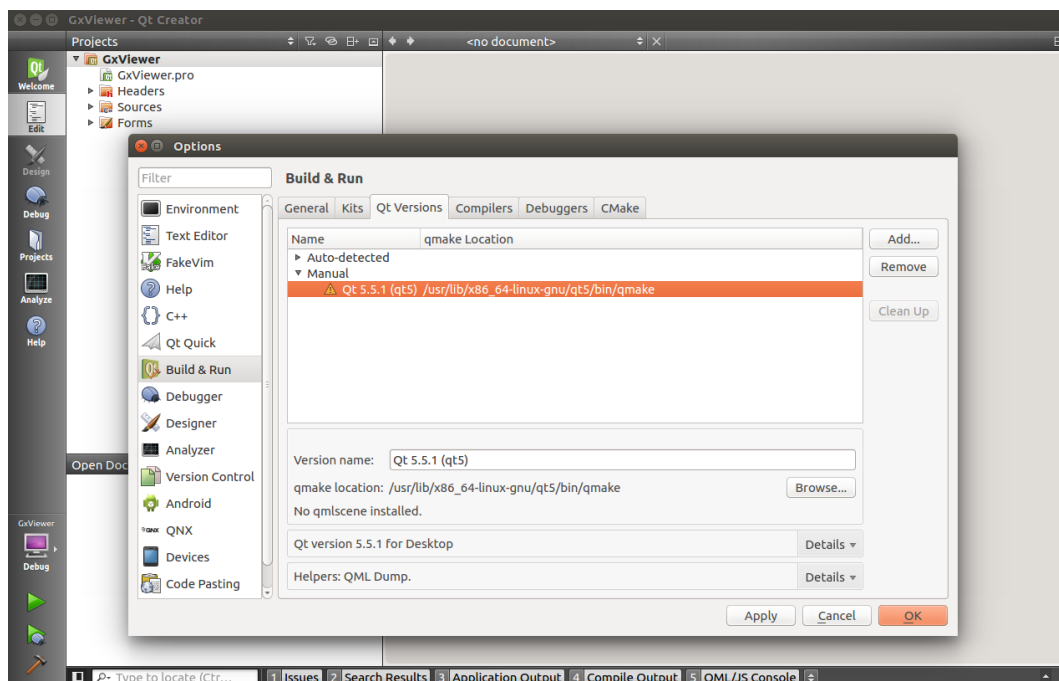


图 2-8

步骤三：qmake 加载成功后，在 Tool -> Options -> Build & Run -> Kits 标签页，为要使用的构建选项的 Qt Version 添加 qmake，如下图。若工程能够正常打开，则可忽略该步骤。

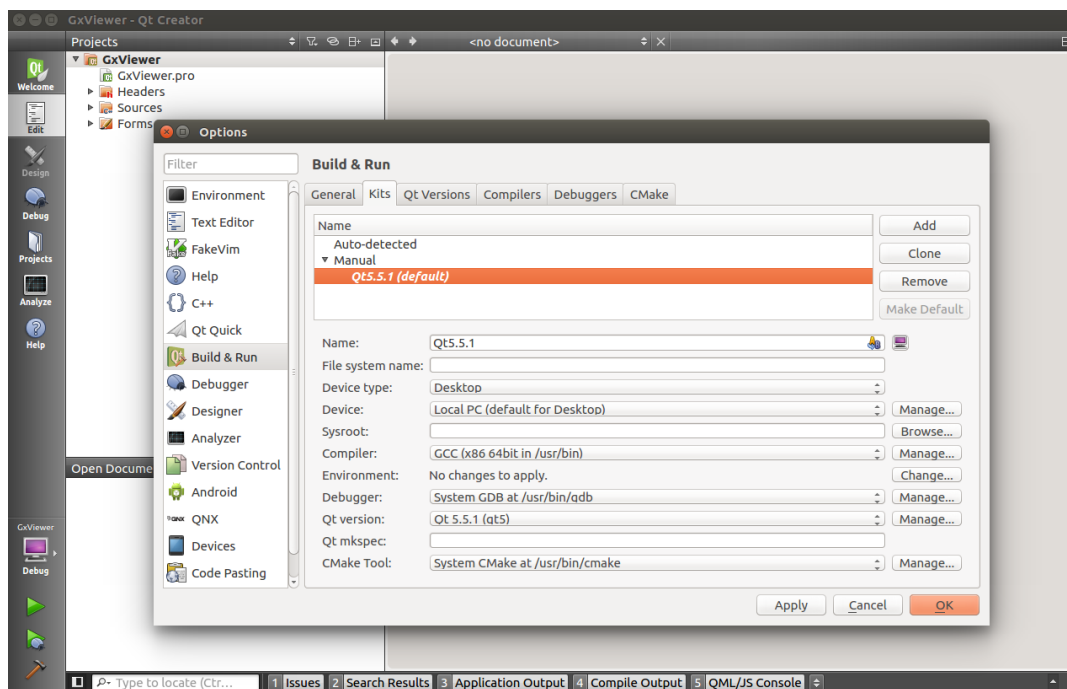


图 2-9

步骤四：工程打开成功后，编译运行即可。

2.1.3. macOS

2.1.3.1. 安装开发环境

安装 daheng macOS sdk 安装包

1) 将提供的 dmg 中后缀为.pkg 安装包双击进行安装。

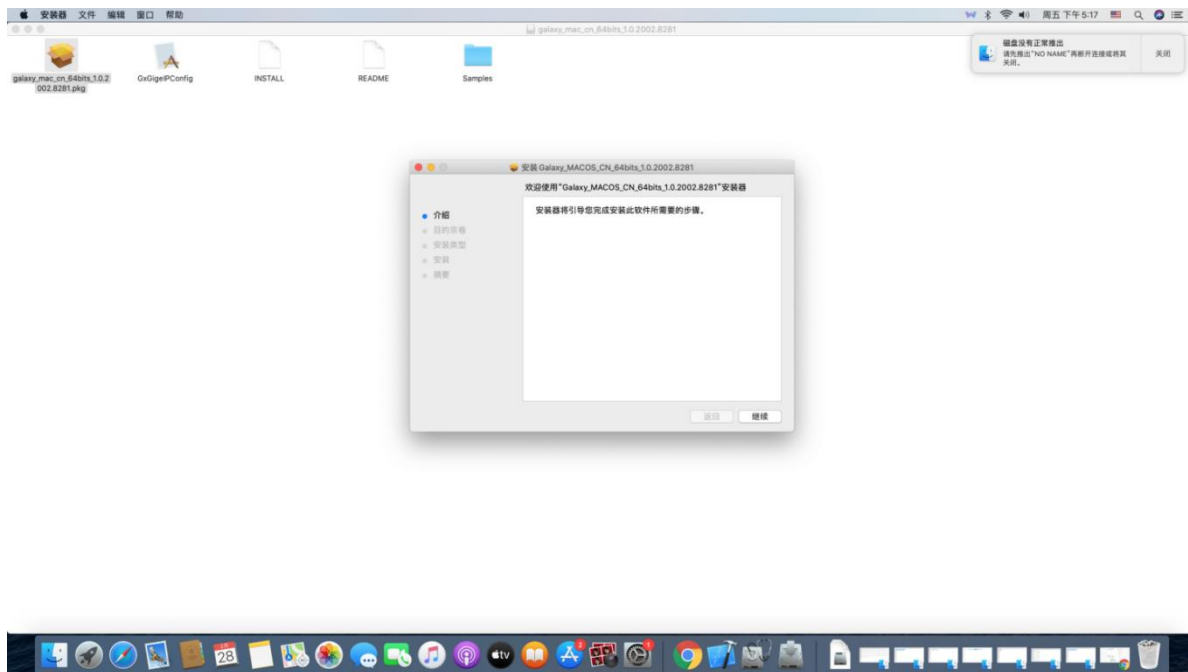


图 2-10

2) 点击继续按钮，选择安装的磁盘，galaxy.framework 安装在该磁盘的/Library/Frameworks/目录下。

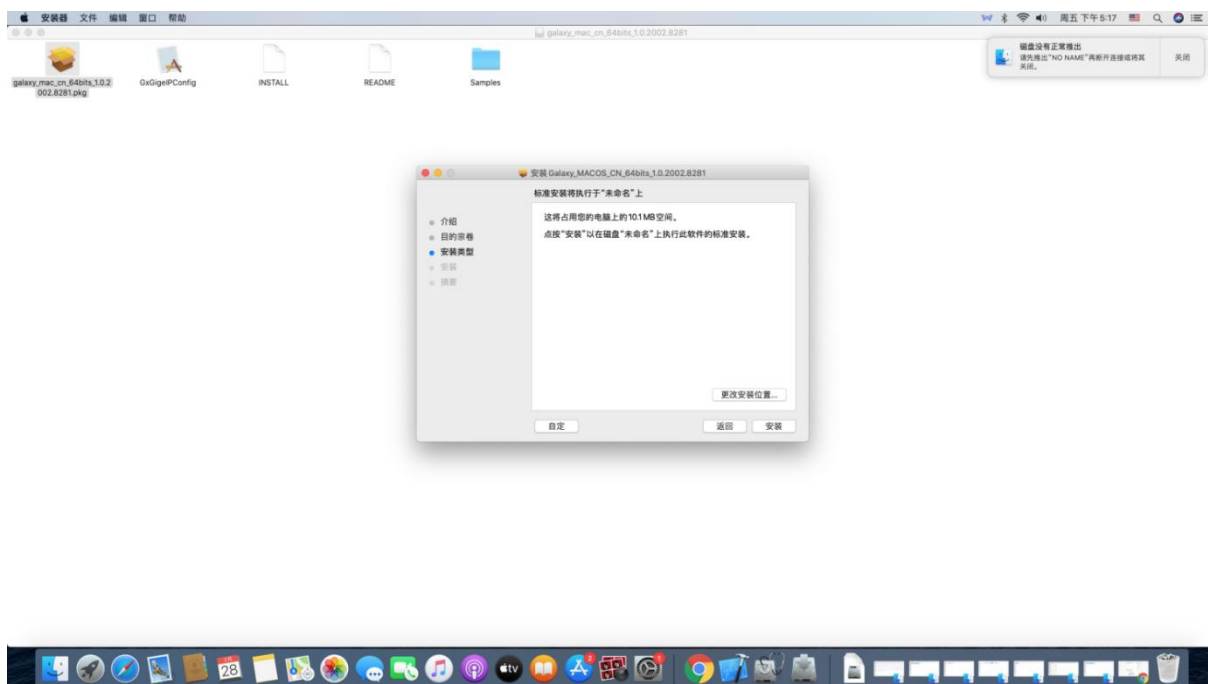


图 2-11

3) 进行身份验证。

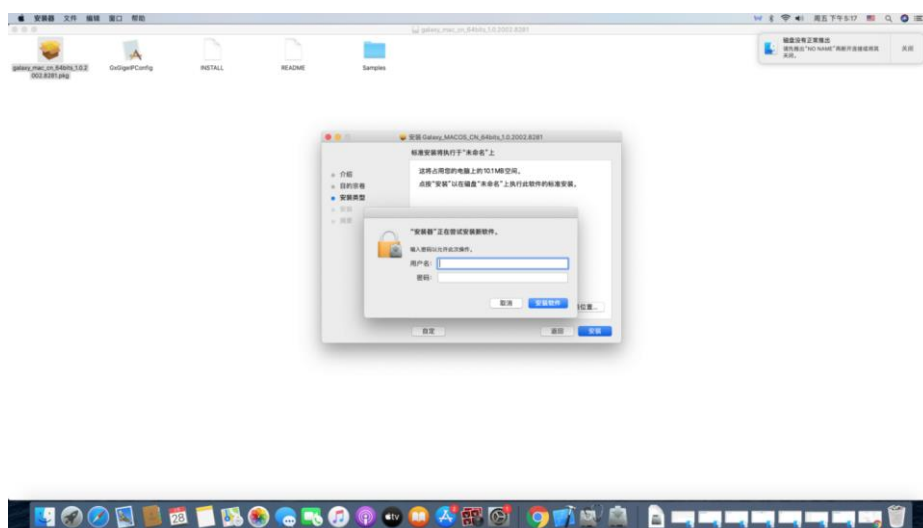


图 2-12

4) 在弹出的安装界面对话框中一直点击继续按钮，直至安装完成。

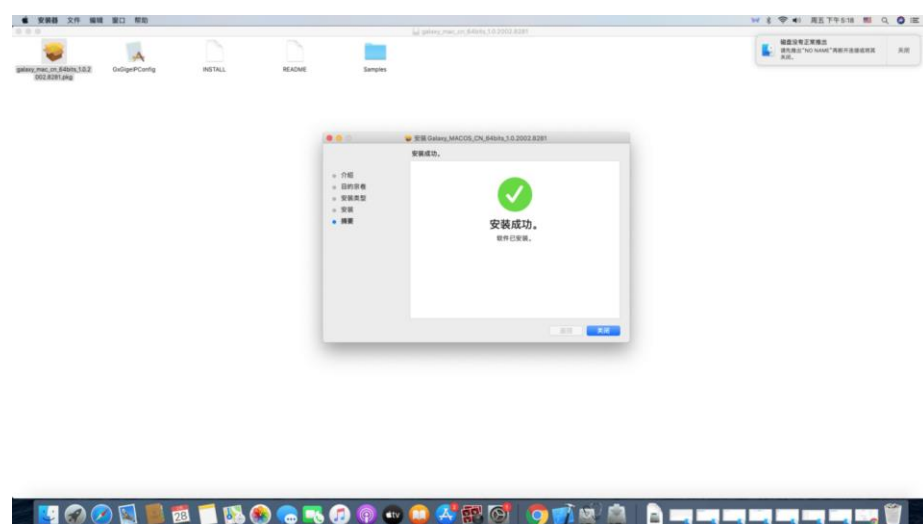


图 2-13

2.1.3.2. 基于 Xcode 的示例程序

安装包中的 sample 程序在 Xcode 11.3(11C29)版本的 Xcode 下可正常编译运行。下面介绍 Xcode 11.3 的 Xcode 集成开发环境的安装方法及配置步骤。

➤ macOS 平台安装基于 Xcode 11.3 的 Xcode

1) 安装

在 App Store 中搜索 Xcode，点击安装按钮。

2) 运行

在启动台中运行 Xcode 即可。

2.1.3.3. Xcode (基于 Xcode 11.3) 下配置环境

步骤一：运行程序后，在菜单栏中选择“File->New->Project”创建一个新工程，显示界面如下图，选择“macOS->Command Line Tool”，在弹出对话框中创建一个全新的工程。

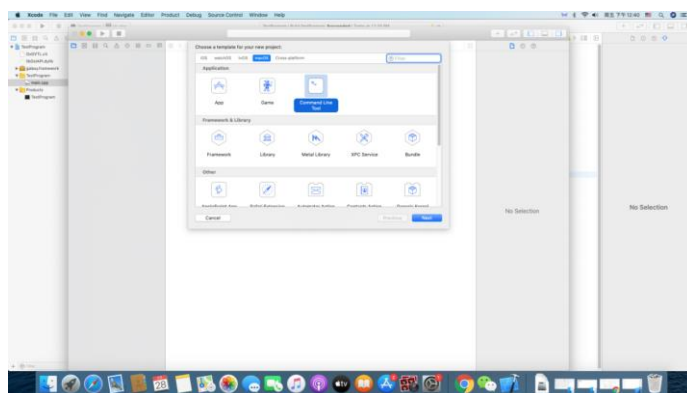


图 2-14

步骤二：工程中添加头文件及库文件。首先添加 galaxy.framework 到工程中，galaxy.framework 的路径在 /Library/Frameworks/ 目录下。右键点击工程名，点击“Add Files to...”按钮，使用快捷键 Command+Shift+G 可访问 galaxy.framework 所在目录，将 galaxy.framework 添加到工程中。然后使用同样的方法再将 /Library/Frameworks/galaxy.framework/Versions/A/Libraries/ 目录下的 GxIAPL.cti 及 libGxIAPL.dylib 添加到工程中。

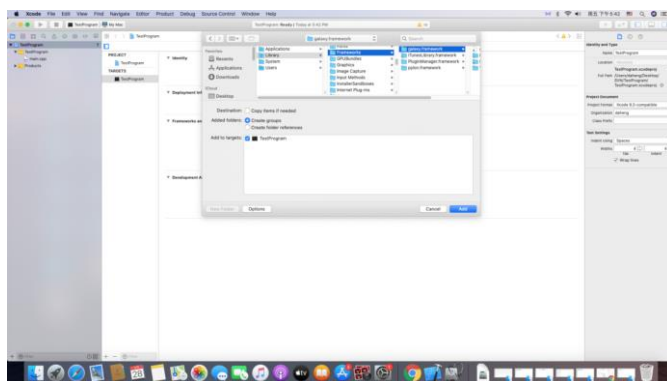


图 2-15

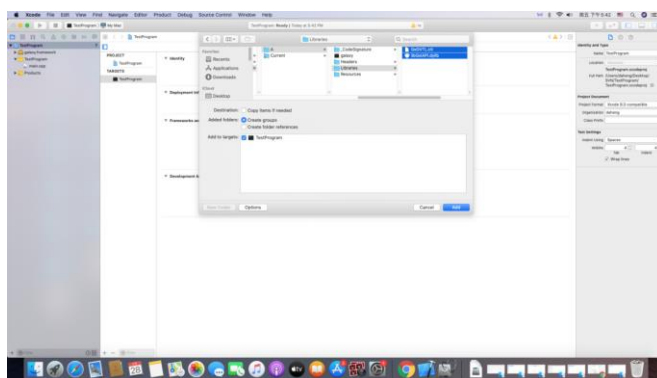


图 2-16

步骤三：加载 Framework 及库文件成功后，鼠标点击工程名，选择 “TestProgram->Build Settings->Linking->Runpath Search Paths”，在该项中添加 galaxy.framework 所在目录：
/Library/Frameworks/。

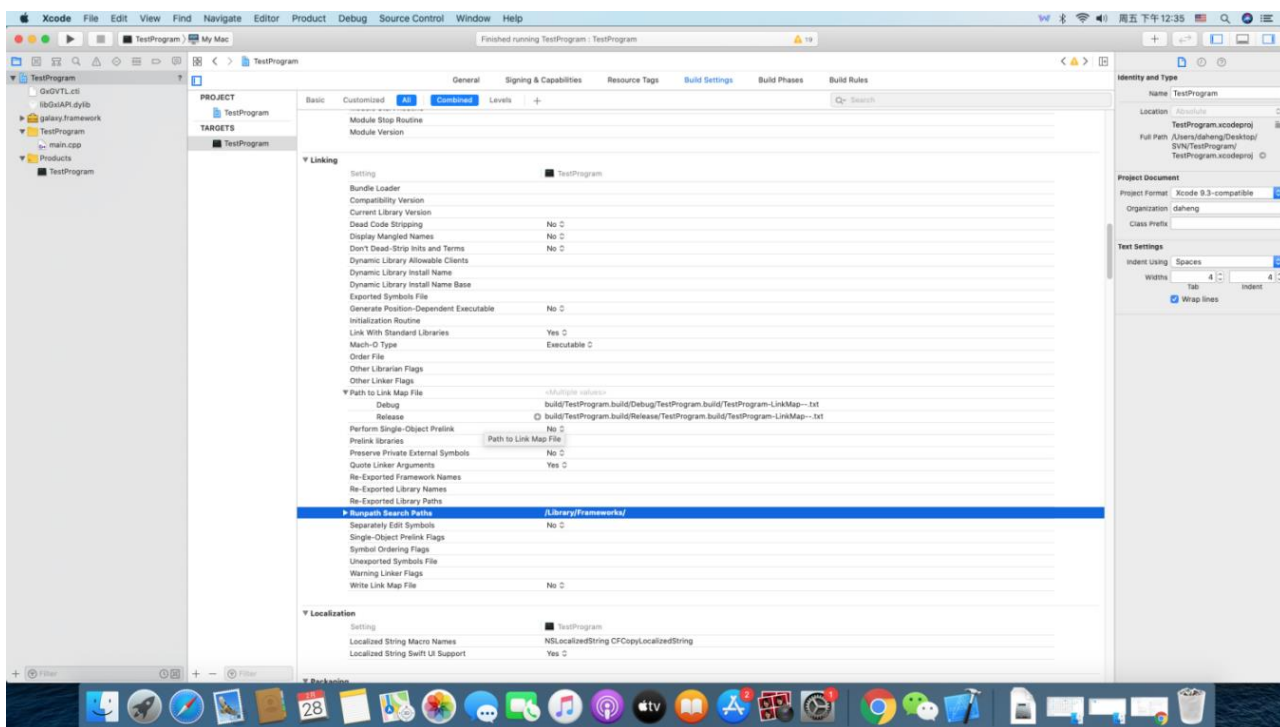


图 2-17

步骤四：引用头文件，编写一个简单程序，编译运行，接口调用成功即证明 Xcode 环境配置成功。

2.2. 快速上手

- 初始化和反初始化 GxIAP 运行时库
- 枚举相机并获取信息
- 配置相机 IP 地址
- 打开关闭相机
- 控制相机功能
- DQBuf 图像采集
- 回调图像采集

2.2.1. 初始化和反初始化 GxIAP 运行时库

在使用 GxIAP 接口([GXCloseLib\(\)](#)、[GXGetLastError\(\)](#)除外)之前必须调用 [GXInitLib\(\)](#)对库进行初始化操作，当用户应用程序退出的时候也必须调用 [GXCloseLib\(\)](#)来释放资源与 [GXInitLib\(\)](#)相对应，本文所有的片段示例代码中默认都进行了初始化和反初始化 GxIAP 库的操作。

代码样例：

```
#include "GxIAPI.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //在起始位置调用 GXInitLib() 进行初始化, 申请资源
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //使用 GxIAPI
    //在结束的时候调用 GXCloseLib() 释放资源
    emStatus = GXCloseLib();
    return 0;
}
```

2.2.2. 枚举相机并获取信息

用户可以调用 [GXUpdateAllDeviceList\(\)](#)接口来枚举当前可用的设备，返回设备个数，然后在不打开相机的前提下使用 [GXGetDeviceInfo\(\)](#)来获取设备基础信息。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    for (uint32_t i = 0; i < ui32DeviceNum; ++i)
    {
        GX_DEVICE_INFO stDeviceInfo;
        GXGetDeviceInfo(i, &stDeviceInfo);
    }
}
```

如果想枚举指定类型的相机，那么还可以使用 [GXUpdateAllDeviceListEx\(\)](#)。如下枚举网络相机

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

emStatus = GXUpdateAllDeviceListEx(GX_TL_TYPE_GEV, &ui32DeviceNum,
                                    1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEVICE_INFO stDeviceInfo;
    unsigned int nDeviceIp;

    //获取第一台设备的网络 IP 信息
```

```
emStatus = GXGetDeviceInfo(1, &stDeviceInfo);
nDeviceIp = stDeviceInfo.DevInfo.stGEVDevInfo.nCurrentIp;
}
```

2.2.3. 获取 Interface 信息

用户通过调用 GXGetInterfaceInfo()接口获取 Interface 的信息，参数类型为 GX_INTERFACE_INFO。

样例代码：

```
//获取 Interface 信息
GX_INTERFACE_INFO stInterfaceInfo = NULL;
GX_STATUS emStatus = GXGetInterfaceInfo(1, &stInterfaceInfo);
```

2.2.4. 获取 Interface 对象

用户通过调用 GXGetInterfaceHandle() 接口获取 Interface，参数类型为整型，下标从 1 开始计数。

样例代码：

```
//获取 Interface 指针
GX_IF_HANDLE hIFHandle = NULL;
GX_STATUS emStatus = GXGetInterfaceHandle(1, &hIFHandle);
```

2.2.5. 配置相机 IP 地址

- 1) 对于 GigE 网络相机，用户可以调用 [GXGigEIpConfiguration\(\)](#)配置相机 IP 地址，使用这种方式设置的 IP 地址在相机掉电重启后仍然有效。调用此函数需要传入目的相机的 MAC 地址，可以通过调用 [GxGetDeviceIpInfo\(\)](#)获得。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

const char* chMAC = "00-21-49-00-00-00";
const char* chIpAddress = "192.168.10.10";
const char* chSubnetMask = "255.255.255.0";
const char* chDefaultGateway = "192.168.10.2";
const char* chUserID = "Daheng Imaging";
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;

//本例以永久 IP 配置方式说明,其他 IP 配置方式类似
//设置为永久 IP 配置方式
emStatus = GXGigEIpConfiguration(chMAC, emIpConfigureMode,
                                   chIpAddress, chSubnetMask,
                                   chDefaultGateway, chUserID);
```

- 2) 用户同样可以调用 [GXGigEForceIp\(\)](#)配置相机 IP 地址，使用这种方式设置的 IP 地址只在本次使用中有效，相机掉电重启后会恢复以前的 IP 地址。调用此函数需要传入目的相机的 MAC 地址，可以通过调用 [GxGetDeviceIpInfo\(\)](#)获得。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
const char* chMAC = "00-21-49-00-00-00";
const char* chIpAddress = "192.168.10.10";
const char* chSubnetMask = "255.255.255.0";
const char* chDefaultGateway = "192.168.10.2";

//ForceIp
emStatus = GXGigEForceIp(chMAC, chIpAddress, chSubnetMask,
                        chDefaultGateway);
```

2.2.6. 打开关闭相机

调用 [GXUpdateAllDeviceList\(\)](#)接口枚举设备后，返回的设备个数如果大于 0，说明当前有可用设备，用户调用 [GxOpenDevice\(\)](#)接口来打开设备。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
GX_OPEN_PARAM stOpenParam;

emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEV_HANDLE hDevice = NULL;

    //打开枚举列表中的第一台设备。
    //假设枚举到了3台可用设备，那么用户可设置stOpenParam参数的pszContent字段为1、2、3
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";

    //通过序列号打开设备
    //stOpenParam.openMode = GX_OPEN_SN;
    //stOpenParam.pszContent = "EA00010002";

    //通过 IP 地址打开设备
    //stOpenParam.openMode = GX_OPEN_IP;
    //stOpenParam.pszContent = "192.168.40.35";

    //通过 MAC 地址打开设备
    //stOpenParam.openMode = GX_OPEN_MAC;
    //stOpenParam.pszContent = "54-04-A6-C2-7C-2F";

    emStatus = GXOpenDevice(&stOpenParam, &hDevice);

    //操作设备：控制、采集
    //关闭设备
    emStatus = GXCloseDevice(hDevice);
}
```

[GxOpenDevice\(\)](#)可用指定访问方式(独占、控制等)、打开方式(通过IP、序列号、MAC、Index等)打开相机。详见后面[GxOpenDevice\(\)](#)接口的解释。

2.2.7. 控制相机功能

为了方便用户开发,演示程序(GalaxyView.exe)提供了相机功能属性的名称和类型,用户可以据此调用配套的接口进行相机功能配置。如下图所示:

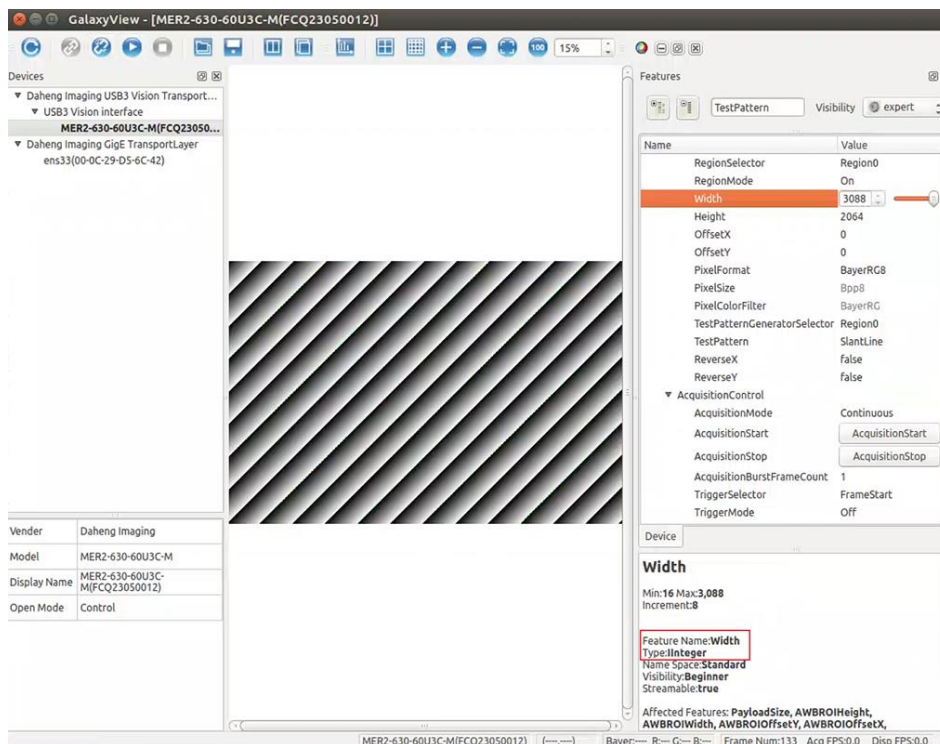


图 2-18

用户可以通过如下介绍,全面了解不同类型属性的访问接口。

整型数据节点相关接口:

[GXGetIntValue\(\)](#): 读取值, 最小值、最大值、步长。

[GXSetIntValue\(\)](#): 设置整型节点值。

代码样例:

```
//如上图 Feature Name 对应的值 Width 为接口的第二个参数
//Type 对应的值 IInteger 表明属性为整形对应 GX_INT_VALUE
//读取图像宽度
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stIntValue;
int64_t i64Width = 0;

//获取图像宽当前值
emStatus = GXGetIntValue(hDevice, "Width", &stIntValue);
i64Width = stIntValue.nCurValue;
```

//设置图像宽度为最小值

```
int64_t i64Value = stIntValue.nMin;
emStatus = GXSetIntValue(hDevice, "Width", i64Value);
```

浮点型数据节点相关接口：

[GXGetFloatValue\(\)](#)：读取当前值，最小值、最大值、步长。

[GXSetFloatValue\(\)](#)：设置浮点类型的节点值。

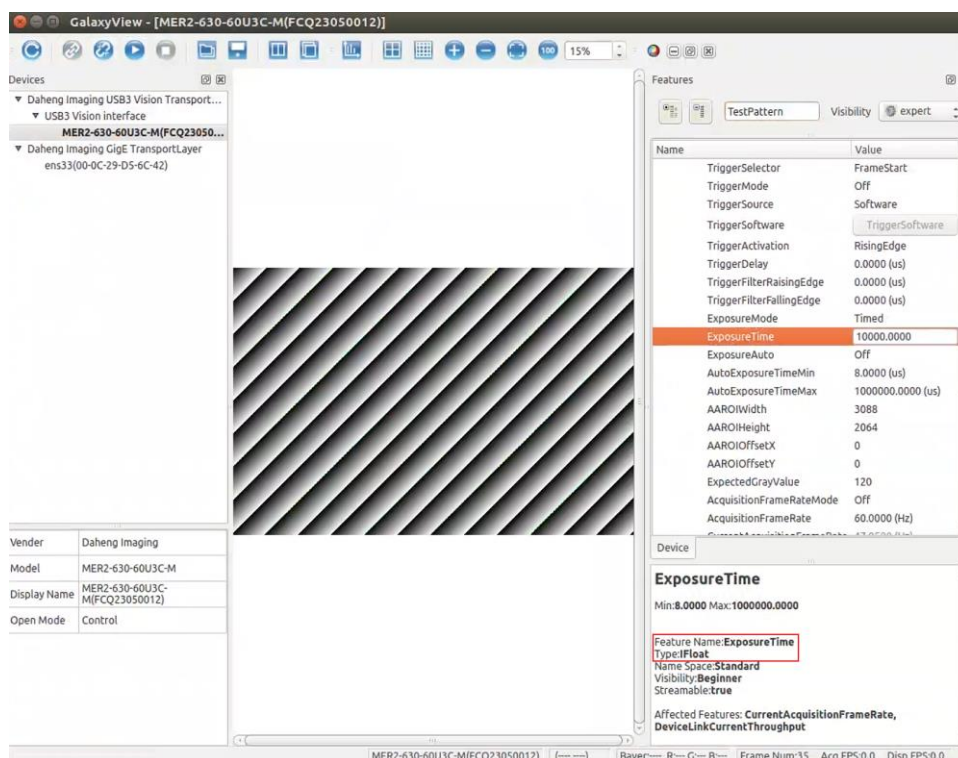


图 2-19

代码样例：

//如上图 Feature Name 对应的值 ExposureTime 为接口的第二个参数

//Type 对应的值 IFloat 表明属性为浮点型形对应 GX_FLOAT_VALUE

//读取曝光时间

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stFloatValue;
```

//读取曝光时间当前值

```
emStatus = GXGetFloatValue(hDevice, "ExposureTime", &stFloatValue);
double dExposureTime = stFloatValue.dCurValue;
```

//设置曝光时间

```
double dValue = 3000;
emStatus = GXSetFloatValue(hDevice, "ExposureTime", dValue);
```


枚举型数据节点相关接口：

[GXGetEnumValue\(\)](#)：读取当前值，枚举项的个数，枚举项的值及枚举项的描述

[GXSetEnumValue\(\)](#)：设置枚举类型的节点值。

[GXSetEnumValueByString\(\)](#)：设置枚举类型的节点值。

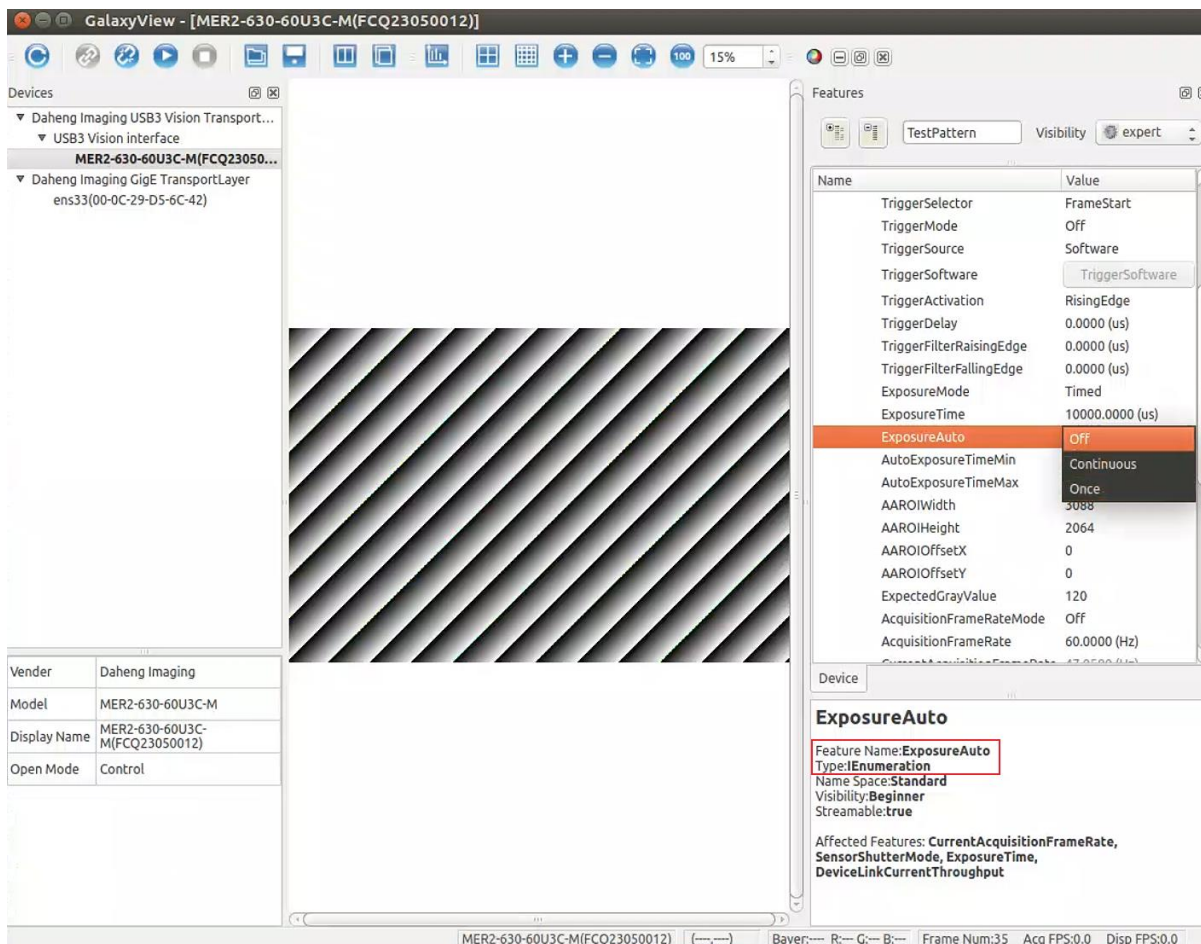


图 2-20

代码样例：

```
//如上图 Feature Name 对应的值为接口的第二个参数
//Type 对应的值 IEnumeration 表明属性为枚举对应 GX_ENUM_VALUE
//读取自动曝光
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_ENUM_VALUE stEnumValue;

//读取自动曝光当前枚举值
emStatus = GXGetEnumValue(hDevice, "ExposureAuto", &stEnumValue);
int64_t i64ExposureAuto = stEnumValue.stCurValue.nCurValue;

//设置自动曝光
int64_t i64Value = GX_EXPOSURE_AUTO_CONTINUOUS;
```

```
emStatus = GXSetEnumValue(hDevice, "ExposureAuto", i64Value);

//设置自动曝光上图中的下拉列表对应此接口的第三个参数
emStatus = GXSetEnumValueByString(hDevice, "ExposureAuto",
                                   "Continuous");
```

布尔型数据节点相关接口：

[GXGetBoolValue\(\)](#)：读取布尔类型的节点值。

[GXSetBoolValue\(\)](#)：设置布尔类型的节点值。

字符串型数据节点相关接口：

[GXGetStringValue\(\)](#)：读取当前值，字符串当前值的长度，单位字节，字符串的最大长度，单位字节。

[GXSetStringValue\(\)](#)：设置字符串类型的节点值。

寄存器型数据节点相关接口：

[GXGetRegisterLength\(\)](#)：查询块儿数据的长度，用此长度申请内存，然后再调用 [GXGetRegisterValue](#) 读取数据。

[GXGetRegisterValue\(\)](#)：读取寄存器类型的节点值。

[GXSetRegisterValue\(\)](#)：设置寄存器类型的节点值。

命令型数据节点相关接口：

[GXSetCommandValue\(\)](#)：发送命令。

2.2.8. DQBuf 图像采集

调用 [GXOpenDevice\(\)](#)接口打开设备后，顺序调用 [GXStreamOn\(\)](#)接口可使流开采和远端设备开采，此时循环调用 [GXDQBuf\(\)](#)、[GXQBuf\(\)](#)（[GXDQAllBufs\(\)](#)、[GXQAllBufs\(\)](#)同理，仅 Linux 支持）可实现连续采集图像。调用 [GXQBuf\(\)](#)接口将图像 buf 放回 GxI API 库后，不能再访问该图像 buf 指针。

代码样例：

```
//-----
//GXDQBuf 接口一次获取一帧图像，本样例代码演示的就是如何用此接口获取一帧图像
//-----
#include "GxI API.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }
```



```

}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}

//打开设备
GX_OPEN_PARAM stOpenParam;
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //定义 GXDQBuf 的传入参数
    GX_FRAME_BUFFER pFrameBuffer;

    //开采
    #ifdef __linux__
    emStatus = GXStreamOn(hDevice);
    #else
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
    #endif
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //调用 GXDQBuf 取一帧图像
        emStatus = GXDQBuf(hDevice, &pFrameBuffer, 1000);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            if (pFrameBuffer->nStatus == GX_FRAME_STATUS_SUCCESS)
            {
                //图像获取成功
                //对图像进行处理
            }

            //调用 GXQBuf 将图像 buf 放回库中继续采图
            emStatus = GXQBuf(hDevice, pFrameBuffer);
        }
    }

    //停采
    #ifdef __linux__
    emStatus = GXStreamOff(hDevice);
    #else
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");
    #endif
}

```

```

    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();
    return 0;
}

```

2.2.9. 回调图像采集

术语：

- 图像处理回调函数——用户编写的图像处理函数，GxI API 规定了函数的返回值、形参等内容（见 GxI API 库说明->类型->[回调函数类型](#)->GXCaptureCallBack()）。
- 注册回调函数——用户调用 [GxRegisterCaptureCallback\(\)](#)接口将用户编写的图像处理回调函数指针传给 GxI API 库（见 [GxRegisterCaptureCallback\(\)](#)接口的详细用法）。
- 注销回调函数——用户调用 [GxUnregisterCaptureCallback\(\)](#)接口通知 GxI API 库释放掉用户注册的回调函数指针。

代码样例：

```

#include "GxI API.h"

//图像回调处理函数
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame->status == GX_FRAME_STATUS_SUCCESS)
    {
        //对图像进行某些操作
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //枚举设备列表
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
}

```

```

}

//打开设备
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus= GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //设置相机的流通道包长属性，提高网络相机的采集性能
    bool bImplementPacketSize = false;
    uint32_t ui32PacketSize = 0;

    //判断设备是否支持流通道数据包功能
    GX_NODE_ACCESS_MODE emAccessMode;
    emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                    &emAccessMode);

    bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
        ? true : false;

    if (bImplementPacketSize)
    {
        //获取当前网络环境的最优包长
        emStatus = GXGetOptimalPacketSize(hDevice, &ui32PacketSize);

        //将最优包长设置为当前设备的流通道包长值
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                ui32PacketSize);
    }

    //注册图像处理回调函数
    emStatus = GXRegisterCaptureCallback(hDevice,
                                         NULL, OnFrameCallbackFun);

    //发送开采命令
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

    //-----
    //
    //在这个区间图像会通过 OnFrameCallbackFun 接口返给用户
    //
    //-----

    //发送停采命令
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

    //注销采集回调

```

```

        emStatus = GXUnregisterCaptureCallback(hDevice);
    }

    emStatus = GXCLOSE_DEVICE(hDevice);
    emStatus = GXCLOSE_LIB();

    return 0;
}

```

另外用户还可以通过 [GxGetImage\(\)](#)接口来获取图像，但是此获取图像的方式不能与回调取图同时使用（见 [GxGetImage\(\)](#)接口的使用详解）。

2.3. 使用千兆网相机调试程序

Windows 的用户在使用 Visual Studio 开发平台在 Debug 模式下调试千兆网相机的时候可能会遇到因为心跳超时而导致设备掉线的情况。应用程序必须以固定的时间间隔发送心跳包到设备，如果设备没有接收到心跳包则会认为当前连接已经断开，从而不会再接收从应用程序发送的任何命令。

当用户正常运行应用程序的时候，底层库会正常发送心跳包，保持和设备的连接状态，但是当用户在应用程序中设置断点调试的时候，程序运行到这些断点时，调试器会暂停所有线程，包括发送心跳包的线程，所以当用户在 Debug 模式下单步调试代码的时候，就不会发送心跳包到设备。

可以通过增加心跳超时时间的方式来解决这个问题。在调试状态下，千兆网传输层软件会自动在打开设备时，将设备超时时间设置为 5 分钟。如果在系统中添加环境变量"GIGE_HEARTBEAT_TIMEOUT"，并且这个环境变量的值大于 0，那么设备心跳超时时间的值会被自动设置为这个环境变量的值。

用户可以通过以下两种方式修改心跳超时时间。

第一种方式，在程序中打开设备后添加如下代码：

```

//hDevice 为设备句柄，已经通过 GXOpenDevice 接口打开设备
//设置心跳时间为 5 分钟
GXSetIntValue(hDevice, "GevHeartbeatTimeout", 300000);

```

第二种方式，在系统中添加环境变量"GIGE_HEARTBEAT_TIMEOUT"，并赋给一个大于零的值，添加完成后使用应用程序打开设备，设备的心跳超时时间就会自动变成此环境变量的值。注意只需要在开发的系统上添加此环境变量，因为无论在 Debug 还是 Release 的应用程序中都起作用。

注意：如果用户将心跳超时时间设置的非常长，当结束程序的时候没有调用 GXCLOSE_DEVICE()接口来关闭设备，或者也没有调用 GXCLOSE_LIB()接口释放资源，这就会导致设备在心跳时间内无法复位，从而导致用户再次尝试打开设备的时候失败。可通过复位或重连设备解决此问题，有两种方式实现设备的复位或重连：

- 1) 在 IP 配置工具中直接选择复位设备或重连设备；
- 2) 通过接口 GXGigEResetDevice()实现设备的复位或重连。

复位设备：等同于给设备掉电上电一次，相机内程序全部重新加载。

重连设备：等同于软件接口关闭设备，执行此操作后，允许用户重新打开设备。

3. 相机功能属性说明

3.1. 设备控制

3.1.1. 设备信息

● 相关属性

属性名称	属性类别	可访问属性	中文名称
DeviceVendorName	STRING	RO	厂商名称
DeviceModelName	STRING	RO	设备型号
DeviceFirmwareVersion	STRING	RO	设备固件版本
DeviceVersion	STRING	RO	设备版本
DeviceSerialNumber	STRING	RO	设备序列号
FactorySettingVersion	STRING	RO	出厂参数版本
DevicePHYVersion	STRING	RO	设备网络芯片版本
DeviceISPFirmwareVersion	STRING	RO	设备 ISP 固件版本
DeviceUserID	STRING	RW	用户自定义名称

● 代码样例

```

//以获取厂商名称为例
//获取厂商名称
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_STRING_VALUE stVendorNameValue;
const char* pszVendorName = NULL;

emStatus = GXGetStringValue(hDevice, "DeviceVendorName",
&stVendorNameValue);

pszVendorName = stVendorNameValue.strCurValue;

//获取自定义名称
GX_STRING_VALUE stUserIDValue;
const char* pszUserID = NULL;

emStatus = GXGetStringValue(hDevice, "DeviceUserID",
&stUserIDValue);
pszUserID = stUserIDValue.strCurValue;

//设置用户自定义名称
emStatus = GXSetStringValue(hDevice, "DeviceUserID", "TestUserID");
    
```

3.1.2. 设备控制

● 名词解释

设备复位：还原设备打开状态为初始状态，同设备重新上电的状态。调用完成后，主机端将失去与当前设备的连接。并且由于是在设备打开状态下才能设置该接口，因此调用完成之后，需要开发人员主动调用GXCloseDevice()接口关闭设备，以释放相应的内存资源。

时间戳频率：只读信息，它表示时间戳计数的频率，其值为 125000000Hz。

时间戳锁存：锁取当前的时间戳值，即从设备开始上电到时间戳锁存命令调用时经历的时间值，该时间值需要通过“时间戳锁存值”来读取。

重置时间戳：复位时间戳计数器，从 0 开始重新计数。

重置时间戳锁存：先锁取当前的时间戳值，然后复位时间戳计数器，从 0 开始重新计数。

设备温度选择：选择设备中要测量温度的位置。

设备温度：设备温度（摄氏度）。它测量的是设备温度选择的位置。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
DeviceReset	COMMAND	WO	设备复位
TimestampTickFrequency	INT	RO	时间戳频率
TimestampLatch	COMMAND	WO	时间戳锁存
TimestampReset	COMMAND	WO	重置时间戳
TimestampLatchReset	COMMAND	WO	重置时间戳锁存
TimestampLatchValue	INT	RO	时间戳锁存值
DeviceTemperatureSelector	ENUM	RW	设备温度选择
DeviceTemperature	FLOAT	RO	设备温度

● 代码样例

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;

//发送重置时间戳命令
emStatus = GXSetCommandValue(hDevice, "TimestampReset");

//发送设备复位命令
emStatus = GXSetCommandValue(hDevice, "DeviceReset");

//获取当前设备温度选择的位置
GX_ENUM_VALUE stValue;
emStatus = GXGetEnumValue(hDevice, "DeviceTemperatureSelector",
                           &stValue);

const char* pszString = stValue.stCurValue.strCurSymbolic;

```

```
//设置当前设备温度选择的位置
emStatus = GXSetEnumValueByString(hDevice,
                                   "DeviceTemperatureSelector",
                                   "Sensor");

//获取当前设备温度选择的位置的温度
GX_FLOAT_VALUE stTemperatureValue;
double dCurValue = 0;
emStatus = GXGetFloatValue(hDevice, "DeviceTemperature",
                           &stTemperatureValue);

dCurValue = stTemperatureValue.dCurValue;
```

3.2. 图像格式

3.2.1. 感兴趣区域

● 名词解释

ROI：感兴趣区域（region of interest），相对于相机传感器的一个可配置的矩形选择区域，相机只输出被选择区域的数据，在矩形框选择范围之外的数据将被忽略。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
SensorWidth	INT	RO	传感器宽度
SensorHeight	INT	RO	传感器高度
WidthMax	INT	RO	最大宽度
HeightMax	INT	RO	最大高度
Width	INT	RW	图像宽度
Height	INT	RW	图像高度
OffsetX	INT	RW	水平偏移
OffsetY	INT	RW	垂直偏移

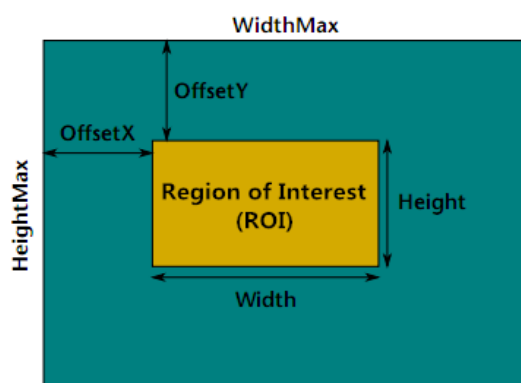


图 3-1 ROI 相关参数

● 效果图



图 3-2 原图



图 3-3 应用 ROI 后

● 代码样例

```
//设置一个 offset 偏移为 ( 0,0 ) , 600×400 尺寸的区域
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Width= 600;
int64_t i64Height= 400;
int64_t i64OffsetX = 0;
int64_t i64OffsetY = 0;
emStatus = GXSetIntValue(hDevice, "Width", i64Width);
emStatus = GXSetIntValue(hDevice, "Height", i64Height);
emStatus = GXSetIntValue(hDevice, "OffsetX", i64OffsetX);
emStatus = GXSetIntValue(hDevice, "OffsetY", i64OffsetY);
```

● 注意事项

1) 有两对属性会影响到 ROI 的最大尺寸：“SensorWidth”和“SensorHeight”。

决定图像传感器的有效分辨率,决定了可用的像素点的个数,它的值是固定的。在默认模式下(没有 Binning, 没有 Decimation, 没有 ROI)。

图像的大小等于：“SensorWidth”×“SensorHeight”

“WidthMax”和“HeightMax”这两个值决定了 ROI 当前可用的最大尺寸,ROI 的宽的最大值可能被

Binning 或者 Decimation 影响。在默认模式下（没有 Binning，没有 Decimation）：

$$\text{"WidthMax"} \times \text{"HeightMax"} = \text{"SensorWidth"} \times \text{"SensorHeight"}$$

2) 为了保证 ROI 的有效性，ROI 的四个属性之间需要遵守关系公式，如下：

$$\text{"OffsetX"} + \text{"Width"} \leq \text{"WidthMax"} \text{ (当前图像的最大宽)}$$

$$\text{"OffsetY"} + \text{"Height"} \leq \text{"HeightMax"} \text{ (当前图像的最大高)}$$

上面两个公式说明了四个属性的最大值是动态变化的，比如调整 Width 的值，会影响 OffsetX 的可调最大值，这种联动关系 GxI API 内部已经实现。

3.2.2. 图像分辨率

● 名词解释

Binning 和 Decimation 是直接作用于相机传感器内部，在相机出图之前进行的二次抽样处理操作，这两个功能通过修改水平或者垂直方向的分辨率达到提高帧率的效果，它和 ROI 的差别是，ROI 是对视野进行裁剪，而 Binning 和 Decimation 则是对整个图像抽行或者合并处理，并不影响这个视野的范围。

➢ Binning：它是一种图像输出模式，分为水平像素 Binning 和垂直像素 Binning。在该模式下，根据用户选择的合并模式将相邻像元的电荷相加，或者取相邻像元电荷的平均值，以一个像素的模式进行输出。使用 Binning 的优点是能将几个像素联合起来作为一个像素使用，提高灵敏度，输出速度，降低分辨率，当行和列同时采用 Binning 时，图像的纵横比并不改变，当采用 2:2 Binning，图像的解析度将减少 75%。

➢ Decimation：像元跳跃式输出，挑选第 N 个（水平或者垂直）像元进行输出，其他的像元均忽略。

➢ 水平像素 Binning 模式：该模式有 Sum 和 Average 两种模式。在 Sum 模式下，将相邻的电荷相加在一起，然后以一个像素模式输出，当像素值大于最大值时取最大值；在 Average 模式下：将相邻电荷相加在一起后取平均值。

➢ 垂直像素 Binning 模式：同理水平像素合并模式。

➢ 抽样行数 DecimationLineNumber：详细参考《水星二代 USB3.0 数字相机应用说明书》8.3 章节。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
BinningHorizontal	INT	RW	水平像素 Binning
BinningVertical	INT	RW	垂直像素 Binning
DecimationHorizontal	INT	RW	水平像素抽样
DecimationVertical	INT	RW	垂直像素抽样
BinningHorizontalMode	ENUM	RW	水平像素 Binning 模式
BinningVerticalMode	ENUM	RW	垂直像素 Binning 模式
DecimationLineNumber	INT	RW	抽样行数
SensorDecimationHorizontal	INT	RW	Sensor 水平像素抽样
SensorDecimationVertical	INT	RW	Sensor 垂直像素抽样

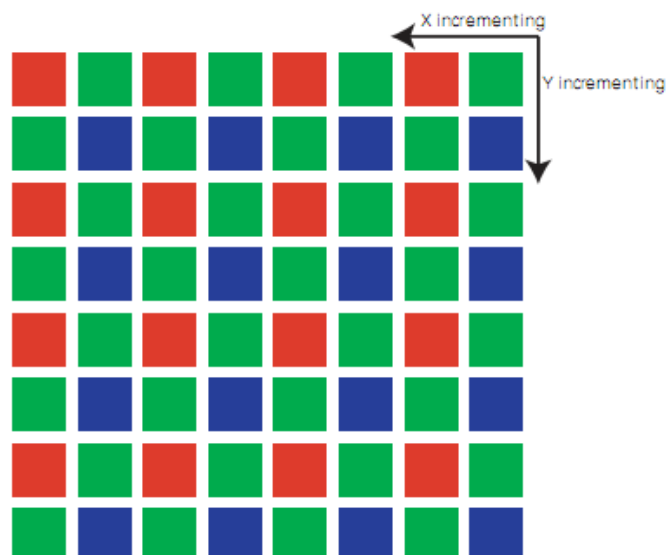


图 3-4 原图

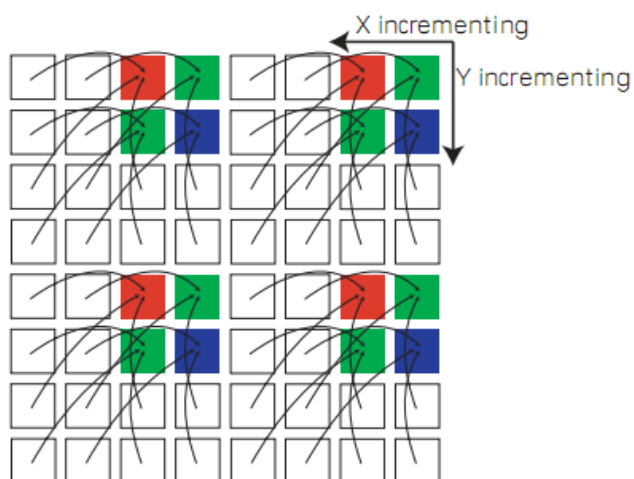


图 3-5 Binning (2x2) 处理

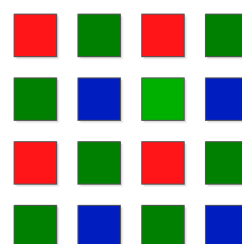


图 3-6 Binning 处理结果

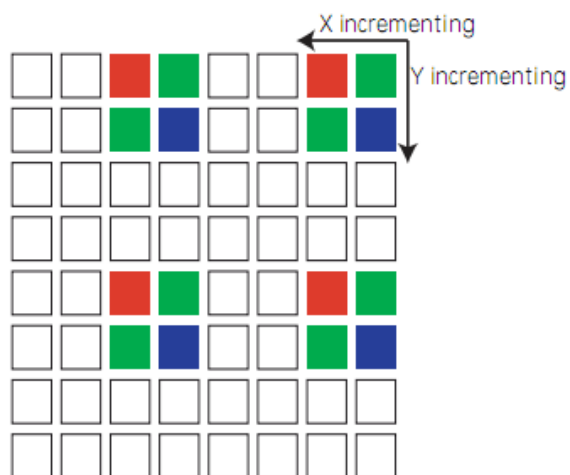


图 3-7 Decimation (2x2) 处理

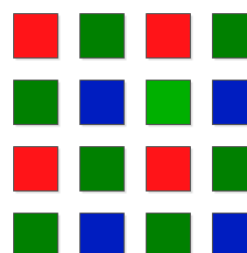


图 3-8 Decimation 处理结果

● 代码样例

```
//配置一个 2×2 的 Binning 和 2×2 的 Decimation
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64BinningH = 2;
int64_t i64BinningV = 2;
int64_t i64DecimationH = 2;
int64_t i64DecimationV = 2;

//设置水平和垂直 Binning 模式为 Sum 模式
emStatus = GXSetEnumValueByString(hDevice, "BinningHorizontalMode",
                                   "Sum");
emStatus = GXSetEnumValueByString(hDevice, "BinningVerticalMode",
                                   "Sum");
emStatus = GXSetIntValue(hDevice, "BinningHorizontal", i64BinningH);
emStatus = GXSetIntValue(hDevice, "BinningVertical", i64BinningV);
emStatus = GXSetIntValue(hDevice, "DecimationHorizontal",
                           i64DecimationH);
emStatus = GXSetIntValue(hDevice, "DecimationVertical",
                           i64DecimationV);
```

● 注意事项

3.2.3. 数据格式

● 名词解释

➤ 数据位深：每一个像元灰度值所占的数据位数，例如 8 位数据表示的灰度值范围是 0~255。RAW 数据格式是 CMOS 或者 CCD 图像传感器将捕捉的光信号转化为数字信号的原始数据格式，这种转换没有经过任何压缩。RAW8 表示输出图像数据位深为 8bit，RAW12 表示输出的图像数据位深为 12bit。

➤ Bayer 颜色转换：Bayer 类型是 RAW 图像数据的排列格式，如图 3-9 所示，图像数据进行处理或者显示时，需要将其通过插值转换为 24 位的 RGB 真彩色图像数据。下面介绍了一种简单的插值算法：

	0	1	2	3	4	5	...
0	R	G	R	G	R	G	...
1	G	B	G	B	G	B	...
2	R	G	R	G	R	G	...
3	G	B	G	B	G	B	...
4	R	G	R	G	R	G	...
5	G	B	G	B	G	B	...
...	...						

图 3-9 Bayer 格式

其中一种简单的转换方式如图 3-10 所示：

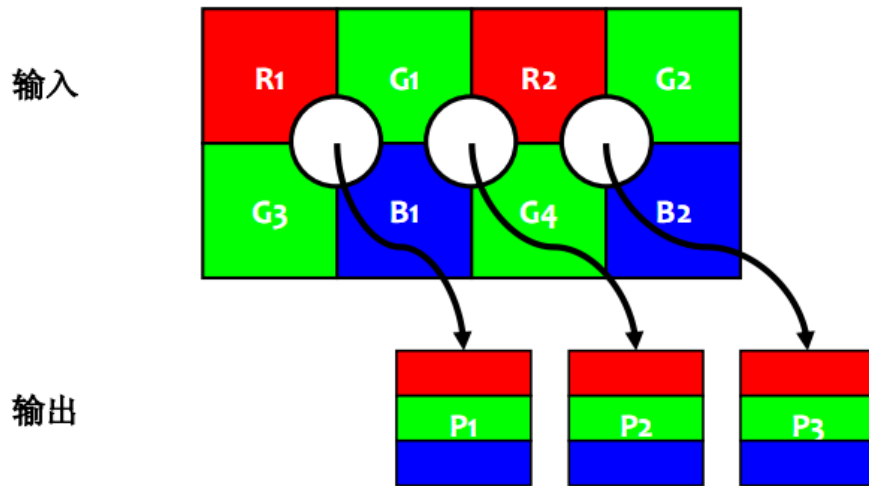


图 3-10 Bayer 转换

像素 1 (P1)	像素 2 (P2)	像素 3 (P3)
$P1_{Red} = R1$	$P2_{Red} = R2$	$P3_{Red} = R2$
$P1_{Green} = \frac{G1 + G3}{2}$	$P2_{Green} = \frac{G1 + G4}{2}$	$P3_{Green} = \frac{G2 + G4}{2}$
$P1_{Blue} = B1$	$P2_{Blue} = B1$	$P3_{Blue} = B2$

图 3-11

● 相关属性

属性名称	属性类别	可访问属性	中文名称
PixelSize	ENUM	RO	像素位深
PixelColorFilter	ENUM	RO	Bayer 格式
PixelFormat	ENUM	RW	像素格式

数据格式：PixelFormat 共 8Byte，32bit。将多种信息集中到一起组成一个单独的信息量。最高两个字节代表了色彩样式（彩色或者黑白），紧接着的两个字节表示数据位深（8 位、10 位等），最低 4 个字节表示编码序号。

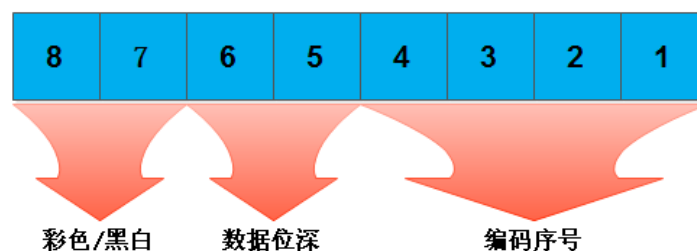


图 3-12 数据格式

● 代码样例

```
//使用 GXGetEnumValue 接口
//查询当前相机支持的"PixelFormat"类型
//参考相关接口说明部分，此处省略
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//读取当前 pixelformat
GX_ENUM_VALUE stPixelFormat;
int64_t i64PixelFormat = 0;
emStatus = GXGetEnumValue(hDevice, "PixelFormat", &stPixelFormat);
i64PixelFormat = stPixelFormat.stCurValue.nCurValue;

//设置 pixelformat 为 bayer 格式为 BG 的位深数据
emStatus = GXSetEnumValueByString(hDevice,
                                   "PixelFormat", "BayerBG10");

//读取当前 pixelsize
GX_ENUM_VALUE stPixelSize;
int64_t i64PixelSize = 0;
emStatus = GXGetEnumValue(hDevice, "PixelSize", &stPixelSize);
i64PixelSize = stPixelSize.stCurValue.nCurValue;

//读取当前 colorfilter
GX_ENUM_VALUE stColorFilter;
int64_t i64ColorFilter = 0;
emStatus = GXGetEnumValue(hDevice, "PixelColorFilter",
                           &stColorFilter);
i64ColorFilter = stColorFilter.stCurValue.nCurValue;
```

3.2.4. 测试图

● 名词解释

测试图：GigE Vision 相机支持三种测试图：灰度值渐变测试图，竖条纹滚动测试图和从斜条纹滚动测试图；USB3 Vision 相机支持三种测试图：灰度值渐变测试图，滚动斜条纹测试图和静止斜条纹测试图；Pallas 系列相机支持三种测试图：灰度值渐变测试图，滚动斜条纹测试图和静止斜条纹测试图。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
TestPatternGeneratorSelector	ENUM	RW	测试图源选择
TestPattern	ENUM	RW	测试图

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//查询当前相机支持什么类型的测试图，请参见 GXGetEnumValue 接口的用法
//本样例假设当前相机支持所有类型的测试图
```

```
//设置为滚动竖条纹测试图
emStatus = GXSetEnumValueByString(hDevice, "TestPattern",
                                   "VerticalLineMoving");

//关闭测试
emStatus = GXSetEnumValueByString(hDevice, "TestPattern", "Off");
```

3.2.5. 帧信息控制

● 名词解释

当开启帧信息后，帧信息会以如下格式追加到图像数据的尾部，用来标识当前图像的信息，如帧号。

开启帧信息之前：

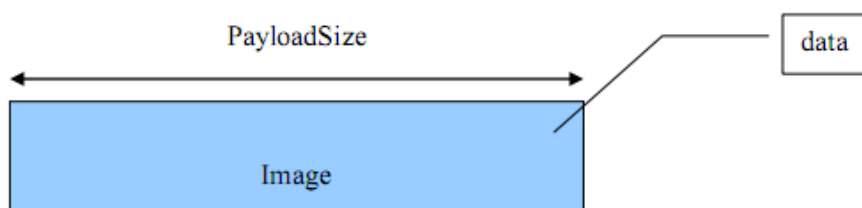


图 3-13 未开启帧信息的图像数据

开启帧信息之后，以 MER-U3x 相机为例：

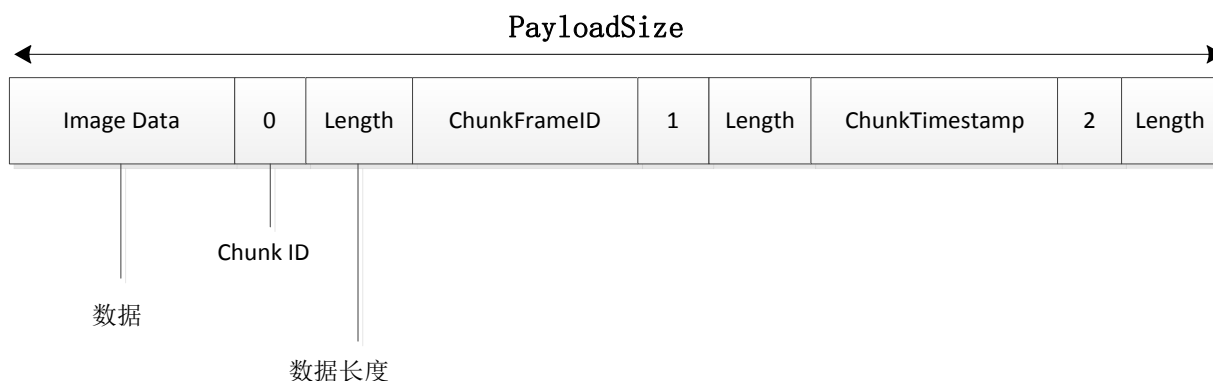


图 3-14 开启帧信息的图像数据

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ChunkModeActive	BOOL	RW	帧信息使能
ChunkSelector	ENUM	RW	帧信息项选择
ChunkEnable	BOOL	RW	单项帧信息使能

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//设置帧信息模式为使能状态
```

```
emStatus = GXSetBoolValue(hDevice, "ChunkModeActive", true);

//查询当前相机支持哪些帧信息，请参见 GXGetEnumDescription 接口的用法
//本样例假设当前相机支持所有类型的帧信息

//选择帧号
emStatus = GXSetEnumValueByString(hDevice, "ChunkSelector",
                                   "FrameID");

//设置帧号为使能状态
emStatus = GXSetBoolValue(hDevice, "ChunkEnable", true);
```

● 注意事项

C 软件接口开放了帧信息的控制接口，但是开放没有获取帧信息的接口，如果用户需要解析帧信息，需要按照上图中的格式，从图像数据的尾部开始解析，具体解析方法和各相机的帧信息排列顺序，请咨询技术支持。

3.2.6. Sensor 曝光模式

● 名词解释

Sensor 曝光模式：指 sensor 采集和处理图像数据的方式。根据设计不同，sensor 可以支持不同的曝光模式，可选值有以下三种。Global：所有的像素点同时曝光，且曝光的时间长度相同；Rolling：所有的像素点曝光的时间长度相同，但每一行的像素点起始时间不同；GlobalReset：所有的像素点同时曝光，但每一行的像素点曝光时间长度不同。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
SensorShutterMode	ENUM	RW	Sensor 曝光模式

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//设置 Sensor 曝光模式为 Global 模式
emStatus = GXSetEnumValueByString(hDevice, "SensorShutterMode",
                                   "Global");
```

3.3. 采集控制

3.3.1. 采集

● 名词解释

➢ 采集模式：设置相机的出图模式，它主要定义了相机在一个完整的采集过程中出图的个数。所谓一个完整的采集过程指的是从发送开采命令开始到发送停采命令结束。

- 采多帧帧数：当采集模式为采多帧模式的时候，这个参数决定了此模式下相机在一个完整的采集过程中出图的个数。
- 采集速度级别：直接控制相机帧率。采集速度级别越大，帧率越大；采集速度级别越小，帧率越小。
- 高速连拍帧数：一次触发采集多帧图像。例如，如果“高速连拍帧数”参数设置为 3，则相机会在一个帧高速连拍开始触发信号后自动输出 3 张图像。
- 采集状态选择：采集状态是用来确定相机是否在等待触发信号。有 FrameTriggerWait 和 AcquisitionTriggerWait 两种模式。FrameTriggerWait：选择此状态后，可以通过查询采集功能状态确定相机是否正在等待帧起始触发信号，AcquisitionTriggerWait：选择此状态后，可以通过查询采集状态功能确定相机是否正在等待多帧采集状态下的触发信号。功能仅在触发模式下使用，对连续采集模式无影响。
- 采集状态：该功能码配合采集状态选择功能使用，具体请参考采集状态选择。在非采集期间（开始采集前，停止采集之后），非触发模式下，查询的值没有任何意义。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
AcquisitionMode	ENUM	RW	采集模式
AcquisitionStart	COMMAND	WO	开采命令
AcquisitionStop	COMMAND	WO	停采命令
AcquisitionFrameCount	INT	RW	多帧采集帧数
AcquisitionSpeedLevel	INT	RW	采集速度级别
AcquisitionBurstFrameCount	INT	RW	高速连拍帧数
AcquisitionStatusSelector	ENUM	RW	采集状态选择
AcquisitionStatus	BOOL	RO	采集状态

● 控制流图

（下图中关于触发模式的概念请参见下一章节——触发）

非触发+连续采集模式：

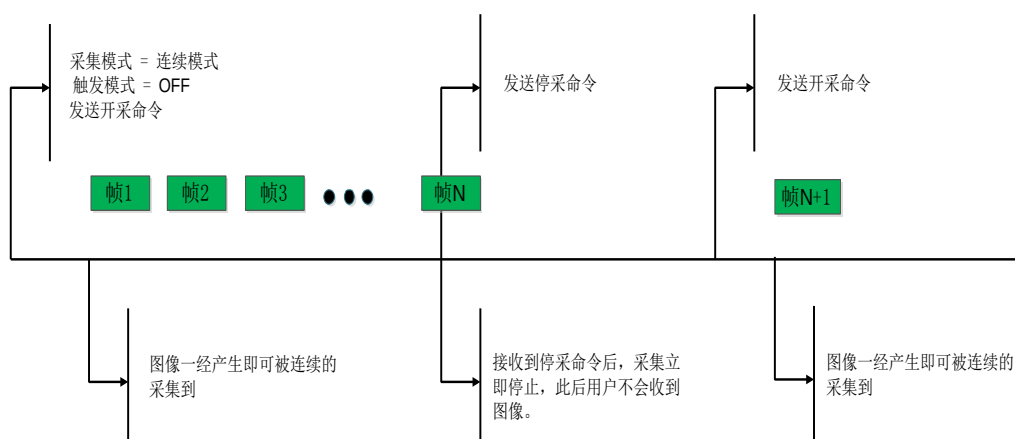


图 3-15 非触发+连续采集模式

触发+连续采集模式：

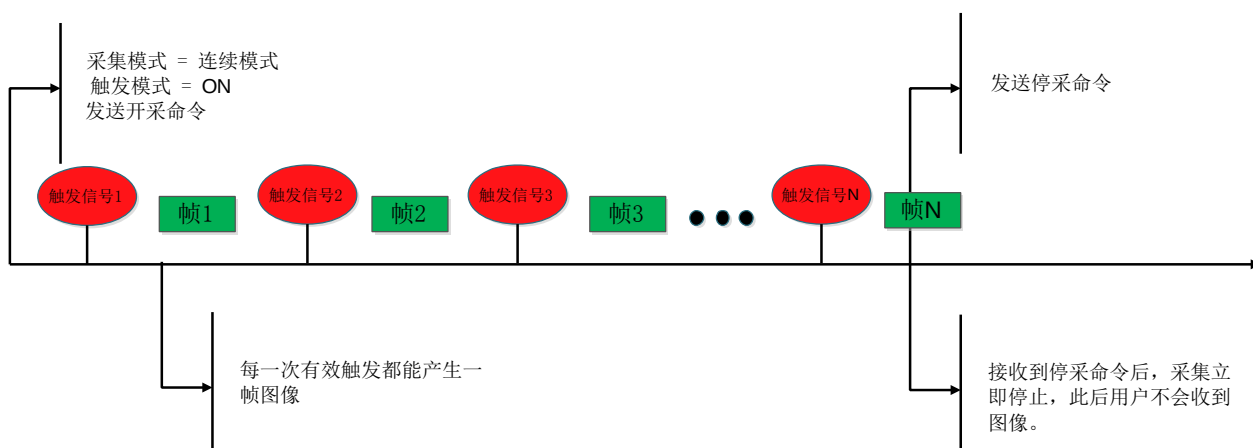


图 3-16 触发+连续采集模式

非触发+多帧模式

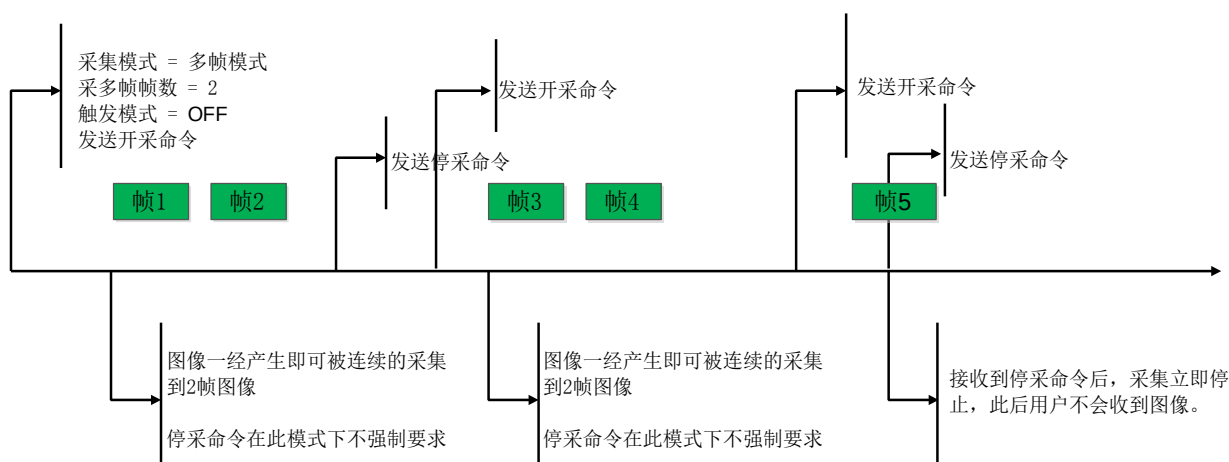


图 3-17 非触发+多帧模式

触发+多帧模式

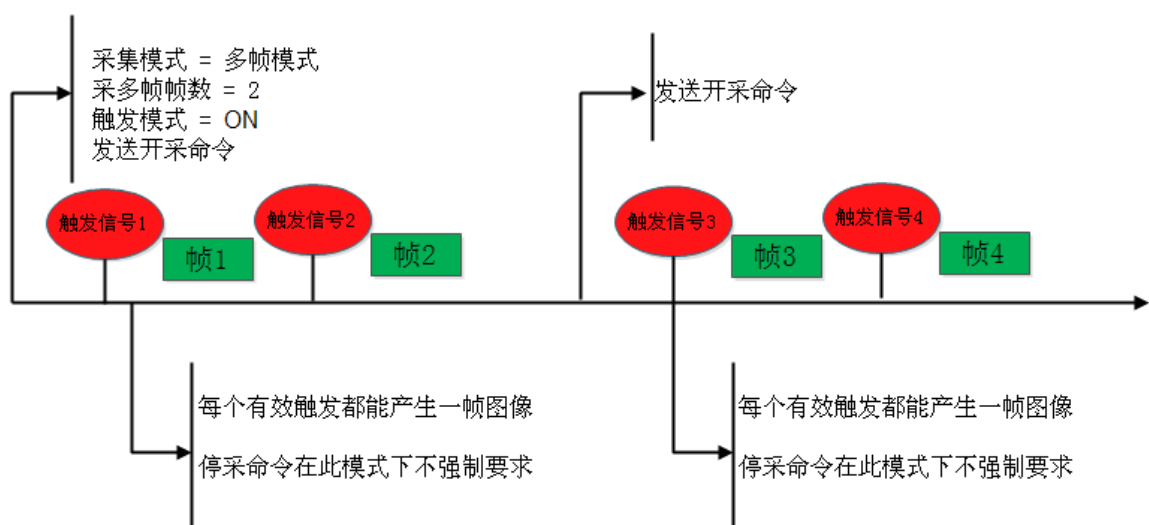


图 3-18 触发+多帧模式

非触发+单帧模式

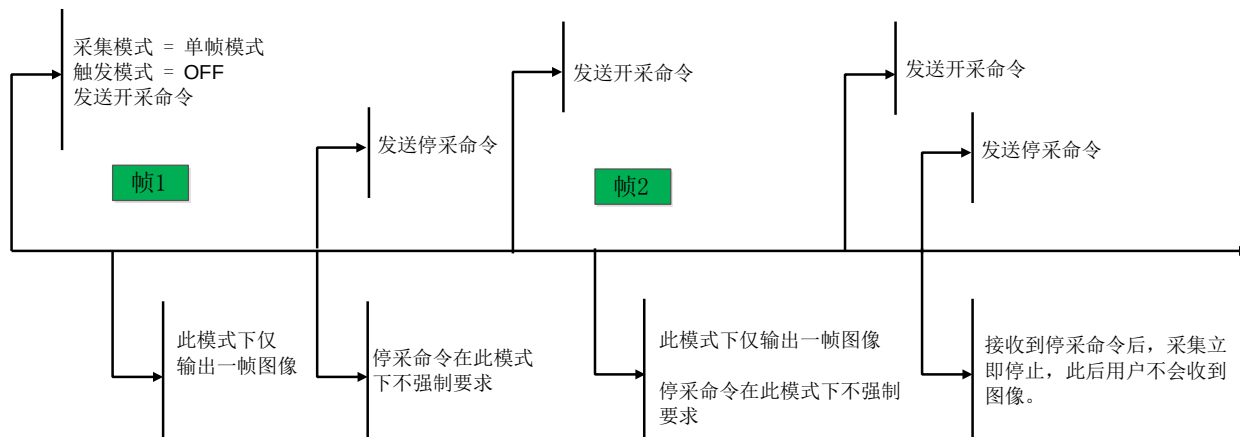


图 3-19 非触发+单帧模式

触发+单帧模式

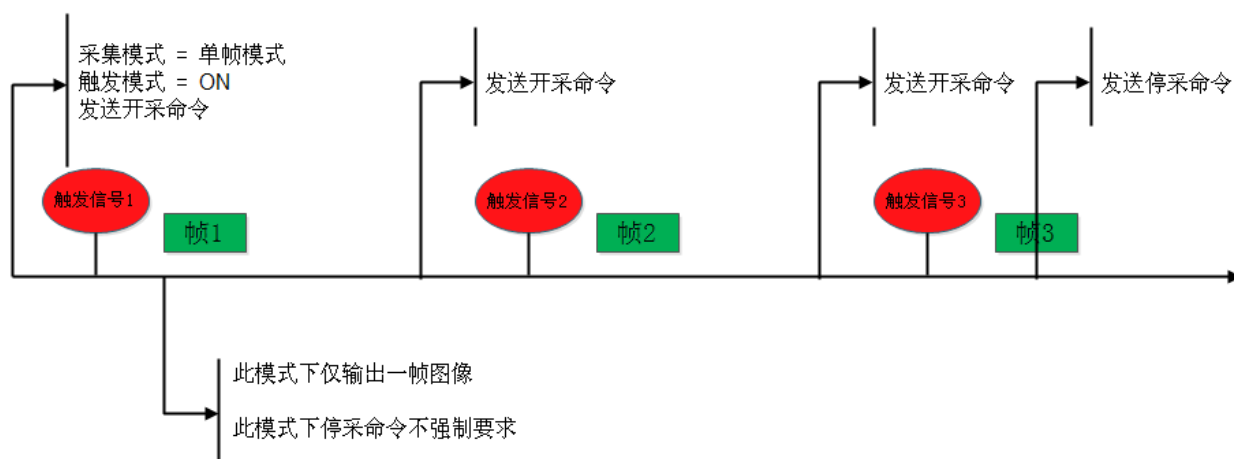


图 3-20 触发+单帧模式

● 代码示例

```
#include "GxIAP.h"

//图像回调处理函数
Static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame->status == 0)
    {
        //对图像进行某些操作
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
```

```

GX_OPEN_PARAM stOpenParam;
uint32_t ui32DeviceNum = 0;

//初始化库
emStatus = GXInitLib();
if (emStatus != GX_STATUS_SUCCESS)
{
    return 0;
}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}

//打开设备
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{

    //设置相机的流通道包长属性，提高网络相机的采集性能
    bool bImplementPacketSize = false;
    uint32_t ui32PacketSize = 0;

    //判断设备是否支持流通道数据包功能
    GX_NODE_ACCESS_MODE emAccessMode;
    emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                    &emAccessMode);
    bImplementPacketSize=(emAccessMode == GX_NODE_ACCESS_MODE_RW)
                        ? true : false;
    if (bImplementPacketSize)
    {
        //获取当前网络环境的最优包长
        emStatus = GXGetOptimalPacketSize(hDevice, &ui32PacketSize);

        //将最优包长设置为当前设备的流通道包长值
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                ui32PacketSize);
    }

    //设置采集模式。一般相机的默认采集模式为连续模式。
    //emStatus = GXSetEnumValueByString(hDevice,
                                        "AcquisitionMode",

```

```

"Continuous");

//注册图像处理回调函数
emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                     OnFrameCallbackFun);

//发送开采命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

//-----
//
//在这个区间图像会通过 OnFrameCallbackFun 接口返给用户
//
//-----
//发送停采命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

//注销采集回调
emStatus= GXUnregisterCaptureCallback(hDevice);
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();
return 0;
}

```

3.3.2. 触发

3.3.2.1. 通用功能

● 名词解释

- 触发源：选择触发信号的来源。软触发或者外触发。
- 触发模式：决定触发信号是否有效。ON 表示触发有效；OFF 表示触发无效。
- 触发极性：触发激活方式，分为上升沿有效或者下降沿有效。
- 软触发命令：软件模拟触发信号。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
TriggerMode	ENUM	RW	触发模式
TriggerSoftware	COMMAND	WO	软触发
TriggerActivation	ENUM	RW	触发极性
TriggerSwitch	ENUM	RW	外触发开关
TriggerSource	ENUM	RW	触发源
TriggerSelector	ENUM	RW	触发类型选择
TriggerDelay	FLOAT	RW	触发延迟

● 代码示例

```
#include "GxIAP.h"

//图像回调处理函数
Static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame->status == 0)
    {
        //对图像进行某些操作
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //枚举设备列表
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //打开设备
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //设置相机的流通道包长属性，提高网络相机的采集性能
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //判断设备是否支持流通道数据包功能
        GX_NODE_ACCESS_MODE emAccessMode;
        emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                        &emAccessMode);
    }
}
```

```

bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                        ? true : false;

if (bImplementPacketSize)
{
    //获取当前网络环境的最优包长
    emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

    //将最优包长设置为当前设备的流通道包长值
    emStatus = GXSetIntValue (hDevice, "GevSCPSPacketSize",
                              ui32PacketSize);
}

//设置触发模式为 ON
emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");

//设置触发激活方式为上升沿
emStatus = GXSetEnumValueByString(hDevice, "TriggerActivation",
                                   "RisingEdge");

//注册图像处理回调函数
emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                     OnFrameCallbackFun);

//发送开采命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
//-----
//
//在发送停采命令前如果产生了有效触发，那么图像
//会通过 OnFrameCallbackFun 接口返给用户
//-----

//发送停采命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

//注销采集回调
emStatus = GXUnregisterCaptureCallback(hDevice);
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();
return 0;
}

```

● 注意事项

3.3.2.2. 高级功能

● 名词解释

➢ 上升沿触发滤波值：上升沿脉冲宽度如果小于此滤波值，那么相机将不会处理这样的触发信号。只有当上升沿脉冲宽度大于、等于此滤波值的时候，才会产生有效触发信号。

➢ 下降沿触发滤波值：下降沿脉冲宽度如果小于此滤波值，那么相机将不会处理这样的触发信号。只有当下降沿脉冲宽度大于、等于此滤波值的时候，才会产生有效触发信号。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
TriggerFilterRaisingEdge	FLOAT	RW	上升沿触发滤波
TriggerFilterFallingEdge	FLOAT	RW	下降沿触发滤波

● 触发波形示意图

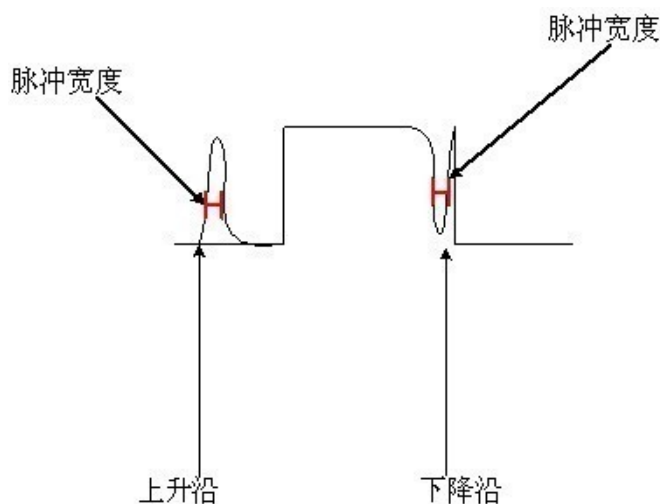


图 3-21 触发波形

● 代码示例

```
//获取上升沿滤波设置范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stRaisingRange;
emStatus = GXGetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                           &stRaisingRange);

//设置上升沿滤波最小值
emStatus = GXSetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                           stRaisingRange.dMin);

//设置上升沿滤波最大值
emStatus = GXSetFloatValue(hDevice, "TriggerFilterRaisingEdge",
                           stRaisingRange.dMax);
```

```

//获取当前上升沿滤波值
double dValueUp = stRaisingRange.dCurValue;

//获取下降沿滤波设置范围
GX_FLOAT_VALUE stFallingRange;
emStatus = GXGetFloatValue(hDevice, "TriggerFilterFallingEdge",
                             &stFallingRange);

//设置下降沿滤波最小值
emStatus = GXSetFloatValue(hDevice, "TriggerFilterFallingEdge",
                             stFallingRange.dMin);

//设置下降沿滤波最大值
emStatus = GXSetFloatValue(hDevice, "TriggerFilterFallingEdge",
                             stFallingRange.dMax);

//获取当前下降沿滤波值
double dValueDown = stFallingRange.dCurValue;
    
```

● 注意事项

3.3.3. 曝光

● 名词解释

曝光模式 :曝光的不同工作方式 ,Timed 模式的时候 ,才可以使用“ExposureTime”、“ExposureAuto”。

曝光时间模式：曝光时间的不同模式，包含 Standard 标准模式和 UltraShort 极小曝光模式；Standard 模式时，才可以使用“ExposureAuto”。

曝光时间：曝光时间即快门速度，指的是传感器的快门从打开到关闭的时间间隔，在这个时间间隔内被摄物体可以在传感器上留下影像。减小曝光时间图像变暗，增大曝光时间图像变亮。

曝光延迟：曝光延迟功能可以有效解决闪光灯延时问题。绝大部分闪光灯从触发到点亮至少有几十微秒以上的延时，当相机工作在小曝光模式下，闪光灯的补光效果就会受影响。通过闪光灯信号和实际曝光开始的延时来实现曝光延迟。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ExposureMode	ENUM	RW	曝光模式
ExposureTimeMode	ENUM	RW	曝光时间模式
ExposureTime	FLOAT	RW	曝光时间
ExposureAuto	ENUM	RW	自动曝光
ExposureDelay	FLOAT	RW	曝光延迟

- 效果图



图 3-22 原图

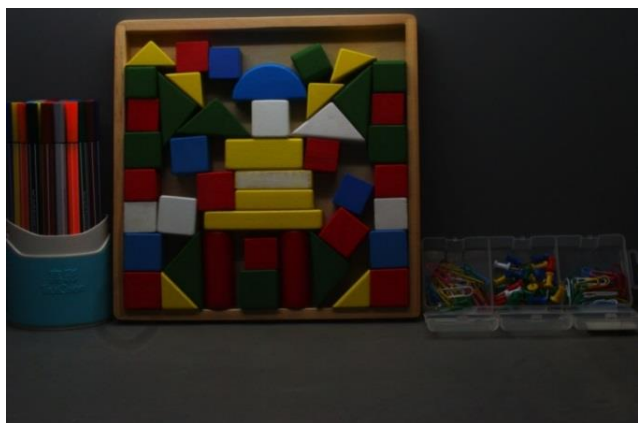


图 3-23 减小曝光时间



图 3-24 增大曝光时间

- 代码样例

```
//获取曝光调节范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stShutterRange;
double dExposureValue = 2.0;

emStatus = GXGetFloatValue(hDevice, "ExposureTime",
                           &stShutterRange);

//设置最小曝光值
emStatus = GXSetFloatValue(hDevice, "ExposureTime",
                           stShutterRange.dMin);

//设置最大曝光值
emStatus = GXSetFloatValue(hDevice, "ExposureTime",
                           stShutterRange.dMax);

//设置连续自动曝光
emStatus = GXSetEnumValueByString(hDevice, "ExposureAuto",
                                   "Continuous");
```

```
//设置曝光延迟为 2μs
emStatus = GXSetFloatValue(hDevice, "ExposureDelay",
                             dExposureValue);

//设置曝光时间模式为极小曝光模式
emStatus = GXSetEnumValueByString(hDevice, "ExposureTimeMode",
                                    "UltraShort");
```

● 注意事项

当外部光源为自然光或者直流电光源时对曝光时间无特殊要求,但是当外部光源为交流电光源时,曝光时间需要与外部光源的交流频率同步(50Hz 光照环境下,曝光时间必须是 1/100s 的整倍数,60Hz 光照环境下,曝光时间必须是 1/120s 的整倍数)。

3.3.4. 传输控制

● 名词解释

当多个相机和主机通过交换机连接时,如果同时触发这些相机采集图像,在传输图像由于交换机的瞬时带宽过大,存储能力有限,会出现数据丢失问题。因此需要使用帧传输延迟避免该问题发生。

在触发模式下,通过设置传输控制模式为“用户控制”,当相机接到软触发命令或者硬触发信号并完成图像采集以后,将图像保存在相机内部的帧存中,等待主机端发送“开始传输”命令以后,相机将采集到的图像传输到主机端。传输延迟时间由主机端决定。在多个相机同时触发的时候,可以为每台相机设置不同的传输延迟时间,以避免交换机的瞬时带宽过大。

● 相关属性

属性名称	属性类别	可访问属	中文名称
TransferControlMode	ENUM	RW	传输控制模式
TransferOperationMode	ENUM	RW	传输操作模式
TransferStart	COMMAND	WO	开始传输
TransferBlockCount	INT	RW	传输帧数

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//前提必须确保触发模式为开启状态
emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");

//设置传输控制模式为用户控制模式
emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                    "UserControlled");

//设置传输操作模式为指定发送帧数
```

```
emStatus = GXSetEnumValueByString(hDevice, "TransferOperationMode",
                                   "MultiBlock");

//设置每次命令输出帧数帧
emStatus = GXSetIntValue(hDevice, "TransferBlockCount", 1);

//开采之后发送软触发信号（或者外触发）
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");

//触发出图之后发送传输命令
emStatus = GXSetCommandValue(hDevice, "TransferStart");
```

- 注意事项

传输控制功能只能工作在触发模式下。

3.3.5. 突发采集模式

- 名词解释

突发采集模式设置是只针对触发模式应用的设置。突发采集模式包括两种模式：标准模式，是指受到接口（GigE 接口或 USB3 接口）带宽限制的一种采集模式；高速模式，是指不受接口（GigE 接口或 USB3 接口）带宽限制的一种采集模式。按照 Sensor 的最大采集能力进行采集，帧率一般大于当前传输能力，主要配合传输控制模式中用户控制模式使用。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
AcquisitionBurstMode	ENUM	RW	突发采集模式

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//1.设置触发的模式为一次触发采集 14 帧
emStatus = GXSetEnumValueByString(hDevice, "TriggerSelector",
                                   "FrameStart");
emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");
emStatus = GXSetIntValue(hDevice, "AcquisitionFrameCount", 14);

//2.设置突发采集模式为高速模式-实现触发采集提速
emStatus = GXSetEnumValueByString(hDevice, "AcquisitionBurstMode",
                                   "HighSpeed");

//3.设置帧存深度为 14
emStatus = GXSetIntValue(hDevice, "FrameBufferCount", 14);

//4.设置传输控制模式为用户控制模式，设置一次传输 14 帧
emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                   "UserControlled");
```

```
emStatus = GXSetEnumValueByString(hDevice, "TransferOperationMode",
                                   "MultiBlock");
emStatus = GXSetIntValue(hDevice, "TransferBlockCount", 14);

//5. 开采，注意：1、2、3、4 需要在打开相机后，开采前完成
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

//6. 发送软触发（或者外触发，在开采后控制发送触发信号即可）
emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");

//7. 发送<开始传输命令>（根据需要触发命令完成后立即发送或延后发）
//注意：需要在本次需要传输的数据传出完成后，再发下一次的触发
emStatus = GXSetCommandValue(hDevice, "TransferStart");
```

● 注意事项

由于传输控制模式为用户控制模式情况下，特别是一次传输多帧的情况下，传输是按接口最大传输速度进行的，由于网卡的性能差别，长时间运行可能出现丢帧，需要增大包间隔，包间隔默认为 0。包间隔设置需要根据以下因素进行不同设置：

- 一次传输的帧数设置。
- 两次传输之间间隔。（推荐 2 次之间的间隔大于 100ms）
- 包长的设置。（推荐使用 8164 包长，注意网卡需要设置为巨帧）

3.3.6. 帧存机制

● 名词解释

帧存覆盖：当相机向内部帧存写入数据的平均带宽大于读出数据的平均带宽时，会出现帧存满的情况。如果帧存满后继续写入图像数据，将覆盖以前帧存里的图像数据。

帧存深度：相机中的帧存个数。

清空帧存：将以前帧存里的数据清空。

- 1) 在触发模式下，如果正在触发采集过程中，发送清帧存命令，将直接屏蔽掉此命令。
- 2) 在被动传输下，如果传输数据过程中，发送清帧存命令，将直接屏蔽掉此命令，不响应。
- 3) 在触发模式为 On，且传输控制在 UserControl 模式下才可用。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
FrameBufferOverwriteActive	BOOL	RW	帧存覆盖使能
FrameBufferCount	INT	RW	帧存深度
FrameBufferFlush	COMMAND	WO	清空帧存

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
//使能帧存覆盖
emStatus = GXSetBoolValue(hDevice, "FrameBufferOverwriteActive", true);
//禁用帧存覆盖
emStatus = GXSetBoolValue(hDevice, "FrameBufferOverwriteActive", false);
//设置触发模式为 ON
emStatus = GXSetEnumValueByString(hDevice, "TriggerMode", "On");
//设置传输控制模式为用户控制模式
emStatus = GXSetEnumValueByString(hDevice, "TransferControlMode",
                                   "UserControlled");
//发送清空帧存信号（在需要时发送清空帧存信号，需谨慎使用）
emStatus = GXSetCommandValue(hDevice, "FrameBufferFlush");
```

3.3.7. 帧率控制

● 名词解释

- 采集帧率调节模式：控制是否开启帧率控制模式，on 为启动帧率控制，off 为不启用帧率控制。详见 GxI API.h 定义。
- 采集帧率：期望的相机采集帧率。
- 当前采集帧率：实际运行时的采集帧率。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
AcquisitionFrameRateMode	ENUM	RW	采集帧率调节模式
AcquisitionFrameRate	FLOAT	RW	采集帧率
CurrentAcquisitionFrameRate	FLOAT	RO	当前采集帧率

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//使能采集帧率调节模式
emStatus = GXSetEnumValueByString(hDevice, "AcquisitionFrameRateMode",
                                   "On");
//设置采集帧率，假设设置为 10.0，用户按照实际需求设置此值
emStatus = GXSetFloatValue(hDevice, "AcquisitionFrameRate", 10.0);
```

● 注意事项

如果“AcquisitionFrameRate”设置的过大，超过了相机实际运行能力，相机会以其自身可达到的最大帧率运行，即“CurrentAcquisitionFrameRate”当前采集帧率的值（帧率受曝光、ROI 等功能的影响）。

3.3.8. 曝光模式

● 名词解释

Timed：定时曝光模式，曝光时间由相机的 ExposureTime 确定。该模式适用于所有型号相机。任何触发模式下都可以使用。

TriggerWidth：触发宽度曝光模式，曝光时间由硬件触发信号的宽度决定，因此该模式仅在外触发的模式下才生效。触发宽度曝光模式适用于改变每帧的曝光时间。

- 1) 如果相机配置的是上升沿触发，则曝光将在触发信号上升时开始，并持续到触发信号下降为止
- 2) 如果相机配置的是下降沿触发，则曝光将在触发信号下降时开始，并持续到触发信号上升为止。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ExposureMode	ENUM	RW	曝光模式

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//设置曝光模式
emStatus = GXSetEnumValueByString(hDevice, "ExposureMode",
                                   "TriggerWidth");

//设置交叠曝光时间最大值
emStatus = GXSetFloatValue(hDevice, "ExposureOverlapTimeMax",
                           621.0);
```

● 注意事项

如果使用了触发宽度曝光模式，请勿以过高的速率发送触发信号，否则，触发信号将被忽略。

在触发宽度曝光模式下，可以通过交叠曝光时间最大值参数，将其设置为你想要使用的最短曝光时间。

3.3.9. 交叠曝光时间最大值

● 名词解释

交叠曝光时间最大值：当相机仍在读取前一帧图像数据时，就开始下一帧新图像的曝光，您可以通过该功能优化交叠图像的采集，可以最大程度地提高相机的帧速率。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ExposureOverlapTimeMax	FLOAT	RW	交叠曝光时间最大值

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//设置曝光模式
emStatus = GXSetEnumValueByString(hDevice, "ExposureMode",
                                   "TriggerWidth");

//设置交叠曝光时间最大值
emStatus = GXSetFloatValue(hDevice, "ExposureOverlapTimeMax",
                           621.0);
```


- 注意事项

交叠曝光时间最大值仅在触发宽度曝光模式下有效。

3.3.10. 多帧灰度控制模式

- 名词解释

多帧灰度控制：通过参数设置，可以使不同帧有不同的灰度（亮度），可配置的参数包括曝光和增益。

多帧灰度控制模式：包括 Off 模式、TwoFrame 模式、FourFrame 模式。

Off 模式：关闭多帧灰度控制。

TwoFrame 模式：可以设置 2 组不同曝光和增益，设置完成后，开采，相机输出 2 组参数循环生效的多帧图像。

FourFrame 模式：可以设置 4 组不同曝光和增益，设置完成后，开采，相机输出 4 组参数循环生效的多帧图像。
- 相关属性

属性名称	属性类别	可访问属性	中文名称
MultiGrayControlMode	ENUM	RW	多帧灰度控制模式
MGCSelector	INT	RW	多帧灰度选择
MGCExposureTime	FLOAT	RW	多帧灰度曝光时间
MCGCGain	FLOAT	RW	多帧灰度增益

- 代码样例


```

GX_STATUS emStatus = GX_STATUS_SUCCESS;

//开启两帧控制模式
emStatus = GXSetEnumValueByString(hDevice, "MultiGrayControlMode",
                                   "TwoFrame");

//设置第一帧曝光时间及增益
emStatus = GXSetIntValue(hDevice, "MGCSelector", 0);
emStatus = GXSetFloatValue(hDevice, "MGCExposureTime", 400000.0);
emStatus = GXSetFloatValue(hDevice, "MCGCGain", 2.0);

//设置第二帧曝光时间及增益
emStatus = GXSetIntValue(hDevice, "MGCSelector", 1);
emStatus = GXSetFloatValue(hDevice, "MGCExposureTime", 1000.0);
emStatus = GXSetFloatValue(hDevice, "MCGCGain", 1.0);
      
```

3.4. 数字 I/O

3.4.1. 引脚控制

● 相关属性

属性名称	属性类别	可访问属性	中文名称
LineSelector	ENUM	RW	引脚选择
LineMode	ENUM	RW	引脚方向
LineInverter	BOOL	RW	引脚电平反转
LineSource	ENUM	RW	引脚输出源
LineStatus	BOOL	RO	引脚状态
LineStatusAll	INT	RO	所有引脚的状态
UserOutputSelector	ENUM	RW	用户自定义输出选择
UserOutputValue	BOOL	RW	用户自定义输出值
PulseWidth	FLOAT	RW	用户自定义脉冲宽度 (Linux)

● 代码样例

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;

//举例引脚选择为 Line2
emStatus = GXSetEnumValueByString(hDevice, "LineSelector", "Line2");

//设置引脚方向为输出
emStatus = GXSetEnumValueByString(hDevice, "LineMode", "Output");

//可选操作引脚电平反转
//emStatus = GXSetBoolValue(hDevice, "LineInverter", true);

//如果设置输出源为闪光灯,则如下代码
emStatus = GXSetEnumValueByString(hDevice, "LineSource", "Strobe");

//如果设置输出源为用户自定义输出,则如下代码
emStatus = GXSetEnumValueByString(hDevice, "LineSource",
                                   "UserOutput0");

//还可以设置用户自定义输出值,如下代码
emStatus = GXSetEnumValueByString(hDevice, "UserOutputSelector",
                                   "UserOutput0");
emStatus = GXSetBoolValue(hDevice, "UserOutputValue", true);

```



```
//查看引脚 Line2 的状态，因为当前 LineSelector 选择的是 Line2，直接查询当前值即可
bool bLineStatus = true;
emStatus = GXGetBoolValue(hDevice, "LineStatus", &bLineStatus);

//查看当前所有引脚状态
GX_INT_VALUE stAllLineStatus;
int64_t i64AllLineStatus = 0;
emStatus = GXGetIntValue(hDevice, "LineStatusAll",
                          &stAllLineStatus);

i64AllLineStatus = stAllLineStatus.nCurValue;
```

3.4.2. USB2.0 相机 I/O 控制

USB2.0 的 I/O 控制比较特殊，与上面一节展示“引脚控制”操作不是同一个操作流程，USB2.0 的 I/O 控制涉及到以下三个功能：

● 名词解释

➢ 用户 I/O 输出模式：分为闪光灯模式和用户自定义模式。闪光灯模式：在此模式下相机发送触发信号来激活闪光灯（触发信号分为上升沿、下降沿两种）；用户自定义模式：在此模式下用户可以自己设定相机恒定的输出电平来做特别处理，比如控制 LED 灯（电平分为高电平或者低电平）。

➢ 输出信号极性：当输出信号模式为闪光灯模式的时候代表上升沿或者下降沿；当输出信号模式为用户自定义模式的时候代表高电平或者低电平。

➢ 闪光灯开关：此开关仅在输出模式为闪光灯模式下才起作用，当设置为开的时候输出触发信号，当设置为关的时候不输出闪光灯信号。

● 相关参数

属性名称	属性类别	可访问属性	中文名称
UserOutputMode	ENUM	RW	用户 I/O 输出模式（USB2.0 相机专用）
UserOutputValue	BOOL	RW	用户自定义输出值（如果是 USB2.0 相机，此值表示输出信号极性：当输出信号模式为闪光灯模式的时候代表上升沿或者下降沿；当输出信号模式为用户自定义模式的时候代表高电平或者低电平）
StrobeSwitch	ENUM	RW	闪光灯开关（USB2.0 相机专用）

● 控制流图

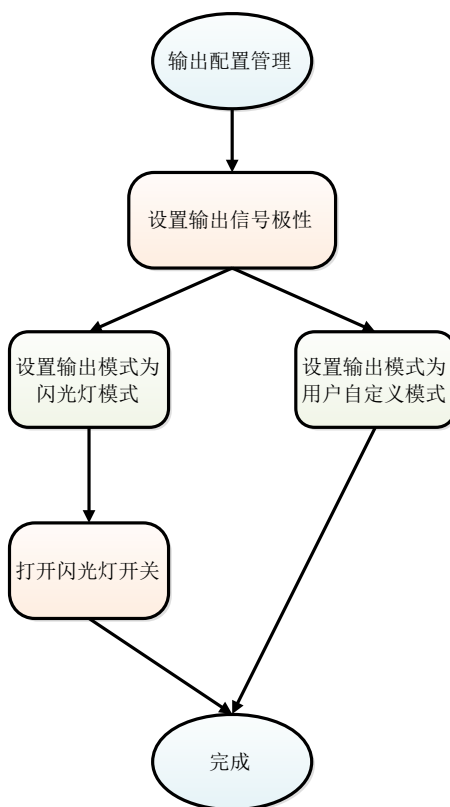


图 3-25 I/O 控制流图

● 代码样例

```

#include "GxIAP.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //枚举设备列表
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
}
    
```

```
//打开设备
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //设置输出信号极性为高
    emStatus = GXSetBoolValue(hDevice, "UserOutputValue", true);

    //设置输出模式为闪光灯模式
    emStatus = GXSetEnumValueByString(hDevice, "UserOutputMode",
                                      "Strobe");

    //打开闪光灯开关
    emStatus = GXSetEnumValueByString(hDevice, "StrobeSwitch", "On");
}

emStatus = GXCloseDevice(hDevice);
GXCloseLib();
return 0;
}
```

- 注意事项

3.5. 计数器和计时器控制

3.5.1. 计时器

- 名词解释

- 计时器：用来测量内部或外部信号的持续时间。
- 计时器持续时间：当计时器达到计时器持续时间时，计时器停止计数，直到新的触发信号产生或发送计时器复位命令。
- 计时器延迟时间：计时开始前延迟时间。
- 计时器触发源：设置触发所选计时器的内部信号。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
TimerSelector	ENUM	RW	计时器选择
TimerDuration	FLOAT	RW	计时器持续时间
TimerDelay	FLOAT	RW	计时器延迟时间
TimerTriggerSource	ENUM	RW	计时器触发源

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//举例计时器选择 Timer1
emStatus = GXSetEnumValueByString(hDevice, "TimerSelector",
                                   "Timer1");

//设置计时器持续时间 100μs , 用户根据实际情况设置
emStatus = GXSetFloatValue(hDevice, "TimerDuration", 100.0);

//设置计时器延迟时间 10μs , 用户根据实际情况设置
emStatus = GXSetFloatValue(hDevice, "TimerDelay", 10.0);

//设置触发源为曝光开始
emStatus = GXSetEnumValueByString(hDevice, "TimerTriggerSource",
                                   "ExposureStart");
```

3.5.2. 计数器

● 名词解释

- 计数器：用来记录内部事件、I/O 外部事件或者时钟滴答计数等。
- 计数器事件触发源：设置哪种事件可以增加所选计数器。目前支持一种模式，“帧开始”事件。
- 计数器复位源：设置哪种信号源将重置计数器。
- 计数器复位信号极性：复位激活方式，分为上升沿有效或者下降沿有效。
- 计数器复位：复位所选计数器。仅当“计数器复位源”选择“Software”时有效。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
CounterSelector	ENUM	RW	计数器选择
CounterEventSource	ENUM	RW	计数器事件触发源
CounterResetSource	ENUM	RW	计数器复位源
CounterResetActivation	ENUM	RW	计数器复位信号极性
CounterReset	COMMAND	WO	计数器复位
CounterTriggerSource	ENUM	RW	计数器触发源
CounterDuration	INT	RW	计数器持续时间
TimerTriggerActivation	ENUM	RW	定时器触发极性

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//举例计数器选择 Counter1
emStatus = GXSetEnumValueByString(hDevice, "CounterSelector",
                                   "Counter1");
```

```
//设置事件触发源为帧开始
emStatus = GXSetEnumValueByString(hDevice, "CounterEventSource",
                                   "FrameStart");

//设置复位源为软触发
emStatus = GXSetEnumValueByString(hDevice, "CounterResetSource",
                                   "Software");

//设置复位信号极性为上升沿有效
emStatus = GXSetEnumValueByString(hDevice, "CounterResetActivation",
                                   "RisingEdge");

//发送复位计数器命令,仅当复位源设置为软触发时有效
emStatus = GXSetCommandValue(hDevice, "CounterReset");

//设置触发源为软触发
emStatus = GXSetEnumValueByString(hDevice, "CounterTriggerSource",
                                   "Software");

//设置计数器持续时间值
emStatus = GXSetIntValue(hDevice, "CounterDuration", 50);

//设置计数器上升沿触发
emStatus = GXSetEnumValueByString(hDevice, "TimerTriggerActivation",
                                   "RisingEdge");
```

3.6. 模拟控制

3.6.1. 增益

增益是用来提高像素值的一个乘法因子,起到的效果就是增加图像亮度。在一些特定的条件下(外部环境等) sensor 达不到期望的饱和度,所以需要调节增益。当然提高增益也会提高噪声,所以提高增益并没有改善实际像素的动态范围。所以当需要提高图像亮度的时候首先考虑调节曝光时间,只有当曝光时间调节已经无法满足要求的时候再去调节增益。

- 名词解释
 - 增益通道选择:调节增益之前需要先选择通道,一般分为 ALL、Red、Green、Blue 四种通道类型。
 - 增益控制方式:分为手动调节方式和自动调节方式两种。
- 相关属性

属性名称	属性类别	可访问属性	中文名称
GainSelector	ENUM	RW	增益通道选择
Gain	FLOAT	RW	增益
GainAuto	ENUM	RW	自动增益

● 效果图



图 3-26 原图

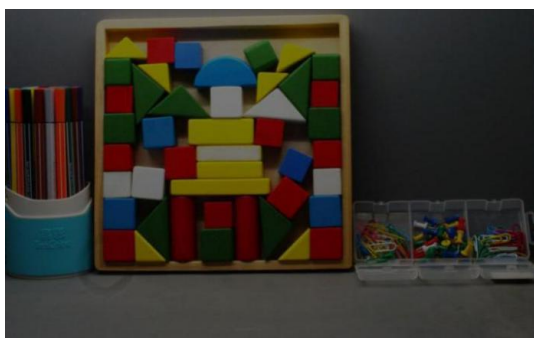


图 3-27 降低增益效果



图 3-28 提高增益效果

● 代码样例

```
//手动增益调节-----
//使用 GXGetEnumValue 接口
//查询当前相机支持的"GainSelector"类型
//参考接口说明部分，此处省略
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//假设当前相机支持"GainSelector"和"All"

//选择增益通道类型
emStatus = GXSetEnumValueByString(hDevice, "GainSelector",
                                   "All");

//status = GXSetEnumValueByString(hDevice, "GainSelector", "Red");
//status = GXSetEnumValueByString(hDevice, "GainSelector", "Green");
//status = GXSetEnumValueByString(hDevice, "GainSelector", "Blue");

//获取增益调节范围
GX_FLOAT_VALUE stGainRange;
emStatus = GXGetFloatValue(hDevice, "Gain", &stGainRange);
//设置最小增益值
emStatus = GXSetFloatValue(hDevice, "Gain", stGainRange.dMin);
```

```
//设置最大增益值
```

```
emStatus = GXSetFloatValue(hDevice, "Gain", stGainRange.dMax);
```

```
//设置连续自动增益
```

```
emStatus = GXSetEnumValueByString(hDevice, "GainAuto", "Continuous");
```

● 注意事项

USB2.0 相机的增益不是 db 单位，是寄存器的数值，具有调节范围，值越大，增益放大倍数越大。

3.6.2. 黑电平

● 名词解释

➢ 黑电平：定义图像数据对应的参考电平，调节黑电平不影响信号的放大倍数，仅仅是对信号进行上下平移。向上调节黑电平，图像变亮；向下调节黑电平，图像变暗。

➢ 黑电平控制方式：手动调节方式、自动调节方式。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
BlackLevelSelector	ENUM	RW	黑电平通道选择
BlackLevel	FLOAT	RW	黑电平
BlackLevelAuto	ENUM	RW	自动黑电平

● 效果图



图 3-29 原图



图 3-30 减小黑电平



图 3-31 增大黑电平

● 代码样例

```
//手动黑电平调节-----
//使用 GXGetEnumValue 接口
//查询当前相机支持的"BlackLevelSelector"类型
//参考接口说明部分，此处省略
//假设当前相机支持"BlackLevelSelector"和"All"
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//选择黑电平通道类型
emStatus = GXSetEnumValueByString(hDevice, "BlackLevelSelector",
                                   "All");
//status= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                "Red");
//status= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                "Green");
//status= GXSetEnumValueByString(hDevice, "BlackLevelSelector",
//                                "Blue");
//获取黑电平调节范围
GX_FLOAT_VALUE stBlackLevelRange;
emStatus = GXGetFloatValue(hDevice, "BlackLevel",
                           &stBlackLevelRange);

//设置最小黑电平值
emStatus = GXSetFloatValue(hDevice, "BlackLevel",
                           stBlackLevelRange.dMin);

//设置最大黑电平值
emStatus = GXSetFloatValue(hDevice, "BlackLevel",
                           stBlackLevelRange.dMax);

//自动黑电平设置-----
//设置连续自动黑电平
emStatus = GXSetEnumValueByString(hDevice, "BlackLevelAuto",
                                   "Continuous");
```

● 注意事项

3.6.3. 白平衡

● 名词解释

➤ 白平衡：在各种不同的色温下，目标物的色彩会产生变化。其中，白色物体变化得最为明显：在室内钨丝灯这样低色温照射下，白色物体看起来会带有橘黄色色调，在这样的光照条件下拍摄出来的景物偏黄；在蔚蓝色天空这样高色温照射下，则会带有蓝色色调，在这种光照条件下拍摄出来的景物偏蓝。为了尽可能减少外来光线对目标颜色造成影响，在不同的色温条件下都能还原出被拍摄目标本来的色彩，需要进行色彩校正，达到正确的色彩平衡，成为白平衡调整。

➤ 白平衡控制方式：手动调节方式、自动调节方式。

➤ 自动白平衡关照环境：此值需要用户依据相机所处环境的光源，自行设置，设置完此值后白平衡效果更佳。

➤ 自动白平衡感兴趣区域：自动白平衡采用白平衡“白点”区域（ROI）中的图像数据计算白平衡系数，然后根据计算的系数对图像的各分量进行处理，使 ROI 区域中的红、绿、蓝三分量的值一致。自动白平衡只对彩色传感器有效。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
BalanceRatioSelector	ENUM	RW	白平衡通道选择
BalanceRatio	FLOAT	RW	白平衡系数
BalanceWhiteAuto	ENUM	RW	自动白平衡
AWBLampHouse	ENUM	RW	自动白平衡光源
AWBROIOffsetX	INT	RW	自动白平衡感兴趣区域 X 坐标
AWBROIOffsetY	INT	RW	自动白平衡感兴趣区域 Y 坐标
AWBROIWidth	INT	RW	自动白平衡感兴趣区域宽度
AWBROIHeight	INT	RW	自动白平衡感兴趣区域高度
LightSourcePreset	ENUM	RW	环境光源预设

● 效果图



图 3-32 D65 环境下图像偏色



图 3-33 进行白平衡调整之后的效果

● 代码样例

```
//手动白平衡调节-----  
  
//使用 GXGetEnumValue 接口  
//查询当前相机支持的"BalanceRatioSelector"类型  
//参考接口说明部分，此处省略  
  
//假设当前相机支持"BalanceRatioSelector"和"Red"  
GX_STATUS emStatus = GX_STATUS_SUCCESS;  
  
//选择白平衡通道  
emStatus = GXSetEnumValueByString(hDevice, "BalanceRatioSelector",  
                                   "Red");  
//emStatus = GXSetEnumValueByString(hDevice,  
//                                   "BalanceRatioSelector",  
//                                   "Green");  
  
//status= GXSetEnumValueByString(hDevice, "BalanceRatioSelector",  
//                                "Blue");  
//获取白平衡调节范围  
GX_FLOAT_VALUE stRatioRange;  
emStatus = GXGetFloatValue(hDevice, "BalanceRatio", &stRatioRange);  
  
//设置最小白平衡系数  
emStatus = GXSetFloatValue(hDevice, "BalanceRatio",  
                           stRatioRange.dMin);  
  
//设置最大白平衡系数  
emStatus = GXSetFloatValue(hDevice, "BalanceRatio",  
                           stRatioRange.dMax);  
  
//设置自动白平衡感兴趣区域(代码中参数为举例，用户根据自己需要自行修改参数值)  
emStatus = GXSetIntValue(hDevice, "AWBROIWidth", 100);  
emStatus = GXSetIntValue(hDevice, "AWBROIHeight", 100);  
emStatus = GXSetIntValue(hDevice, "AWBROIOffsetX", 0);  
emStatus = GXSetIntValue(hDevice, "AWBROIOffsetY", 0);  
  
//自动白平衡设置-----  
  
//设置自动白平衡光照环境，比如当前相机所处环境为荧光灯  
emStatus = GXSetEnumValueByString(hDevice, "AWBLampHouse",  
                                   "Fluorescence");  
  
//设置连续自动白平衡  
emStatus = GXSetEnumValueByString(hDevice, "BalanceWhiteAuto",  
                                   "Continuous");  
  
//环境光源预设设置-----
```

```
//设置环境光源预设为 Daylight-6500K
emStatus = GXSetEnumValueByString(hDevice, "LightSourcePreset",
                                   "Daylight6500K");
```

● 注意事项

如果自动白平衡功能失效，可能是因为自动白平衡感兴趣区域太小，找不到白点造成。

3.7. 传输层控制

3.7.1. PayloadSize

● 名词解释

PayloadSize：此参数表示相机当前输出的每一帧图像数据大小（单位：字节），用户获取图像，分配 buffer 大小依据此值。当关闭帧信息的时候，输出图像不附带帧信息，此时 payloadSize 的大小就代表图像大小；当开启帧信息以后，输出图像附带帧信息，此时 payloadSize 的大小等于图像大小加帧信息大小。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
PayloadSize	INT	RO	数据大小

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stValue;
int64_t i64Value = 0;

emStatus = GXGetIntValue(hDevice, "PayloadSize", &stValue);
i64Value = stValue.nCurValue;
```

● 注意事项

用户分配 buffer 获取图像数据的时候，强烈建议用户使用 PayloadSize 的值。

3.7.2. IP 配置方式

● 相关属性

属性名称	属性类别	可访问属性	中文名称
GevCurrentIPConfigurationLLA	BOOL	RO	LLA 方式配置 IP
GevCurrentIPConfigurationDHCP	BOOL	RW	DHCP 方式配置 IP
GevCurrentIPConfigurationPersistentIP	BOOL	RW	永久 IP 方式配置 IP

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//本例以永久 IP 配置方式说明,其他 IP 配置方式类似
//设置为永久 IP 配置方式
emStatus = GXSetBoolValue(hDevice,
                           "GevCurrentIPConfigurationPersistentIP",
                           true);
```

3.7.3. 预估带宽

- 名词解释

预估带宽：在相机当前设置条件下，传输图像数据所需要的网络带宽。

- 相关参数

属性名称	属性类别	可访问属性	中文名称
EstimatedBandwidth	INT	RO	预估带宽

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//读取当前的预估带宽
GX_INT_VALUE stValue;
int64_t i64Value = 0;
emStatus = GXGetIntValue(hDevice, "EstimatedBandwidth", &stValue);
i64Value = stValue.nCurValue;
```

3.7.4. 设备心跳超时时间

- 名词解释

设备心跳超时时间：如果在此时间范围内，设备端接收不到来自主控主机的 GVCP 命令包，将会断开与主控主机的连接。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
GevHeartbeatTimeout	INT	RW	心跳超时时间

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//读取当前的设备心跳时间
GX_INT_VALUE stTimeValue;
emStatus = GXGetIntValue(hDevice, "GevHeartbeatTimeout",
                        &stTimeValue);

//读取设备心跳时间可设置范围参见 GXGetIntValue 接口的使用方法
//设置设备心跳时间
emStatus = GXSetIntValue(hDevice, "GevHeartbeatTimeout",
                        stTimeValue.nCurValue);
```

- 注意事项

强烈推荐用户使用默认值，除非特殊情况才允许用户根据现场环境修改此值。

3.7.5. 包长

- 名词解释

包长：指 GigE Vision 相机向主机端传输流通道数据的网络包大小，以字节为单位。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
GevSCPSPacketSize	INT	RW	流通道包长

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//读取包长
GX_INT_VALUE stValue;
int64_t i64PacketSize = 0;
emStatus = GXGetIntValue(hDevice, "GevSCPSPacketSize", &stValue);
i64PacketSize = stValue.nCurValue;
```

3.7.6. 包间隔

● 名词解释

包间隔：用于控制 GigE Vision 相机传输图像流数据的带宽。包间隔是在流通道传输的相邻网络数据包之间插入的空闲时钟个数。增加包间隔能够降低相机对网络带宽的占用率,同时也有可能降低了相机的帧率。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
GevSCPD	INT	RW	流通道包间隔

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//读取包间隔
int64_t i64PacketDelay = 0;
GX_INT_VALUE stValue;
emStatus = GXGetIntValue(hDevice, "GevSCPD", &stValue);
i64PacketDelay = stValue.nCurValue;
```

3.7.7. 链接速度

● 名词解释

链接速度：指当前工作的网络环境是千兆网还是百兆网。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
GevLinkSpeed	INT	RO	连接速度

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//读取当前的连接速度
GX_INT_VALUE stValue;
int64_t i64Value = 0;
emStatus = GXGetIntValue(hDevice, "GevLinkSpeed", &stValue);
```

```
i64Value = stValue.nCurValue;
```

3.8. 带宽控制

● 名词解释

USB3 Vision 相机提供带宽控制功能，用来控制单台设备的带宽上限。当设备链路带宽限制大于当前设备采集带宽值时，当前设备采集带宽将不发生改变；当设备链路带宽限制小于当前设备采集带宽时，当前设备采集带宽将会下降到设备链路带宽限制以下；当前设备采集带宽可以从相机中读取。

例如：当前设备采集带宽是 35000000Bps，设备链路带宽限制为 40000000Bps，当前设备采集带宽仍为 35000000Bps；当前设备采集带宽是 70000000Bps，设备链路带宽限制为 40000000Bps，当前设备采集带宽将变为 40000000Bps。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
DeviceLinkSelector	INT	RW	设备链路选择
DeviceLinkThroughputLimitMode	ENUM	RW	设备带宽限制模式
DeviceLinkThroughputLimit	INT	RW	设备带宽限制
DeviceLinkCurrentThroughput	INT	RO	当前设备采集带宽

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//开启带宽限制模式
emStatus = GXSetEnumValueByString(hDevice,
                                   "DeviceLinkThroughputLimitMode", "On");

//查询当前的带宽值
GX_INT_VALUE stLinkThroughputVal;
emStatus = GXGetIntValue(hDevice, "DeviceLinkCurrentThroughput",
                          &stLinkThroughputVal);

//设置带宽限制值
int64_t i64LinkThroughputLimitVal = 40000000;
emStatus = GXSetIntValue(hDevice, "DeviceLinkThroughputLimit",
                          i64LinkThroughputLimitVal);
```

● 注意事项

当开启设备带宽限制模式或者更改设备链路带宽限时，需要对设备进行停采操作。

3.9. 事件控制

● 名词解释

当特定的情形发生时，相机将产生对应的“事件”，并将事件消息发送到 PC 机，告知 PC 有“事件”

发生了。比如，在水星千兆网系列相机中，相机会在以下几种情况下产生并传递事件：曝光结束、图像帧数据丢弃、触发信号溢出、图像帧存数据不为空和事件队列溢出。每种事件都有相应的使能位，在默认情况下，相机的所有事件使能都为关闭状态。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
EventSelector	ENUM	RW	事件源选择
EventNotification	ENUM	RW	事件使能
EventExposureEnd	INT	RO	曝光结束事件 ID
EventExposureEndTimestamp	INT	RO	曝光结束事件时间戳
EventExposureEndFrameID	INT	RO	曝光结束事件帧 ID
EventBlockDiscard	INT	RO	数据包丢失事件 ID
EventBlockDiscardTimestamp	INT	RO	数据包丢失事件时间戳
EventBlockDiscardFrameID	INT	RO	数据包丢失事件帧 ID
EventOverrun	INT	RO	事件队列溢出事件 ID
EventOverrunTimestamp	INT	RO	事件队列溢出事件时间戳
EventFrameStartOvertrigger	INT	RO	触发信号被屏蔽事件 ID
EventFrameStartOvertriggerTimestamp	INT	RO	触发信号被屏蔽事件时间戳
EventFrameStartOvertriggerFrameID	INT	RO	触发信号被屏蔽事件帧 ID
EventBlockNotEmpty	INT	RO	帧存不为空事件 ID
EventBlockNotEmptyTimestamp	INT	RO	帧存不为空事件时间戳
EventBlockNotEmptyFrameID	INT	RO	帧存不为空事件帧 ID
EventInternalError	INT	RO	内部错误事件 ID
EventInternalErrorTimestamp	INT	RO	内部错误事件时间戳
EventFrameBurstStartOvertrigger	INT	RO	多帧触发屏蔽事件 ID
EventFrameBurstStartOvertriggerFrameID	INT	RO	多帧触发屏蔽事件帧 ID
EventFrameBurstStartOvertriggerTimestamp	INT	RO	多帧触发屏蔽事件时间戳
EventFrameStartWait	INT	RO	帧等待事件 ID
EventFrameStartWaitFrameID	INT	RO	帧等待事件帧 ID
EventFrameStartWaitTimestamp	INT	RO	帧等待事件时间戳
EventFrameBurstStartWait	INT	RO	多帧等待事件 ID
EventFrameBurstStartWaitTimestamp	INT	RO	多帧等待事件时间戳
EventFrameBurstStartWaitFrameID	INT	RO	多帧等待事件帧 ID

● 代码样例

参见 [GXRegisterFeatureCallbackByString\(\)](#) 接口介绍。

3.10. 查找表控制

- 名词解释

查找表：查找表为像素值转化的映射表，其与像素值一一对应，用来改变每个像素值输出。既可用于高位深图像向低位深图像的转化，也可用于同等位深图像单纯的像素值转化。查找表给予用户最大的灵活性，用以实现像素值的线性和非线性变换（如对数修正和指数修正），而无需消耗系统中宝贵的计算资源。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
LUTSelector	ENUM	RW	查找表选择
LUTValueAll	BUFFER	RW	查找表内容
LUTEnable	BOOL	RW	查找表使能
LUTIndex	INT	RW	查找表索引
LUTValue	INT	RW	查找表值

- 代码样例

参见 [GXGetRegisterValue\(\)](#)、[GXSetRegisterValue\(\)](#)两个接口说明。

- 注意事项

3.11. 用户配置

这一节描述的是对相机所有参数配置的控制，允许用户保存或者加载厂商参数组或者用户参数组。

- 名词解释

参数组类型：用户选择一种参数组类型（可能是厂商默认参数组或用户参数组 0），进行保存或者加载操作。

加载参数组：将用户选定的参数组加载到相机。

保存参数组：将相机当前的参数保存到用户选定的参数组里面。

启动参数组：设置设备上电启动的时候使用的参数组。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
UserSetSelector	ENUM	RW	参数组选择
UserSetLoad	COMMAND	WO	加载参数组
UserSetSave	COMMAND	WO	保存参数组
UserSetDefault	ENUM	RW	启动参数组

● 代码样例

```
//使用 GXGetEnumValue 接口
//查询当前相机支持的"UserSetSelector"类型
//参考接口说明部分，此处省略

//假设当前相机支持厂商默认参数组、用户参数组 0，共两种参数组
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//选择加载厂商参数组
emStatus = GXSetEnumValueByString(hDevice, "UserSetSelector",
                                   "Default");
emStatus = GXSetCommandValue(hDevice, "UserSetLoad");

//选择加载用户参数组 0
emStatus = GXSetEnumValueByString(hDevice, "UserSetSelector",
                                   "UserSet0");
emStatus = GXSetCommandValue(hDevice, "UserSetLoad");

//设置启动参数组为厂商参数组
emStatus = GXSetEnumValueByString(hDevice, "UserSetDefault",
                                   "Default");
```

● 注意事项

3.12. 设置相机 IP

3.12.1. 配置静态 IP 地址

● 名词解释

- 静态 IP：以静态 IP 方式配置相机 IP。
- DHCP：以 DHCP 方式配置相机 IP，即使用 DHCP 服务器分配的 IP。
- LLA：以 LLA(Link-Local Address)方式配置相机 IP。

● 相关参数

GX_IP_CONFIGURE_STATIC_IP：以静态 IP 方式配置相机 IP

GX_IP_CONFIGURE_DHCP：以 DHCP 方式配置相机 IP

GX_IP_CONFIGURE_LLA：以 LLA 方式配置相机 IP

GX_IP_CONFIGURE_DEFAULT：以默认方式配置相机 IP，选用此种方式，相机内部会将以上三种配置方式全部使能，但仍会以静态 IP 方式配置相机

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
//示例 MAC 地址，实际相机 MAC 地址可用 GXGetDeviceInfo 获得
char chArrMAC[] = "00-21-49-00-00-00";

char chArrIpAddress[] = "192.168.10.10";
char chArrSubnetMask[] = "255.255.255.0";
char chArrDefaultGateway[] = "192.168.10.2";
char chArrUserID[] = "Daheng Imaging";
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;

//本例以永久 IP 配置方式说明，其他 IP 配置方式类似
//设置为永久 IP 配置方式
emStatus = GXGigEIpConfiguration(chArrMAC, emIpConfigureMode,
                                   chArrIpAddress, chArrSubnetMask,
                                   chArrDefaultGateway, chArrUserID);
```

● 注意事项

- 调用此接口前，必须先进行枚举操作，并且执行此操作时相机不能是打开状态。
- 当选用 GX_IP_CONFIGURE_STATIC_IP 和 GX_IP_CONFIGURE_DEFAULT 参数配置 IP 时，传入接口的 IP 地址、子网掩码、默认网关等参数指针不允许为 NULL。
- 当选用 GX_IP_CONFIGURE_LLA 和 GX_IP_CONFIGURE_DHCP 参数配置 IP 时，传入接口的 IP 地址、子网掩码、默认网关等参数指针可以为 NULL。
- 用户自定义名称（UserID）允许的最大长度是 16 个字符；使用任何一种方式配置相机 IP，传入接口的用户自定义名称（UserID）参数指针均可以是 NULL。

3.12.2. Force IP

● 名词解释

Force IP：使用 Force IP 可以临时改变相机的 IP 地址，仅对本次使用有效，当相机掉电重启后会恢复原来的 IP。

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//示例 MAC 地址，实际相机 MAC 地址可用 GXGetDeviceIPInfo 获得
char chArrMAC[] = "00-21-49-00-00-00";
char chArrIpAddress[] = "192.168.10.10";
char chArrSubnetMask[] = "255.255.255.0";
char chArrDefaultGateway[] = "192.168.10.2";

//ForceIp
emStatus = GXGigEForceIp(chArrMAC, chArrIpAddress, chArrSubnetMask,
                          chArrDefaultGateway);
```

● 注意事项

调用此接口前，必须先进行枚举操作，并且执行此操作时相机不能是打开状态。

3.13. 其他

3.13.1. 自动曝光自动增益相关功能

在采集过程中，相机能够根据外界环境的光线变化，在一定范围内自动调节增益值和曝光时间，以最大限度达到用户的期望灰度，这就是自动增益和自动曝光功能。在默认参数下，相机根据整幅图像来计算并调节亮度，使感兴趣区域内的图像亮度达到期望值。

可变区域的自动增益和自动曝光功能非常灵活，用户可以根据不同的应用场景设定图像亮度的期望值，在一些背光或者图像局部亮度差异较大的应用场合，用户可以根据需要划定感兴趣区域，相机将根据设定值智能选取增益和曝光的调节比例，保证最好的图像质量。此外，用户还可以设定曝光时间调节的上下限和增益调节的上下限等参数。

3.13.1.1. 期望灰度值

● 名词解释

期望灰度：此参数是自动曝光和自动增益调节的一个基准参数，自动功能调节的最终期望是达到用户设置的期望灰度值。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ExpectedGrayValue	INT	RW	期望灰度值

● 代码样例

```

//获取期望灰度值调节范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stGrayValueRange;
emStatus = GXGetIntValue(hDevice, "ExpectedGrayValue",
                        &stGrayValueRange);

//设置最小期望灰度值
emStatus = GXSetIntValue(hDevice, "ExpectedGrayValue",
                        stGrayValueRange.nMin);

//设置最大期望灰度值
emStatus = GXSetIntValue(hDevice, "ExpectedGrayValue",
                        stGrayValueRange.nMax);

```

● 注意事项

3.13.1.2. 光照环境

● 名词解释

光照环境：指的是相机工作的外部环境，分为自然光、50Hz 交流电光源、60Hz 交流电光源。自动功

能根据外部光照条件更好的进行适应调节。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
AALightEnvironment	ENUM	RW	2A 光照环境

● 代码样例

```
//使用 GXGetEnumValue 接口
//查询当前相机支持的"AALightEnvironment"类型
//参考接口说明部分，此处省略
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//选择增益通道类型
emStatus = GXSetEnumValueByString(hDevice, "AALightEnvironment",
                                   "NatureLight");
//emStatus = GXSetEnumValueByString(hDevice, "AALightEnvironment",
//                                   "AC60HZ");
```

● 注意事项

3.13.1.3. 统计区域

● 名词解释

统计区域 :相机支持可变区域的自动功能统计 ,相机默认是整幅图像的区域进行统计 ,已达到期望灰度 ,而统计区域功能可支持用户自动选择要进行自动功能的部分区域 ,这样比较灵活。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
AAROIWidth	INT	RW	自动调节感兴趣区域宽度
AAROIHeight	INT	RW	自动调节感兴趣区域高度
AAROIOffsetX	INT	RW	自动调节感兴趣区域 X 坐标
AAROIOffsetY	INT	RW	自动调节感兴趣区域 Y 坐标

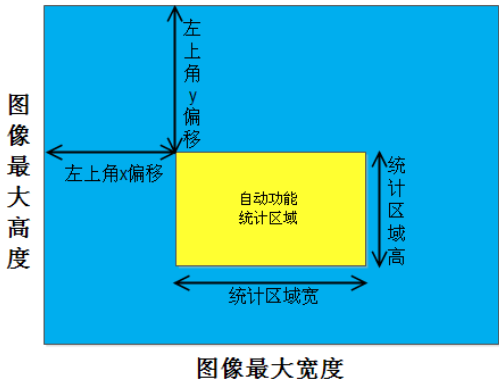


图 3-34 统计区域

● 代码样例

```
//获取调节范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stROIWidthRange;
GX_INT_VALUE stROIHeightRange;
GX_INT_VALUE stROIYRange;
GX_INT_VALUE stROIYRange;
emStatus = GXGetIntValue(hDevice, "AAROIWidth", &stROIWidthRange);
emStatus = GXGetIntValue(hDevice, "AAROIHeight", &stROIHeightRange);
emStatus = GXGetIntValue(hDevice, "AAROIOffsetX", &stROIYRange);
emStatus = GXGetIntValue(hDevice, "AAROIOffsetY", &stROIYRange);

//设置统计区域为整幅图像
emStatus = GXSetIntValue(hDevice, "AAROIWidth", stROIWidthRange.nMax);
emStatus = GXSetIntValue(hDevice, "AAROIHeight",
                           stROIHeightRange.nMax);
emStatus = GXSetIntValue(hDevice, "AAROIOffsetX", 0);
emStatus = GXSetIntValue(hDevice, "AAROIOffsetY", 0);
```

● 注意事项

- 1) 统计区域只是为自动曝光和自动增益的计算提供一个统计范围，相机根据此范围内的数据情况进行计算得出调整参数，然后应用于整幅画面，而不仅仅应用于所选区域。
- 2) 为了保证 AAROI 的有效性，AAROI 的四个属性的可调范围如下公式：

$$0 < \text{AAROIOffsetX} \leq \text{Width (图像最大宽)} - \text{AAROIWidth}$$

$$0 < \text{AAROIOffsetY} \leq \text{Height (图像最大高)} - \text{AAROIHeight}$$

$$0 < \text{AAROIWidth} \leq \text{Width (图像最大宽)} - \text{AAROIOffsetX}$$

$$0 < \text{AAROIHeight} \leq \text{Height (图像最大高)} - \text{AAROIOffsetY}$$

3.13.1.4. 自动调节范围

● 名词解释

- 自动曝光范围：当开启自动曝光模式后，相机会在一定范围内动态调节曝光值来适应当前环境，用户可以调节这个范围。
- 自动增益范围：当开启自动增益模式后，相机会在一定范围内动态调节增益值来适应当前环境，用户可以调节这个范围。
- 自动曝光：相机的一种工作模式，在此模式下相机自动根据外界环境在一定范围内调整曝光时间已达到最佳图像质量。
- 自动增益：相机的一种工作模式，在此模式下相机自动根据外界环境在一定范围内调整增益值已达到最佳图像质量。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
AutoGainMin	FLOAT	RO	自动增益最小值
AutoGainMax	FLOAT	RO	自动增益最大值
AutoExposureTimeMin	FLOAT	RW	自动曝光最小值
AutoExposureTimeMax	FLOAT	RW	自动曝光最大值

● 代码样例

```
//获取调节范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stAutoGainMinRange;
GX_FLOAT_VALUE stAutoGainMaxRange;
GX_FLOAT_VALUE stAutoExposureMinRange;
GX_FLOAT_VALUE stAutoExposureMaxRange;
emStatus = GXGetFloatValue(hDevice, "AutoGainMin",
                           &stAutoGainMinRange);
emStatus = GXGetFloatValue(hDevice, "AutoGainMax",
                           &stAutoGainMaxRange);
emStatus = GXGetFloatValue(hDevice, "AutoExposureTimeMin",
                           &stAutoExposureMinRange);
emStatus = GXGetFloatValue(hDevice, "AutoExposureTimeMax",
                           &stAutoExposureMaxRange);

//设置调节边界值
emStatus = GXSetFloatValue(hDevice, "AutoGainMin",
                           stAutoGainMinRange.dMin);
emStatus = GXSetFloatValue(hDevice, "AutoGainMax",
                           stAutoGainMinRange.dMax);
emStatus = GXSetFloatValue(hDevice, "AutoExposureTimeMin",
                           stAutoExposureMinRange.dMin);
emStatus = GXSetFloatValue(hDevice, "AutoExposureTimeMax",
                           stAutoExposureMaxRange.dMax);
```

● 注意事项

为了保证自动功能范围值的正确合法性，对自动功能可调范围进行了一些特别处理：

比如“AutoGainMin”（自动增益功能的最小值），这个最小值本身也是有调节范围的，这个调节范围的最大值不能大于自动增益功能的最大值的当前值；同理，“AutoGainMax”（自动增益功能的最大值），这个值也是有可调范围的，这个可调范围的最小值不能小于自动增益功能的最小值的当前值。

3.13.2. 坏点校正

● 名词解释

坏点：Sensor 表面可能存在极少的坏点或者死点，这样就不能正确的反应实际的色彩，在进行插值时这些坏点还会将周边的色彩污染，为了解决这个问题，厂商采用软件的方法对图像进行预处理，消除坏点。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
DeadPixelCorrect	ENUM	RW	坏点校正

● 代码样例

```
//开启自动坏点校正
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSetEnumValueByString(hDevice, "DeadPixelCorrect", "On");

//关闭自动坏点校正
emStatus = GXSetEnumValueByString(hDevice, "DeadPixelCorrect", "Off");
```

● 注意事项

3.13.3. ADC Level (DigitalShift)

● 名词解释

假设当前相机为 10 位输出或者 12 位输出，为了方便图像显示，需要从 10 位或者 12 位数据中选择 8 位进行输出显示。当相机输出图像不是 8 位时，此功能可以用于选择相机输出哪 8 位图像。

在有些设备中将 ADC Level 叫为数字移位，即 DigitalShift。

假如当前相机输出 10 位：共三个转换级别

Level0 : 0~7bit

Level1 : 1~8bit

Level2 : 2~9bit

假如当前相机输出 12 位：共五个转换级别

Level0 : 0~7bit

Level1 : 1~8bit

Level2 : 2~9bit

Level3 : 3~10bit

Level4 : 4~11bit

ADC Level : 级别选择越低，图像越亮，噪声越大；级别选择越高，图像越暗，噪声越小。

DigitalShift : 数值越低，图像越暗，噪声越小；数值越大，图像越亮，噪声越大。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
ADCLevel	INT	RW	AD 转换级别
DigitalShift	INT	RW	数字移位

- 效果图（假设相机输出 10 位，我们要挑选 8 位输出）



图 3-35 选择 Level0 (0~7bit)



图 3-36 选择 Level1 (1~8bit)



图 3-37 选择 Level2 (2~9bit)

- 代码样例

```
//获取转换级别范围(示例同样适用于选位输出"DigitalShift")
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stADCLevelRange;
emStatus = GXGetIntValue(hDevice, "ADCLevel", &stADCLevelRange);
//设置最低转换级别
emStatus = GXSetIntValue(hDevice, "ADCLevel", stADCLevelRange.nMin);
//设置最高转换级别
emStatus = GXSetIntValue(hDevice, "ADCLevel", stADCLevelRange.nMax);
```

- 注意事项

假如当前相机本身输出时 10 位，用户设置相机输出格式也是 10 位，那么就涉及不到选择 8 位的情况，那么这时候 ADC Level 功能将不可使用。只有相机本身输出 10 位数据，但是用户要求相机输出 8 位数据的时候才需要选择输出哪 8 位。

3.13.4. 消隐控制

● 名词解释

水平消隐（行消隐）：在将光信号转换为电信号的扫描过程中，扫描总是从图像的左上角开始，水平向前行进，同时扫描点也以较慢的速率向下移动。当扫描点到达图像右侧边缘时，扫描点快速返回左侧，重新开始在第 1 行的起点下面进行第 2 行扫描，行与行之间的返回过程称为水平消隐(HBlank)。

垂直消隐（场消隐）：一幅完整的图像扫描信号，由水平消隐间隔分开的行信号序列构成，称为一帧。扫描点扫描完一帧后，要从图像的右下角返回到图像的左上角，开始新一帧的扫描，这一时间间隔，叫做垂直消隐，也称场消隐(VBlank)。

● 相关属性

属性名称	属性类别	可访问属性	中文名称
HBlanking	INT	RW	水平消隐
VBlanking	INT	RW	垂直消隐

● 代码样例

```
//获取水平消隐范围
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stHBRange;
emStatus = GXGetIntValue(hDevice, "HBlanking", &stHBRange);
//设置最小水平消隐
emStatus = GXSetIntValue(hDevice, "HBlanking", stHBRange.nMin);
//设置最大水平消隐
emStatus = GXSetIntValue(hDevice, "HBlanking", stHBRange.nMax);
//获取垂直消隐范围
GX_INT_VALUE stVBRange;
emStatus = GXGetIntValue(hDevice, "VBlanking", &stVBRange);
//设置最小垂直消隐
emStatus = GXSetIntValue(hDevice, "VBlanking", stVBRange.nMin);
//设置最大垂直消隐
emStatus = GXSetIntValue(hDevice, "VBlanking", stVBRange.nMax);
```

● 注意事项

增大水平消隐或者垂直消隐都会降低帧率。反之，则会提高帧率。

3.13.5. 用户加密区

● 名词解释

用户加密区是为保护自有知识产权的应用而专门设置的。加密区是用户自定义的，当加密区打开时，可以被当做普通参数区使用；当加密区关闭后，则必须使用密钥打开。用户可以通过保险箱与自己的软件更紧密的联系在一起，从而提高解密难度，保护自有知识产权。产品出厂时加密区是打开的。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
UserPassword	STRING	RW	用户密码
VerifyPassword	STRING	RW	校验用户密码
UserData	BUFFER	RW	加密区内容

- 代码样例

具体使用方法请联系技术支持 (support@daheng-imaging.com)

- 注意事项

使用加密区仅能提高一些解密难度,不能确保不被破解,因此大恒公司不对用户因保险箱被破解导致的损失承担任何法律责任。

3.13.6. 用户数据区

- 名词解释

用户数据区是为用户预留出来的一定范围的数据区域,该区域可读可写,共 16K 字节大小,分为 4 个数据段,每个数据段为 4K 字节大小。读写数据段时,首先需要设置数据段编号,然后根据数据、数据长度来读写所设置的数据段,数据写入后立即保存到相机 Flash 区域中。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
DataFieldSelector	ENUM	RW	用户选择 Flash 数据区域
DataFieldValue	BUFFER	RW	用户区内容

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//申请长度为 4K 的 buffer, 该 buffer 需要由用户填充数据
size_t nLength = 4096;
uint8_t* pui8SetBuffer = new uint8_t[nLength];

GX_NODE_ACCESS_MODE emAccessMode = GX_NODE_ACCESS_MODE_NI;
emStatus = GXGetNodeAccessMode(hDevice, "DataFieldSelector",
                                &emAccessMode);
if (emAccessMode != GX_NODE_ACCESS_MODE_NI)
{
    //设置 Flash 数据区域为编号 1
    emStatus = GXSetEnumValueByString(hDevice, "DataFieldSelector",
                                       "DataField_1");

    //设置用户区内容
    emStatus = GXSetRegisterValue(hDevice, "DataFieldValue",
                                   pui8SetBuffer, nLength);

    //获取用户区内容
```

```

        emStatus = GXGetRegisterValue(hDevice, "DataFieldValue",
                                       pui8SetBuffer, &nLength);
    }
    else
    {
        //设置用户区内容
        emStatus = GXSetRegisterValue(hDevice, "DataFieldValueAll",
                                       pui8SetBuffer, nLength);

        //获取用户区内容
        emStatus = GXGetRegisterValue(hDevice, "DataFieldValueAll",
                                       pui8SetBuffer, &nLength);
    }

    //操作完成后
    if(NULL != pui8SetBuffer)
    {
        delete[] pui8SetBuffer;
        pui8SetBuffer = NULL;
    }

```

- 注意事项

3.13.7. 取消参数范围限制

- 名词解释

通过设置取消参数范围限制后,可以使部分有参数范围限制的功能范围变大,支持的范围达到不会使相机图像或功能出现异常的范围。影响的功能节点有白平衡分量、曝光、增益、自动曝光、自动增益、锐度、黑电平的范围。

- 相关属性

属性名称	属性类别	可访问属性	中文名称
RemoveParameterLimit	ENUM	RW	取消参数范围限制

- 代码样例

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
//开启取消参数范围限制
emStatus = GXSetEnumValueByString(hDevice, "RemoveParameterLimit",
                                   "On");

//关闭取消参数范围限制
emStatus = GXSetEnumValueByString(hDevice, "RemoveParameterLimit",
                                   "Off");

```

- 注意事项

3.13.8. 平场校正

- 名词解释

阴影：成像器件各像元响应不一致造成中心与四个边缘的亮度不一致以及由于 LENS 在周边入射角度不足，导致颜色出现偏差的现象，一般表现中心和四周偏角的颜色不一致。为了解决这一现象，采用软件处理的方法对图像进行预处理，消除阴影。

- 相关属性

属性名	属性类别	可访问属性	中文名称
FlatFieldCorrection	ENUM	RW	平场校正开关
FFCLoad	BUFFER	RO	获取平场校正参数
FFCSave	BUFFER	WO	设置平场校正参数

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//开启平场校正
emStatus = GXSetEnumValueByString(hDevice, "FlatFieldCorrection",
                                   "On");

//获取平场校正参数的长度
size_t nSize = 0;
emStatus = GXGetRegisterLength(hDevice, "FFCLoad", &nSize);

//申请一段内存，用于保存平场校正参数
char* pszBuffer = new char[nSize];

//从设备中加载出平场校正参数
emStatus = GXGetRegisterValue(hDevice, "FFCLoad",
                              (uint8_t *)pszBuffer, &nSize);

//将平场校正参数保存到设备中
emStatus = GXSetRegisterValue(hDevice, "FFCSave",
                              (uint8_t *)pszBuffer, nSize);

//关闭平场校正
emStatus = GXSetEnumValueByString(hDevice, "FlatFieldCorrection",
                                   "Off");

//操作完成后
If (NULL != pszBuffer)
{
    delete[] pszBuffer;
    pszBuffer = NULL;
}
```

4. 本地设备控制

4.1. 相关参数

属性名称	属性类别	可访问属性	中文名称
DeviceCommandTimeout	INT	RW	命令超时
DeviceCommandRetryCount	INT	RW	命令重试次数

4.2. 示例代码

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;

//以设置命令重试次数为例
uint64_t ui64CommandRetryCount = 0;
emStatus = GXSetIntValue(hDevice, "DeviceCommandRetryCount",
                        ui64CommandRetryCount);

```

4.3. 注意事项

5. 流层控制

5.1. 统计参数

5.1.1. 相关参数

属性名称	属性类别	可访问属性	中文名称
StreamAnnouncedBufferCount	INT	RO	声明的 Buffer 个数
StreamDeliveredFrameCount	INT	RO	接收帧个数
StreamLostFrameCount	INT	RO	不足导致的丢帧个数
StreamIncompleteFrameCount	INT	RO	接收的残帧个数
StreamDeliveredPacketCount	INT	RO	接收到的包数
StreamResendPacketCount	INT	RO	重传包个数
StreamRescuedPacketCount	INT	RO	重传成功包个数
StreamResendCommandCount	INT	RO	重传命令次数
StreamUnexpectedPacketCount	INT	RO	异常包个数
StreamMissingBlockIDCount	INT	RO	BlockID 丢失个数

5.1.2. 注意事项

在采集过程中，用户可以随时读取这些参数来观测当前驱动的采集状态。

5.1.3. 代码样例

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE hDataStream = 0;
emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDataStream);

//以读取接收到的帧个数为例
uint64_t ui64GetFrameCount = 0;
GX_INT_VALUE stValue;
emStatus = GXGetIntValue(hDataStream, "StreamDeliveredFrameCount",
                        &stValue);
ui64GetFrameCount = stValue.nCurValue;

```

5.2. 控制参数

5.2.1. 相关属性

● GigE Vision 相机相关设置参数

属性名称	属性类别	可访问属性	中文名称
MaxPacketCountInOneBlock	INT	RW	数据块最大重传包数
MaxPacketCountInOneCommand	INT	RW	一次重传命令最大包含的包数

ResendTimeout	INT	RW	重传超时时间
MaxWaitPacketCount	INT	RW	最大等待包数
ResendMode	ENUM	RW	重传模式
BlockTimeout	INT	RW	数据块超时时间
MaxNumQueueBuffer	INT	RW	采集队列最大 Buffer 个数
StreamBufferHandlingMode	ENUM	RW	Buffer 处理模式

“MaxPacketCountInOneBlock”：数据块最大重传包数，一帧图像允许的重传包数最大值，如果某帧图像统计发送重传包太多而超过此值，则不会再发送重传命令，而是直接判定残帧。此值为 0 表示关闭此功能。

“MaxPacketCountInOneCommand”：一次重传命令最大包含的包数，接收过程中判定到丢包，如果本次丢包太多，超过此值，则不会发送重传命令，而是直接判断残帧。此值为0表示关闭此功能。

“ResendTimeout”：重传超时时间，发送重传命令之后，等待重传包到来的等待时间，如果在此时间内重传包没有到来，则判断残帧。

“MaxWaitPacketCount”：最大等待包数，如果某帧图像还没接收完，但是下一帧图像已经到来，如果下一帧图像接收了N个包，上一帧还没接收完，则强制结束上一帧，将其判断为残帧。此值为0表示关闭此功能。

“ResendMode”：重传模式，控制重传开关。

“BlockTimeout”：数据块超时时间，帧接收的总时间，如果超过此时间还未接收完毕，则直接判断为残帧。

“MaxNumQueueBuffer”：采集队列最大Buffer个数，它可调节底层实际参与数据接收的buffer个数，在多相机高分辨率同时采集失败的时候，可以适当减小此参数的值，这样可以保证多相机都可同时采集，但是高帧率采集的情况下此值如果过小，可能会有丢帧。具体用法请联系技术支持。

“StreamBufferHandlingMode”：对于当前流通道，可能支持的Buffer处理模式。GigE Vision支持三种模式，这三种模式的定义及功能简述如下：

“StreamBufferHandlingMode”（OldestFirst）：默认值。图像缓冲区遵守先进先出的原则，所有的缓冲区全部填满后，新的图像数据会被丢弃，直到用户完成已经填满图像数据的缓冲区处理。

典型应用场景是，要求接收到相机采集的每帧图像，不丢帧。该模式实现不丢帧，还需要图像数据的传输和处理的速度尽量快（至少小于帧周期）。

“StreamBufferHandlingMode”（OldestFirstOverwrite）：同样遵守先进先出的原则。与OldestFirst模式的区别是，当所有的缓冲区全部填满后，SDK将主动丢弃缓冲区中时间戳最旧的一帧图像缓冲区，用于接收新的图像数据。

典型应用场景是，不要求接收相机采集的每帧图像，应用环境下图像传输与处理的速度较慢。

“StreamBufferHandlingMode”（NewestOnly）：该模式下用户拿到的始终是SDK接收到的最新图。SDK每接收到一帧新的图像数据，就会主动丢弃旧时间戳的图像，因此当用户图像处理不及时或者速度较慢时，就会出现丢帧。

该模式主要应用场合是，对图像采集与显示实时性要求比较高，且不要求接收到相机采集的每帧图像。但是受相机的采集帧率和内部缓存，以及传输速度、用户使用场景的限制，SDK接收到的最新图与相机最新曝光的图像可能有延迟。

● USB3 Vision 相机相关设置参数

属性名称	属性类别	可访问属性	中文名称
StreamTransferSize	INT	RW	传输数据块大小
StreamTransferNumberUrb	INT	RW	传输数据块数量
StreamBufferHandlingMode	ENUM	RW	Buffer 处理模式

“StreamTransferSize”：传输数据块大小，USB3 Vision相机传输时每个数据块的大小。

“StreamTransferNumberUrb”：传输数据块数量，此参数限制了单台设备实际映射到系统内核的数据块数量，默认为64，该值设置过小会影响传输效率；但是，可以映射到系统内核的总的数据块数量是有限的，多台相机共同使用时，可以适当减小该值，增加可同时采集的设备数量。

“StreamBufferHandlingMode”：对于当前流通道，可能支持的Buffer处理模式。USB3 Vision支持的模式与GigE Vision一致。

5.2.2. 注意事项

以上控制参数只能在停采之后进行修改，采集过程中不允许修改控制参数。

5.2.3. 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE hDataStream = 0;
emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDataStream);

//以设置重传超时时间为例
uint64_t ui64ResendTimeout = 128;
emStatus = GXSetIntValue (hDataStream, "ResendTimeout",
                           ui64ResendTimeout);
```


6. 受相机型号影响的功能

相机类型：MER-040-60UM/C	
1	【UC/UM】调整“Width”的值会联动更新“HBlanking”的当前值和最小值 关系式如下： $HBlanking_min = MAX(-21, 0x236 - Width + 1)$ $HBlanking_cur = MAX(HBlanking_cur, HBlanking_min)$
2	【UM】“BinningHorizontal”和“BinningVertical”二者具有联动关系

表 6-1 MER-040-60UM/C

相机类型：MER-500-7UM/C	
1	在调节 Binning 功能的时候，需要 Decimation 的水平和垂直必须都是 0 在调节 Decimation 功能的时候，需要 Binning 的水平和垂直必须都是 0
2	Binning 的水平参数只能设置 0、1、3，不能设置 2

表 6-2 MER-500-7UM/C

7. GxI API 库说明

7.1. 类型

7.1.1. 数据类型

名称	描述
Int8_t	有符号 8 位整数
Int16_t	有符号 16 位整数
Int32_t	有符号 32 位整数
Int64_t	有符号 64 位整数
uint8_t	无符号 8 位整数
uint16_t	无符号 16 位整数
uint32_t	无符号 32 位整数
uint64_t	无符号 64 位整数

7.1.2. 句柄类型

名称	描述
GX_PORT_HANDLE	通用句柄
GX_IF_HANDLE	Interface 句柄
GX_DEV_HANDLE	设备句柄， 通过 GXOpenDevice 获取，通过此句柄进行控制与采集
GX_DS_HANDLE	DataStream 句柄
GX_LOCAL_DEV_HANDLE	本地设备层句柄
GX_EVENT_CALLBACK_HANDLE	设备事件回调句柄， 注册设备相关事件回调函数，比如设备掉线回调函数
GX_FEATURE_CALLBACK_BY_STRING_HANDLE	设备属性更新回调句柄
GX_FEATURE_CALLBACK_HANDLE	设备属性更新回调句柄， 注册设备属性更新回调函数的时候获取

7.1.3. 回调函数类型

名称	描述
typedef void (GX_STDC * GXCaptureCallBack) (GX_FRAME_CALLBACK_PARAM *pFrameData)	采集回调函数类型

typedef void (GX_STDC *GXDeviceOfflineCallBack) (void *pUserParam)	掉线回调函数类型
typedef void (GX_STDC *GXFeatureCallBack) (GX_FEATURE_ID_CMD nFeatureID , void *pUserParam)	设备属性更新回调函数类型
typedef void (GX_STDC *GXFeatureCallBackByString) (const char* pszFeatureName, void *pUserParam)	设备属性更新回调函数类型

7.2. 常量

7.2.1. GX_STATUS_LIST

```
typedef enum GX_STATUS_LIST
{
    GX_STATUS_SUCCESS           = 0,
    GX_STATUS_ERROR             = -1,
    GX_STATUS_NOT_FOUND_TL      = -2,
    GX_STATUS_NOT_FOUND_DEVICE = -3,
    GX_STATUS_OFFLINE           = -4,
    GX_STATUS_INVALID_PARAMETER = -5,
    GX_STATUS_INVALID_HANDLE    = -6,
    GX_STATUS_INVALID_CALL      = -7,
    GX_STATUS_INVALID_ACCESS    = -8,
    GX_STATUS_NEED_MORE_BUFFER = -9,
    GX_STATUS_ERROR_TYPE        = -10,
    GX_STATUS_OUT_OF_RANGE      = -11,
    GX_STATUS_NOT_IMPLEMENTED   = -12,
    GX_STATUS_NOT_INIT_API      = -13,
    GX_STATUS_TIMEOUT           = -14
}GX_STATUS_LIST;
```

名称	描述
GX_STATUS_SUCCESS	成功
GX_STATUS_ERROR	不希望发生的未明确指明的内部错误
GX_STATUS_NOT_FOUND_TL	找不到 TL 库
GX_STATUS_NOT_FOUND_DEVICE	找不到设备
GX_STATUS_OFFLINE	当前设备为掉线状态
GX_STATUS_INVALID_PARAMETER	无效参数，一般是指针为 NULL 或输入的 IP 等参数格式无效
GX_STATUS_INVALID_HANDLE	无效句柄
GX_STATUS_INVALID_CALL	无效的接口调用,专指软件接口逻辑错误
GX_STATUS_INVALID_ACCESS	功能当前不可访问或设备访问模式错误
GX_STATUS_NEED_MORE_BUFFER	用户申请的 buffer 不足：读操作时用户输入 buffersize 小于实际需要

GX_STATUS_ERROR_TYPE	用户使用的 FeatureID 类型错误，比如整型接口使用了浮点型的功能码
GX_STATUS_OUT_OF_RANGE	用户写入的值越界
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_NOT_INIT_API	没有调用初始化接口
GX_STATUS_TIMEOUT	超时错误

7.2.2. GX_FRAME_STATUS

```
enum GX_FRAME_STATUS_LIST
{
    GX_FRAME_STATUS_SUCCESS          = 0,
    GX_FRAME_STATUS_INCOMPLETE       = -1
};
typedef int32_t GX_FRAME_STATUS;
```

名称	描述
GX_FRAME_STATUS_SUCCESS	正常帧
GX_FRAME_STATUS_INCOMPLETE	残帧

7.2.3. GX_ACCESS_MODE

```
typedef enum GX_ACCESS_MODE
{
    GX_ACCESS_READONLY          =2,
    GX_ACCESS_CONTROL           =3,
    GX_ACCESS_EXCLUSIVE          =4
}GX_ACCESS_MODE;
typedef int32_t GX_ACCESS_MODE_CMD;
```

名称	描述
GX_ACCESS_READONLY	以只读方式打开设备
GX_ACCESS_CONTROL	以控制方式打开设备
GX_ACCESS_EXCLUSIVE	以独占方式打开设备

7.2.4. GX_OPEN_MODE

```
typedef enum GX_OPEN_MODE
{
    GX_OPEN_SN                  =0,
    GX_OPEN_IP                  =1,
    GX_OPEN_MAC                  =2,
    GX_OPEN_INDEX               =3,
    GX_OPEN_USERID              =4
}GX_OPEN_MODE;
typedef int32_t GX_OPEN_MODE_CMD;
```

名称	描述
GX_OPEN_SN	通过序列号打开设备
GX_OPEN_IP	通过 IP 地址打开设备
GX_OPEN_MAC	通过 MAC 地址打开设备
GX_OPEN_INDEX	通过设备序号 (从 1 开始 , 1、2、3、4...) 打开设备
GX_OPEN_USERID	通过用户自定义 ID 打开设备

7.2.5. GX_IP_CONFIGURE_MODE_LIST

```
typedef enum GX_IP_CONFIGURE_MODE_LIST
{
    GX_IP_CONFIGURE_DHCP ,
    GX_IP_CONFIGURE_LLA ,
    GX_IP_CONFIGURE_STATIC_IP ,
    GX_IP_CONFIGURE_DEFAULT
}GX_IP_CONFIGURE_MODE_LIST;
typedef int32_t GX_IP_CONFIGURE_MODE;
```

名称	描述
GX_IP_CONFIGURE_DHCP	启用 DHCP 模式 , 由 DHCP 服务器分配 IP 地址
GX_IP_CONFIGURE_LLA	启用 LLA 方式分配 IP 地址
GX_IP_CONFIGURE_STATIC_IP	使用静态 IP 方式配置 IP 地址
GX_IP_CONFIGURE_DEFAULT	使用默认方式配置 IP 地址

7.2.6. GX_RESET_DEVICE_MODE

```
typedef enum GX_RESET_DEVICE_MODE
{
    GX_MANUFACTURER_SPECIFIC_RECONNECT    = 0x1,
    GX_MANUFACTURER_SPECIFIC_RESET        = 0x2
}GX_RESET_DEVICE_MODE;
typedef int32_t GX_RESET_DEVICE_MODE;
```

名称	描述
GX_MANUFACTURER_SPECIFIC_RECONNECT	设备重连 , 等同于软件接口关闭设备
GX_MANUFACTURER_SPECIFIC_RESET	设备复位 , 等同于给设备掉电上电一次

7.2.7. GX_TL_TYPE_LIST

```
typedef enum GX_TL_TYPE_LIST
{
    GX_TL_TYPE_UNKNOWN    = 0,
    GX_TL_TYPE_USB        = 1,
    GX_TL_TYPE_GEV        = 2,
    GX_TL_TYPE_U3V        = 4,
```

```

GX_TL_TYPE_CXP          = 8
}GX_TL_TYPE_LIST;
typedef int32_t GX_TL_TYPE;

```

名称	描述
GX_TL_TYPE_UNKNOWN	未知类型设备
GX_TL_TYPE_USB	USB2.0
GX_TL_TYPE_GEV	GEV
GX_TL_TYPE_U3V	U3V
GX_TL_TYPE_CXP	CXP

7.2.8. GX_NODE_ACCESS_MODE

```

typedef enum GX_NODE_ACCESS_MODE
{
    GX_NODE_ACCESS_MODE_NI      = 0,
    GX_NODE_ACCESS_MODE_NA      = 1,
    GX_NODE_ACCESS_MODE_WO      = 2,
    GX_NODE_ACCESS_MODE_RO      = 3,
    GX_NODE_ACCESS_MODE_RW      = 4,
    GX_NODE_ACCESS_MODE_UNDEF   = 5
}GX_NODE_ACCESS_MODE;

```

名称	描述
GX_NODE_ACCESS_MODE_NI	不支持
GX_NODE_ACCESS_MODE_NA	不可读写
GX_NODE_ACCESS_MODE_WO	只写
GX_NODE_ACCESS_MODE_RO	只读
GX_NODE_ACCESS_MODE_RW	可读写
GX_NODE_ACCESS_MODE_UNDEF	未定义

7.2.9. GX_DEVICE_CLASS

```

enum GX_DEVICE_CLASS_LIST
{
    GX_DEVICE_CLASS_UNKNOWN      = 0,
    GX_DEVICE_CLASS_USB2        = 1,
    GX_DEVICE_CLASS_GEV         = 2,
    GX_DEVICE_CLASS_U3V         = 3,
    GX_DEVICE_CLASS_SMART        = 4,
    GX_DEVICE_CLASS_CXP          = 5
};
typedef int32_t GX_DEVICE_CLASS;

```

名称	描述
GX_DEVICE_CLASS_UNKNOWN	未知设备种类
GX_DEVICE_CLASS_USB2	USB2.0 设备
GX_DEVICE_CLASS_GEV	千兆网设备 (GigE Vision)
GX_DEVICE_CLASS_U3V	USB3.0 设备 (USB3 Vision)
GX_DEVICE_CLASS_SMART	Smart 设备
GX_DEVICE_CLASS_CXP	CXP 设备

7.2.10. GX_LOG_TYPE_LIST

```
enum GX_LOG_TYPE_LIST
{
    GX_LOG_TYPE_OFF = 0x00000000,
    GX_LOG_TYPE_FATAL= 0x00000001,
    GX_LOG_TYPE_ERROR= 0x00000010,
    GX_LOG_TYPE_WARN= 0x00000100,
    GX_LOG_TYPE_INFO= 0x00001000,
    GX_LOG_TYPE_DEBUG = 0x00010000,
    GX_LOG_TYPE_TRACE = 0x00100000
};
typedef uint32_t GX_LOG_TYPE_LIST;
```

名称	描述
GX_LOG_TYPE_OFF	不生成任何类型日志
GX_LOG_TYPE_FATAL	生成 FATAL 类型日志
GX_LOG_TYPE_ERROR	生成 ERROR 类型日志
GX_LOG_TYPE_WARN	生成 WARNING 类型日志
GX_LOG_TYPE_INFO	生成 INFO 类型日志
GX_LOG_TYPE_DEBUG	生成 DEBUG 类型日志
GX_LOG_TYPE_TRACE	生成 TRACE 类型日志

7.3. 结构体

7.3.1. GX_OPEN_PARAM

相关接口：[GXOpenDevice\(\)](#)

此结构体是打开设备接口专用的结构体。

```
typedef struct GX_OPEN_PARAM
{
    cha *pszContent;
    GX\_OPEN\_MODE\_CMD openMode;
    GX\_ACCESS\_MODE\_CMD accessMode;
}GX_OPEN_PARAM;
```

名称	描述
pszContent	标准 C 字符串，由 openMode 决定，可能是一个 IP 地址或者是相机序列号等等
openMode	打开设备的方式，通过 SN、IP、MAC 等方式打开相机，参考 GX_OPEN_MODE
accessMode	访问设备的方式，只读、控制、独占等方式，参考 GX_ACCESS_MODE

7.3.2. GX_FRAME_CALLBACK_PARAM

相关接口：[GXRegisterCaptureCallback\(\)](#)

这是采集回调处理函数的形参类型，用户编写的采集回调处理函数的形参类型必须为此类型。

```
typedef struct GX_FRAME_CALLBACK_PARAM
{
    void*          pUserParam;
    GX\_FRAME\_STATUS status;
    const void*    plmgBuf;
    int32_t        nImgSize;
    int32_t        nWidth;
    int32_t        nHeight;
    int32_t        nPixelFormat;
    uint64_t        nFrameID;
    uint64_t        nTimestamp;
    int32_t        reserved[1];
}GX_FRAME_CALLBACK_PARAM;
```

名称	描述
pUserParam	用户私有数据指针
status	标示回调函数传回的图像状态，参考 GX_FRAME_STATUS
plmgBuf	图像数据地址（开启帧信息后，plmgBuf 包含图像数据和帧信息数据）
nImgSize	数据大小，单位字节（开启帧信息后，nImgSize 为图像数据大小+帧信息大小）
nWidth	图像的宽
nHeight	图像的高
nPixelFormat	图像的 PixFormat
nFrameID	图像的帧号
nTimestamp	图像的时间戳
reserved	4 字节，保留

7.3.3. GX_FRAME_DATA

相关接口：[GXGetImage\(\)](#)

```
typedef struct GX_FRAME_DATA
{
    GX\_FRAME\_STATUS    nStatus;
    void*              plmgBuf;
    int32_t            nWidth;
```



```

        int32_t          nHeight;
        int32_t          nPixelFormat;
        int32_t          nImgSize;
        uint64_t         nFrameID;
        uint64_t         nTimestamp;
        uint64_t*        pUserParam;
        int32_t          reserved [3];
    }GX_FRAME_DATA;

```

名称	描述
nStatus	标示获取的图像的状态，参考 GX_FRAME_STATUS
pImgBuf	图像数据指针（开启帧信息后，pImgBuf 包含图像数据和帧信息数据）
nWidth	图像宽度
nHeight	图像高度
nPixelFormat	图像格式
nImgSize	数据大小（开启帧信息后，nImgsize 为图像数据大小+帧信息大小）
nFrameID	图像的帧号
nTimestamp	图像的时间戳
nOffsetX	图像 X 方向偏移
nOffsetY	图像 Y 方向偏移
reserved	4 字节，保留

7.3.4. GX_FRAME_BUFFER

相关接口：[GXDQBuf\(\)](#)、[GXQBuf\(\)](#)、[GXDQAllBufs\(\)](#)

windows:

```

typedef struct GX_FRAME_BUFFER
{
    uint64_t          nFrameID;
    uint64_t          nTimestamp;
    uint64_t          nBufID;
    void_64          pImgBuf;
    GX\_FRAME\_STATUS  nStatus;
    int32_t           nWidth;
    int32_t           nHeight;
    int32_t           nPixelFormat;
    int32_t           nImgSize;
    int32_t           nOffsetX;
    int32_t           nOffsetY;
    int32_t           reserved[33];
}GX_FRAME_BUFFER;
typedef GX_FRAME_BUFFER* PGX_FRAME_BUFFER;

```

Linux :

```
typedef struct GX_FRAME_BUFFER
{
    GX\_FRAME\_STATUS      nStatus;
    void*                plmgBuf;
    int32_t              nWidth;
    int32_t              nHeight;
    int32_t              nPixelFormat;
    int32_t              nImgSize;
    uint64_t             nFrameID;
    uint64_t             nTimestamp;
    uint64_t             nBufID;
    int32_t              nOffsetX;
    int32_t              nOffsetY;
    int32_t              reserved[16];
}GX_FRAME_BUFFER;
typedef GX_FRAME_BUFFER* PGX_FRAME_BUFFER;
```

名称	描述
nStatus	标示获取的图像的状态，参考 GX_FRAME_STATUS
plmgBuf	图像数据指针（开启帧信息后，plmgBuf 包含图像数据和帧信息数据）
nWidth	图像宽度
nHeight	图像高度
nPixelFormat	图像格式
nImgSize	数据大小（开启帧信息后，nImgsize 为图像数据大小+帧信息大小）
nFrameID	图像的帧号
nTimestamp	图像的时间戳
nBufID	BufID
nOffsetX	图像 X 方向偏移
nOffsetY	图像 Y 方向偏移
reserved	64 字节，保留

7.3.5. GX_REGISTER_STACK_ENTRY

相关接口：[GXReadRemoteDevicePortStacked\(\)](#), [GXWriteRemoteDevicePortStacked\(\)](#)

描述了属性值对应的寄存器地址及值信息。

```
typedef struct GX_REGISTER_STACK_ENTRY
{
    uint64_t    nAddress;
    void*       pBuffer;
    size_t      nSize;
} GX_REGISTER_STACK_ENTRY;
```

名称	描述
nAddress	寄存器地址
pBuffer	寄存器内存指针
nSize	寄存器内存长度

7.3.6. GX_INTERFACE_INFO

相关接口：[GXGetInterfaceInfo\(\)](#)

描述了 INTERFACE 信息。

```
typedef struct GX_INTERFACE_INFO
{
    GX_TL_TYPE          emTLayerType;
    unsigned int         nReserved[4];
    union
    {
        GX_CXP_INTERFACE_INFO    stCXPIFInfo;
        GX_GEV_INTERFACE_INFO    stGEVIFInfo;
        GX_U3V_INTERFACE_INFO    stU3VIFInfo;
        GX_USB_INTERFACE_INFO     stUSBIFInfo;
        unsigned int             nReserved[64];
    }IFInfo;
}GX_INTERFACE_INFO;
```

名称	描述
emTLayerType	设备传输层协议类型
nReserved	保留字段
stCXPIFInfo	CXP 采集卡信息
stGEVIFInfo	GEV 采集卡信息
stU3VIFInfo	U3V 采集卡信息
stUSBIFInfo	USB 采集卡信息
nReserved	保留字段

7.3.6.1. GX_CXP_INTERFACE_INFO

```
typedef struct GX_CXP_INTERFACE_INFO
{
    unsigned char    chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned int     ui32InitFlag;
    unsigned int     nPCIEInfo;
    unsigned int     nReserved[64];
}GX_CXP_INTERFACE_INFO;
```

名称	描述
chInterfaceID	Interface 唯一标识符
chDisplayName	显示名称
chSerialNumber	序列号
ui32InitFlag	初始化状态
nPCIEInfo	采集卡的 PCIE 插槽信息
nReserved	预留

7.3.6.2. GX_GEV_INTERFACE_INFO

```
typedef struct GX_GEV_INTERFACE_INFO
{
    unsigned char    chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char             szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int     nReserved[64];
}GX_GEV_INTERFACE_INFO;
```

名称	描述
chInterfaceID	Interface 唯一标识符
chDisplayName	显示名称
chSerialNumber	序列号
szDescription	卡描述
nReserved	预留

7.3.6.3. GX_U3V_INTERFACE_INFO

```
typedef struct GX_U3V_INTERFACE_INFO
{
    unsigned char    chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char             szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int     nReserved[64];
}GX_U3V_INTERFACE_INFO;
```

名称	描述
chInterfaceID	Interface 唯一标识符
chDisplayName	显示名称
chSerialNumber	序列号

szDescription	卡描述
nReserved	预留

7.3.6.4. GX_USB_INTERFACE_INFO

```
typedef struct GX_USB_INTERFACE_INFO
{
    unsigned char    chInterfaceID[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDisplayName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    char             szDescription[GX_INFO_LENGTH_256_BYTE];
    unsigned int      nReserved[64];
}GX_USB_INTERFACE_INFO;
```

名称	描述
chInterfaceID	Interface 唯一标识符
chDisplayName	显示名称
chSerialNumber	序列号
szDescription	卡描述
nReserved	预留

7.3.7. GX_DEVICE_INFO

相关接口：[GXGetDeviceInfo\(\)](#)

描述了设备信息。

```
typedef struct GX_DEVICE_INFO
{
    GX_DEVICE_CLASS    emDevType;
    unsigned int        nReserved[4];
    {
        GX_CXP_DEVICE_INFO    stCXPDevInfo;
        GX_GEV_DEVICE_INFO    stGEVDevInfo;
        GX_U3V_DEVICE_INFO    stU3VDevInfo;
        GX_USB_DEVICE_INFO    stUSBDevInfo;
        unsigned int          nReserved[256];
    }DevInfo;
}GX_DEVICE_INFO;
```

名称	描述
emDevType	设备类型
nReserved	保留字段
stCXPDevInfo	CXP 设备信息
stGEVDevInfo	GEV 设备信息
stU3VDevInfo	U3V 设备信息

stUSBDevInfo	USB 设备信息
nReserved	保留字段

7.3.7.1. GX_CXP_DEVICE_INFO

```
typedef struct GX_CXP_DEVICE_INFO
{
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
}GX_CXP_DEVICE_INFO;
```

名称	描述
chVendorName	供应商名字
chModelName	型号名字
chManufacturerInfo	厂商信息
chDeviceVersion	设备版本
chSerialNumber	序列号
chUserDefinedName	用户自定义名字
nReserved	保留字段

7.3.7.2. GX_GEV_DEVICE_INFO

```
typedef struct GX_GEV_DEVICE_INFO
{
    unsigned int     nIpCfgOption;
    unsigned int     nIpCfgCurrent;
    unsigned int     nCurrentIp;
    unsigned int     nCurrentSubNetMask;
    unsigned int     nDefaultGateWay;
    unsigned int     nNetExport;
    uint64_t         nMacAddress;
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
}GX_GEV_DEVICE_INFO;
```

名称	描述
nIpCfgOption	支持的 IP 配置
nIpCfgCurrent	当前 IP 配置，参考支持的 IP 配置说明
nCurrentIp	当前 IP 地址
nCurrentSubNetMask	当前子网掩码
nDefaultGateWay	当前网关
nNetExport	网卡 IP 地址
nMacAddress	MAC 地址
chVendorName	供应商名字
chModelName	型号名字
chManufacturerInfo	厂商信息
chDeviceVersion	设备版本
chSerialNumber	序列号
chUserDefinedName	用户自定义名字
nReserved	保留字段

7.3.7.3. GX_U3V_DEVICE_INFO

```
typedef struct GX_U3V_DEVICE_INFO
{
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
}GX_U3V_DEVICE_INFO;
```

名称	描述
chVendorName	供应商名字
chModelName	型号名字
chManufacturerInfo	厂商信息
chDeviceVersion	设备版本
chSerialNumber	序列号
chUserDefinedName	用户自定义名字
nReserved	保留字段

7.3.7.4. GX_USB_DEVICE_INFO

```
typedef struct GX_USB_DEVICE_INFO
```

```
{
    unsigned char    chVendorName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chModelName[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chManufacturerInfo[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chDeviceVersion[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chSerialNumber[GX_INFO_LENGTH_64_BYTE];
    unsigned char    chUserDefinedName[GX_INFO_LENGTH_64_BYTE];
    unsigned int     nReserved[64];
}GX_USB_DEVICE_INFO;
```

名称	描述
chVendorName	供应商名字
chModelName	型号名字
chManufacturerInfo	厂商信息
chDeviceVersion	设备版本
chSerialNumber	序列号
chUserDefinedName	用户自定义名字
nReserved	保留字段

7.3.8. GX_INT_VALUE

相关接口：[GXGetIntValue\(\)](#)

描述了属性值信息。

```
typedef struct GX_INT_VALUE
{
    int64_t nCurValue;
    int64_t nMin;
    int64_t nMax;
    int64_t nInc;
    int32_t reserved[16];
}GX_INT_VALUE;
```

名称	描述
nCurValue	当前值
nMin	最小值
nMax	最大值
nInc	步长
reserved	保留

7.3.9. GX_FLOAT_VALUE

相关接口：[GXGetFloatValue\(\)](#)

描述了属性值信息。

```
typedef struct GX_FLOAT_VALUE
```



```
{
    double  dCurValue;
    double  dMin;
    double  dMax;
    double  dInc;
    bool     bIncIsValid;
    char     szUnit[GX_INFO_LENGTH_8_BYTE];
    int32_t reserved[16];
}GX_FLOAT_VALUE;
```

名称	描述
dCurValue	当前值
dMin	最小值
dMax	最大值
dInc	步长
szUnit	单位
reserved	保留

7.3.10. GX_STRING_VALUE

相关接口：[GXGetStringValue\(\)](#)

描述了属性值信息。

```
typedef struct GX_STRING_VALUE
{
    char     strCurValue[GX_INFO_LENGTH_256_BYTE];
    int64_t nMaxLength;
    int32_t reserved[4];
}GX_STRING_VALUE;
```

名称	描述
strCurValue	当前值
nMaxLength	最大长度
reserved	保留

7.3.11. GX_ENUM_VALUE

相关接口：[GXGetEnumValue\(\)](#)

描述了属性值信息。

```
typedef struct GX_ENUM_VALUE
{
    GX\_ENUM\_VALUE\_DES stCurValue;
    uint32_t             nSupportedNum;
```

```

GX_ENUM_VALUE_DES nArrySupportedValue[128];
int32_t reserved[16];
}GX_ENUM_VALUE;
    
```

名称	描述
stCurValue	当前值
nSupportedNum	支持的子项个数
nArrySupportedValue	支持的子项的值
reserved	保留

7.3.11.1. GX_ENUM_VALUE_DES

```

typedef struct GX_ENUM_VALUE_DES
{
    int64_t nCurValue;
    char strCurSymbolic[GX_INFO_LENGTH_128_BYTE];
    int32_t reserved[4];
}GX_ENUM_VALUE_DES;
    
```

名称	描述
nCurValue	当前值
strCurSymbolic	当前值的符号名称
reserved	保留

7.3.12. GX_GIGE_ACTION_COMMAND_RESULT

描述了 Action 命令返回的状态信息。

```

typedef struct GX_GIGE_ACTION_COMMAND_RESULT
{
    char strDeviceAddress[16];
    int32_t nStatus;
}GX_GIGE_ACTION_COMMAND_RESULT;
    
```

名称	描述
strDeviceAddress	当前返回 ack 的设备 IP (点分 10 进制的 IPv4)
nStatus	设备返回的 ACTION 状态，错误来源于 GigeVision 协议 0 : 标识命令发送成功。 0x8013 :设备未与主时钟进行时间同步。在执行该接口前，必须启用“ PtpEnable” , 并且确保 “PtpStatus” 为 “Master” 或者 “Slave” (表明已经与主时钟同步)。 0x8015 :设备队列或数据包数据已溢出。strDeviceAddress 对应的设备正在执行上一个 GXGigEIssueScheduledActionCommand 请求时，再次收到新的 GXGigEIssueScheduledActionCommand 请求，会返回此错误。 0x8016 : GXGigEIssueScheduledActionCommand 发出的 Scheduled Action Command 已过时。

7.4. 接口

7.4.1. 获取 SDK 版本信息

7.4.1.1. GXGetLibVersion (仅 Linux 支持)

声明：

GX_EXTC const char* GX_STDC GXGetLibVersion()

意义：

获取接口库版本号。

形参：

无。

返回值：

字符串类型的接口库版本号。

代码样例：

```
#include "GxI API.h"

int main(int argc, char* argv[])
{
    //获取接口库版本号
    const char* pszLibVersion = GXGetLibVersion();
    return 0;
}
```

7.4.2. 初始化及销毁

7.4.2.1. GXInitLib

声明：

GX_API GXInitLib()

意义：

初始化设备库，进行一些资源申请操作。使用 GxI API 与相机进行交互前必须调用此接口进行初始化操作，当停止 GxI API 对设备进行的所有控制之后必须调用 [GXCloseLib\(\)](#) 释放资源。

形参：无。

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误

GX_STATUS_NOT_FOUND_TL 找不到TL库

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

调用其他接口([GXCloseLib\(\)](#)、[GXGetLastError\(\)](#)除外)时必须先调用 [GXInitLib\(\)](#) 进行初始化操作，否则返回 GX_STATUS_NOT_INIT_API 错误。

代码样例：

```
#include "GxI API.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //在起始位置调用 GXInitLib() 进行初始化，申请资源
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //使用 GxI API
    //...

    //在结束的时候调用 GXCloseLib() 释放资源
    emStatus = GXCloseLib();

    return 0;
}
```

7.4.2.2. GXCloseLib

声明：

GX_API GXCloseLib()

意义：

关闭设备库，释放资源。当停止 GxI API 对设备进行的所有控制之后，必须调用此接口来释放资源，与 [GXInitLib\(\)](#) 对应。

形参：

无。

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
#include "GxI API.h"
int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //在起始位置调用 GXInitLib() 进行初始化，申请资源
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }
}
```

```

    }

    //使用 GxI API
    //...

    //在结束的时候调用 GXCloseLib() 释放资源
    emStatus = GXCloseLib();
    return 0;
}

```

7.4.2.3. GXSetLogType

声明：

GX_API GXSetLogType(const uint32_t ui32LogType)

意义：

设置可以生成的日志类型。

注意：

调用此接口之前必须先进行初始化。

形参：

[in] ui32LogType 日志类型，详情见 [GX_LOG_TYPE_LIST](#)

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_ACCESS	访问配置文件出错

上面没有涵盖到的，不常见的错误情况请参见[GX_STATUS_LIST](#)。

代码样例：

```

#include "GxI API.h"
int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;

    //在起始位置调用 GXInitLib() 进行初始化，申请资源
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //设置生成 ERROR、WARN、INFO 三种类型日志
    uint32_t ui32LogType = GX_LOG_TYPE_ERROR | GX_LOG_TYPE_WARN
                          | GX_LOG_TYPE_INFO;

    emStatus = GXSetLogType(ui32LogType);
    if (emStatus != GX_STATUS_SUCCESS)
    {

```

```

        //设置日志类型失败处理
    }

    //设置不生成任何日志
    uint32_t ui32LogType = GX_LOG_TYPE_OFF;
    emStatus = GXSetLogType(ui32LogType);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        //设置日志类型失败处理
    }

    //在结束的时候调用 GXCloseLib() 释放资源
    emStatus = GXCloseLib();
    return 0;
}

```

7.4.2.4. GXGetLogType

声明：

GX_API GXGetLogType(uint32_t* pui32Value)

意义：

获取生成的日志类型。

注意：

调用此接口之前必须先进行初始化。

形参：

[out] pui32Value 可以生成的日志类型，详情见 [GX_LOG_TYPE_LIST](#)

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_ACCESS	访问配置文件出错
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见[GX_STATUS_LIST](#)。

代码样例：

```

emStatus = GXInitLib();

uint32_t ui32LogType;
emStatus = GXGetLogType(&ui32LogType);
if (emStatus != GX_STATUS_SUCCESS)
{
    //获取可生成的日志类型失败
}

if (ui32LogType & GX_LOG_TYPE_OFF)
{

```

```

        //不生成任何日志
    }

    if (ui32LogType & GX_LOG_TYPE_ERROR)
    {
        //生成 ERROR 类型的日志
    }
    ...

    emStatus = GXCloseLib();

```

7.4.2.5. GXGetLastError

声明：

```

GX_API GXGetLastError (GX_STATUS *pErrorCode,
                        char *pszErrText,
                        size_t *pnSize)

```

意义：

获取程序最新的错误描述信息。

形参：

[out] pErrorCode	获取错误码, 如果不想获取此值, 那么此参数可以传 NULL
[out] pszErrText	返回错误信息缓冲区地址
[in,out] pnSize	错误信息缓冲区地址大小, 单位字节
如果pszErrText为NULL:	
[out] pnSize	返回实际需要的buffer大小
如果pszErrText非NULL:	
[in] pnSize	为实际分配的buffer大小
[out] pnSize	返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功, 没有发生错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小

上面没有涵盖到的, 不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_STATUS emErrCode = GX_STATUS_SUCCESS;
char* pszTemp = NULL;
size_t nSize = 0;

//传入 NULL 指针, 获取实际长度, 然后申请 buffer 再获取描述信息
GXGetLastError(&emErrCode, NULL, &nSize);
pszTemp = new char[nSize];
emStatus = GXGetLastError(&emErrCode, pszTemp, &nSize);

```

```
//操作完成后
If (NULL != pszTemp)
{
    delete[] pszTemp;
    pszTemp = NULL;
}
```

7.4.2.6. GXUpdateAllDeviceList

声明：

```
GX_API GXUpdateAllDeviceList (uint32_t* punNumDevices,
                               uint32_t nTimeOut)
```

意义：

全网枚举当前可用的所有设备，并且获取设备个数。

形参：

[out] <i>punNumDevices</i>	用来返回设备个数的地址指针，不能为 NULL 指针。
[in] <i>nTimeOut</i>	枚举的超时时间(单位ms)，如果在用户指定超时时间内成功枚举到设备，则立即返回；如果在用户指定超时时间内没有枚举到设备，则一直等待，直到达到用户指定的超时时间返回。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

//枚举设备个数；超时时间受用户使用环境限制，用户可自行设置，不局限于 1000ms
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000)
```

7.4.2.7. GXUpdateAllDeviceListEx

声明：

```
GX_API GXUpdateAllDeviceListEx (uint64_t nTLType, uint32_t* punNumDevices,
                                uint32_t nTimeOut)
```

意义：

全网枚举当前指定类型的所有设备，并且获取设备个数。

形参：

[in] <i>nTLType</i>	枚举特定类型的设备，参见 GX_TL_TYPE_LIST
[out] <i>punNumDevices</i>	用来返回设备个数的地址指针，不能为 NULL 指针

[in] *nTimeOut* 枚举的超时时间(单位ms)，如果在用户指定超时时间内成功枚举到设备，则立即返回；如果在用户指定超时时间内没有枚举到设备，则一直等待，直到达到用户指定的超时时间返回

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误
 GX_STATUS_NOT_INIT_API 没有调用GXInitLib()初始化库
 GX_STATUS_INVALID_PARAMETER 用户输入的指针为NULL
 上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;

//枚举千兆网设备个数,超时时间受用户使用环境限制,用户可自行设置,不局限于1000ms
emStatus = GXUpdateAllDeviceListEx(GX_TL_TYPE_GEV, &ui32DeviceNum,
                                    1000);
```

7.4.2.8. GXGetDeviceInfo

声明：

GX_API GXGetDeviceInfo(uint32_t nIndex, GX_DEVICE_INFO* pstDeviceInfo)

意义：

获取序号为 nIndex 设备的基础信息。

形参：

[in] *nIndex* 设备序号
 [out] *pstDeviceInfo* 设备信息结构体指针

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误
 GX_STATUS_NOT_INIT_API 没有调用GXInitLib()初始化库
 GX_STATUS_INVALID_PARAMETER 用户输入的index越界
 上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

此接口的作用是获取枚举到设备信息接口，调用之前需要调用[GXUpdateAllDeviceList\(\)](#)、[GXUpdateAllDeviceListEx\(\)](#)接口。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEVICE_INFO stInfo;
```

```

//获取第一台设备的信息
emStatus = GXGetDeviceInfo(1, &stInfo);
//获取千兆网设备的信息为例子
unsigned int nIp = stInfo.stGEVDevInfo.nCurrentIp;
uint64_t nMacAddress = stInfo.stGEVDevInfo.nMacAddress;

}

```

7.4.2.9. GXOpenDevice

声明：

**GX_API GXOpenDevice (GX_OPEN_PARAM* pOpenParam,
GX_DEV_HANDLE* phDevice)**

意义：

通过指定唯一标识打开设备，例如指定 SN、IP、MAC、Index 等。

形参：

[in] pOpenParam	用户配置的打开设备参数 参见 GX_OPEN_PARAM 结构体定义
[out] phDevice	接口返回的设备句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_NOT_FOUND_DEVICE	没有找到与指定信息匹配的设备
GX_STATUS_INVALID_ACCESS	设备无法在当前访问方式下打开

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

建议使用该函数前建议先调用 [GXUpdateAllDeviceList\(\)](#)进行一次枚举，以保证库内部设备列表与当前设备一致。

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_OPEN_PARAM stOpenParam;

//访问方式
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
//stOpenParam.accessMode = GX_ACCESS_READONLY;
//stOpenParam.accessMode = GX_ACCESS_CONTROL;

//通过序列号
stOpenParam.openMode = GX_OPEN_SN;
stOpenParam.pszContent = "EA00010002";

```

```
//通过 IP 地址
//stOpenParam.openMode = GX_OPEN_IP;
//stOpenParam.pszContent = "192.168.40.35";

//通过 MAC 地址
//stOpenParam.openMode = GX_OPEN_MAC;
//stOpenParam.pszContent = "54-04-A6-C2-7C-2F";

//通过枚举号 1、2、3...
//stOpenParam.openMode = GX_OPEN_INDEX;
//stOpenParam.pszContent = "1";

GX_DEV_HANDLE hDevice = NULL;
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
```

7.4.2.10. GXCLOSEDEVICE

声明：

GX_API GXCLOSEDEVICE (GX_DEV_HANDLE hDevice)

意义：

指定设备句柄关闭设备。

形参：

[in] hDevice	用户指定的要关闭的设备句柄
	hDevice 可通过 GXOpenDevice() 接口获取

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄，或重复关闭设备

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

关闭已经被关闭的设备句柄，返回 GX_STATUS_INVALID_HANDLE 错误。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DEV_HANDLE hDevice = NULL;
GX_OPEN_PARAM stOpenParam;
uint32_t ui32DeviceNum = 0;

//枚举设备个数，超时时间 1000ms [超时时间用户自行设置，没有推荐值]
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    //设置打开设备结构体参数
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
```

```

    stOpenParam.pszContent = "1";

    GX_DEV_HANDLE hDevice = NULL;
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //操作设备：控制、采集
        //...
        //关闭设备
        emStatus = GXCloseDevice(hDevice);
    }
}

```

7.4.2.11. GXGetInterfaceNum

声明：

GX_API GXGetInterfaceNum(uint32_t* punNumInterfaces);

意义：

获取 Interface 数量。

注意：

调用此接口之前必须先进行枚举

形参：

[in,out] punNumInterfaces 保存 Interface 个数指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32InterFaces;
emStatus = GXGetInterfaceNum(&ui32InterFaces);

```

7.4.2.12. GXGetInterfaceInfo

声明：

GX_API GXGetInterfaceInfo(uint32_t nIndex, GX_INTERFACE_INFO* pstInterfaceInfo);

意义：

获取指定索引的 Interface 的信息。

注意：

此接口的作用是获取枚举到的Interface信息，调用之前需要调用[GXUpdateAllDeviceList\(\)](#)、[GXUpdateAllDeviceListEx\(\)](#)接口。

形参：

[in] <i>nIndex</i>	Interface 次序，从开始 1
[in,out] <i>pstInterfaceInfo</i>	指向存储 Interface 信息结构体的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_INTERFACE_INFO stInfo;
emStatus = GXGetInterfaceInfo(1, &stInfo);
```

7.4.2.13. GXGetInterfaceHandle

声明：

GX_API GXGetInterfaceHandle(uint32_t nIndex, GX_IF_HANDLE* phIF);

意义：

获取指定索引的 Interface 句柄。

注意：

此接口的作用是获取枚举到的Interface句柄，调用之前需要调用[GXUpdateAllDeviceList\(\)](#)、[GXUpdateAllDeviceListEx\(\)](#)接口。

形参：

[in] <i>nIndex</i>	Interface 次序，从开始 1
[in,out] <i>phIF</i>	指向 Interface 句柄的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_IF_HANDLE hIF;
emStatus = GXGetInterfaceHandle (1, &hIF);
```

7.4.2.14. GXGetParentInterfaceFromDev

声明：

GX_API GXGetParentInterfaceFromDev(GX_DEV_HANDLE hDevice, GX_IF_HANDLE* phIF);

意义：

指定设备句柄获取所属的 Interface 句柄

形参：

[in] <i>hDevice</i>	设备句柄
[in,out] <i>phIF</i>	Interface 句柄

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_IF_HANDLE hIF = NULL;
emStatus = GXGetParentInterfaceFromDev (hDevice, &hIF);
```

7.4.2.15. GXGetLocalDeviceHandleFromDev

声明：

**GX_API GXGetLocalDeviceHandleFromDev(GX_DEV_HANDLE hDevice,
GX_LOCAL_DEV_HANDLE* phLocalDev);**

意义：

获取本地设备句柄。

形参：

[in] <i>hDevice</i>	设备句柄
[in,out] <i>phLocalDev</i>	指向本地设备句柄的指针

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_LOCAL_DEV_HANDLE hLocalDev = NULL;
emStatus = GXGetLocalDeviceHandleFromDev (hDevice, &hLocalDev);
```

7.4.2.16. GXGetDataStreamNumFromDev

声明：

GX_API GXGetDataStreamNumFromDev(GX_DEV_HANDLE hDevice, uint32_t* pnDSNum);

意义：

获取设备支持的流通道个数。

形参：

[in] <i>hDevice</i>	设备句柄
[in,out] <i>pnDSNum</i>	指向流通道个数的指针

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DSNum = 0;
emStatus = GXGetDataStreamNumFromDev (hDevice, &ui32DSNum);
```

7.4.2.17. GXGetDataStreamHandleFromDev

声明：

```
GX_API GXGetDataStreamHandleFromDev(GX_DEV_HANDLE hDevice,
                                     uint32_t nDSIndex, GX_DS_HANDLE* phDS);
```

意义：

获取指定设备句柄的流通道 ID 句柄。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>nDSIndex</i>	设备流序号，序号从 1 开始
[in,out] <i>phDS</i>	设备流句柄

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_DS_HANDLE hDS = 0;
emStatus = GXGetDataStreamHandleFromDev (hDevice, 1, &hDS);
```

7.4.3. 设备参数设置

7.4.3.1.通用读写接口

7.4.3.1.1. GXGetIntValue

声明：

```

GX_API GXGetIntValue (GX_PORT_HANDLE hPort,
                      const char* strName,
                      GX_INT_VALUE* pstIntValue)
    
```

意义：

获取整型数据节点信息。

形参：

[in] hPort	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] strName	节点名称
[out] pstIntValue	指向整型值信息结构体的指针
	参见 GX_INT_VALUE 结构体定义

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_VALUE stIntValue;
emStatus = GXGetIntValue(hDevice, "Width", &stIntValue);
int64_t i64Width = stIntValue.nCurValue;
    
```

7.4.3.1.2. GXSetIntValue

声明：

```

GX_API GXSetIntValue(GX_PORT_HANDLE hPort, const char* strName,
                    int64_t i64Value)
    
```

意义：

设置整型数据节点值。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)
[in] <i>strName</i>	节点名称
[in] <i>i64Value</i>	用户将要设置的值

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_OUT_OF_RANGE	用户传入值越界，比最小值小，或者比最大值大，或者不是步长整数倍
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 200;

emStatus = GXSetIntValue(hDevice, "Width", i64Value);
    
```

7.4.3.1.3. GXGetFloatValue

声明：

```

GX_API GXGetFloatValue(GX_PORT_HANDLE hPort,
                        const char* strName,
                        GX_FLOAT_VALUE* pstFloatValue);
    
```

意义：

获取浮点型数据节点信息。

形参：

[in] <i>hPort</i>	句柄(Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)
[in] <i>strName</i>	节点名称
[out] <i>pstFloatValue</i>	指向浮点型值信息结构体的指针 参见 GX_FLOAT_VALUE 结构体定义

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库

GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_VALUE stFloatValue;
emStatus = GXGetFloatValue(hDevice, "ExposureTime", &stFloatValue);
double dExposureTime = stFloatValue.dCurValue;
```

7.4.3.1.4. GXSetFloatValue

声明：

```
GX_API GXSetFloatValue (GX_PORT_HANDLE hPort,
                        const char* strName, double dValue);
```

意义：

设置浮点型数据节点值。

形参：

[in] hPort	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] strName	节点名称
[in] dValue	用户将要设置的值

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_OUT_OF_RANGE	用户传入值越界，比最小值小，或者比最大值大
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 3000;
emStatus = GXSetFloatValue(hDevice, "ExposureTime", dValue);
```

7.4.3.1.5. GXGetEnumValue

声明：

```
GX_API GXGetEnumValue(GX_PORT_HANDLE hPort, const char* strName,
                      GX_ENUM_VALUE* pstEnumValue);
```

意义：

获取枚举型数据节点信息。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] <i>strName</i>	节点名称
[out] <i>pstEnumValue</i>	指向枚举型值信息结构体的指针 参 GX_ENUM_VALUE 结构体定义

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_ENUM_VALUE stEnumValue;
emStatus = GXGetEnumValue(hDevice, "GainAuto", &stEnumValue);
int64_t i64GainAuto = stEnumValue.stCurValue.nCurValue;
```

7.4.3.1.6. GXSetEnumValue

声明：

```
GX_API GXSetEnumValue(GX_PORT_HANDLE hPort,
                      const char* strName, int64_t i64Value);
```

意义：

设置枚举型数据节点值。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] <i>strName</i>	节点名称

[in] <i>i64Value</i>	用户将要设置的值
----------------------	----------

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_OUT_OF_RANGE</code>	用户传入值越界
<code>GX_STATUS_INVALID_ACCESS</code>	不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = GX_GAIN_AUTO_CONTINUOUS;
emStatus = GXSetEnumValue(hDevice, "GainAuto", i64Value);
```

7.4.3.1.7. GXSetEnumValueByString

声明：

```
GX_API GXSetEnumValueByString(GX_PORT_HANDLE hPort,  
                                const char* strName,  
                                const char* strValue);
```

意义：

设置枚举型数据节点值。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
[in] <i>strName</i>	节点名称
[in] <i>strValue</i>	指向用户将要设置的值的指针

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_OUT_OF_RANGE</code>	用户传入值越界
<code>GX STATUS INVALID ACCESS</code>	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX STATUS LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSetEnumValueByString (hDevice,
                                     "GainAuto",
                                     "Continuous");
```

7.4.3.1.8. GXGetBoolValue

声明：

```
GX_API GXGetBoolValue (GX_PORT_HANDLE hPort,
                        const char* strName, bool* pbValue);
```

意义：

获取布尔型数据节点信息。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
[in] <i>strName</i>	节点名称
[out] <i>pbValue</i>	指向返回的布尔值的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = false;
emStatus = GXGetBoolValue (hDevice, "ReverseX", &bTemp);
```

7.4.3.1.9. GXSetBoolValue

声明：

```
GX_API GXSetBoolValue(GX_PORT_HANDLE hPort, const char* strName, bool bValue);
```

意义：

设置布尔型数据节点值。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
-------------------	---

[in] *strName*

节点名称

[in] *bValue*

用户将要设置的布尔值

返回值：

GX_STATUS_SUCCESS

操作成功，没有发生错误

GX_STATUS_NOT_INIT_API

没有调用GXInitLib()初始化库

GX_STATUS_INVALID_HANDLE

用户传入非法的句柄

GX_STATUS_NOT_IMPLEMENTED

当前不支持的功能

GX_STATUS_ERROR_TYPE

用户传入的featureID类型错误

GX_STATUS_INVALID_ACCESS

当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = true;
emStatus = GXSetBoolValue(hDevice, "ReverseX", bTemp);
    
```

7.4.3.1.10. GXGetStringValue

声明：

GX_API GXGetStringValue(GX_PORT_HANDLE hPort,

const char* strName,

GX_STRING_VALUE* pstStringValue);

意义：

获取字符串型数据节点信息。

形参：

[in] *hPort*

句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）

[in] *strName*

节点名称

[out] *pstStringValue*

指向字符串型值信息结构体的指针

参见 [GX_STRING_VALUE](#) 结构体定义

返回值：

GX_STATUS_SUCCESS

操作成功，没有发生错误

GX_STATUS_NOT_INIT_API

没有调用GXInitLib()初始化库

GX_STATUS_INVALID_HANDLE

用户传入非法的句柄

GX_STATUS_NOT_IMPLEMENTED

当前不支持的功能

GX_STATUS_ERROR_TYPE

用户传入的featureID类型错误

GX_STATUS_INVALID_PARAMETER

用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_STRING_VALUE stVendorNameValue;
const char* pszVendorName = NULL;
emStatus = GXGetStringValue(hDevice, "DeviceVendorName",
                            &stVendorNameValue);
pszVendorName = stVendorNameValue.strCurValue;
```

7.4.3.1.11. GXSetStringValue

声明：

```
GX_API GXSetStringValue(GX_PORT_HANDLE hPort, const char* strName,
                        const char* strValue);
```

意义：

设置字符串类型数据节点值。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
[in] <i>strName</i>	节点名称
[in] <i>strValue</i>	指向用户将要设置的值的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL
GX_STATUS_OUT_OF_RANGE	用户写入内容超过字符串最大长度
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char* pszText = "test";
emStatus = GXSetStringValue(hDevice, "DeviceModelName", pszText);
```

7.4.3.1.12. GXSetCommandValue

声明：

```
GX_API GXSetCommandValue(GX_PORT_HANDLE hPort, const char* strName);
```

意义：

发送命令。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
[in] <i>strName</i>	节点名称

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_ACCESS	当前不可访问，不能发送命令

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSetCommandValue(hDevice, "TriggerSoftware");
```

7.4.3.1.13. GXGetNodeAccessMode

声明：

```
GX_API GXGetNodeAccessMode(GX_PORT_HANDLE hPort, const char* strName,
GX_NODE_ACCESS_MODE* pemAccessMode);
```

意义：

获取数据节点的读写属性。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、 DataStream 句柄)
[in] <i>strName</i>	节点名称
[in,out] <i>pemAccessMode</i>	节点读写属性 参见 GX_NODE_ACCESS_MODE 结构体定义

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_NODE_ACCESS_MODE emAccessMode;
emStatus = GXGetNodeAccessMode(hDevice, "Width", &emAccessMode);
```


7.4.3.1.14. GXGetRegisterLength

声明：

```
GX_API GXGetRegisterLength(GX_PORT_HANDLE hPort, const char* strName,
                             size_t* pnSize);
```

意义：

获取寄存器型数据节点数据长度。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)
[in] <i>strName</i>	节点名称
[in,out] <i>pnSize</i>	保存数据长度的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nDataFieldValue;
emStatus = GXGetRegisterLength(hDevice, "DataFieldValue",
                                &nDataFieldValue);
```

7.4.3.1.15. GXGetRegisterValue

声明：

```
GX_API GXGetRegisterValue(GX_PORT_HANDLE hPort, const char* strName,
                             uint8_t* pBuffer, size_t* pnSize);
```

意义：

获取寄存器型数据节点值。

形参：

[in] <i>hPort</i>	句柄(Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)
[in] <i>strName</i>	节点名称
[out] <i>pBuffer</i>	指向用户申请的数据内存地址指针
[in,out] <i>pnSize</i>	表示用户输入的缓冲区地址的长度

如果*pBuffer*为NULL：

[out] <i>pnSize</i>	为实际需要的buffer大小
---------------------	----------------

如果*pBuffer*非NULL：

[in] <i>pnSize</i>	为用户分配的buffer大小
[out] <i>pnSize</i>	返回实际填充buffer大小

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)。

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nLength = 0;
emStatus = GXGetRegisterLength(hDevice, "DataFieldValue", &nLength);
uint8_t *pui8Buffer = new uint8_t[nLength];
emStatus = GXGetRegisterValue(hDevice, "DataFieldValue", pui8Buffer,
                              &nLength);

//操作完成后
if(NULL != pui8Buffer)
{
    delete[] pui8Buffer;
    pui8Buffer = NULL;
}
    
```

7.4.3.1.16. GXSetRegisterValue

声明：

GX_API GXSetRegisterValue(GX_PORT_HANDLE hPort, const char* strName, uint8_t* pBuffer, size_t nSize);

意义：

设置寄存器型数据节点值。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] <i>strName</i>	节点名称
[in] <i>pBuffer</i>	指向用户将要设置的数据内存地址指针
[in] <i>pnSize</i>	表示用户输入的缓冲区地址的长度

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

size_t nLength = 0;
emStatus = GXGetRegisterLength(hDevice, "DataFieldValue", &nLength);

uint8_t* pui8Buffer = new uint8_t[nLength];
emStatus = GXSetRegisterValue(hDevice, "DataFieldValue",
                               pui8Buffer, nLength);

//操作完成后
If (NULL != pui8Buffer)
{
    delete[] pui8Buffer;
    pui8Buffer = NULL;
}
```

7.4.3.2. 寄存器读写接口

7.4.3.2.1. GXReadPort

声明：

```
GX_API GXReadPort(GX_PORT_HANDLE hPort,
                  uint64_t ui64Address,
                  void* pBuffer, size_t* piSize);
```

意义：

通过寄存器地址读取内存。

注意：

千兆网/万兆网设备读取到的是大端，其他设备读取到的是小端。

形参：

[in] hPort	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] ui64Address	相机属性的寄存器地址
[in,out] pBuffer	指向读取到的内存值的指针地址
[in] piSize	待读取内存的长度

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
//相机属性值对应的地址
//以下地址为假地址仅供参考，具体地址可咨询技术支持
```

```
uint64_t ui64Address = 0xFFFF;

//申请存储相机属性值内存
size_t nSize = 4;
void* pBuffer = new int;

//批量获取相机属性值
emStatus = GXReadPort(hDevice, ui64Address, pBuffer, &nSize);

//操作完成后
if (NULL != pBuffer)
{
    delete pBuffer;
    pBuffer = NULL;
}
```

7.4.3.2.2. GXWritePort

声明：

```
GX_API GXWritePort(GX_PORT_HANDLE hPort, uint64_t ui64Address,
void* pBuffer, size_t* piSize);
```

意义：

通过寄存器地址写内存。

注意：

- 1) 千兆网相机设置的属性值需转换为大端后设置。
- 2) 调用当前接口后，使用 [GXGetEnumValue\(\)](#)、[GXGetIntValue\(\)](#)、[GXGetBoolValue\(\)](#)等接口获取到的节点值仍为修改前值，可使用 [GXReadPort\(\)](#)或 [GXReadPortStacked\(\)](#)接口获取最新的属性值。

形参：

[in] hPort	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] ui64Address	相机属性的寄存器地址
[in] pBuffer	指向写入的内存值的指针地址
[in] piSize	待读取内存的长度

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//相机属性值对应的地址
```

```
//以下地址为假地址仅供参考，具体地址可咨询技术支持
uint64_t ui64Address = 0xFFFF;

//申请存储相机属性值内存
int32_t i32Value = 1;
size_t nSize = sizeof(i32Value);

//设置相机属性值
emStatus = GXWritePort(hDevice, ui64Address, &i32Value, &nSize);

//操作完成后
if (NULL != pBuffer)
{
    delete[] pBuffer;
    pBuffer = NULL;
}
```

7.4.3.2.3. GXReadPortStacked

声明：

```
GX_API GXReadPortStacked(GX_PORT_HANDLE hPort,
                          GX_REGISTER_STACK_ENTRY* pstEntries,
                          size_t *piSize);
```

意义：

批量读取用户指定的多个内存地址。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in,out] <i>pstEntries</i>	用来指向需要获取相机属性的地址结构体内存，返回属性的值 参见 GX_REGISTER_STACK_ENTRY 结构体定义
[in,out] <i>piSize</i>	用来返回成功获取到相机属性值的个数

注意：

- 1) 批量读取的内存只支持4个字节为单位。
- 2) 千兆网/万兆网设备读取到的是大段，其他设备读取到的是小段。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
//以下事例为获取 3 个属性值（可根据自己的需求进行修改）
size_t nStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//属性地址（当前仅支持整型）
//以下地址为假地址仅供参考，具体地址可咨询技术支持
pstStackEntry[0].nAddress = 0xFFFF;

//属性值长度（当前仅支持整型）
pstStackEntry[0].nSize = 4;

//申请存储相机属性值内存
pstStackEntry[0].pBuffer = chBufferWidth;

pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX;

//批量获取多个相机属性值
emStatus = GXReadPortStacked(hDevice, pstStackEntry, &nStackEntry);
```

7.4.3.2.4. GXWritePortStacked

声明：

```
GX_API GXWritePortStacked(GX_PORT_HANDLE hPort,
                           const GX_REGISTER_STACK_ENTRY* pstEntries,
                           size_t *piSize);
```

意义：

批量写入用户指定的多个内存地址。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
[in] <i>pstEntries</i>	用来指向需要设置相机属性的地址结构体内存 参见 GX_REGISTER_STACK_ENTRY 结构体定义
[in,out] <i>piSize</i>	用来返回成功写入的相机属性个数

注意：

- 1) 千兆网相机设置的相机属性值需转换为大端后设置。

- 2) 调用当前接口后，使用 [GXGetEnumValue\(\)](#)、[GXGetIntValue\(\)](#)、[GXGetBoolValue\(\)](#)等接口获取到的节点值仍为修改前值，可使用 [GXReadPort\(\)](#)或 [GXReadPortStacked\(\)](#)接口获取最新的属性值。
- 3) 批量写入的内存只支持 4 个字节为单位。

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//以下事例为获取 3 个属性值（可根据自己的需求进行修改）
size_t nStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//属性地址（当前仅支持整型）
//以下地址为假地址仅供参考，具体地址可咨询技术支持
pstStackEntry[0].nAddress = 0xFFFF;

//相机属性值长度（当前仅支持整型）
pstStackEntry[0].nSize = 4;

//申请存储相机属性值内存
pstStackEntry[0].pBuffer = chBufferWidth;

//以下值可根据自己的需求进行修改
pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX;

//批量设置用户指定的多个相机属性值
emStatus = GXWritePortStacked(hDevice, pstStackEntry, &nStackEntry);
```

7.4.4. 设备功能接口

7.4.4.1. GXGetDevicePersistentIpAddress

声明：

```

GX_API GXGetDevicePersistentIpAddress(GX_DEV_HANDLE hDevice,
                                       char*      pszIP,
                                       size_t*    pnIPLength,
                                       char*      pszSubNetMask,
                                       size_t*    pnSubNetMaskLength,
                                       char*      pszDefaultGateWay,
                                       size_t*    pnDefaultGateWayLength);
    
```

意义：

获取设备的永久 IP 信息。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pszIP</i>	设备永久IP字符串地址
[in, out] <i>pnIPLength</i>	设备永久IP地址字符串长度,单位字节。
[in] <i>pnIPLength</i>	用户buffer大小
[out] <i>pnIPLength</i>	实际填充大小
[in] <i>pszSubNetMask</i>	设备永久子网掩码字符串地址
[in, out] <i>pnSubNetMaskLength</i>	设备永久子网掩码字符串长度
[in] <i>pnSubNetMaskLength</i>	用户buffer大小
[out] <i>pnSubNetMaskLength</i>	实际填充大小
[in] <i>pszDefaultGateWay</i>	设备永久网关字符串地址
[in, out] <i>pnDefaultGateWayLength</i>	设备永久网关字符串长度
[in] <i>pnDefaultGateWayLength</i>	用户buffer大小
[out] <i>pnDefaultGateWayLength</i>	实际填充大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chArrIP[32] = {0};
size_t nIPLen = 32;
char chArrSubNetMask[32] = {0};
size_t nSubNetMaskLen = 32;
char chArrDefaultGateWay[32] = {0};
    
```



```
size_t nDefaultGateWayLen = 32;
emStatus = GXGetDevicePersistentIpAddress(hDevice,
                                           chArrIP,
                                           &nIPLen,
                                           chArrSubNetMask,
                                           &nSubNetMaskLen,
                                           chArrDefaultGateWay,
                                           &nDefaultGateWayLen);
```

7.4.4.2. GXSetDevicePersistentIpAddress

声明：

```
GX_API GXSetDevicePersistentIpAddress(GX_DEV_HANDLE hDevice,
                                       const char* pszIP,
                                       const char* pszSubNetMask,
                                       const char* pszDefaultGateWay);
```

意义：

设置设备的永久 IP 信息。

形参：

[in] hDevice	设备句柄
[in] pszIP	设备永久IP字符串，末尾'\0'
[in] pszSubNetMask	设备永久子网掩码字符串，末尾'\0'
[in] pszDefaultGateWay	设备永久网关字符串，末尾'\0'

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//代码中的 IP 地址信息仅是例子，实际值由用户自己设定
emStatus = GXSetDevicePersistentIpAddress(hDevice,
                                           "192.168.1.2",
                                           "255.255.255.0",
                                           "192.168.1.1");
```

7.4.4.3. GXGigElpConfiguration

声明：

```
GX_API GXGigElpConfiguration(const char* pszDeviceMacAddress,
                              GX_IP_CONFIGURE_MODE emIpConfigMode,
                              const char* pszIpAddress,
                              const char* pszSubnetMask,
                              const char* pszDefaultGateway,
                              const char* pszUserID);
```

意义：

配置相机静态 IP 地址。

形参：

[in] <i>pszDeviceMacAddress</i>	设备 MAC 地址
[in] <i>emIpConfigMode</i>	IP 配置方式
[in] <i>pszIpAddress</i>	待设置的 IP 地址
[in] <i>pszSubnetMask</i>	待设置的子网掩码
[in] <i>pszDefaultGateway</i>	待设置的默认网关
[in] <i>pszUserID</i>	待设置的用户自定义名称

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_NOT_FOUND_DEVICE</code>	没有找到设备
<code>GX_STATUS_ERROR</code>	操作失败
<code>GX_STATUS_INVALID_ACCESS</code>	拒绝访问
<code>GX_STATUS_TIMEOUT</code>	操作超时

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";
char chIpAddress[] = "192.168.10.10";
char chSubnetMask[] = "255.255.255.0";
char chDefaultGateway[] = "192.168.10.2";
char chUserID[] = "Daheng Imaging";
GX_IP_CONFIGURE_MODE emIpConfigureMode = GX_IP_CONFIGURE_STATIC_IP;
emStatus = GXGigEIpConfiguration(chMAC,
                                emIpConfigureMode,
                                chIpAddress,
                                chSubnetMask,
                                chDefaultGateway,
                                chUserID);
```

7.4.4.4. GXGigEForceIp

声明：

```
GX_API GXGigEForceIp(const char* pszDeviceMacAddress,
                    const char* pszIpAddress,
                    const char* pszSubnetMask,
                    const char* pszDefaultGateway);
```

意义：

执行 Force IP 操作。

形参：

[in] <i>pszDeviceMacAddress</i>	设备 MAC 地址
[in] <i>pszIpAddress</i>	待设置的 IP 地址
[in] <i>pszSubnetMask</i>	待设置的子网掩码
[in] <i>pszDefaultGateway</i>	待设置的默认网关

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_NOT_FOUND_DEVICE</code>	没有找到设备
<code>GX_STATUS_ERROR</code>	操作失败
<code>GX_STATUS_INVALID_ACCESS</code>	拒绝访问
<code>GX_STATUS_TIMEOUT</code>	操作超时

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";
char chIpAddress[] = "192.168.10.10";
char chSubnetMask[] = "255.255.255.0";
char chDefaultGateway[] = "192.168.10.2";
emStatus = GXGigEForceIp(chMAC, chIpAddress, chSubnetMask,
chDefaultGateway);
```

7.4.4.5. GXGigEResetDevice

声明：

```
GX_API GXGigEResetDevice(const char* pszDeviceMacAddress,
GX_RESET_DEVICE_MODE ui32FeatureInfo);
```

意义：

执行设备重连或复位操作。详见 2.3 章节。

设备重连通常应用在调试千兆网相机时，设备已经被打开，此时程序异常，而后立即重新打开设备报错（因为调试心跳 5 分钟，设备依然处于打开状态），这时可通过设备重连功能，使设备处于未打开状态，而后再次打开设备即可成功。

设备复位通常应用在相机状态异常，此时设备重连功能也无法生效，也不具备给设备重新上电的条件，可尝试使用设备复位功能，使设备掉电再上电。设备复位后，需要重新执行枚举、打开设备操作。

注意：

- 1) 设备复位时间需要 1s 左右，因此需要确保 1s 后再调用枚举接口。
- 2) 如果设备正在正常采集，禁止使用设备复位和重连功能，否则会导致设备掉线

形参：

[in] <i>pszDeviceMacAddress</i>	设备 MAC 地址
[in] <i>ui32FeatureInfo</i>	重置设备模式

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_PARAMETER</code>	无效参数
<code>GX_STATUS_NOT_FOUND_DEVICE</code>	没有找到设备
<code>GX_STATUS_ERROR</code>	操作失败
<code>GX_STATUS_INVALID_ACCESS</code>	拒绝访问
<code>GX_STATUS_TIMEOUT</code>	操作超时

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

设备重连代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

char chMAC[] = "00-21-49-00-00-00";
//设备重连
emStatus = GXGigEResetDevice(chMAC,
                             GX_MANUFACTURER_SPECIFIC_RECONNECT);

//此时可重新打开设备
GX_DEV_HANDLE hDevice = NULL;
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
```

设备复位代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char chMAC[] = "00-21-49-00-00-00";
//设备复位
emStatus = GXGigEResetDevice(chMAC, GX_MANUFACTURER_SPECIFIC_RESET);

//此时需要等待 1s 后再直接重新枚举设备
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
```

7.4.4.6. GXFeatureSave

声明：

GX_API GXFeatureSave(GX_PORT_HANDLE hPort, const char* strFileName);

意义：

保存当前采集卡或相机配置参数到文本文件。

形参：

[in] <i>hPort</i>	句柄（Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄）
-------------------	--

[in] strFileName	待生成的配置文件的路径
------------------	-------------

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
-------------------	-------------

GX STATUS INVALID PARAMETER	无效参数
-----------------------------	------

GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
--------------------------	-----------

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const char* pszFilePath = "feature_save.txt";
emStatus = GXFeatureSave(hDevice, pszFilePath);
```

7.4.4.7. GXFeatureSaveW

声明：

GX_API GXFeatureSaveW(GX_PORT_HANDLE hPort, const wchar_t* strFileName);

意义：

保存当前采集卡或相机配置参数到文本文件。

形参：

[in] *hPort* 句柄 (Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)

[in] strFileName	待生成的配置文件的路径(宽字节)
------------------	------------------

返回值：

GX STATUS SUCCESS	操作成功，没有发生错误
-------------------	-------------

GX STATUS INVALID PARAMETER 无效参数

GX STATUS INVALID HANDLE	用户传入非法的句柄
--------------------------	-----------

上面没有涵盖到的，不常见的错误情况请参见 [GX STATUS LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const wchar_t* pszFilePath = L"feature_save.txt";
emStatus = GXFeatureSaveW(hDevice, pszFilePath);
```

7.4.4.8. GXFeatureLoad

声明：

GX_API GXFeatureLoad(GX_PORT_HANDLE hPort, const char* strFileName, bool bVerify);

意义：

配置文件中的参数加载到采集卡或相机。

形参：

[in] *hPort* 句柄 (Interface 句柄、远端设备句柄、本地设备句柄、
DataStream 句柄)

[in] <i>strFileName</i>	配置文件的路径
[in] <i>bVerify</i>	如果此值为 true，所有导入进去的值将会被读出进行校验是否一致

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const char* pszFilePath = "feature_save.txt";
emStatus = GXFeatureLoad(hDevice, pszFilePath, true);
```

7.4.4.9. GXFeatureLoadW

声明：

GX_API GXFeatureLoadW(GX_PORT_HANDLE hPort, const char* strFileName, bool bVerify);

意义：

配置文件中的参数加载到采集卡或相机。

形参：

[in] <i>hPort</i>	句柄 (Interface 句柄、远端设备句柄、本地设备句柄、DataStream 句柄)
[in] <i>strFileName</i>	配置文件的路径 (宽字节)
[in] <i>bVerify</i>	如果此值为 true，所有导入进去的值将会被读出进行校验是否一致

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
const wchar_t* pszFilePath = L"feature_save.txt";
emStatus = GXFeatureLoadW(hDevice, pszFilePath, true);
```

7.4.4.10. GXExportConfigFile

声明：

GX_API GXExportConfigFile (GX_DEV_HANDLE hDevice, const char * pszFilePath);

意义：

- 1) 导出相机当前配置到文本文件。

- 2) 配合 GalaxyView 的导入设备配置功能使用。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pszFilePath</i>	待生成的配置文件的路径

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXExportConfigFile(hDevice, "feature_save.txt");
```

7.4.4.11. GXImportConfigFile

声明：

```
GX_API GXImportConfigFile(GX_DEV_HANDLE hDevice,
                           const char * pszFilePath,
                           bool bVerify = false);
```

意义：

- 1) 将配置文件中的参数导入到相机。
- 2) 配合 GalaxyView 的导出设备配置功能使用。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pszFilePath</i>	配置文件的路径
[in] <i>bVerify</i>	如果此值为 true，所有导入进去的值将会被读出进行校验是否一致

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXImportConfigFile(hDevice, "feature_save.txt");
```

7.4.4.12. GXExportConfigFileW

声明：

```
GX_API GXExportConfigFileW (GX_DEV_HANDLE hDevice, const wchar_t * pszFilePath);
```

意义：

- 1) 导出相机当前配置到文本文件（UNICODE 接口）。
- 2) 配合 GalaxyView 的导入设备配置功能使用。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pszFilePath</i>	待生成的配置文件的路径

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXExportConfigFileW(hDevice, L"feature_save.txt");
```

7.4.4.13. GXImportConfigFileW

声明：

```
GX_API GXImportConfigFileW(GX_DEV_HANDLE hDevice,
                           const wchar_t * pszFilePath,
                           bool bVerify = false);
```

意义：

- 1) 将配置文件中的参数导入到相机（UNICODE 接口）。
- 2) 配合 Demo 的导出设备配置功能使用。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pszFilePath</i>	配置文件的路径
[in] <i>bVerify</i>	如果此值为 true，所有导入进去的值将会被读出进行校验是否一致

致

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXImportConfigFileW(hDevice, L"feature_save.txt");
```


7.4.4.14. GXGigElssueActionCommand

声明：

```
GX_API GXGigElssueActionCommand(uint32_t nDeviceKey, uint32_t nGroupKey,
                                  uint32_t nGroupMask,
                                  const char* strBroadcastAddress,
                                  const char* strSpecialAddress,
                                  uint32_t nTimeoutMs, uint32_t* pnNumResults,
                                  GX_GIGE_ACTION_COMMAND_RESULT* pstResults );
```

意义：

通过发送 ActionCommond 命令使网络上的相机同时执行 action 动作。

形参：

[in] <i>nDeviceKey</i>	设备密钥，对应设备的 “ActionDeviceKey” 属性
[in] <i>nGroupKey</i>	组密钥，对应设备的 “ActionGroupKey” 属性
[in] <i>nGroupMask</i>	组掩码，对应设备的 “ActionGroupMask” 属性，该值不能为 0
[in] <i>strBroadcastAddress</i>	发送 action 命令的目的 IP，可为广播、子网广播、单播
[in] <i>strSpecialAddress</i>	发送 action 命令的源 IP，用于明确的表明从哪个网络适配器上发送命令。如果不指定则所有网络适配器的每一个 IP 都会发送当前 action 命令
[in] <i>nTimeoutMs</i>	0 表示不需要等待 ack 返回；非 0 表明最大等待 ack 返回时间，时间到后有参数 pNumResults 返回实际收到的 ack 个数。
[in][out] <i>pNumResults</i>	表明期望返回的 ack 个数。此值表示为用户期望得到设备执行 action 命令反馈的预期 ack 个数。如果 nTimeoutMs 为 0，则忽略此参数。因此，如果 nTimeoutMs 为 0，则此参数可以为 NULL。
[in][out] <i>pResults</i>	一个包含*pNumResults 元素的数组，用于保存操作命令结果状态。缓冲区从头开始填充。如果收到的结果少于可用的结果项，则不会更改剩余结果。如果 nTimeoutMs 为 0，则忽略此参数。因此，如果 nTimeoutMs 为 0，则此参数可以为 NULL。 详见： GX_GIGE_ACTION_COMMAND_RESULT 。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

7.4.4.15. GXGigElssueScheduledActionCommand

声明：

```

GX_API GXGigElssueScheduledActionCommand (uint32_t nDeviceKey, uint32_t nGroupKey,
                                           uint32_t nGroupMask,
                                           uint64_t nActiontimeNs,
                                           const char* strBroadcastAddress,
                                           const char* strSpecialAddress,
                                           uint32_t nTimeoutMs,
                                           uint32_t* pnNumResults,
                                           GX_GIGE_ACTION_COMMAND_RESULT* pstResults);
    
```

意义：

通过发送 ActionCommond 命令在某个绝对时间点，使网络上的相机同时执行 action 动作。

形参：

[in] <i>nDeviceKey</i>	设备密钥，对应设备的 “ActionDeviceKey” 属性
[in] <i>nGroupKey</i>	组密钥，对应设备的 “ActionGroupKey” 属性
[in] <i>nGroupMask</i>	组掩码，对应设备的 “ActionGroupMask” 属性，该值不能为 0
[in] <i>nActiontimeNs</i>	执行操作的时间（以纳秒为单位）。实际值可以为所使用的主时钟值上预计的延时时间。主时钟值可以通过在从一组相机设备中锁存时间戳值 GXSetCommandValue(hDevice, "TimestampLatch") 后读取时间戳值 GXGetIntValue(hDevice, "TimestampLatchValue", &pstIntValue) 来获取一组同步相机设备的主时钟值。
[in] <i>strBroadcastAddress</i>	发送 action 命令的目的 IP，可为广播、子网广播、单播
[in] <i>strSpecialAddress</i>	发送 action 命令的源 IP，用于明确的表明从哪个网络适配器上发送命令。如果不指定则所有网络适配器的每一个 IP 都会发送当前 action 命令
[in] <i>nTimeoutMs</i>	0 表示不需要等待 ack 返回；非 0 表明最大等待 ack 返回时间，时间到后有参数 pNumResults 返回实际收到的 ack 个数。
[in][out] <i>pNumResults</i>	表明期望返回的 ack 个数。此值表示为用户期望得到设备执行 action 命令反馈的预期 ack 个数。如果 nTimeoutMs 为 0，则忽略此参数。因此，如果 nTimeoutMs 为 0，则此参数可以为 NULL。
[in][out] <i>pResults</i>	一个包含*pNumResults 元素的数组，用于保存操作命令结果状态。缓冲区从头开始填充。如果收到的结果少于可用的结果项，则不会更改剩余结果。如果 nTimeoutMs 为 0，则忽略此参数。因此，如果 nTimeoutMs 为 0，则此参数可以为 NULL。

详见：[GX_GIGE_ACTION_COMMAND_RESULT](#)。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

7.4.5. 设备采集接口

7.4.5.1. GXSetAcquisitionBufferNumber

声明：

```
GX_API GXSetAcquisitionBufferNumber(GX_DEV_HANDLE hDevice,
                                     uint64_t nBufferNum);
```

意义：

设置采集 buffer 个数。

形参：

[in] hDevice	设备句柄
[in] nBufferNum	用户设置的采集 buffer 个数

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户传入的输入参数无效
GX_STATUS_INVALID_CALL	发送开采命令后，不能设置采集buffer个数

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

- 1) 此接口是可选接口，不是开采流程的必须环节。
- 2) 发送开采命令后不允许设置采集 buffer 个数，若开采后设置 buffer 个数，则返回 GX_STATUS_INVALID_CALL。
- 3) 用户设置采集 buffer 个数必须大于 0，若设置的 buffer 个数为 0，则返回 GX_STATUS_INVALID_PARAMETER。
- 4) 用户设置采集 buffer 个数要注意合理性 若设置的 buffer 个数过多 则在采集时占用的内存较大；若设置的 buffer 个数过少，则在采集时虽然占用的内存较小，但会出现由于 buffer 不足导致丢帧现象。
- 5) 用户一旦设置过 buffer 个数 ,而且成功采集过 ,则此 buffer 个数值将一直有效 ,直到关闭设备为止。

代码样例：

```
#include "GxI API.h"

//图像回调处理函数
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame->status == GX_FRAME_STATUS_SUCCESS)
    {
        //对图像进行某些操作
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //枚举设备列表
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //打开设备
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //设置相机的流通道包长属性，提高网络相机的采集性能
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //判断设备是否支持流通道数据包功能
        GX_NODE_ACCESS_MODE emAccessMode;
        emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                        &emAccessMode);
    }
}
```

```

        bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                                ? true : false;

    if (bImplementPacketSize)
    {
        //获取当前网络环境的最优包长
        emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

        //将最优包长设置为当前设备的流通道包长值
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                ui32PacketSize);
    }

    //设置采集 buffer 个数
    emStatus = GXSetAcquisitionBufferNumber(hDevice, 10);

    //注册图像处理回调函数
    emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                        OnFrameCallbackFun);

    //开采
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");

    //-----
    //
    //在这个区间图像会通过 OnFrameCallbackFun 接口返给用户
    //
    //-----

    //停采
    emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

    //注销采集回调
    emStatus = GXUnregisterCaptureCallback(hDevice);
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();

return 0;
}

```

7.4.5.2. GXStreamOn (仅 Linux 支持)

声明：

GX_API GXStreamOn(GX_DEV_HANDLE hDevice);

意义：

开采，包括流开采和设备开采。

形参：

[in] <i>hDevice</i>	设备句柄
---------------------	------

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_ACCESS	设备访问模式错误
GX_STATUS_ERROR	不希望发生的未明确指明的内部错误

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

GXStreamOn 会开启所有流通道。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXStreamOn(hDevice);
```

7.4.5.3. GXStreamOff (仅 Linux 支持)

声明：

GX_API GXStreamOff(GX_DEV_HANDLE hDevice) ;

意义：

停采，包括流停采和设备停采。

形参：

[in] <i>hDevice</i>	设备句柄
---------------------	------

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_ACCESS	设备访问模式错误
GX_STATUS_INVALID_CALL	无效的接口调用
GX_STATUS_ERROR	不希望发生的未明确指明的内部错误

上面没有涵盖到的，不常见的错误情况请参见 [GX STATUS LIST](#)

注意事项：

GXStreamOff()会关闭所有流通道。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXStreamOff(hDevice);
```

7.4.5.4. GXDQBuf

声明：

```
GX_API GXDQBuf(GX_DEV_HANDLE hDevice,
                GX_FRAME_BUFFER* ppFrameBuffer,
                uint32_t nTimeOut);
```

意义：

零拷贝方式采集单帧，获取到图像后需要调用 [GXQBuf\(\)](#) 接口将图像归还给采集系统，。

形参：

[in] <i>hDevice</i>	设备句柄
[out] <i>ppFrameBuffer</i>	接口输出的图像数据的地址指针
[in] <i>nTimeOut</i>	取图的超时时间（单位 ms）

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL
GX_STATUS_INVALID_CALL	未开采或注册回调，不允许调用该接口
GX_STATUS_TIMEOUT	采图超时错误
GX_STATUS_ERROR	不希望发生的未明确指明的内部错误

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)。

注意事项：

- 1) 开采前，不允许调用 GXDQBuf()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 2) 注册采集回调后，不允许调用 GXDQBuf()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 3) GXDQBuf()接口需要与 GXQBuf()接口配合使用，否则无法持续采集图像。

代码样例：

```
//-----
//GXDQBuf 接口一次获取一帧图像，本样例代码演示的就是如何用此接口获取一帧图像
//-----
#include "GxI API.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
```

```

{
    return 0;
}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}

//打开设备
GX_OPEN_PARAM stOpenParam;
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //定义 GXDQBuf 的传入参数
    GX_FRAME_BUFFER pFrameBuffer;

    //开采
    #ifdef __linux__
    emStatus = GXStreamOn(hDevice);
    #else
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
    #endif
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //调用 GXDQBuf 取一帧图像
        emStatus = GXDQBuf(hDevice, &pFrameBuffer, 1000);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            if(pFrameBuffer-> nStatus==GX_FRAME_STATUS_SUCCESS)
            {
                //图像获取成功
                //对图像进行处理...
            }

            //调用 GXQBuf 将图像 buf 放回库中继续采图
            emStatus = GXQBuf(hDevice, pFrameBuffer);
        }
    }

    //停采
    #ifdef __linux__
    emStatus = GXStreamOff(hDevice);
    #else

```



```

        emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");
    #endif
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();
return 0;
}

```

7.4.5.5. GXQBuf

声明：

```

GX_API GXQBuf(GX_DEV_HANDLE hDevice,
              GX_FRAME_BUFFER pFrameBuffer);

```

意义：

将使用 GXDQBuf()接口获取的图像（零拷贝）归还给采集系统。

形参：

[in] *hDevice* 设备句柄

[in] *pFrameBuffer* 待放回 GxI API 库的图像数据 Buf 指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL/传入不合理指针
GX_STATUS_INVALID_CALL	未开采或注册回调，不允许调用该接口

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

- 1) 开采前，不允许调用 GXQBuf()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 2) 注册采集回调后，不允许调用 GXQBuf()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 3) GXDQBuf()接口需要与 GXQBuf()接口配合使用，否则无法持续采集图像；
- 4) GXQBuf()接口将图像 buf 放回 GxI API 库后，不能再访问该图像 buf 指针。

代码样例：

见 [GXDQBuf\(\)](#)示例程序。

7.4.5.6. GXDQAllBufs（仅 Linux 支持）

声明：

```

GX_API GXDQAllBufs(GX_DEV_HANDLE hDevice,
                  GX_FRAME_BUFFER *ppFrameBufferArray,
                  uint32_t nFrameBufferArraySize,
                  uint32_t *pnFrameCount,
                  uint32_t nTimeOut);

```

意义：

在开始采集之后，将使用 GXDQBuf()接口获取的所有图像的 buf (零拷贝) 归还给采集系统。获取到的图像数据数组中的存图顺序是从旧到新，即 ppFrameBufferArray[0] 存储的是最旧的图，ppFrameBufferArray[nFrameCount - 1]存储的是最新的图。

形参：

[in] hDevice	设备句柄
[out] ppFrameBufferArray	图像数据指针的数组
[in] nFrameBufferArraySize	图像数组申请个数
[out] pnFrameCount	返回实际填充图像个数
[in] nTimeOut	取图的超时时间（单位 ms）

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL
GX_STATUS_INVALID_CALL	未开采或注册回调，不允许调用该接口
GX_STATUS_NEED_MORE_BUFFER	用户申请的buffer不足：读操作时用户输入buffersize小于实际需要
GX_STATUS_TIMEOUT	采图超时错误
GX_STATUS_ERROR	不希望发生的未明确指明的内部错误

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

- 1) 开采前，不允许调用 GXDQAllBufs()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 2) 注册采集回调后，不允许调用 GXDQAllBufs()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 3) GXDQAllBufs()接口需要与 GXQAllBufs()接口配合使用，否则无法持续采集图像；
- 4) 图像数据指针的数组大小应大于等于图像采集 Buffer 个数（默认值为 5），否则会返回 GX_STATUS_NEED_MORE_BUFFER 错误。

代码样例：

```
#include "GxI API.h"
#include <stdint.h>

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    uint32_t ui32DeviceNum = 0;
```

```
//初始化库
emStatus = GXInitLib();
if (status != GX_STATUS_SUCCESS)
{
    return 0;
}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (emStatus <= 0))
{
    return 0;
}

//打开设备
GX_OPEN_PARAM stOpenParam;
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //定义接收图像数组
    //图像采集buffer个数默认为5, 图像数组大小应大于等于图像采集buffer个数
    GX_FRAME_BUFFER pFrameBuffer[5];

    //定义实际填充图像个数
    uint32_t ui32FrameCount = 0;

    //开采
    emStatus = GXStreamOn(hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //调用 GXDQAllBufs 获取队列中所有图像
        emStatus = GXDQAllBufs(hDevice, pFrameBuffer, 5, &nFrameCount,
                                1000);
        if (emStatus == GX_STATUS_SUCCESS)
        {
            for (int i = 0; i < ui32FrameCount; i++)
            {
                if (pFrameBuffer[i] != NULL &&
                    pFrameBuffer[i]-> nStatus == GX_FRAME_STATUS_SUCCESS)
                {
                    //第 i 幅图像获取成功
                    //对图像进行处理
                }
            }
            //调用 GXQAllBufs 将获取到的所有图像 buf 放回库中继续采图
        }
    }
}
```

```

        emStatus = GXQAllBufs(hDevice);
    }

    //停采
    emStatus = GXStreamOff(hDevice);
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();

return 0;
}

```

7.4.5.7. GXQAllBufs (仅 Linux 支持)

声明：

GX_API GXQAllBufs(GX_DEV_HANDLE hDevice)

意义：

在开始采集之后，通过此接口可以将获取到的所有图像数据 Buf 放回 GxI API 库，继续用于采集。

形参：

[in] <i>hDevice</i>	设备句柄
---------------------	------

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX STATUS INVALID CALL	未开采或注册回调，不允许调用该接口

注意事项：

- 1) 开采前，不允许调用 GXQAllBufs()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 2) 注册采集回调后，不允许调用 GXQAllBufs()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误；
- 3) GXQAllBufs()接口需要与 GXDQAllBufs()接口配合使用，否则无法持续采集图像；
- 4) GXQAllBufs()接口会将所有通过 GXDQAllBufs()获取到的图像 buf 放回 GxIAPI 库，此时不能再访问这些图像 buf 指针。

代码样例：

见 [GXDQAllBufs\(\)](#) 示例程序。

7.4.5.8. GXRegisterCaptureCallback

声明：

[illegible]

意义：

注册采集回调函数，与 [GXUnregisterCaptureCallback\(\)](#)接口对应。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>pUserParam</i>	指向用户将在回调处理函数中使用的私有数据指针，可通过 GX_FRAME_CALLBACK_PARAM.pUserParam 参数获取
[in] <i>callBackFun</i>	用户将要注册的回调处理函数，函数类型参见 GXCaptureCallBack()

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_INVALID_PARAMETER</code>	用户传入指针为NULL
<code>GX_STATUS_INVALID_CALL</code>	发送开采命令后，不能注册采集回调函数

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

发送开采命令后不允许注册回调函数，若开始采集后注册回调返回 `GX_STATUS_INVALID_CALL`。

代码样例：

```
#include "GxI API.h"

//图像回调处理函数
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame-> status == GX_FRAME_STATUS_SUCCESS)
    {
        //对图像进行某些操作
    }
    return;
}

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }
}
```

```

}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}

//打开设备
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //设置相机的流通道包长属性，提高网络相机的采集性能
    bool bImplementPacketSize = false;
    uint32_t ui32PacketSize = 0;

    //判断设备是否支持流通道数据包功能
    GX_NODE_ACCESS_MODE emAccessMode;
    emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                    &emAccessMode);
    bImplementPacketSize = (emAccessMode ==
                            GX_NODE_ACCESS_MODE_RW)? true : false;

    if (bImplementPacketSize)
    {
        //获取当前网络环境的最优包长
        emStatus = GXGetOptimalPacketSize(hDevice,
                                            &ui32PacketSize);

        //将最优包长设置为当前设备的流通道包长值
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                  ui32PacketSize);
    }

    //注册图像处理回调函数
    emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                         OnFrameCallbackFun);

    //发送开采命令
    emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
    //-----
    //
    //在这个区间图像会通过 OnFrameCallbackFun 接口返给用户
    //
    //-----
}

```

```

        //发送停采命令
        emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

        //注销采集回调
        emStatus = GXUnregisterCaptureCallback(hDevice);
    }
    emStatus = GXCloseDevice(hDevice);
    emStatus = GXCloseLib();

    return 0;
}

```

7.4.5.9. GXUnregisterCaptureCallback

声明：

GX_API GXUnregisterCaptureCallback(GX_DEV_HANDLE hDevice)

意义：

注销采集回调函数，与 [GXRegisterCaptureCallback\(\)](#)接口对应。

形参：

[in] hDevice 设备句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_CALL	发送停采命令之前，不能注销采集回调函数

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

停采之前不允许注销回调，若未停采而执行注销回调操作返回 GX_STATUS_INVALID_CALL。

代码样例：

```

#include "GxI API.h"
//图像回调处理函数
static void GX_STDC OnFrameCallbackFun(GX_FRAME_CALLBACK_PARAM* pFrame)
{
    if (pFrame-> status == GX_FRAME_STATUS_SUCCESS)
    {
        //对图像进行某些操作
    }
    return;
}

int main (int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
}

```

```

GX_OPEN_PARAM stOpenParam;
uint32_t ui32DeviceNum = 0;

//初始化库
emStatus = GXInitLib();
if (emStatus != GX_STATUS_SUCCESS)
{
    return 0;
}

//枚举设备列表
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}

//打开设备
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //设置相机的流通道包长属性，提高网络相机的采集性能
    bool bImplementPacketSize = false;
    uint32_t ui32PacketSize = 0;

    //判断设备是否支持流通道数据包功能
    GX_NODE_ACCESS_MODE emAccessMode;
    emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                    &emAccessMode);

    bImplementPacketSize = (emAccessMode ==
                            GX_NODE_ACCESS_MODE_RW)? true : false;

    if (bImplementPacketSize)
    {
        //获取当前网络环境的最优包长
        emStatus = GXGetOptimalPacketSize(hDevice, &ui32PacketSize);

        //将最优包长设置为当前设备的流通道包长值
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                   ui32PacketSize);
    }

    //注册图像处理回调函数
    emStatus = GXRegisterCaptureCallback(hDevice, NULL,
                                          OnFrameCallbackFun);
}

```



```

//发送开泵命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
//-----
//
//在这个区间图像会通过 OnFrameCallbackFun 接口返给用户
//
//-----
//发送停泵命令
emStatus = GXSetCommandValue(hDevice, "AcquisitionStop");

//注销采集回调
emStatus = GXUnregisterCaptureCallback(hDevice);
}
emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();

return 0;
}

```

7.4.5.10. GXGetImage

声明：

**GX_API GXGetImage(GX_DEV_HANDLE hDevice,
GX_FRAME_DATA *pFrameData, int32_t nTimeout);**

意义：

在开始采集之后，通过此接口可以直接获取图像，注意此接口不能与回调采集方式混用。

形参：

[in] hDevice	设备句柄
[in,out] pFrameData	指向用户传入的用来接收图像数据的地址指针
[in] nTimeout	取图的超时时间（单位 ms）

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_CALL	用户注册采集回调函数之后，又调用GXGetImage()接口取图
GX_STATUS_INVALID_PARAMETER	用户传入图像地址指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

注册采集回调后，不允许调用 GXGetImage()接口，若调用会返回 GX_STATUS_INVALID_CALL 错误。
当使用高分辨率相机进行高速采集的时候，因为 GXGetImage()接口内部有 buffer 拷贝会影响传输性能，建议用户在此种情况下使用回调采集方式。

代码样例：

```
//-----
//GXGetImage 接口一次获取一帧图像，本样例代码演示的就是如何用此接口获取一帧图像
//-----
#include "GxI API.h"

int main(int argc, char* argv[])
{
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;

    //初始化库
    emStatus = GXInitLib();
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return 0;
    }

    //枚举设备列表
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    //打开设备
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //设置相机的流通道包长属性，提高网络相机的采集性能
        bool bImplementPacketSize = false;
        uint32_t ui32PacketSize = 0;

        //判断设备是否支持流通道数据包功能
        GX_NODE_ACCESS_MODE emAccessMode;
        emStatus = GXGetNodeAccessMode(hDevice, "GevSCPSPacketSize",
                                         &emAccessMode);
        bImplementPacketSize = (emAccessMode == GX_NODE_ACCESS_MODE_RW)
                                ? true : false;

        if (bImplementPacketSize)
        {
            //获取当前网络环境的最优包长
            emStatus = GXGetOptimalPacketSize(hDevice, &ui32PacketSize);

            //将最优包长设置为当前设备的流通道包长值
        }
    }
}
```

```
        emStatus = GXSetIntValue(hDevice, "GevSCPSPacketSize",
                                ui32PacketSize);
    }

    GX_INT_VALUE stPayloadSize;
    //获取图像 buffer 大小, 下面动态申请内存
    emStatus = GXGetIntValue(hDevice, "PayloadSize", &stPayloadSize);

    if(emStatus==GX_STATUS_SUCCESS&&stPayloadSize.nCurValue>0)
    {
        //定义 GXGetImage 的传入参数
        GX_FRAME_DATA stFrameData;

        //根据获取的图像 buffer 大小 m_nPayloadSize 申请 buffer
        stFrameData.pImgBuf= malloc((size_t) stPayloadSize.nCurValue);

        //发送开始采集命令
        emStatus = GXSetCommandValue(hDevice, "AcquisitionStart");
        if (emStatus == GX_STATUS_SUCCESS)
        {
            //调用 GXGetImage 取一帧图像
            while(GXGetImage(hDevice, &stFrameData, 100) != GX_STATUS_SUCCESS)
            {
                Sleep(10);
            }
            if (stFrameData.nStatus == GX_FRAME_STATUS_SUCCESS)
            {
                //图像获取成功
                //对图像进行处理...
            }
        }
    }

    //发送停止采集命令
    emStatus = GXSetCommandValue (hDevice, "AcquisitionStop");

    //释放图像缓冲区 buffer
    free(stFrameData.pImgBuf);
}

emStatus = GXCloseDevice(hDevice);
emStatus = GXCloseLib();

return 0;
}
```

7.4.5.11. GXFlushQueue

声明：

GX_API GXFlushQueue(GX_DEV_HANDLE hDevice);

意义：

清空图像输出队列中的缓存图像。

形参：

[in] *hDevice* 设备句柄

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误
 GX_STATUS_NOT_INIT_API 没有调用GXInitLib()初始化库
 GX_STATUS_INVALID_HANDLE 用户传入非法的句柄
 上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

如果用户处理图像的速度较慢，库内会残留上次采集过程的缓存图像，特别在触发模式下，用户发送完触发之后，获取到的是旧图，如果用户想获取到当前触发对应的图像，需要在发送触发之前调用GXFlushQueue()接口，先清空图像输出队列。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXFlushQueue(hDevice);
```

7.4.5.12. GXGetOptimalPacketSize (仅 Windows 支持)

声明：

GX_API GXGetOptimalPacketSize (GX_DEV_HANDLE hDevice, uint32_t* punPacketSize);

意义：

获取当前网络环境的最优包长。

最优包长是指网络相机在当前网络环境中允许的流通道包长最大值。建议用户在打开网络相机后，通过该接口获取最优包长，并设置网络相机的流通道包长属性，以提高网络相机的采集性能。

形参：

[in] *hDevice* 设备句柄
 [out] *punPacketSize* 用来返回获取到的最优包长

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误
 GX_STATUS_INVALID_PARAMETER 无效参数
 GX_STATUS_INVALID_HANDLE 用户传入非法的句柄
 GX_STATUS_NOT_IMPLEMENTED 当前不支持的功能
 上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32PacketSize = 0;
```

```
//获取当前网络环境的最优包长
emStatus = GXGetOptimalPacketSize (hDevice, &ui32PacketSize);

//将最优包长设置为当前设备的流通道包长值

emStatus = GXSetIntValue (hDevice, "GevSCPSPacketSize", ui32PacketSize);
```

7.4.5.13. GXGetPayLoadSize

声明：

```
GX_API GXGetPayLoadSize(GX_DS_HANDLE hDStream, uint32_t* punPacketSize);
```

意义：

获取设备流 PayLoadSize。

形参：

[in] <i>hDStream</i>	设备流句柄
[in,out] <i>punPacketSize</i>	PayLoadSize 指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32PacketSize = 0;
emStatus = GXGetPayLoadSize (hDStream, &ui32PacketSize);
```

7.4.5.14. GXRegisterBuffer

声明：

```
GX_API GXRegisterBuffer(GX_DEV_HANDLE hDevice,
                        void* pBuffer,
                        size_t pBufferSize,
                        void* pUserParam);
```

意义：

注册用户申请的 buffer 用于图像采集，用户可自主管理 buffer 内存。

注意：

- 1) 用户申请的 buffer 大小不能自行计算，需要从 [GXGetPayLoadSize\(\)](#)接口获取，否则可能导致异常。POLs 采集卡等产品基于性能考虑要求采集使用的内存大小和首地址对齐，因此用户在使用该接口时需要按照对齐字节数属性 StreamBufferAlignment 的要求对首地址和大小对齐。
- 2) 注册 buffer 个数较少时，采集驱动完成了单个采集（buffer 处于消费者队列），驱动内无 buffer 可用可能导致丢帧。因此建议注册 buffer 数量至少是 3 个，并且尽快完成图像的使用。另外 POLs

等硬件要求最小 AnnounceBuffer 数量必须大于等于 3，如果用户 register 的 Buffer 数量不足，则会导致开采失败。

- 3) 调用 [GXRegisterBuffer\(\)](#)接口后，如果用户修改了相机或采集卡的参数，导致 PayloadSize 发生变化，需要重新注销掉已注册的 buffer，根据新长度重新申请内存并注册。
- 4) [GXRegisterBuffer\(\)](#)为可选接口，若不调用此接口注册 buffer，开采后 API 库会自动申请 buffer 并开采，不影响采集流程。

形参：

[in] hDevice	设备句柄
[in] pBuffer	用户申请的 buffer 指针
[in] nSize	用户申请的 buffer 长度
[in] pUserParam	用户自定义参数，设置后可从 GX_FRAME_DATA .pUserParam 参数获取

返回值：

GX_STATUS_SUCCESS 操作成功，没有发生错误

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//获取用户注册的 buffer 长度及地址对齐值
uint64_t u64BufferSize = 0;
size_t uBufAddrAlignment = 0;
emStatus = GXGetPayloadSize(hDStream, &u64BufferSize);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}

GX_INT_VALUE stIntValue;
emStatus = GXGetIntValue(hStreamFeatureControl,
                        "StreamBufferAlignment", &stIntValue);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}
uBufAddrAlignment = stIntValue.nCurValue;

//根据地址对齐值申请获取到的 buffer 长度
char* pBuffer = _aligned_malloc(u64BufferSize, uBufAddrAlignments);
if(NULL == pBuffer)
{
    //异常处理...
```

```

}

//用户自定义数据，若无赋空即可
void* pUserParam = NULL;

//注册 buffer
emStatus = GXRegisterBuffer(hDevice, pBuffer, u64BufferSize
                             pUserParam);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}

//开采...
//图像处理...
//停采...

//反注册 buffer...

//释放内存
if(NULL != pBuffer)
{
    _aligned_free(pBuffer);
    pBuffer = NULL;
}

```

7.4.5.15. GXUnRegisterBuffer

声明：

GX_API GXUnRegisterBuffer(GX_DEV_HANDLE hDevice, void* pBuffer)

意义：

注销用户使用 [GXRegisterBuffer\(\)](#) 接口注册的 buffer。

注意：此接口需要在停止采集后调用，开采过程中调用会影响正常采集流程。

形参：

[in] hDevice	设备句柄
[in] pBuffer	用户申请的 buffer 指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
-------------------	-------------

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;

//获取用户注册的 buffer 长度及地址对齐值
uint64_t u64BufferSize = 0;

```

```

size_t  uBufAddrAlignment = 0;
emStatus = GXGetPayloadSize(hDStream, &u64BufferSize);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}

GX_INT_VALUE stIntValue;
emStatus = GXGetIntValue(hStreamFeatureControl,
                        "StreamBufferAlignment", &stIntValue);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}
uBufAddrAlignment = stIntValue.nCurValue;

//根据地址对齐值申请获取到的 buffer 长度
char* pBuffer = _aligned_malloc(u64BufferSize, uBufAddrAlignment);
if(NULL == pBuffer)
{
    //异常处理...
}

//用户自定义数据，若无赋空即可
void* pUserParam = NULL;

//注册 buffer
emStatus = GXRegisterBuffer(hDevice, pBuffer, u64BufferSize ,
                        pUserParam);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}

//开采...
//图像处理...
//停采...

//反注册 buffer...
emStatus = GXUnRegisterBuffer(hDevice, pBuffer);
if(GX_STATUS_SUCCESS != emStatus)
{
    //异常处理...
}

//释放内存
if(NULL != pBuffer)

```



```
{
    _aligned_free(pBuffer);
    pBuffer = NULL;
}
```

7.4.6. 设备事件接口

7.4.6.1. GXRegisterDeviceOfflineCallback

声明：

```
GX_API GXRegisterDeviceOfflineCallback(GX_DEV_HANDLE hDevice,
                                        void* pUserParam,
                                        GXDeviceOfflineCallBack callBackFun,
                                        GX_EVENT_CALLBACK_HANDLE* pHCallback);
```

意义：

目前水星千兆网相机提供了设备掉线通知事件机制，用户可以通过此接口注册事件处理回调函数。

形参：

[in] hDevice	设备句柄
[in] pUserParam	用户私有参数
[in] callBackFun	用户事件处理回调函数，函数类型参见 GXDeviceOfflineCallBack()
[out] pHCallback	掉线回调函数句柄，此句柄用来注销回调函数使用。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	不支持的事件ID或者回调函数非法

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取掉线事件

```
#include "GxI API.h"

//掉线回调处理函数
static void GX_STDC OnDeviceOfflineCallbackFun(void* pUserParam)
{
    //收到掉线通知后，由用户主动发送通知主线程停止采集或者关闭设备等操作...
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
```

```
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //定义掉线回调函数句柄
    GX_EVENT_CALLBACK_HANDLE hCB;

    //注册设备掉线回调函数
    GXRegisterDeviceOfflineCallback(hDevice, NULL,
    OnDeviceOfflineCallbackFun, &hCB);
    //-----
    //
    //事件通知会通过 OnDeviceOfflineCallbackFun 接口返给用户
    //
    //-----
    //注销设备掉线事件回调
    GXUnregisterDeviceOfflineCallback(hDevice, hCB);
}
emStatus = GXCLOSEDevice(hDevice);
GXCloseLib();
return 0;
}
```

7.4.6.2. GXUnregisterDeviceOfflineCallback

声明：

```
GX_API GXUnregisterDeviceOfflineCallback(GX_DEV_HANDLE hDevice,
GX_EVENT_CALLBACK_HANDLE hCallBack);
```

意义：

注销事件处理回调函数。

形参：

[in] hDevice	设备句柄
[in] hCallBack	设备掉线回调函数句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取掉线事件

```
#include "GxI API.h"
//掉线回调处理函数
static void GX_STDC OnDeviceOfflineCallbackFun(void* pUserParam)
{
    //收到掉线通知后，由用户主动发送通知主线程停止采集或者关闭设备等操作...
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_DEV_HANDLE hDevice = NULL;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //定义掉线回调函数句柄
        GX_EVENT_CALLBACK_HANDLE hCB;

        //注册设备掉线回调函数
        GXRegisterDeviceOfflineCallback(hDevice, NULL,
                                         OnDeviceOfflineCallbackFun, &hCB);

        //-----
        //
        //事件通知会通过 OnDeviceOfflineCallbackFun 接口返给用户
        //
        //-----

        //注销设备掉线事件回调
        GXUnregisterDeviceOfflineCallback(hDevice, hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}
```

7.4.6.3. GXRegisterFeatureCallbackByString

声明：

```
GX_API GXRegisterFeatureCallbackByString(GX_PORT_HANDLE hPort,
                                          void* pUserParam,
                                          GXFeatureCallBackByString callBackFun,
                                          const char* strfeatureName,
                                          GX_FEATURE_CALLBACK_BY_STRING_HANDLE *pHCallBack);
```

意义：

注册属性更新回调函数。

形参：

[in] <i>hPort</i>	句柄
[in] <i>pUserParam</i>	用户私有参数
[in] <i>callBackFun</i>	用户事件处理回调函数，函数类型参见 GXFeatureCallBackByString()
[in] <i>strfeatureName</i>	节点名称
[out] <i>pHCallBack</i>	属性更新回调函数句柄，用来注销回调函数

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	不支持的ID或者回调函数非法

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取远端设备事件

```
#include "GxI API.h"
GX_DEV_HANDLE hDevice = NULL;
//远端设备事件回调处理函数
static void GX_STDC OnFeatureCallbackFun(const char* strFeatureName,
                                          void* pUserParam)
{
    if (strFeatureName == "EventExposureEnd")
    {
        //获取此帧曝光结束事件对应的时间戳和帧 ID 等事件数据
        GX_INT_VALUE stIntValue;
        GXGetIntValue(hDevice, "EventExposureEndFrameID", &stIntValue);
    }
    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
```

```

emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
{
    return 0;
}
stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
stOpenParam.openMode = GX_OPEN_INDEX;
stOpenParam.pszContent = "1";
emStatus = GXOpenDevice(&stOpenParam, &hDevice);
if (emStatus == GX_STATUS_SUCCESS)
{
    //选择曝光结束事件
    GXSetEnumValueByString(hDevice, "EventSelector",
                           "ExposureEnd");

    //开启曝光结束事件
    GXSetEnumValueByString(hDevice, "EventNotification", "On");

    //定义属性更新回调函数句柄
    GX_FEATURE_CALLBACK_HANDLE hCB;

    //注册曝光结束事件回调函数
    GXRegisterFeatureCallbackByString(hDevice,
                                      NULL,
                                      OnFeatureCallbackFun,
                                      "EventExposureEnd",
                                      &hCB);

    //-----
    //
    //事件通知会通过 OnFeatureCallbackFun 接口返给用户
    //
    //-----

    //注销曝光结束事件回调
    GXUnregisterFeatureCallbackByString(hDevice, "EventExposureEnd",
                                       hCB);
}
emStatus = GXCloseDevice(hDevice);
GXCloseLib();
return 0;
}

```

7.4.6.4. GXUnregisterFeatureCallbackByString

声明：

```

GX_API GXUnregisterFeatureCallbackByString(GX_PORT_HANDLE hPort,
                                           const char* strfeatureName,
                                           GX_FEATURE_CALLBACK_BY_STRING_HANDLE hCallback);

```

意义：

注销设备属性更新回调函数。

注意：

与 GXRegisterFeatureCallback() 配套使用，每次注册都必须有相应的注销与之对应。

形参：

[in] <i>hPort</i>	句柄
[in] <i>strfeatureName</i>	节点名称
[in] <i>hCallBack</i>	属性更新回调函数句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用 GXInitLib() 初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	不支持的事件ID

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取远端设备事件

```
#include "GxI API.h"
GX_DEV_HANDLE hDevice = NULL;
//远端设备事件回调处理函数
static void GX_STDC OnFeatureCallbackFun(const char* strFeatureName,
                                          void* pUserParam)
{
    if (strFeatureName == "EventExposureEnd")
    {
        //获取此帧曝光结束事件对应的时间戳和帧 ID 等事件数据
        GX_INT_VALUE stIntValue;
        GXGetIntValue(hDevice, "EventExposureEndFrameID", &stIntValue);
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
```

```

//选择曝光结束事件
GXSetEnumValueByString(hDevice, "EventSelector",
                        "ExposureEnd");

//开启曝光结束事件
GXSetEnumValueByString(hDevice, "EventNotification", "On");

//定义属性更新回调函数句柄
GX_FEATURE_CALLBACK_HANDLE hCB;

//注册曝光结束事件回调函数
GXRegisterFeatureCallbackByString(hDevice, NULL,
                                   OnFeatureCallbackFun,
                                   "EventExposureEnd",
                                   &hCB);

//-----
//
//事件通知会通过 OnFeatureCallbackFun 接口返给用户
//
//-----
//注销曝光结束事件回调
GXUnregisterFeatureCallbackByString(hDevice, "EventExposureEnd",
                                    hCB);
}
emStatus = GXCloseDevice(hDevice);
GXCloseLib();
return 0;
}

```

7.4.6.5. GXFlushEvent

声明：

GX_API GXFlushEvent (GX_DEV_HANDLE hDevice);

意义：

清空设备事件，比如帧曝光结束事件数据队列。

形参：

[in] hDevice 设备句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项：

库内部事件数据的接收和处理采用缓存机制，如果用户接收、处理事件的速度慢于事件产生的速度，事件数据就会在库内积累，会影响用户获取实时事件数据。如果用户想获取实时事件数据，需要先调用 GXFlushEvent() 接口清空事件缓存数据。此接口一次性清空所有事件数据。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXFlushEvent(hDevice);
```

7.4.6.6. GXGetEventNumInQueue

声明：

GX_API GXGetEventNumInQueue (GX_DEV_HANDLE hDevice, uint32_t *pnEventNum);

意义：

获取当前远端设备事件队列缓存里面的事件个数。

形参：

[in] hDevice	设备句柄
[in] pnEventNum	事件个数指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用 GXInitLib() 初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户输入的指针为 NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EventNum = 0;
emStatus = GXGetEventNumInQueue(hDevice, &ui32EventNum);
```

7.5. 不推荐使用定义

7.5.1. 类型

7.5.2. 常量

7.5.2.1. GX_ACCESS_STATUS

```
typedef enum GX_ACCESS_STATUS
{
    GX_ACCESS_STATUS_UNKNOWN    = 0,
    GX_ACCESS_STATUS_READWRITE  = 1,
    GX_ACCESS_STATUS_READONLY   = 2,
    GX_ACCESS_STATUS_NOACCESS   = 3,
}GX_ACCESS_STATUS;
```



```
typedef int32_t GX_ACCESS_STATUS_CMD;
```

名称	描述
GX_ACCESS_STATUS_UNKNOWN	设备当前状态未知
GX_ACCESS_STATUS_READWRITE	设备当前可读可写
GX_ACCESS_STATUS_READONLY	设备当前仅支持读
GX_ACCESS_STATUS_NOACCESS	设备当前既不支持读，又不支持写

7.5.2.2. GX_FEATURE_LEVEL

```
typedef enum GX_FEATURE_LEVEL
{
    GX_FEATURE_LEVEL_REMOTE_DEV    =0x00000000,
    GX_FEATURE_LEVEL_TL            =0x01000000,
    GX_FEATURE_LEVEL_IF            =0x02000000,
    GX_FEATURE_LEVEL_DEV           =0x03000000,
    GX_FEATURE_LEVEL_DS            =0x04000000,
}GX_FEATURE_LEVEL;
```

名称	描述
GX_FEATURE_LEVEL_REMOTE_DEV	Remote Device 层
GX_FEATURE_LEVEL_TL	System 层
GX_FEATURE_LEVEL_IF	Interface 层
GX_FEATURE_LEVEL_DEV	Device 层
GX_FEATURE_LEVEL_DS	DataStream 层

7.5.2.3. GX_FEATURE_TYPE

```
typedef enum GX_FEATURE_TYPE
{
    GX_FEATURE_INT        =0x10000000,
    GX_FEATURE_FLOAT      =0x20000000,
    GX_FEATURE_ENUM       =0x30000000,
    GX_FEATURE_BOOL       =0x40000000,
    GX_FEATURE_STRING     =0x50000000,
    GX_FEATURE_BUFFER     =0x60000000,
    GX_FEATURE_COMMAND    =0x70000000,
}GX_FEATURE_TYPE;
```

名称	描述
GX_FEATURE_INT	整型
GX_FEATURE_FLOAT	浮点型
GX_FEATURE_ENUM	枚举型
GX_FEATURE_BOOL	布尔型

GX_FEATURE_STRING	字符串型
GX_FEATURE_BUFFER	块儿数据型
GX_FEATURE_COMMAND	命令型

7.5.3. 结构体

7.5.3.1. GX_INT_RANGE

相关接口：[GXGetIntRange\(\)](#)

描述了整型值的最大值、最小值、步长。

```
typedef struct GX_INT_RANGE
{
    int64_t nMin;
    int64_t nMax;
    int64_t nInc;
    int32_t reserved[8];
}GX_INT_RANGE;
```

名称	描述
nMin	最小值
nMax	最大值
nInc	步长
reserved	32 字节，保留

7.5.3.2. GX_FLOAT_RANGE

相关接口：[GXGetFloatRange\(\)](#)

描述了浮点型值的最大值、最小值、步长、单位。

```
typedef struct GX_FLOAT_RANGE
{
    double  dMin;
    double  dMax;
    double  dInc;
    char     szUnit[GX_INFO_LENGTH_8_BYTE];
    bool     bInclsValid;
    int8_t   reserved[31];
}GX_FLOAT_RANGE;
```

名称	描述
dMin	最小值
dMax	最大值
dInc	步长
szUnit	8 字节，单位

bInclsValid	1 字节，步长是否支持
reserved	31 字节，保留

7.5.3.3. GX_ENUM_DESCRIPTION

相关接口：[GXGetEnumDescription\(\)](#)

描述了枚举类型的所有枚举项的值与描述信息。

```
typedef struct GX_ENUM_DESCRIPTION
{
    int64_t nValue;
    char    szSymbolic[GX_INFO_LENGTH_64_BYTE];
    int32_t reserved[8];
}GX_ENUM_DESCRIPTION;
```

名称	描述
nValue	枚举项的值
szSymbolic	64 字节，枚举项的字符描述信息
reserved	32 字节，保留

7.5.3.4. GX_DEVICE_IP_INFO

相关接口：[GXGetDeviceIPInfo\(\)](#)

此结构体表示了网络相机才有的一些网络相关的属性描述。

```
typedef struct GX_DEVICE_IP_INFO
{
    char szDeviceID[GX_INFO_LENGTH_64_BYTE + 4];
    char szMAC[GX_INFO_LENGTH_32_BYTE];
    char szIP[GX_INFO_LENGTH_32_BYTE];
    char szSubNetMask[GX_INFO_LENGTH_32_BYTE];
    char szGateWay[GX_INFO_LENGTH_32_BYTE];
    char szNICMAC[GX_INFO_LENGTH_32_BYTE];
    char szNICIP[GX_INFO_LENGTH_32_BYTE];
    char szNICSubNetMask[GX_INFO_LENGTH_32_BYTE];
    char szNICGateWay[GX_INFO_LENGTH_32_BYTE];
    charszNICDescription[GX_INFO_LENGTH_128_BYTE + 4];
    char reserved[512];
}GX_DEVICE_IP_INFO;
```

名称	描述
szDeviceID	64+4 字节，设备唯一标识
szMAC	32 字节，MAC 地址
szIP	32 字节，IP 地址
szSubNetMask	32 字节，子网掩码
szGateWay	32 字节，网关

szNICMAC	32 字节，对应网卡的 MAC 地址
szNICIP	32 字节，对应网卡的 IP 地址
szNICSubNetMask	32 字节，对应网卡的子网掩码
szNICGateWay	32 字节，对应网卡的网关
szNICDescription	128+4 字节，对应网卡的描述
reserved	512 字节，保留

7.5.3.5. GX_DEVICE_BASE_INFO

相关接口：[GXGetAllDeviceBaseInfo\(\)](#)

此结构体代表设备的基础信息，无论 USB 相机还是网络相机都共有的一些属性描述。

```
typedef struct GX_DEVICE_BASE_INFO
{
    char szVendorName[GX_INFO_LENGTH_32_BYTE];
    char szModelName[GX_INFO_LENGTH_32_BYTE];
    char szSN[GX_INFO_LENGTH_32_BYTE];
    char szDisplayName[GX_INFO_LENGTH_128_BYTE + 4];
    char szDeviceID[GX_INFO_LENGTH_64_BYTE + 4];
    char szUserID[GX_INFO_LENGTH_64_BYTE + 4];
    GX\_ACCESS\_STATUS\_CMD accessStatus;
    GX\_DEVICE\_CLASS deviceClass;
    char reserved[300];
}GX_DEVICE_BASE_INFO;
```

名称	描述
szVendorName	32 字节，厂商名称
szModelName	32 字节，设备类型名称
szSN	32 字节，设备序列号
szDisplayName	128+4 字节，设备展示名称
szUserID	64+4 字节，用户自定义名称
szDeviceID	64+4 字节，设备唯一标识
accessStatus	4 字节，设备当前支持的访问状态，参见 GX_ACCESS_STATUS 的定义
deviceClass	4 字节，设备种类，比如 USB2.0、GEV
reserved	300 字节，保留

7.5.4. 接口

7.5.4.1. GXReadRemoteDevicePortStacked (仅 Windows 支持)

声明：

```
GX_API GXReadRemoteDevicePortStacked(GX_DEV_HANDLE hDevice,
                                        GX_REGISTER_STACK_ENTRY* pstEntries,
                                        size_t *piSize);
```

意义：

批量读取用户指定的多个相机属性值，当前仅支持整型（4 字节）。

注意：

获取到千兆网相机属性值为大端数据。

形参：

[in] <i>hDevice</i>	设备句柄
[in,out] <i>pstEntries</i>	用来指向需要获取相机属性的地址结构体内存，返回属性的值
[in,out] <i>piSize</i>	用来返回成功获取到相机属性值的个数

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//以下事例为获取 3 个属性值（可根据自己的需求进行修改）
size_t sStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//属性地址（当前仅支持整型）
//以下地址为假地址仅供参考，具体地址可咨询技术支持
pstStackEntry[0].nAddress = 0xFFFF;

//属性值长度（当前仅支持整型）
pstStackEntry[0].nSize = 4;

//申请存储相机属性值内存
pstStackEntry[0].pBuffer = chBufferWidth ;

pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight ;
```

```
pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX ;

//批量获取多个相机属性值
emStatus = GXReadRemoteDevicePortStacked(hDevice, pstStackEntry,
&sStackEntry);
```

7.5.4.2. GXWriteRemoteDevicePortStacked (仅 Windows 支持)

声明：

```
GX_API GXWriteRemoteDevicePortStacked(GX_DEV_HANDLE hDevice,
const GX_REGISTER_STACK_ENTRY* pstEntries,
size_t *piSize);
```

意义：

批量设置用户指定的多个相机属性值，当前仅支持整型（4 字节）。

注意：

- 1) 千兆网相机设置的相机属性值需转换为大端后设置。
- 2) 调用当前接口后，使用 GXGetEnum()、GXGetInt()、GXGetBool()等接口获取到的节点值仍为修改前值，可使用 GXReadRemoteDevicePort()或 GXReadRemoteDevicePortStacked()接口获取最新的属性值。

形参：

[in] hDevice	设备句柄
[in] pstEntries	用来指向需要设置相机属性的地址结构体内存
[in,out] piSize	用来返回成功写入的相机属性个数

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//以下事例为获取 3 个 U3 相机属性值（可根据自己的需求进行修改）
size_t sStackEntry = 3;
GX_REGISTER_STACK_ENTRY pstStackEntry[3];
char chBufferWidth[256] = {0};
char chBufferHeight[256] = {0};
char chBufferOffsetX[256] = {0};

//属性地址（当前仅支持整型）
//以下地址为假地址仅供参考，具体地址可咨询技术支持
```

```

pstStackEntry[0].nAddress = 0xFFFF;

//相机属性值长度（当前仅支持整型）
pstStackEntry[0].nSize = 4;

//申请存储相机属性值内存
pstStackEntry[0].pBuffer = chBufferWidth ;

//以下值可根据自己的需求进行修改
pstStackEntry[1].nAddress = 0xFFFF;
pstStackEntry[1].nSize = 4;
pstStackEntry[1].pBuffer = chBufferHeight;

pstStackEntry[2].nAddress = 0xFFFF;
pstStackEntry[2].nSize = 4;
pstStackEntry[2].pBuffer = chBufferOffsetX ;

//批量设置用户指定的多个相机属性值
emStatus = GXWriteRemoteDevicePortStacked(hDevice, pstStackEntry,
                                             &sStackEntry);

```

7.5.4.3. GXReadRemoteDevicePort

声明：

**GX_API GXReadRemoteDevicePort (GX_DEV_HANDLE hDevice, uint64_t ui64Address,
void *pBuffer, size_t *piSize)**

意义：

读取用户指定寄存器的值。

注意：

获取的千兆网相机属性值为大端数据。

形参：

[in] hDevice	设备句柄
[in] ui64Address	相机属性的寄存器地址
[out] pBuffer	相机属性值的内存，不能为 NULL
[in,out] piSize	相机属性值的内存大小，不能为 NULL

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
//相机属性值对应的地址
//以下地址为假地址仅供参考，具体地址可咨询技术支持
uint64_t ui64Address = 0xFFFF;

//申请存储相机属性值内存
size_t nSize = 4;
void *pBuffer = new int;

//批量获取相机属性值（U3 相机）
emStatus = GXReadRemoteDevicePort(hDevice, ui64Address,
                                   pBuffer, &nSize);

//操作完成后
if(NULL != pBuffer)
{
    delete pBuffer;
    pBuffer = NULL;
}
```

7.5.4.4. GXWriteRemoteDevicePort

声明：

GX_API GXWriteRemoteDevicePort(GX_DEV_HANDLE hDevice, uint64_t ui64Address, const void *pBuffer, size_t *piSize);

意义：

设置用户指定寄存器的值。

注意：

- 1) 千兆网相机设置的属性值需转换为大端后设置。
- 2) 调用当前接口后，使用 GXGetEnum()、GXGetInt()、GXGetBool()等接口获取到的节点值仍为修改前值，可使用 GXReadRemoteDevicePort()或 GXReadRemoteDevicePortStacked()接口获取最新的属性值。

形参：

[in] hDevice	设备句柄
[in] ui64Address	相机属性的寄存器地址
[in] pBuffer	相机属性值的内存，不能为 NULL
[in, out] piSize	相机属性值的内存大小，不能为 NULL

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_INVALID_PARAMETER	无效参数
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//相机属性值对应的地址
//以下地址为假地址仅供参考，具体地址可咨询技术支持
uint64_t ui64Address = 0xFFFF;

//申请存储相机属性值内存
int32_t i32Value = 1;
size_t nSize = sizeof(i32Value);

//设置相机属性值
emStatus = GXWriteRemoteDevicePort(hDevice, ui64Address,
                                     &i32Value, &nSize);

//操作完成后
if (NULL != pBuffer)
{
    delete pBuffer;
    pBuffer = NULL;
}
```

7.5.4.5. GXUnregisterFeatureCallback

声明：

```
GX_API GXUnregisterFeatureCallback(GX_DEV_HANDLE hDevice,
                                    GX_FEATURE_ID_CMD featureID,
                                    GX_FEATURE_CALLBACK_HANDLE hCallBack);
```

意义：

注销设备属性更新回调函数。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码
[in] hCallBack	属性更新回调函数句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	不支持的事件ID

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取远端设备事件

```
#include "GxI API.h"
GX_DEV_HANDLE hDevice = NULL;
```

```
//远端设备事件回调处理函数
static void GX_STDC OnFeatureCallbackFun(GX_FEATURE_ID_CMD featureID,
                                         void* pUserParam)
{
    if (featureID == GX_INT_EVENT_EXPOSUREEND)
    {
        //获取此帧曝光结束事件对应的时间戳和帧 ID 等事件数据
        int64_t i64FrameID=0;
        GXGetInt(hDevice,GX_INT_EVENT_EXPOSUREEND_FRAMEID, &i64FrameID);
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }

    GX_OPEN_PARAM stOpenParam;
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //选择曝光结束事件
        GXSetEnum(hDevice,GX_ENUM_EVENT_SELECTOR,
                  GX_ENUM_EVENT_SELECTOR_EXPOSUREEND);

        //开启曝光结束事件
        GXSetEnum(hDevice,GX_ENUM_EVENT_NOTIFICATION,
                  GX_ENUM_EVENT_NOTIFICATION_ON);

        //定义属性更新回调函数句柄
        GX_FEATURE_CALLBACK_HANDLE hCB;

        //注册曝光结束事件回调函数
        GXRegisterFeatureCallback(hDevice,
                                  NULL,
                                  OnFeatureCallbackFun,
                                  GX_INT_EVENT_EXPOSUREEND,
                                  &hCB);
    }
}
```

```

//-----
//
//事件通知会通过 OnFeatureCallbackFun 接口返给用户
//
//-----

//注销曝光结束事件回调
GXUnregisterFeatureCallback(hDevice, GX_INT_EVENT_EXPOSUREEND, hCB);
}
emStatus = GXCloseDevice(hDevice);
GXCloseLib();
return 0;
}

```

7.5.4.6. GXRegisterFeatureCallback

声明：

```

GX_API GXRegisterFeatureCallback(GX_DEV_HANDLE hDevice,
                                void* pUserParam,
                                GXFeatureCallbackcallBackFun,
                                GX_FEATURE_ID_CMD featureID,
                                GX_FEATURE_CALLBACK_HANDLE *pHCallback);

```

意义：

注册设备属性更新回调函数，当设备属性发生当前值的更新，或者可访问属性等变化的时候，激发调用此回调函数。

形参：

[in] hDevice	设备句柄
[in] pUserParam	用户私有参数
[in] callBackFun	用户事件处理回调函数，函数类型参见 GXFeatureCallback()
[in] featureID	设备功能码
[out] pHCallback	属性更新回调函数句柄，用来注销回调函数

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	不支持的ID或者回调函数非法

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：获取远端设备事件

```

#include "GxI API.h"
GX_DEV_HANDLE hDevice = NULL;

//远端设备事件回调处理函数
static void GX_STDC OnFeatureCallbackFun(GX_FEATURE_ID_CMD featureID,

```

```

                                void* pUserParam)
{
    if (featureID == GX_INT_EVENT_EXPOSUREEND)
    {
        //获取此帧曝光结束事件对应的时间戳和帧 ID 等事件数据
        int64_t i64FrameID=0;
        GXGetInt(hDevice, GX_INT_EVENT_EXPOSUREEND_FRAMEID, &i64FrameID);
    }

    return;
}

int main(int argc, char* argv[])
{
    GXInitLib();
    GX_STATUS emStatus = GX_STATUS_SUCCESS;
    GX_OPEN_PARAM stOpenParam;
    uint32_t ui32DeviceNum = 0;
    emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
    if ((emStatus != GX_STATUS_SUCCESS) || (ui32DeviceNum <= 0))
    {
        return 0;
    }
    stOpenParam.accessMode = GX_ACCESS_EXCLUSIVE;
    stOpenParam.openMode = GX_OPEN_INDEX;
    stOpenParam.pszContent = "1";
    emStatus = GXOpenDevice(&stOpenParam, &hDevice);
    if (emStatus == GX_STATUS_SUCCESS)
    {
        //选择曝光结束事件
        GXSetEnum(hDevice, GX_ENUM_EVENT_SELECTOR,
                  GX_ENUM_EVENT_SELECTOR_EXPOSUREEND);

        //开启曝光结束事件
        GXSetEnum(hDevice, GX_ENUM_EVENT_NOTIFICATION,
                  GX_ENUM_EVENT_NOTIFICATION_ON);

        //定义属性更新回调函数句柄
        GX_FEATURE_CALLBACK_HANDLE hCB;

        //注册曝光结束事件回调函数
        GXRegisterFeatureCallback(hDevice,
                                  NULL,
                                  OnFeatureCallbackFun,
                                  GX_INT_EVENT_EXPOSUREEND,
                                  &hCB);

        //-----
        //
        //事件通知会通过 OnFeatureCallbackFun 接口返给用户
        //
        //-----
    }
}

```

```

        //注销曝光结束事件回调
        GXUnregisterFeatureCallback(hDevice, GX_INT_EVENT_EXPOSUREEND, hCB);
    }
    emStatus = GXCloseDevice(hDevice);
    GXCloseLib();
    return 0;
}

```

7.5.4.7. GXSendCommand

声明：

```

GX_API GXSendCommand(GX_DEV_HANDLE hDevice,
                     GX_FEATURE_ID featureID);

```

意义：

发送命令码。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_ACCESS	当前不可访问，不能发送命令

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
emStatus = GXSendCommand(hDevice, GX_COMMAND_TRIGGER_SOFTWARE);

```

7.5.4.8. GXGetBufferLength

声明：

```

GX_API GXGetBufferLength(GX_DEV_HANDLE hDevice,
                         GX_FEATURE_ID featureID,
                         size_t* pnSize);

```

意义：

获取 Buffer 类型值的数据长度 ,单位字节 ,然后用户再根据获取的长度申请内存 ,再调用 GXGetBuffer() 来获取 Buffer 数据。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID

[out] *pnSize* 指向返回的长度值的指针，长度单位字节

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nLength = 0;
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_FRAME_INFORMATION,
                             &nLength);
```

7.5.4.9. GXGetBuffer

声明：

```
GX_API GXGetBuffer (GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    uint8_t* pBuffer,
                    size_t* pnSize);
```

意义：

获取 Buffer 数据。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pBuffer</i>	指向用户申请的 Buffer 数据内存地址指针
[in,out] <i>pnSize</i>	表示用户输入的缓冲区地址的长度

如果*pBuffer*为NULL：

[out] <i>pnSize</i>	为实际需要的buffer大小
---------------------	----------------

如果*pBuffer*非NULL：

[in] <i>pnSize</i>	为用户分配的buffer大小
[out] <i>pnSize</i>	返回实际填充buffer大小

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄

GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小
上面没有涵盖到的，不常见的错误情况请参见 GX_STATUS_LIST	

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
//以读取查找表为例
//读取查找表长度
size_t nLutLength = 0;
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);

//申请 buffer
uint8_t *pui8GetLutBuffer = new uint8_t[nLutLength];

//读取查找表内容
emStatus = GXGetBuffer(hDevice, GX_BUFFER_LUT_VALUEALL,
                       pui8GetLutBuffer, &nLutLength);

//操作完成后
If (NULL != pui8GetLutBuffer)
{
    delete[] pui8GetLutBuffer;
    pui8GetLutBuffer = NULL;
}

```

7.5.4.10. GXSetBuffer

声明：

```

GX_API GXSetBuffer(GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   uint8_t* pBuffer,
                   size_t nSize);

```

意义：

设置 Buffer 数据。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[in] <i>pBuffer</i>	指向用户将要设置的 Buffer 数据内存地址指针
[in] <i>nSize</i>	表示用户输入的缓冲区地址的长度

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL
GX_STATUS_OUT_OF_RANGE	用户写入内容超过字符串最大长度
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
//以设置查找表为例
//读取查找表长度
size_t nLutLength = 0;
emStatus = GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);

//申请 buffer
uint8_t *pui8SetLutBuffer = new uint8_t[nLutLength];
//设置查找表内容（比如可以从本地读取一个查找表文件）
//此处不举例如何设置查找表 buffer 内容，请用户安装自己需要设置查找表内容

//设置查找表
emStatus = GXSetBuffer(hDevice, GX_BUFFER_LUT_VALUEALL,
                       pui8SetLutBuffer, nLutLength);

//操作完成后
If (NULL != pui8SetLutBuffer)
{
    delete[] pui8SetLutBuffer;
    pui8SetLutBuffer = NULL;
}

```

7.5.4.11. GXGetStringMaxLength

声明：

```

GX_API GXGetStringMaxLength(GX_DEV_HANDLE hDevice,
                             GX_FEATURE_ID featureID,
                             size_t* pnSize);

```

意义：

获取字符串类型值的最大长度，单位字节，用户根据获取的长度信息再申请内存，然后调用GXGetString()获取字符串信息。此接口获取的是字符串的最大可能长度（包含结束符'\0'），但是字符串的

实际长度可能没有这么长，如果用户想按照字符串的实际长度申请 buffer，请调用 GXGetStringLength()接口获取字符串实际长度。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pnSize</i>	指向返回的长度值的指针，长度值带末位'\0'，长度单位字节

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
emStatus = GXGetStringMaxLength(hDevice,
                                GX_STRING_DEVICE_VENDOR_NAME, &nSize);

```

7.5.4.12. GXGetString

声明：

```

GX_API GXGetString(GX_DEV_HANDLE hDevice,
                   GX_FEATURE_ID featureID,
                   char* pszContent,
                   size_t* pnSize)

```

意义：

获取字符串类型值的内容。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	要操作的功能码
[out] <i>pszContent</i>	指向用户申请的字符串缓存地址
[in,out] <i>pnSize</i>	表示用户输入的字符串缓冲区地址的长度
如果 <i>pszContent</i> 为NULL：	
[out] <i>pnSize</i>	为实际需要的buffer大小
如果 <i>pszContent</i> 非NULL：	
[in] <i>pnSize</i>	为用户分配的buffer大小

[out] pnSize

返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小

上面没有涵盖到的，不常见的错误情况请参见GX_STATUS_LIST

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
//首先获取字符串长度
emStatus = GXGetStringLength(hDevice, GX_STRING_DEVICE_VENDOR_NAME,
                             &nSize);

//根据获取的长度申请内存
char *pszText = new char[nSize];

emStatus = GXGetString(hDevice, GX_STRING_DEVICE_VENDOR_NAME,
                       pszText, &nSize);

//操作完成后
if(NULL != pszText)
{
    delete[] pszText;
    pszText = NULL;
}
    
```

7.5.4.13. GXSetString

声明：

GX_API GXSetString(GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID,
char* pszContent)

意义：

设置字符串值内容。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID

[in] *pszContent* 指向用户将要设置的字符串地址，字符串末尾结束符'\0'

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户传入指针为NULL
GX_STATUS_OUT_OF_RANGE	用户写入内容超过字符串最大长度
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
char* pszText = "test";
emStatus = GXSetString(hDevice, GX_STRING_USER_PASSWORD, pszText);
```

7.5.4.14. GXGetStringLength

声明：

**GX_API GXGetStringLength(GX_DEV_HANDLE hDevice,
GX_FEATURE_ID featureID, size_t* pnSize)**

意义：

获取字符串类型值的当前值长度，单位字节，用户根据获取的长度信息再申请内存，然后调用GXGetString()获取字符串信息。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pnSize</i>	指向返回的长度值的指针，长度值带末位'\0'，长度单位字节

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
size_t nSize = 0;
```

```
emStatus = GXGetStringLength(hDevice, GX_STRING_DEVICE_VENDOR_NAME,
                              &nSize);
```

7.5.4.15. GXSetBool

声明：

```
GX_API GXSetBool(GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID,
                  bool bValue)
```

意义：

设置布尔类型值。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID
[in] bValue	用户将要设置的布尔值

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = true;
emStatus = GXSetBool(hDevice, GX_BOOL_REVERSE_X, bTemp);
```

7.5.4.16. GXGetBool

声明：

```
GX_API GXGetBool(GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID, bool* pbValue)
```

意义：

获取布尔类型值。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID
[out] pbValue	指向返回的布尔值的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
-------------------	-------------

GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bTemp = false;
emStatus = GXGetBool(hDevice, GX_BOOL_REVERSE_X, &bTemp);
```

7.5.4.17. GXSetEnum

声明：

```
GX_API GXSetEnum (GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID,
                  int64_t nValue)
```

意义：

设置枚举值。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID
[in] nValue	用户将要设置的枚举值，值的范围可通过 GX_ENUM_DESCRIPTION 中的 nValue 获取。

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_OUT_OF_RANGE	用户传入值越界
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = GX_GAIN_AUTO_CONTINUOUS;
emStatus = GXSetEnum(hDevice, GX_ENUM_GAIN_AUTO, i64Value);
```

7.5.4.18. GXGetEnumEntryNums

声明：

```
GX_API GXGetEnumEntryNums(GX_DEV_HANDLE hDevice,
                             GX_FEATURE_ID featureID,
                             uint32_t* pnEntryNums)
```

意义：

获取枚举项的可选项的个数。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	要操作的功能码
[out] <i>pnEntryNums</i>	指向返回的个数的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EntryNums = 0;
emStatus = GXGetEnumEntryNums(hDevice, GX_ENUM_GAIN_AUTO,
                               &ui32EntryNums);
```

7.5.4.19. GXGetEnumDescription

声明：

```
GX_API GXGetEnumDescription(GX_DEV_HANDLE hDevice,
                             GX_FEATURE_ID featureID,
                             GX_ENUM_DESCRIPTION* pEnumDescription,
                             size_t* pnBufferSize)
```

意义：

获取枚举类型值的描述信息：枚举项的个数及每一项的值及字符串描述。参见 [GX_ENUM_DESCRIPTION](#) 结构体。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pEnumDescription</i>	数组指针，用来盛放返回的枚举描述信息

[in,out] *pnBufferSize* 用户传入的GX_ENUM_DESCRIPTION数组的大小，大为字节

如果*pEnumDescription*为NULL：

[out] *pnBufferSize* 为实际需要的buffer大小

如果*pEnumDescription*非NULL：

[in] *pnBufferSize* 为用户分配的buffer大小

[out] *pnBufferSize* 返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32EntryNums = 0;

//首先获取有多少项
emStatus = GXGetEnumEntryNums(hDevice, GX_ENUM_GAIN_AUTO,
                               &ui32EntryNums);

//根据获取的项的个数来申请内存
GX_ENUM_DESCRIPTION* pstEnumDescription =
    new GX_ENUM_DESCRIPTION[ui32EntryNums];

size_t nSize = ui32EntryNums * sizeof(GX_ENUM_DESCRIPTION);
emStatus = GXGetEnumDescription(hDevice, GX_ENUM_GAIN_AUTO,
                                pstEnumDescription, &nSize);

//操作完成后
If (NULL != pstEnumDescription)
{
    delete[] pstEnumDescription;
    pstEnumDescription = NULL;
}
```

7.5.4.20. GXGetEnum

声明：

```
GX_API GXGetEnum(GX_DEV_HANDLE hDevice,  
                  GX_FEATURE_ID featureID,  
                  int64_t* pnValue)
```

意义：

获取当前枚举值。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pnValue</i>	指向返回结果的指针

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL
<code>GX_STATUS_INVALID_ACCESS</code>	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;  
int64_t i64Value = 0;  
emStatus = GXGetEnum(hDevice, GX_ENUM_GAIN_AUTO, &i64Value);
```

7.5.4.21. GXSetFloat

声明：

```
GX_API GXSetFloat (GX_DEV_HANDLE hDevice,  
                   GX_FEATURE_ID featureID, double dValue)
```

意义：

设置浮点类型值。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[in] <i>dValue</i>	用户将要设置的浮点值

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
--------------------------------	-------------

GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_OUT_OF_RANGE	用户传入值越界，比最小值小，或者比最大值大
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 3000;
emStatus = GXSetFloat(hDevice, GX_FLOAT_EXPOSURE_TIME, dValue);
```

7.5.4.22. GXGetFloatRange

声明：

```
GX_API GXGetFloatRange (GX_DEV_HANDLE hDevice,
                        GX_FEATURE_ID featureID,
                        GX_FLOAT_RANGE* pFloatRange)
```

意义：

获取 Float 类型值的最小值、最大值、步长、单位等描述信息。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID
[out] pFloatRange	浮点型值的描述结构体指针，参见 GX_FLOAT_RANGE

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读取浮点型范围

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_FLOAT_RANGE stFloatRange;
emStatus = GXGetFloatRange(hDevice, GX_FLOAT_EXPOSURE_TIME,
                          &stFloatRange);
```

7.5.4.23. GXGetFloat

声明：

```
GX_API GXGetFloat (GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    double* pdValue)
```

意义：

获取浮点类型值。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pdValue</i>	指向返回的浮点值的指针

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL
<code>GX_STATUS_INVALID_ACCESS</code>	当前不可访问，不能读

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
double dValue = 0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_EXPOSURE_TIME, &dValue);
```

7.5.4.24. GXSetInt

声明：

```
GX_API GXSetInt (GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID,
                  int64_t nValue)
```

意义：

设置 Int 类型值。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[in] <i>nValue</i>	用户将要设置的值

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
--------------------------------	-------------

GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_OUT_OF_RANGE	用户传入值越界，比最小值小，或者比最大值大，或者不是步长整数倍
GX_STATUS_INVALID_ACCESS	当前不可访问，不能写

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 200;
emStatus = GXSetInt(hDevice, GX_INT_WIDTH, i64Value);

```

7.5.4.25. GXGetFeatureName

声明：

```

GX_API GXGetFeatureName (GX_DEV_HANDLE hDevice,
                          GX_FEATURE_ID featureID,
                          char* pszName,
                          size_t* pnSize)

```

意义：

获取功能码对应的字符串描述。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pszName</i>	用户输入的字符串缓冲区地址,字符串长度包含末尾结束符'\0'
[in,out] <i>pnSize</i>	用户输入的表示字符串缓冲区地址的长度,单位字节。

如果用户输入的 *pszName* 为 NULL：

[out] <i>pnSize</i>	返回需要的实际长度
---------------------	-----------

如果用户输入的*pszName*非NULL：

[in] <i>pnSize</i>	为用户分配的buffer大小
[out] <i>pnSize</i>	返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

char* pszName = NULL;
size_t nSize = 0;
//传入 NULL 获取要求的 buffer 长度，然后申请 buffer，再获取内容
GXGetFeatureName(hDevice, GX_FLOAT_GAIN, NULL, &nSize);
pszName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, GX_FLOAT_GAIN, pszName,
                             &nSize);

//操作完成后
if(NULL != pszName)
{
    delete[] pszName;
    pszName = NULL;
}
```

7.5.4.26. GXIsImplemented

声明：

```
GX_API GXIsImplemented (GX_DEV_HANDLE hDevice,
                        GX_FEATURE_ID featureID,
                        bool* pblsImplemented)
```

意义：

查询当前相机是否支持某功能，相机不支持某功能的概念有两个：

- 1) 通过查询相机寄存器，查到当前相机确实不支持此功能。
- 2) 当前相机描述文件中根本就没有此功能的描述。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pblsImplemented</i>	用来返回是否支持的结果，如果支持则返回 true，如果不支持则返回 false

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
```

```
bool bIsImplemented = false;

//参数 hDevice 已经通过 GXOpenDevice 获取，其他代码样例中出现的
//hDevice 意义同此处，后面将不再进行说明
emStatus = GXIsImplemented(hDevice, GX_FLOAT_GAIN, &bIsImplemented);
```

7.5.4.27. GXIsReadable

声明：

```
GX_API GXIsReadable(GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    bool* pbIsReadable)
```

意义：

查询某功能码当前是否可读。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pbIsReadable</i>	用来返回结果 ,如果可读则返回 true ,如果不可读则返回 false

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bIsReadable = false;
emStatus = GXIsReadable(hDevice, GX_FLOAT_GAIN, &bIsReadable);
```

7.5.4.28. GXIsWritable

声明：

```
GX_API GXIsWritable(GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    bool* pbIsWritable)
```

意义：

查询某功能码当前是否可写。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID

[out] *pIsWritable* 用来返回结果 如果可写则返回 true 如果不可写则返回 false

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
bool bIsWritable = false;
emStatus = GXIsWritable(hDevice, GX_FLOAT_GAIN, &bIsWritable);
```

7.5.4.29. GXGetIntRange

声明：

```
GX_API GXGetIntRange(GX_DEV_HANDLE hDevice,
                    GX_FEATURE_ID featureID,
                    GX_INT_RANGE* pIntRange)
```

意义：

获取 Int 类型值的最小值、最大值、步长。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pIntRange</i>	范围描述结构体，参见 GX_INT_RANGE

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_HANDLE</code>	用户传入非法的句柄
<code>GX_STATUS_NOT_IMPLEMENTED</code>	当前不支持的功能
<code>GX_STATUS_ERROR_TYPE</code>	用户传入的featureID类型错误
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL
<code>GX_STATUS_INVALID_ACCESS</code>	当前不可访问，不能读取整型范围

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
GX_INT_RANGE stIntRange;
emStatus = GXGetIntRange(hDevice, GX_INT_WIDTH, &stIntRange);
```

7.5.4.30. GXGetInt

声明：

```
GX_API GXGetInt (GX_DEV_HANDLE hDevice,
                  GX_FEATURE_ID featureID,
                  int64_t* pnValue)
```

意义：

获取 Int 类型值的当前值。

形参：

[in] <i>hDevice</i>	设备句柄
[in] <i>featureID</i>	功能码 ID
[out] <i>pnValue</i>	指向返回当前值的指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NOT_IMPLEMENTED	当前不支持的功能
GX_STATUS_ERROR_TYPE	用户传入的featureID类型错误
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_ACCESS	当前不可访问，不能读
上面没有涵盖到的，不常见的错误情况请参见 GX_STATUS_LIST	

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
int64_t i64Value = 0;
emStatus = GXGetInt(hDevice, GX_INT_WIDTH, &i64Value);
```

7.5.4.31. GXGetDeviceIPInfo

声明：

```
GX_API GXGetDeviceIPInfo(uint32_t nIndex, GX_DEVICE_IP_INFO* pstDeviceIPInfo)
```

意义：

指定设备索引，获取设备的网络信息。

形参：

[in] <i>nIndex</i>	设备序号
[out] <i>pstDeviceIPInfo</i>	设备网络信息结构体指针

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的index越界

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

调用该函数获取设备信息前必须调用 [GxUpdateAllDeviceList\(\)](#)接口进行一次枚举，否则可能造成用户获取的设备信息与当前实际连接的设备不一致。

代码样例：

```

GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEVICE_IP_INFO stIPInfo;

    //获取第一台设备的网络信息
    emStatus = GXGetDeviceIPInfo(1, &stIPInfo);
}
    
```

7.5.4.32. GXGetAllDeviceBaseInfo

声明：

**GX_API GXGetAllDeviceBaseInfo (GX_DEVICE_BASE_INFO* pDeviceInfo,
size_t* pnBufferSize)**

意义：

获取所有设备的基础信息。

形参：

[out] pDeviceInfo	设备信息结构体指针
[in,out] pnBufferSize	设备信息结构体缓冲区大小，单位字节
如果pDeviceInfo为NULL：	
[out] pnBufferSize	返回实际大小
如果pDeviceInfo非NULL：	
[in] pnBufferSize	为用户分配buffer大小
[out] pnBufferSize	返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

调用该函数获取设备信息前必须调用 [GxUpdateAllDeviceList\(\)](#)接口进行一次枚举，否则易造成用户获取的设备信息与当前实际连接的设备不一致。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
uint32_t ui32DeviceNum = 0;
emStatus = GXUpdateAllDeviceList(&ui32DeviceNum, 1000);
if ((emStatus == GX_STATUS_SUCCESS) && (ui32DeviceNum > 0))
{
    GX_DEVICE_BASE_INFO *pstBaseinfo = new GX_DEVICE_BASE_INFO[ui32DeviceNum];
    size_t nSize = ui32DeviceNum * sizeof(GX_DEVICE_BASE_INFO);

    //获取所有设备的基础信息
    emStatus = GXGetAllDeviceBaseInfo(pstBaseinfo, &nSize);

    //操作完成后
    if (NULL != pstBaseinfo)
    {
        delete[]pstBaseinfo;
        pstBaseinfo = NULL;
    }
}
```

7.5.4.33. GXOpenDeviceByIndex

声明：

GX_API GXOpenDeviceByIndex (uint32_t nDeviceIndex, GX_DEV_HANDLE* phDevice)

意义：

通过序号打开设备，从 1 开始。

形参：

[in] <i>nDeviceIndex</i>	设备序号，从 1 开始，例如：1、2、3、4...
[out] <i>phDevice</i>	接口返回的设备句柄

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_OUT_OF_RANGE	输入设备序号越界

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

注意事项:

设备序号从 1 开始。

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

GX_DEV_HANDLE hDevice = NULL;
//打开第一个设备
emStatus = GXOpenDeviceByIndex(1, &hDevice);
```

7.5.4.34. GXUpdateDeviceList

声明：

```
GX_API GXUpdateDeviceList (uint32_t* punNumDevices,  
                           uint32_t nTimeOut)
```

意义：

子网枚举当前可用的所有设备，并且获取设备个数。

形参：

[out] <i>punNumDevices</i>	用来返回设备个数的地址指针，不能为 NULL 指针。
[in] <i>nTimeOut</i>	枚举的超时时间(单位ms)，如果在用户指定超时时间内成功枚举到设备，则立即返回；如果在用户指定超时时间内没有枚举到设备，则一直等待，直到达到用户指定的超时时间返回。

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库
<code>GX_STATUS_INVALID_PARAMETER</code>	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;  
uint32_t ui32DeviceNum = 0;  
  
//枚举设备个数；超时时间受用户使用环境限制，用户可自行设置，不局限于 1000ms  
emStatus = GXUpdateDeviceList(&ui32DeviceNum, 1000);
```

7.5.4.35. GXGetFeatureNameList

声明：

```
GX_API GXGetFeatureNameList (GX_PORT_HANDLE      hPort,  
                             GX_FEATURE_NAME *  pFeatureNameList,  
                             uint32_t *         pCount)
```

意义：

获取所有节点名称。

形参：

[in] <i>hPort</i>	句柄
[in] <i>pFeatureNameList</i>	节点名称存储位置
[in, out] <i>pCount</i>	给定pFeatureNameList数组容量，返回实际写入的个数

返回值：

<code>GX_STATUS_SUCCESS</code>	操作成功，没有发生错误
<code>GX_STATUS_NOT_INIT_API</code>	没有调用GXInitLib()初始化库

GX_STATUS_INVALID_PARAMETER
GX_STATUS_INVALID_HANDLE

用户输入的指针为NULL
用户传入非法的句柄，或者关闭已经被关闭的设备

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_STATUS      emStatus = GX_STATUS_SUCCESS;
GX_FEATURE_NAME stFeatureNameList;
uint32_t       ui32Count = 0;

emStatus = GXGetFeatureNameList(hDevice, &stFeatureNameList,
                                &ui32Count);

```

7.5.4.36. GXGetFeatureName

声明：

```

GX_API GXGetFeatureName (GX_DEV_HANDLE    hDevice,
                        GX_FEATURE_ID_CMD featureID,
                        char*              pszName,
                        size_t*            pnSize)

```

意义：

获取功能控制码对应的字符串描述。

形参：

[in] hDevice	设备句柄
[in] featureID	功能码 ID
[in] pszName	用户输入的字符串缓冲区地址,字符串长度包含结束符'\0'
[in, out] pnSize	用户输入的字符串缓冲区地址的长度,单位字节
如果用户输入的pszName为NULL：	
[out] pnSize	返回需要的实际长度
如果用户输入的pszName非NULL：	
[in] pnSize	为用户分配的buffer大小
[out] pnSize	返回实际填充buffer大小

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib()初始化库
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_NEED_MORE_BUFFER	用户分配的buffer过小

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```

GX_ENUM_VALUE stEnumValue;
GX_FEATURE_ID_CMD emFeatureID = GX_INT_TIMESTAMP_TICK_FREQUENCY;

```

```
size_t nSize = 0;

emStatus = GXGetFeatureName(hDevice, emFeatureID, NULL, &nSize);

char* pFeatureName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, emFeatureID, pFeatureName,
                             &nSize);

//.....

delete[] pFeatureName;
```

7.5.4.37. GXGetFeatureType

声明：

```
GX_API GXGetFeatureType(GX_PORT_HANDLE hPort,
                        const char *      strName,
                        GX_FEATURE_TYPE * type)
```

意义：

获取节点的类型。

形参：

[in] <i>hPort</i>	句柄
[in] <i>strName</i>	节点名称
[out] <i>type</i>	节点类型

返回值：

GX_STATUS_SUCCESS	操作成功，没有发生错误
GX_STATUS_NOT_INIT_API	没有调用GXInitLib初始化库
GX_STATUS_INVALID_HANDLE	用户传入非法的句柄
GX_STATUS_INVALID_PARAMETER	用户输入的指针为NULL

上面没有涵盖到的，不常见的错误情况请参见 [GX_STATUS_LIST](#)

代码样例：

```
GX_ENUM_VALUE stEnumValue;
GX_FEATURE_ID_CMD emFeatureID = GX_INT_TIMESTAMP_TICK_FREQUENCY;
size_t nSize = 0;

emStatus = GXGetFeatureName(hDevice, emFeatureID, NULL, &nSize);

char* pFeatureName = new char[nSize];
emStatus = GXGetFeatureName(hDevice, emFeatureID, pFeatureName,
                             &nSize);

GX_FEATURE_TYPE emType;
emStatus = GXGetFeatureType(hDevice, pFeatureName, &emType);
if( NULL != pFeatureName)
{
    delete[] pFeatureName;
    pFeatureName = NULL;
}
```

8. 图像处理接口说明

图像处理接口主要包括图像像素格式转换、颜色质量提升等功能接口。图像处理接口函数的原型声明在文件 DxImageProc.h 中。

8.1. 类型

8.1.1. 数据类型

名称	描述
VxInt8	8 位有符号整型
VxInt16	16 位有符号整型
VxInt32	32 位有符号整型
VxInt64	64 位有符号整型
VxUInt8	8 位无符号整型
VxUInt16	16 位无符号整型
VxUInt32	32 位无符号整型

8.2. 常量

8.2.1. 状态码

```
typedef enum tagDX_STATUS
{
    DX_OK = 0,
    DX_PARAMETER_INVALID = -101,
    DX_PARAMETER_OUT_OF_BOUND = -102,
    DX_NOT_ENOUGH_SYSTEM_MEMORY = -103,
    DX_NOT_FIND_DEVICE = -104,
    DX_STATUS_NOT_SUPPORTED = -105,
    DX_CPU_NOT_SUPPORT_ACCELERATE = -106,
    DX_PARAMETER_NOT_INITIALIZED = -107
} DX_STATUS;
```

名称	描述
DX_OK	操作成功
DX_PARAMETER_INVALID	输入参数无效
DX_PARAMETER_OUT_OF_BOUND	参数越界
DX_NOT_ENOUGH_SYSTEM_MEMORY	系统内存不足
DX_NOT_FIND_DEVICE	没有找到设备
DX_STATUS_NOT_SUPPORTED	不支持的格式

DX_CPU_NOT_SUPPORT_ACCELERATE	CPU 不支持加速
DX_PARAMETER_NOT_INITIALIZED	参数没有初始化
DX_FILE_OPEN_FAIL	文件打开失败
DX_FILE_READ_FAIL	文件读取失败
DX_FILE_VERSION_NOSUPPORT	文件版本号错误
DX_FILE_CONTENT_NOT_CALIB	文件内容包含没有标定的参数
DX_FILE_CONTENT_NOT_ENOUGH	文件内容缺失
DX_FILE_CONTENT_MODEL_ERR	标定模型错误
DX_FILE_CONTENT_NOT_LOAD	没有加载标定文件
DX_FILE_CONTENT_ERR	文件内容错误
DX_CALC_ERR	计算错误

8.2.2. 像素 Bayer 格式

```
typedef enum tagDX_PIXEL_COLOR_FILTER
{
    NONE    = 0,
    BAYERRG = 1,
    BAYERGB = 2,
    BAYERGR = 3,
    BAYERBG = 4
} DX_PIXEL_COLOR_FILTER;
```

名称	描述
NONE	非 Bayer 格式
BAYERRG	第一行以 RG 开始
BAYERGB	第一行以 GB 开始
BAYERGR	第一行以 GR 开始
BAYERBG	第一行以 BG 开始

8.2.3. Bayer 转换类型

```
typedef enum tagDX_BAYER_CONVERT_TYPE
{
    RAW2RGB_NEIGHBOUR = 0,
    RAW2RGB_ADAPTIVE = 1,
    RAW2RGB_NEIGHBOUR3 = 2,
    RAW2RGB_WEIGHT = 3
} DX_BAYER_CONVERT_TYPE;
```

名称	描述
RAW2RGB_NEIGHBOUR	邻域平均插值算法
RAW2RGB_ADAPTIVE	边缘自适应插值算法
RAW2RGB_NEIGHBOUR3	更大区域的邻域平均插值算法
RAW2RGB_WEIGHT	权重插值算法

8.2.4. 有效数据位

```
typedef enum tagDX_VALID_BIT
{
    DX_BIT_0_7    = 0,
    DX_BIT_1_8    = 1,
    DX_BIT_2_9    = 2,
    DX_BIT_3_10   = 3,
    DX_BIT_4_11   = 4
} DX_VALID_BIT;
```

如果原始 raw 图是 8 位（0~7），只能选择 DX_BIT_0_7 算法；原始 raw 图为 10 位（0~9）时，转换成 8 位时可以选择 DX_BIT_0_7，DX_BIT_1_8 和 DX_BIT_2_9 三种算法；原始 raw 图为 12 位（0~11），转换成 8 位数据时可以选择 DX_BIT_0_7，DX_BIT_1_8，DX_BIT_2_9，DX_BIT_3_10 和 DX_BIT_4_11 全部五种算法。

名称	描述
DX_BIT_0_7	取 0~7 位
DX_BIT_1_8	取 1~8 位
DX_BIT_2_9	取 2~9 位
DX_BIT_3_10	取 3~10 位
DX_BIT_4_11	取 4~11 位

8.2.5. 图像实际位数

```
typedef enum tagDX_ACTUAL_BITS
{
    DX_ACTUAL_BITS_8  = 8,
    DX_ACTUAL_BITS_10 = 10,
    DX_ACTUAL_BITS_12 = 12,
    DX_ACTUAL_BITS_14 = 14,
    DX_ACTUAL_BITS_16 = 16
} DX_ACTUAL_BITS;
```

图像数据的实际位数，即位宽，每个像素所占用的字节数。

名称	描述
DX_ACTUAL_BITS_8	8 位

DX_ACTUAL_BITS_10	10 位
DX_ACTUAL_BITS_12	12 位
DX_ACTUAL_BITS_14	14 位
DX_ACTUAL_BITS_16	16 位

8.2.6. 图像镜像翻转类型

```
typedef enum DX_IMAGE_MIRROR_MODE
{
    HORIZONTAL_MIRROR = 0,
    VERTICAL_MIRROR    = 1
}DX_IMAGE_MIRROR_MODE;
```

名称	描述
HORIZONTAL_MIRROR	水平镜像
VERTICAL_MIRROR	垂直镜像

8.2.7. 彩色图像 RGB 顺序

```
typedef enum DX_RGB_CHANNEL_ORDER
{
    DX_ORDER_RGB = 0,
    DX_ORDER_BGR = 1
}DX_RGB_CHANNEL_ORDER;
```

名称	描述
DX_ORDER_RGB	彩色图像顺序为 RGB
DX_ORDER_BGR	彩色图像顺序为 BGR

8.2.8. 图像格式转换句柄

```
#define IMAGE_CONVERT_DECLARE_HANDLE(name) \
struct name##_; typedef struct name##_ *name
IMAGE_CONVERT_DECLARE_HANDLE(DX_IMAGE_FORMAT_CONVERT_HANDLE);
```

8.2.9. 系统参数

```
typedef enum DX_SYSTEM_PARAM
{
    THREAD_NUM = 0,
    SSSE3_ENABLE = 1,
    NEON_ENABLE = 2
} DX_SYSTEM_PARAM;
```

名称	描述
THREAD_NUM	线程数量

SSSE3_ENABLE	SSSE3 使能
NEON_ENABLE	NEON 使能

8.3. 结构体

8.3.1. 黑白图像处理功能设置结构体

```
typedef struct MONO_IMG_PROCESS
{
    bool      bDefectivePixelCorrect;
    bool      bSharpness;
    bool      bAccelerate;
    float fSharpFactor;
    VxUInt8   *pProLut;
    VxUInt16   nLutLength;
    VxUInt8   arrReserved[32];
} MONO_IMG_PROCESS;
```

接口 DxMono8ImgProcess 输入参数。

名称	描述
bDefectivePixelCorrect	坏点校正开关
bSharpness	锐化开关
bAccelerate	加速使能
fSharpFactor	锐化强度因子
pProLut	查找表 Buffer
nLutLength	Lut Buffer 长度
arrReserved[32]	保留 32 字节

8.3.2. 彩色图像处理功能设置结构体

```
typedef struct COLOR_IMG_PROCESS
{
    bool bDefectivePixelCorrect;
    bool bDenoise;
    bool bSharpness;
    bool bAccelerate;
    VxInt16 *parrCC;
    VxUInt8 nCCBufLength;
    float fSharpFactor;
    VxUInt8 *pProLut;
    VxUInt16 nLutLength;
    DX_BAYER_CONVERT_TYPE cvType;
    DX_PIXEL_COLOR_FILTER emLayOut;
    bool bFlip;
    VxUInt8 arrReserved[32];
}
```

```
} COLOR_IMG_PROCESS;
```

接口 DxRaw8ImgProcess 输入参数。

名称	描述
bDefectivePixelCorrect	坏点校正开关
bDenoise	降噪开关
bSharpness	锐化开关
bAccelerate	加速使能
*parrCC	色彩处理参数数组地址
nCCBufLength	parrCC 长度 (sizeof(VxInt16)*9)
fSharpFactor	锐化强度因子
pProLut	查找表 Buffer
nLutLength	Lut Buffer 长度
cvType	插值方法选择
emLayOut	BAYER 格式
bFlip	翻转标志
arrReserved[32]	保留 32 字节

8.3.3. 平场校正处理功能设置结构体

```
typedef struct FLAT_FIELD_CORRECTION_PROCESS
{
    void                *pBrightBuf;
    void                *pDarkBuf;
    VxUInt32            nImgWid;
    VxUInt32            nImgHei;
    DX_ACTUAL_BITS      nActualBits;
    DX_PIXEL_COLOR_FILTER emBayerType;
} FLAT_FIELD_CORRECTION_PROCESS;
```

接口 DxGetFFCCoefficients 输入参数。

名称	描述
pBrightBuf	亮场图像 Buffer
pDarkBuf	暗场图像 Buffer
nImgWid	图像宽
nImgHei	图像高
nActualBits	实际位深
emBayerType	Bayer 格式

8.3.4. 颜色校正系数用户设置结构体

```
typedef struct COLOR_TRANSFORM_FACTOR
{
    float fGain00;
    float fGain01;
    float fGain02;

    float fGain10;
    float fGain11;
    float fGain12;

    float fGain20;
    float fGain21;
    float fGain22;
} COLOR_TRANSFORM_FACTOR;
```

接口 DxCalcUserSetCCParam 输入参数。

名称	描述
fGain00	作用于红色像素的红色通道增益
fGain01	作用于绿色像素的红色通道增益
fGain02	作用于蓝色像素的红色通道增益
fGain10	作用于红色像素的绿色通道增益
fGain11	作用于绿色像素的绿色通道增益
fGain12	作用于蓝色像素的绿色通道增益
fGain20	作用于红色像素的蓝色通道增益
fGain21	作用于绿色像素的蓝色通道增益
fGain22	作用于蓝色像素的蓝色通道增益

8.3.5. 静态坏点校正功能参数结构体

```
typedef struct STATIC_DEFECT_CORRECTION
{
    VxUInt32          nImgWidth;
    VxUInt32          nImgHeight;
    VxUInt32          nImgOffsetX;
    VxUInt32          nImgOffsetY;
    VxUInt32          nImgWidthMax;
    DX_PIXEL_COLOR_FILTER nBayerType;
    DX_ACTUAL_BITS     emActualBits;
} STATIC_DEFECT_CORRECTION;
```

接口 DxStaticDefectCorrection 输入参数。

名称	描述
nImgWidth	当前图像的宽度
nImgHeight	当前图像的高度
nImgOffsetX	当前图像的水平偏移
nImgOffsetY	当前图像的垂直偏移
nImgWidthMax	设备支持的最大宽度
nBayerType	图像像素格式
emActualBits	图像像素实际有效位

8.4. 接口

8.4.1. DxRaw12PackedToRaw16

声明：

VxInt32 DHDECL DxRaw12PackedToRaw16 (void* pInputBuffer, void* pOutputBuffer, VxUInt32 nWidth, VxUInt32 nHeight)

意义：

该函数用于将 Raw12 Packed 格式数据转换为 Raw16 数据。

形参：

pInputBuffer 指向原始图像 12 位 Packed 数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区，需新开辟，不能使用原始数据 buffer，开辟大小为图像宽 x 图像高 x2

nWidth 图像宽

nHeight 图像高

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 600;
int32_t i32Height = 400;
BYTE *pRaw16Buf = new BYTE[i32Width * i32Height * 2];
//开辟输出 buffer
if (pRaw16Buf == NULL)
{
    return ;
}
else
{
    memset(pRaw16Buf, 0, i32Width * i32Height * 2 * sizeof(BYTE));
    //缓冲区初始化
}
```

```
//pPackedRaw12Buf 为原始图像数据
VxInt32 DxStatus = DxRaw12PackedToRaw16(pPackedRaw12Buf, pRaw16Buf,
                                          i32Width, i32Height);

if (DxStatus != DX_OK)
{
    if (pRaw16Buf != NULL)
    {
        delete []pRaw16Buf;
        pRaw16Buf = NULL;
    }
    return ;
}

//对 Raw16 数据进行处理
//.....

//处理完成
if(pRaw16Buf != NULL)
{
    delete []pRaw16Buf;
    pRaw16Buf = NULL;
}
```

8.4.2. DxRaw10PackedToRaw16

声明：

**VxInt32 DHDECL DxRaw10PackedToRaw16 (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight)**

意义：

该函数用于将 Raw10 Packed 格式数据转换为 Raw16 数据。

形参：

pInputBuffer 指向原始图像 10 位 Packed 数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区，需新开辟，不能使用原始数据 buffer，
开辟大小为图像宽 x 图像高 x2

nWidth 图像宽

nHeight 图像高

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 600;
int32_t i32Height = 400;
BYTE *pRaw16Buf = new BYTE[i32Width * i32Height * 2];
//开辟输出 buffer
if(pRaw16Buf == NULL)
{
```

```

        return ;
    }
    else
    {
        memset(pRaw16Buf, 0, i32Width * i32Height * 2 * sizeof(BYTE));
        //缓冲区初始化
    }
    //pPackedRaw10Buf 为原始图像数据
    VxInt32 DxStatus = DxRaw10PackedToRaw16(pPackedRaw10Buf, pRaw16Buf,
                                              i32Width, i32Height);

    if (DxStatus != DX_OK)
    {
        if (pRaw16Buf != NULL)
        {
            delete []pRaw16Buf;
            pRaw16Buf = NULL;
        }

        return ;
    }

    //对 Raw16 数据进行处理
    //.....

    //处理完成
    if (pRaw16Buf != NULL)
    {
        delete []pRaw16Buf;
        pRaw16Buf = NULL;
    }
}

```

8.4.3. DxRaw8toRGB24

声明：

**VxInt32 DHDECL DxRaw8toRGB24 (void *pInputBuffer, void *pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight,
DX_BAYER_CONVERT_TYPE cvtype,
DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip)**

意义：

该函数用于将 Bayer 图像转换为 RGB 图像。

形参：

pInputBuffer 指向原始图像 8 位数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区（RGB 数据），开辟大小为图像宽 x 图像高 x3

nWidth 图像宽

nHeight 图像高

cvtype 转换算法类型

*nBayerType*Bayer 图像格式类型

bFlip 是否翻转；true，将图像进行上下翻转；FALSE，不翻转

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 600;
int32_t i32Height = 400;

//输出图像 RGB 数据
BYTE *pRGB24Buf = new BYTE[i32Width * i32Height * 3];
if (pRGB24Buf == NULL)
{
    return ;
}
else
{
    //缓冲区初始化
    memset(pRGB24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
}

DX_BAYER_CONVERT_TYPE emType = RAW2RGB_NEIGHBOUR; //选择插值算法
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG; //选择图像 Bayer 格式
bool bFlip = true;
//pRaw8Buf 为原始图像数据
VxInt32 DxStatus = DxRaw8toRGB24(pRaw8Buf, pRGB24Buf, i32Width,
                                i32Height, emType, emBayerType,
                                bFlip);

if (DxStatus != DX_OK)
{
    if (pRGB24Buf != NULL)
    {
        delete []pRGB24Buf;
        pRGB24Buf = NULL;
    }

    return;
}

//对 RGB24 数据进行处理

if (pRGB24Buf != NULL)
{
    delete []pRGB24Buf;
    pRGB24Buf = NULL;
}
```

8.4.4. DxRaw8toRGB24Ex

声明：

```
VxInt32 DHDECL DxRaw8toRGB24Ex (void *pInputBuffer, void *pOutputBuffer,
                                VxUInt32 nWidth, VxUInt32 nHeight,
                                DX_BAYER_CONVERT_TYPE cvtype,
                                DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip,
                                DX_RGB_CHANNEL_ORDER emChannelOrder)
```

意义：

该函数用于将 Bayer 图像转换为 RGB 图像，图像的通道顺序可选择为 RGB 或者 BGR。

形参：

pInputBuffer 指向原始图像 8 位数据缓冲区

pOutputBuffer 指向目标图像 24 位数据缓冲区，开辟大小为图像宽 x 图像高 x 3

nWidth 图像宽

nHeight 图像高

cvtype 转换算法类型

nBayerType Bayer 图像格式类型

bFlip 是否翻转；true，将图像进行上下翻转；false，不翻转

emChannelOrder 输出图像的 RGB 通道顺序

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 600;
int32_t i32Height = 400;

//输出图像 BGR 数据
BYTE *pBGR24Buf = new BYTE[i32Width * i32Height * 3];
if (pBGR24Buf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pBGR24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
}

DX_BAYER_CONVERT_TYPE emType = RAW2RGB_NEIGHBOUR; //选择插值算法
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG; //选择图像 Bayer 格式
bool bFlip = true;
DX_RGB_CHANNEL_ORDER emChannelOrder = DX_ORDER_BGR;
```



```
//pRaw8Buf 为原始图像数据
VxInt32 DxStatus = DxRaw8toRGB24Ex (pRaw8Buf, pBGR24Buf, i32Width,
                                     i32Height, emType, emBayerType,
                                     bFlip, emChannelOrder);

if (DxStatus != DX_OK)
{
    if (pBGR24Buf != NULL)
    {
        delete []pBGR24Buf;
        pBGR24Buf = NULL;
    }

    return ;
}

//对 BGR 通道顺序的图像数据进行处理

if (pBGR24Buf!= NULL)
{
    delete []pBGR24Buf;
    pBGR24Buf = NULL;
}
```

8.4.5. DxRotate90CW8B

声明：

**VxInt32 DHDECL DxRotate90CW8B (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight)**

意义：

该函数对 8bit 图像进行顺时针(ClockWise)旋转 90 度操作。对于 Bayer 图像，旋转后将改变 Bayer 格式。例如，BAYER_RG 类型的 Raw 图顺时针旋转后变成 BAYER_GR 类型。旋转后图像的宽等于原始图像的高，高等于原始图像的宽。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nWidth 原始图像的宽度

nHeight 原始图像的高度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//BAYER_RG 的 Raw 图像进行旋转，宽为 1600，高为 1234
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pTemp = new BYTE[i32Width * i32Height];
```

```

if (pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE));
}

//顺时针旋转 90 度
//pRaw8Buf 为原始图像数据
VxInt32 DxStatus = DxRotate90CW8B(pRaw8Buf, pTemp, i32Width,
                                   i32Height);

if (DxStatus != DX_OK)
{
    if (pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }

    return ;
}
else
{
    //拷贝的原始图像缓冲区
    memcpy(pRaw8Buf, pTemp, i32Width * i32Height * sizeof(BYTE));
}
if (pTemp != NULL)
{
    delete []pTemp;
    pTemp = NULL;
}

//此时 , pRaw8Buf 中的数据类型为 BAYER_GR, 图像宽为 1234 , 高为 1600

```

8.4.6. DxRotate90CCW8B

声明：

**VxInt32 DHDECL DxRotate90CCW8B (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight)**

意义：

该函数对 8bit 灰度图像进行逆时针(Counter ClockWise)旋转 90 度操作。对于 Bayer 图像，旋转后将改变 Bayer 格式。例如，BAYER_RG 类型的 Raw 图逆时针旋转后变成 BAYER_GB 类型。旋转后图像的宽等于原始图像的高，高等于原始图像的宽。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nWidth 原始图像的宽度

nHeight 原始图像的高度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//BAYER_RG 的 Raw 图像进行旋转，宽为 1600，高为 1234
int32_t i32Width = 1600;
int32_t i32Height = 1234;
BYTE* pTemp = new BYTE[i32Width * i32Height];
if (pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE));
}
//逆时针旋转 90 度
//pRaw8Buf 为原始图像数据
VxInt32 DxStatus = DxRotate90CCW8B(pRaw8Buf, pTemp, i32Width,
                                     i32Height);

if (DxStatus != DX_OK)
{
    if (pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }
    return ;
}
else
{
    memcpy(pRaw8Buf, pTemp, i32Width * i32Height * sizeof(BYTE));
    //拷贝的原始图像缓冲区
}

if (pTemp != NULL)
{
    delete []pTemp;
    pTemp = NULL;
}

//此时，pRaw8Buf 中的数据类型为 BAYER_GR, 图像宽为 1234，高为 1600
```

8.4.7. DxRotate90CW16B

声明：

**VxInt32 DHDECL DxRotate90CW16B (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight)**

意义：

该函数对 16bit 图像进行顺时针 (ClockWise) 旋转 90 度操作。对于 Bayer 图像，旋转后将改变 Bayer 格式。例如，BAYER_RG 类型的 Raw 图顺时针旋转后变成 BAYER_GR 类型。旋转后图像的宽等于原始图像的高，高等于原始图像的宽。

形参：

<i>pInputBuffer</i>	指向原始图像数据缓冲区
<i>pOutputBuffer</i>	指向目标图像数据缓冲区
<i>nWidth</i>	原始图像的宽度
<i>nHeight</i>	原始图像的高度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//BAYER_RG 的 Raw 图像进行旋转，宽为 1600，高为 1234
int32_t i32Width = 1600;
int32_t i32Height = 1234;

unsigned * pTemp = new BYTE[i32Width * i32Height * 2];
if(pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE) * 2);
}

//顺时针旋转 90 度
//pRaw16Buf 为原始图像数据
VxInt32 DxStatus = DxRotate90CW16B(pRaw16Buf, pTemp, i32Width,
                                     i32Height);

if (DxStatus != DX_OK)
{
    if (pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }

    return ;
}
else
{
    //拷贝的原始图像缓冲区
```

```
memcpy(pRaw16Buf, pTemp, i32Width * i32Height * sizeof(BYTE) * 2);
}
if (pTemp != NULL)
{
    delete []pTemp;
    pTemp = NULL;
}
//此时, pRaw16Buf 中的数据类型为 BAYER_GR, 图像宽为 1234, 高为 1600
```

8.4.8. DxRotate90CCW16B

声明：

**VxInt32 DHDECL DxRotate90CCW16B (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight)**

意义：

该函数对 16bit 灰度图像进行逆时针(Counter ClockWise)旋转 90 度操作。对于 Bayer 图像，旋转后将改变 Bayer 格式。例如，BAYER_RG 类型的 Raw 图逆时针旋转后变成 BAYER_GB 类型。旋转后图像的宽等于原始图像的高，高等于原始图像的宽。

形参：

<i>pInputBuffer</i>	指向原始图像数据缓冲区
<i>pOutputBuffer</i>	指向目标图像数据缓冲区
<i>nWidth</i>	原始图像的宽度
<i>nHeight</i>	原始图像的高度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//BAYER_RG 的 Raw 图像进行旋转, 宽为 1600, 高为 1234
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pTemp = new BYTE[i32Width * i32Height * 2];
if (pTemp == NULL)
{
    return;
}
else
{
    memset(pTemp, 0, i32Width * i32Height * sizeof(BYTE) * 2);
}
//逆时针旋转 90 度
//pRaw16Buf 为原始图像数据
VxInt32 DxStatus = DxRotate90CCW16B(pRaw16Buf, pTemp, i32Width,
                                     i32Height);

if (DxStatus != DX_OK)
{
```

```

        if (pTemp != NULL)
        {
            delete []pTemp;
            pTemp = NULL;
        }
        return ;
    }
    else
    {
        memcpy(pRaw16Buf, pTemp, i32Width * i32Height * sizeof(BYTE) * 2);
        //拷贝的原始图像缓冲区
    }

    if (pTemp != NULL)
    {
        delete []pTemp;
        pTemp = NULL;
    }

    //此时，pRaw16Buf 中的数据类型为 BAYER_GR, 图像宽为 1234，高为 1600

```

8.4.9. DxBrightness

声明：

**VxInt32 DHDECL DxBrightness (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nImagesize, VxInt32 nFactor)**

意义：

该函数对输入图像进行亮度调节处理，输入图像为 24bitRGB 图像或者 8bit 灰度图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nImagesize 输入图像 buffer 长度，以字节为单位(对于 RGB 图像：图像宽 x 图像高 x3)

nFactor 亮度调节因子，值范围：-150 ~ 150

0：亮度没有变化

大于 0：增加亮度

小于 0：减小亮度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//对 24bitRGB 图像进行亮度调节
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];
if (pRGBOutputBuf == NULL)
{

```

```

        return;
    }
    else
    {
        memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
    }

    //pRGBInputBuf 为原始图像数据
    VxInt32 DxStatus = DxBrightness((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                    i32Width*i32Height*3, 50);
    if (DxStatus != DX_OK)
    {
        return ;
    }

    //对 8bit 灰度图像进行亮度调节
    BYTE* pYOutputbuf = new BYTE[i32Width * i32Height];
    if (pYOutputbuf == NULL)
    {
        return;
    }
    else
    {
        memset(pYOutputbuf, 0, i32Width * i32Height * sizeof(BYTE));
    }

    VxInt32 DxStatus = DxBrightness((BYTE*)pYInputbuf, pYOutputbuf,
                                    i32Width*i32Height, 50);
    if (DxStatus != DX_OK)
    {
        return ;
    }

```

效果图：

如图 8-1 为未进行亮度调节的图像，图 8-2 亮度调节前图像 为亮度调节后的图像。

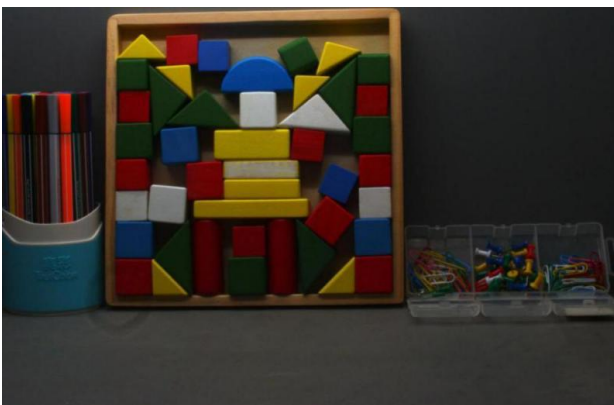


图 8-1 亮度调节前图像



图 8-2 亮度调节后图像

8.4.10. DxContrast

声明：

```
VxInt32 DHDECL DxContrast (void* pInputBuffer, void* pOutputBuffer,  
                           VxUInt32 nImagesize, VxInt32 nFactor)
```

意义：

该函数对输入图像进行对比度调节，输入图像为 24bit RGB 图像或者 8bit 灰度图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nImagesize 输入图像 buffer 长度，以字节为单位(对于 RGB 图像：图像宽 x 图像高 x3)

nFactor 对比度调节参数，值范围：-50 ~ 100

0：对比度没有变化

大于 0：增加对比度

小于 0：减小对比度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//对 24bitRGB 图像进行对比度调节  
int32_t i32Width = 1600;  
int32_t i32Height = 1234;  
  
BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];  
if (pRGBOutputBuf == NULL)  
{  
    return;  
}  
else  
{  
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));  
}  
  
// pRGBInputBuf 为原始图像数据  
VxInt32 DxStatus = DxContrast ((BYTE*)pRGBInputBuf, pRGBOutputBuf,  
                               i32Width*i32Height*3, 50);  
  
if (DxStatus != DX_OK)  
{  
    return ;  
}  
  
//对 8bit 灰度图像进行对比度调节  
BYTE* pYOutputbuf = new BYTE[i32Width * i32Height];  
if (pYOutputbuf == NULL)  
{  
    return;  
}
```



```

    }
    else
    {
        memset(pYOutputbuf, 0, i32Width * i32Height * sizeof(BYTE));
    }

    //pYInputBuf 为原始图像数据
    VxInt32 DxStatus = DxContrast ((BYTE*)pYInputbuf, pYOutputbuf,
                                   i32Width*i32Height, 50);

    if (DxStatus != DX_OK)
    {
        return ;
    }

```

效果图：

如下图,图 8-3 为关闭对比度调节时图像,图 8-3 对比度调节前图像 为打开并调节对比度后的图像。



图 8-3 对比度调节前图像



图 8-4 对比度调节后图像

8.4.11. DxSharpen24B

声明：

**VxInt32 DHDECL DxSharpen24B (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight, float fFactor)**

意义：

该函数对输入图像进行锐化处理，输入图像为 24bitRGB 图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 经过锐化处理的 RGB 图像 buffer

nWidth 图像宽

nHeight 图像高

fFactor 锐化调节参数，范围：0.1 ~ 5.0

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//对 24bitRGB 图像进行锐化操作

```

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE [i32Width * i32Height * 3];
if (pRGBOutputBuf == NULL)
{
    return;
}
else
{
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE) * 3);
}

//pRGBInputBuf 为原始图像数据

VxInt32 DxStatus = DxSharpen24B ((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                i32Width, i32Height, 2.0);

if (DxStatus != DX_OK)
{
    return ;
}
```

效果图：

如下图，错误!未找到引用源。为锐化前图像，为锐化后图像。



图 8-5 锐化后图像



图 8-6 锐化后图像

8.4.12. DxSharpenMono8

声明：

VxInt32 DHDECL DxSharpenMono8 (void* pInputBuffer, void* pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight, float fFactor)

意义：

该函数对输入图像进行锐化处理，输入图像为 8bitRGB 图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

<i>pOutputBuffer</i>	经过锐化处理的 RGB 图像 buffer
<i>nWidth</i>	图像宽
<i>nHeight</i>	图像高
<i>fFactor</i>	锐化调节参数，范围：0.1 ~ 5.0

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//对 8bit 图像进行锐化操作
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pOutputBuf = new BYTE [i32Width * i32Height];
if(pOutputBuf == NULL)
{
    return;
}
else
{
    memset(pOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//pInputBuf 为原始图像数据

VxInt32 DxStatus = DxSharpenMono8 ((BYTE*)pInputBuf, pOutputBuf,
                                     i32Width,i32Height, 2.0);

if (DxStatus != DX_OK)
{
    return ;
}
    
```

8.4.13. DxSaturation

声明：

VxInt32 DHDECL DxSaturation (void* pInputBuffer, void* pOutputBuffer, VxUInt32 nImagesize, VxInt32 nFactor)

意义：

该函数对输入图像进行饱和度调节，输入图像为 24bitRGB 图像。

形参：

- pInputBuffer* 指向原始图像数据缓冲区
- pOutputBuffer* 经过饱和度调节图像的 buffer
- nImagesize* 输入图像 buffer 长度，以字节为单位。(对于 RGB 图像：图像宽 x 图像高 x3)
- nFactor*饱和度调节参数，值范围：0 ~ 128，其中：
 - 64：饱和度没有变化
 - 大于 64：增加饱和度

小于 64：减小饱和度

128：饱和度为当前两倍

0：黑白图像

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//对 24bitRGB 图像进行饱和度调节
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE* pRGBOutputBuf = new BYTE[i32Width * i32Height];
if (pRGBOutputBuf == NULL)
{
    return;
}
else
{
    memset(pRGBOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//pRGBInputBuf 为原始图像数据
VxInt32 DxStatus = DxSaturation((BYTE*)pRGBInputBuf, pRGBOutputBuf,
                                i32Width*i32Height, 90);

if (DxStatus != DX_OK)
{
    return ;
}
```

效果图：

如下图，图 8-7 为饱和度调节前图像，图 8-7 饱和度调节前图像 为饱和度调节后图像。



图 8-7 饱和度调节前图像



图 8-8 饱和度调节后图像

8.4.14. DxGetWhiteBalanceRatio

声明：

**VxInt32 DHDECL DxGetWhiteBalanceRatio (void *pInputBuffer, VxUInt32 nWidth,
VxUInt32 nHeight, double* dRatioR,
double* dRatioG, double* dRatioB)**

意义：

该函数用于获取白平衡系数。输入为 24bit RGB 图像，输出为白平衡系数。为了计算准确，输入的图像应该是客观“白色”区域。

形参：

pInputBuffer “白色” 图像数据缓冲区

nWidth 图像宽度

nHeight 图像高度

dRatioR 红分量白平衡系数

dRatioG 绿分量白平衡系数

dRatioB 蓝分量白平衡系数

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
double dRatioR = 1.0;
double dRatioG = 1.0;
double dRatioB = 1.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//求取白平衡系数
//pRGBInputBuf 为原始图像数据
VxInt32 DxStatus =DxGetWhiteBalanceRatio((BYTE*)pRGBInputBuf,
                                           i32Width,i32Height,
                                           &dRatioR, &dRatioG,
                                           &dRatioB);

if (DxStatus != DX_OK)
{
    return ;
}
```

效果图：

如图 8-9 所示为没有进行白平衡操作的 RGB 图像：



图 8-9 没有进行白平衡操作的 RGB 图像

- 1) 如果调用此函数求白平衡系数，为了准确，应该给 RGB 图像中一个客观“白色”区域子图像给该函数，如图 8-10 中的画框区域。（该区域的实际物体为白色积木），进行白平衡操作后的图像如图 8-11。
- 2) 调用方法：（pRGBBuf 为图 8-10 中框内子图像 buffer，nWidth 为子图像宽，nHeight 为子图像高）需创建新缓冲区存储子图像 buffer，然后将新缓冲区传递给 DxGetWhiteBalanceRatio。



图 8-10 选取的“白色”区域

计算出了 R、G、B 三个分量的白平衡系数后，进行白平衡操作方法如下：

```
//添加宏定义，放在头文件

//<判断位数据范围
#define CLIP8(a)  (((a) & 0xFFFFFFFF00) ? (((a) < 0) ? 0 : 255) : (a))

int i = 0;
int j = 0;
int nPos = 0;
//白平衡查找表
BYTE arrRLut[256];
BYTE arrGLut[256];
BYTE arrBLut[256];

//三个像素点
int R = 0;
int G = 0;
int B = 0;
double dRatioR = 1.0;
double dRatioG = 1.0;
double dRatioB = 1.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//计算白平衡系数查找表
for(i = 0; i<256; i++)
{
    //计算白平衡后的值
```

```

R = i * dRatioR;
G = i * dRatioG;
B = i * dRatioB;

arrRLut[i] = CLIP8(R);
arrGLut[i] = CLIP8(G);
arrBLut[i] = CLIP8(B);
}
//进行白平衡操作
for (i = 0; i < i32Height; i++)
{
    for(j = 0; j < i32Width; j++)
    {
        nPos = 3 * i * i32Width + 3 * j;
        pRGBBuf[nPos + 0] = arrRLut[pRGBBuf[nPos + 0]];
        pRGBBuf[nPos + 1] = arrGLut[pRGBBuf[nPos + 1]];
        pRGBBuf[nPos + 2] = arrBLut[pRGBBuf[nPos + 2]];
    }
}

```



图 8-11 进行白平衡后的图像

8.4.15. DxAutoRawDefectivePixelCorrect

声明：

**VxInt32 DHDECL DxAutoRawDefectivePixelCorrect (void* pRawImgBuf, VxUInt32 nWidth,
VxUInt32 nHeight, VxInt32 nBitNum)**

意义：

该函数自动对 Raw 图像进行实时坏点检测和校正，输入图像可以是 8bit Raw 图或者 16bit Raw 图。该函数支持 Bayer 格式的 Raw 图像。参数 nBitNum 指数据的实际位数，若为 8 位 Raw 图，则输入 8，若是 16 位 Raw 图，则输入数据的实际位数。例如，12 位数据的 Raw 图像，其占两个字节（16 位），但实际位数为 12，此时输入 12。该函数不支持 Packed 格式的 Raw 图，对于 Packed 格式，需先用函数 DxRaw10PackedToRaw16 或函数 DxRaw10PackedToRaw16 转换成 Raw16 格式。由于是实时检测和校正，当开启该功能时，每帧图像均需进行检测校正处理。

形参：

pRawImgBuf 图像输入 buffer（8bit 或 16bitRaw）

nWidth 图像宽

nHeight 图像高

nBitNum 数据的实际位数

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

if(bAutoCorrect) //是否开启自动坏点校正，pPaw16Buf 为 16 位 raw 图
{
    // pPaw16Buf 为输入图像数据
    VxInt32 DxStatus= DxAutoRawDefectivePixelCorrect (pPaw16Buf,
                                                         i32Width,
                                                         i32Height,
                                                         12);

    if (DxStatus != DX_OK)
    {
        if (pRaw16Buf != NULL)
        {
            delete []pRaw16Buf;
            pRaw16Buf = NULL;
        }

        return;
    }
}

//对 Raw 图像处理
//.....
```

8.4.16. DxRaw16toRaw8

声明：

VxInt32 DHDECL DxRaw16toRaw8 (void *pInputBuffer, void *pOutputBuffer, VxUInt32 nWidth, VxUInt32 nHeight, DX_VALID_BIT nValidBits);

意义：

该函数将 Raw16 图像（实际位数为 16 位，有效位为 10 位或者 12 位）转换成 Raw8 位图像（实际位数和有效位数都是 8 位）。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nWidth 图像宽

nHeight 图像高

nValidBits 数据有效位数

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

BYTE *pRaw8Buf = new BYTE[i32Width * i32Height]; //开辟输出 buffer
if (pRaw8Buf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pRaw8Buf, 0, i32Width * i32Height * sizeof(BYTE));
}
//pRaw16Buf 为输入图像数据
VxInt32 DxStatus = DxRaw16toRaw8(pRaw16Buf, pRaw8Buf,
                                   i32Width, i32Height,
                                   DX_BIT_4_11);

if (DxStatus != DX_OK)
{
    if (pRaw8Buf != NULL)
    {
        delete []pRaw8Buf;
        pRaw8Buf = NULL;
    }

    return;
}

//对 Raw8 数据进行处理
// .....
//处理完成
if (pRaw8Buf != NULL)
{
    delete []pRaw8Buf;
    pRaw8Buf = NULL;
}
```

8.4.17. DxRGB48toRGB24

声明：

```
VxInt32 DHDECL DxRGB48toRGB24 (void *pInputBuffer, void *pOutputBuffer,
                                VxUInt32 nWidth, VxUInt32 nHeight,
                                DX_VALID_BIT nValidBits);
```

意义：

该函数将 RGB 48 位彩色图像数据转换为 RGB 24 位图像数据，由于是三个通道同时进行处理，每个通道将 16 位数据（有效位数可能是 12 位，10 位等）转换为 8 位数据。

形参：

pInputBuffer 指向原始图像数据缓冲区（缓冲区大小为 $nWidth * nHeight * 3 * 2 \text{ BYTE}$ ）

pOutputBuffer 指向目标图像数据缓冲区（缓冲区大小为 $nWidth * nHeight * 3 \text{ BYTE}$ ）

nWidth 图像宽

nHeight 图像高

nValidBits 数据有效位数

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//开辟输出 buffer
BYTE *pRGB24Buf = new BYTE[i32Width * i32Height * 3];
if(pRGB24Buf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pRGB24Buf, 0, i32Width * i32Height * 3 * sizeof(BYTE));
}
//pPaw48Buf 为输入图像数据
VxInt32 DxStatus = DxRGB48toRGB24(pRGB48Buf, pRGB24Buf, i32Width,
                                   i32Height, DX_BIT_4_11);

if (DxStatus != DX_OK)
{
    if (pRGB24Buf != NULL)
    {
        delete []pRGB24Buf;
        pRGB24Buf = NULL;
    }

    return;
}

//对 Raw8 数据进行处理
// .....
```

```
//处理完成
if (pRGB24Buf != NULL)
{
    delete []pRGB24Buf;
    pRGB24Buf = NULL;
}
```

8.4.18. DxRaw16toRGB48

声明：

```
VxInt32 DHDECL DxRaw16toRGB48 (void *pInputBuffer, void *pOutputBuffer,
                                VxUInt32 nWidth, VxUInt32 nHeight,
                                DX_ACTUAL_BITS nActualBits,
                                DX_BAYER_CONVERT_TYPE cvtype,
                                DX_PIXEL_COLOR_FILTER nBayerType,
                                bool bFlip)
```

意义：

Raw16 图像（每个像素占 16 位）转换成 RGB48 图像（每个 RGB 分量各占 16 位），可以调用函数 DxRGB48toRGB24 将 RGB48 图像转为 RGB24 图像，便于显示。

形参：

pInputBuffer 指向原始图像数据缓冲区
pOutputBuffer 指向目标图像数据缓冲区
nWidth 图像宽
nHeight 图像高
nActualBits 数据有效位数
cvtype 转换算法类型
nBayerTypeBayer 图像格式类型
bFlip 是否翻转；true，将图像进行上下翻转；FALSE，不翻转

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//输出图像 RGB 数据
BYTE *pRGB48Buf = new BYTE[i32Width * i32Height * 3 * 2];
if (pRGB48Buf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
```

```

    memset(pRGB48Buf, 0, i32Width * i32Height * 3 * 2 * sizeof(BYTE));
}

DX_BAYER_CONVERT_TYPE emType= RAW2RGB_NEIGHBOUR; //选择插值算法
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG; //选择图像 Bayer 格式
DX_ACTUAL_BITS emActualBits = DX_ACTUAL_BITS_16; //实际位宽
bool bFlip = true;
//pPaw16Buf 为输入图像数据
VxInt32 DxStatus = DxRaw16toRGB48(pRaw16Buf, pRGB48Buf, i32Width,
                                   i32Height, emActualBits,
                                   emType, emBayerType, bFlip);

if (DxStatus != DX_OK)
{
    if (pRGB48Buf != NULL)
    {
        delete []pRGB48Buf;
        pRGB48Buf = NULL;
    }

    return;
}

//对 RGB48 数据进行处理
// .....

if (pRGB48Buf != NULL)
{
    delete []pRGB48Buf;
    pRGB48Buf = NULL;
}

```

8.4.19. DxRaw8toARGB32

声明：

```

VxInt32 DHDECL DxRaw8toARGB32 (void *pInputBuffer, void *pOutputBuffer,
                                VxUInt32 nWidth, VxUInt32 nHeight,
                                int nStride, DX_BAYER_CONVERT_TYPE cvtype,
                                DX_PIXEL_COLOR_FILTER nBayerType, bool bFlip,
                                VxUInt8 nAlpha)

```

意义：

该函数将 Raw8 图像转换成 ARGB32 图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区（缓冲区大小为 $nWidth * nHeight * 4$ ）

nWidth 图像宽

nHeight 图像高

nStride Bitmap 的扫描宽度

cvtype 转换算法类型

nBayerType Bayer 图像格式类型

bFlip 是否翻转；true，将图像进行上下翻转；FALSE，不翻转

nAlpha Alpha 通道值（取值范围为 0~255）

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//输出图像 ARGB 数据
BYTE *pARGB32Buf = new BYTE[i32Width * i32Height * 4];
if (pARGB32Buf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pARGB32Buf, 0, i32Width * i32Height * sizeof(BYTE));
}

DX_BAYER_CONVERT_TYPE emType= RAW2RGB_NEIGHBOUR; //选择插值算法
DX_PIXEL_COLOR_FILTER emBayerType = BAYERRG; //选择图像 Bayer 格式
int32_t i32Stride = i32Width; //扫描宽度
bool bFlip = true;
VxUInt8 nAlpha = 127;
// pPaw8Buf 为输入图像数据
VxInt32 DxStatus = DxRaw8toARGB32 (pRaw8Buf, pARGB32Buf, i32Width,
                                     i32Height, i32Stride, emType,
                                     emBayerType, bFlip, nAlpha);

if (DxStatus != DX_OK)
{
    if (pARGB32Buf!= NULL)
    {
        delete []pARGB32Buf;
        pARGB32Buf = NULL;
    }
    return;
}

//对 ARGB32 数据进行处理
//.....
```

```
if (pARGB32Buf != NULL)
{
    delete []pARGB32Buf;
    pARGB32Buf = NULL;
}
```

8.4.20. DxGetContrastLut

声明：

VxInt32 DHDECL DxGetContrastLut (int nContrastParam, void *pContrastLut,int *pLutLength);

意义：

该函数为计算对比度查找表，用于图像质量提升函数的输入，仅支持 24 位 RGB 图像。

形参：

nContrastParam 对比度调节参数，通过 GxI API 库中的 GX_INT_CONTRAST_PARAM 码获取

pContrastLut 对比度查找表

pLutLength 对比度查找表长度，单位是字节

如果用户输入的长度值小于当前实际的长度值，返回错误，并且 *pLutLength* 返回实际的长度值

如果用户输入的 *pContrastLut* 为 NULL，返回成功，*pLutLength* 返回实际的长度值

如果操作成功，*pLutLength* 返回实际的长度值

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int64_t i64ContrastParam = 0;

//判断当前相机是否支持对比度获取
bool bIsImplemented = false;
GX_STATUS emStatus = GXIsImplemented(hDevice, GX_INT_CONTRAST_PARAM,
&bIsImplemented);

if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

if (bIsImplemented)
{
    //获取对比度调节参数
    emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM, &i64ContrastParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }
}
else
{
    i64ContrastParam = 0;
```

```

}

//获取对比度查找表的长度
int32_t i32LutLength = 0;
VxInt32 DxStatus= DxGetContrastLut((int)i64ContrastParam, NULL,
                                   &i32LutLength);

if(DxStatus != DX_OK)
{
    return;
}

//为对比度查找表申请空间
BYTE *pContrastLut = new BYTE[i32LutLength];
if (pContrastLut == NULL)
{
    return;
}

//计算对比度查找表
DxStatus = DxGetContrastLut((int)i64ContrastParam, pContrastLut,
                             &i32LutLength);
if (DxStatus != DX_OK)
{
    if (pContrastLut!= NULL)
    {
        delete []pContrastLut;
        pContrastLut = NULL;
    }
    return;
}

//图像处理
// .....

if (pContrastLut!= NULL)
{
    delete []pContrastLut;
    pContrastLut= NULL;
}

```

8.4.21. DxGetGammatLut

声明：

```
VxInt32 DHDECL DxGetGammatLut (double dGammaParam, void *pGammaLut,
                                int *pLutLength);
```

意义：

该函数为计算 Gamma 查找表，用于图像质量提升函数的输入，仅支持 24 位 RGB 图像。

形参：

dGammaParam Gamma 调节参数，通过 GxI API 库中的 GX_FLOAT_GAMMA_PARAM 码获取

pGammaLut Gamma 查找表

pLutLength Gamma 查找表长度，单位是字节

如果用户输入的长度值小于当前实际的长度值，返回错误，并且 *pLutLength* 返回实际的长度值

如果用户输入的 *pGammaLut* 为 NULL，返回成功，*pLutLength* 返回实际的长度值

如果操作成功，*pLutLength* 返回实际的长度值

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32LutLength = 0;
double dGammaParam = 0.0;
//判断当前相机是否支持 Gamma 获取
bool bIsImplemented = false;
GX_STATUS emStatus = GXIsImplemented(hDevice, GX_FLOAT_GAMMA_PARAM,
&bIsImplemented);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}
if (bIsImplemented)
{
    //获取 Gamma 调节参数
    emStatus = GXGetFloat(hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
    if (emStatus != GX_STATUS_SUCCESS)
    {
        return;
    }
}
else
{
    dGammaParam = 1;
}

//获取 Gamma 查找表的长度
VxInt32 DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
if (DxStatus != DX_OK)
{
    return;
}

//为 Gamma 查找表申请空间
BYTE* pGammaLut= new BYTE[i32LutLength];
if (pGammaLut== NULL)
{
    return;
}
```



```
//计算 Gamma 查找表
DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
if (DxStatus != DX_OK)
{
    if (pGammaLut!= NULL)
    {
        delete []pGammaLut;
        pGammaLut= NULL;
    }
    return;
}

//处理图像
//.....
if (pGammaLut!= NULL)
{
    delete []pGammaLut;
    pGammaLut= NULL;
}
```

8.4.22. DxImageImprovment

声明：

```
VxInt32 DHDECL DxImageImprovment (void *pInputBuffer, void *pOutputBuffer,
                                   VxUInt32 nWidth, VxUInt32 nHeight,
                                   VxInt64 nColorCorrectionParam,
                                   void *pContrastLut, void *pGammaLut);
```

意义：

该函数对输入图像进行图像质量的提升，输入图像为 24bitRGB 图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nWidth 图像宽

nHeight 图像高

nColorCorrectionParam 颜色校正值,可通过 GxI API 库中的 GX_INT_COLOR_CORRECTION_PARAM 码获取，也可以设置为 0

pContrastLut 对比度查找表，可以通过 DxGetContrastLut 函数计算，只需要计算一次，也可以设置为 NULL

pGammaLut Gamma 查找表,通过 DxGetGammatLut 函数计算,只需要计算一次,也可以设置为 NULL

nColorCorrectionParam、*pContrastLut*、*pGammaLut* 可以进行不同的组合，实现不同的效果，组合情况如表 8-1 所示：

编号	nColorCorrectionParam	pContrastLut	pGammaLut	效果
----	-----------------------	--------------	-----------	----

1	非 0	非 NULL	非 NULL	对图像进行颜色校正，对比度，Gamma 调节（此时图像质量最好）
2	非 0	NULL	非 NULL	对图像进行颜色校正，Gamma 调节
3	非 0	非 NULL	NULL	对图像进行颜色校正，对比度调节
4	0	非 NULL	非 NULL	对图像进行对比度，Gamma 调节
5	非 0	NULL	NULL	对图像进行颜色校正调节
6	0	非 NULL	NULL	对图像进对比度调节
7	0	NULL	非 NULL	对图像进行 Gamma 调节

表 8-1

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//获取对比度调节参数
VxInt32 DxStatus = DX_OK;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
int32_t i32LutLength = 0;
double dGammaParam = 0.0;
BYTE* pGammaLut;
BYTE* pContrastLut;
GX_STATUS emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM,
                                &i64ContrastParam );

if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取颜色校正调节参数
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &i64ColorCorrectionParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取 Gamma 调节参数
emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
if (emStatus != GX_STATUS_SUCCESS)
{
```

```

        return;
    }

do
{
    //获取 Gamma 查找表的长度
    DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //为 Gamma 查找表申请空间
    pGammaLut = new BYTE[i32LutLength];
    if (pGammaLut== NULL)
    {
        DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //计算 Gamma 查找表
    DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //获取对比度查找表的长度
    DxStatus= DxGetContrastLut((int)i64ContrastParam, NULL,
                                &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //为对比度查找表申请空间
    pContrastLut = new BYTE[i32LutLength];
    if (pContrastLut == NULL)
    {
        DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //计算对比度查找表
    DxStatus = DxGetContrastLut((int)i64ContrastParam, pContrastLut,
                                &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }
}

```

```

}while(0);

//设置查找表失败，释放资源
if (DxStatus != DX_OK)
{
    if (pGammaLut != NULL)
    {
        delete[] pGammaLut;
        pGammaLut = NULL;
    }
    if(pContrastLut != NULL)
    {
        delete[] pContrastLut;
        pContrastLut = NULL;
    }
    return;
}

//输出图像 ARGB 数据
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
if (pOutputBuffer == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}

//提升图像的质量
//pInputBuffer 为输入图像数据
DxStatus = DxImageImprovment(pInputBuffer,pOutputBuffer,i32Width,i32Height,
                             i64ColorCorrectionParam, pContrastLut, pGammaLut);

if (pGammaLut!= NULL)
{
    delete []pGammaLut;
    pGammaLut= NULL;
}
if (pContrastLut!= NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}

```

效果图：

如下图，图 8-12 为图像质量提升前图像，图 8-13 图像质量提升前图像为图像质量提升后图像。



图 8-12 图像质量提升前图像



图 8-13 图像质量提升后图像

8.4.23. DxARGBImageImprovment

声明：

VxInt32 DHDECL DxARGBImageImprovment (void *pInputBuffer, void *pOutputBuffer, VxUInt32 nWidth, VxUInt32 nHeight, VxInt64 nColorCorrectionParam, void *pContrastLut, void *pGammaLut);

意义：

该函数对输入图像进行图像质量的提升，输入图像为 ARGB 图像。

形参：

- pInputBuffer* 指向原始图像数据缓冲区
- pOutputBuffer* 指向目标图像数据缓冲区
- nWidth* 图像宽
- nHeight* 图像高
- nColorCorrectionParam* 颜色校正值,可通过 GxI API 库中的 GX_INT_COLOR_CORRECTION_PARAM 码获取，也可以设置为 0
- pContrastLut* 对比度查找表，可以通过 DxGetContrastLut 函数计算，只需要计算一次，也可以设置为 NULL
- pGammaLut* Gamma 查找表，可以通过 DxGetGammatLut 函数计算，只需要计算一次，也可以设置为 NULL

nColorCorrectionParam、pContrastLut、pGammaLut 可以进行不同的组合，实现不同的效果，组合情况如表 8-2 所示：

编号	nColorCorrectionParam	pContrastLut	pGammaLut	效果
1	非 0	非 NULL	非 NULL	对图像进行颜色校正，对比度，Gamma 调节（此时图像质量最好）
2	非 0	NULL	非 NULL	对图像进行颜色校正，Gamma 调节

3	非 0	非 NULL	NULL	对图像进行颜色校正，对比度调节
4	0	非 NULL	非 NULL	对图像进行对比度，Gamma 调节
5	非 0	NULL	NULL	对图像进行颜色校正调节
6	0	非 NULL	NULL	对图像进对比度调节
7	0	NULL	非 NULL	对图像进行 Gamma 调节

表 8-2

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 DxStatus = DX_OK;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
int32_t i32LutLength = 0;
double dGammaParam = 0.0;
BYTE* pGammaLut;
BYTE* pContrastLut;

//获取对比度调节参数
GX_STATUS emStatus = GXGetInt (hDevice, GX_INT_CONTRAST_PARAM,
                                &i64ContrastParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取颜色校正调节参数
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &i64ColorCorrectionParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取 Gamma 调节参数
emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

do
{

```

```

//获取 Gamma 查找表的长度
VxInt32 DxStatus= DxGetGammaLut (dGammaParam , NULL,
                                &i32LutLength);

if (DxStatus != DX_OK)
{
    break;
}

//为 Gamma 查找表申请空间
pGammaLut = new BYTE[i32LutLength];
if (pGammaLut == NULL)
{
    DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
    break;
}

//计算 Gamma 查找表
DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
if (DxStatus != DX_OK)
{
    break;
}

//获取对比度查找表的长度
DxStatus= DxGetContrastLut((int)i64ContrastParam, NULL,
                           &i32LutLength);

if (DxStatus != DX_OK)
{
    break;
}

//为对比度查找表申请空间
pContrastLut = new BYTE[i32LutLength];
if (pContrastLut == NULL)
{
    DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
    break;
}

//计算对比度查找表
DxStatus = DxGetContrastLut((int)i64ContrastParam, pContrastLut,
                             &i32LutLength);

if (DxStatus != DX_OK)
{
    break;
}
}while(0);

//设置查找表失败，释放资源
if (DxStatus != DX_OK)

```

```

{
    if (pGammaLut != NULL)
    {
        delete[] pGammaLut;
        pGammaLut = NULL;
    }
    if (pContrastLut != NULL)
    {
        delete[] pContrastLut;
        pContrastLut = NULL;
    }
    return;
}

//输出图像 ARGB 数据
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
if (pOutputBuffer == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}
//提升图像的质量
// pInputBuffer 为输入图像数据
DxStatus = DxARGBImageImprovment(pInputBuffer, pOutputBuffer,
                                   i32Width, i32Height,
                                   i64ColorCorrectionParam,
                                   pContrastLut, pGammaLut);

if (pGammaLut != NULL)
{
    delete []pGammaLut;
    pGammaLut = NULL;
}
if (pContrastLut != NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}
}

```

8.4.24. DxImageImprovmentEx

声明：

**VxInt32 DHDECL DxImageImprovmentEx (void *pInputBuffer, void *pOutputBuffer,
VxUInt32 nWidth, VxUInt32 nHeight,
VxInt64 nColorCorrectionParam,
void *pContrastLut, void *pGammaLut ,
DX_RGB_CHANNEL_ORDER emChannelOrder);**

意义：

该函数对输入图像进行图像质量的提升，输入图像为 24 位图像，图像的通道顺序可选择为 RGB 或者 BGR。

形参：

- pInputBuffer* 指向原始图像数据缓冲区
- pOutputBuffer* 指向目标图像数据缓冲区
- nWidth* 图像宽
- nHeight* 图像高
- nColorCorrectionParam* 颜色校正值,可通过 GxI API 库中的 GX_INT_COLOR_CORRECTION_PARAM 码获取，也可以设置为 0
- pContrastLut* 对比度查找表，可以通过 DxGetContrastLut 函数计算，只需要计算一次，也可以设置为 NULL
- pGammaLut* Gamma 查找表，通过 DxGetGammatLut 函数计算，只需要计算一次，也可以设置为 NULL
- emChannelOrder* 输出图像的 RGB 通道顺序

其中，nColorCorrectionParam、pContrastLut、pGammaLut 可以进行不同的组合，实现不同的效果，组合情况如表 8-3 所示：

编号	nColorCorrectionParam	pContrastLut	pGammaLut	效果
1	非 0	非 NULL	非 NULL	对图像进行颜色校正，对比度，Gamma 调节（此时图像质量最好）
2	非 0	NULL	非 NULL	对图像进行颜色校正，Gamma 调节
3	非 0	非 NULL	NULL	对图像进行颜色校正，对比度调节
4	0	非 NULL	非 NULL	对图像进行对比度，Gamma 调节
5	非 0	NULL	NULL	对图像进行颜色校正调节
6	0	非 NULL	NULL	对图像进对比度调节
7	0	NULL	非 NULL	对图像进行 Gamma 调节

表 8-3

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 DxStatus = DX_OK;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
int32_t i32LutLength = 0;
```

```
double dGammaParam = 0.0;
BYTE* pGammaLut;
BYTE* pContrastLut;

//获取对比度调节参数
GX STATUS emStatus = GXGetInt (hDevice, GX_INT CONTRAST_PARAM,
                                &i64ContrastParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取颜色校正调节参数
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &i64ColorCorrectionParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

//获取 Gamma 调节参数
emStatus = GXGetFloat (hDevice, GX_FLOAT_GAMMA_PARAM, &dGammaParam);
if (emStatus != GX_STATUS_SUCCESS)
{
    return;
}

do
{
    //获取 Gamma 查找表的长度
    DxStatus= DxGetGammaLut(dGammaParam, NULL, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }

    //为 Gamma 查找表申请空间
    pGammaLut = new BYTE[i32LutLength];
    if (pGammaLut== NULL)
    {
        DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
        break;
    }

    //计算 Gamma 查找表
    DxStatus = DxGetGammaLut(dGammaParam, pGammaLut, &i32LutLength);
    if (DxStatus != DX_OK)
    {
        break;
    }
}
```

```

//获取对比度查找表的长度
DxStatus= DxGetContrastLut ((int)i64ContrastParam, NULL,
                             &i32LutLength);

if (DxStatus != DX_OK)
{
    break;
}

//为对比度查找表申请空间
pContrastLut = new BYTE[i32LutLength];
if (pContrastLut == NULL)
{
    DxStatus= DX_NOT_ENOUGH_SYSTEM_MEMORY;
    break;
}

//计算对比度查找表
DxStatus = DxGetContrastLut ((int)i64ContrastParam, pContrastLut,
                             &i32LutLength);

if (DxStatus != DX_OK)
{
    break;
}
}while(0);

//设置查找表失败，释放资源
if (DxStatus != DX_OK)
{
    if (pGammaLut != NULL)
    {
        delete[] pGammaLut;
        pGammaLut = NULL;
    }
    if(pContrastLut != NULL)
    {
        delete[] pContrastLut;
        pContrastLut = NULL;
    }
    return;
}

DX_RGB_CHANNEL_ORDER emChannelOrder = DX_ORDER_RGB;

//输出图像 ARGB 数据
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
if (pOutputBuffer == NULL)
{
    return;
}
else
{

```

```
//缓冲区初始化
memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}

//提升图像的质量
// pInputBuffer 为输入图像数据
DxStatus = DxImageImprovementEx(pInputBuffer, pOutputBuffer, i32Width,
                                i32Height, i64ColorCorrectionParam,
                                pContrastLut, pGammaLut, emChannelOrder);

if (pGammaLut != NULL)
{
    delete []pGammaLut;
    pGammaLut = NULL;
}

if (pContrastLut != NULL)
{
    delete []pContrastLut;
    pContrastLut = NULL;
}
```

8.4.25. DxImageMirror

声明：

```
VxInt32 DHDECL DxImageMirror (void *pInputBuffer, void *pOutputBuffer,
                              VxUInt32 nWidth, VxUInt32 nHeight,
                              DX_IMAGE_MIRROR_MODE emMirrorMode)
```

意义：

该函数为产生一个与原图像在水平方向或者垂直方向相对称的镜像图像，输入图像为 8 位的 Raw 图像或 8 位的黑白图像。

形参：

pInputBuffer 指向原始图像数据缓冲区

pOutputBuffer 指向目标图像数据缓冲区

nWidth 图像宽

nHeight 图像高

emMirrorMode 图像镜像翻转方式，HORIZONTAL_MIRROR 和 VERTICAL_MIRROR，即水平方向镜像翻转和垂直方向镜像翻转

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
```

```
//输出图像 ARGB 数据
BYTE * pOut8BitBuf = new BYTE[i32Width * i32Height];
if (pOut8BitBuf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOut8BitBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//图像水平镜像翻转（输入和输出不能为同一个 buffer）
VxInt32 DxStatus = DxImageMirror((BYTE*)pIn8BitBuf, (BYTE*)pOut8BitBuf,
                                i32Width, i32Height, HORIZONTAL_MIRROR);

if (DxStatus != DX_OK)
{
    return ;
}

//图像垂直镜像翻转（输入和输出不能为同一个 buffer）
//pIn8BitBuf 为输入图像数据
DxStatus = DxImageMirror((BYTE*)pIn8BitBuf, (BYTE*)pOut8BitBuf,
                        i32Width, i32Height, VERTICAL_MIRROR);

if (DxStatus != DX_OK)
{
    return ;
}
```

效果图：

如下图，图 8-14 为原始 8 位黑白图像，图 8-15 为水平镜像图像，图 8-16 为垂直镜像图像。

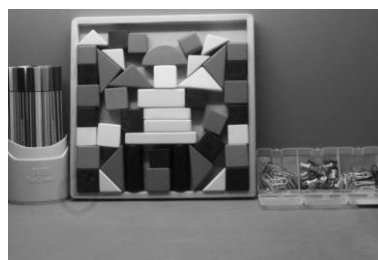


图 8-14 原始图像



图 8-15 水平镜像图像

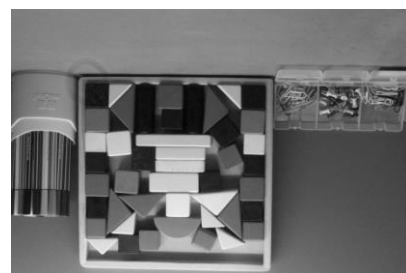


图 8-16 垂直镜像图像

8.4.26. DxImageMirror16B

声明：

```
VxInt32 DHDECL DxImageMirror16B (void *pInputBuffer, void *pOutputBuffer,
                                   VxUInt32 nWidth, VxUInt32 nHeight,
                                   DX_IMAGE_MIRROR_MODE emMirrorMode)
```

意义：

该函数为产生一个与原图像在水平方向或者垂直方向相对称的镜像图像，输入图像为 16 位的 Raw 图像或 16 位的黑白图像。

形参：

<i>pInputBuffer</i>	指向原始图像数据缓冲区
<i>pOutputBuffer</i>	指向目标图像数据缓冲区
<i>nWidth</i>	图像宽
<i>nHeight</i>	图像高
<i>emMirrorMode</i>	图像镜像翻转方式，HORIZONTAL_MIRROR 和 VERTICAL_MIRROR，即水平方向镜像翻转和垂直方向镜像翻转

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;

//输出图像 ARGB 数据
BYTE * pOut8BitBuf = new BYTE[i32Width * i32Height * 2];
if (pOut8BitBuf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOut8BitBuf, 0, i32Width * i32Height * sizeof(BYTE) * 2);
}

//图像水平镜像翻转（输入和输出不能为同一个 buffer）
VxInt32 DxStatus = DxImageMirror16B((BYTE*)pIn16BitBuf, (BYTE*)pOut16BitBuf,
                                     i32Width, i32Height, HORIZONTAL_MIRROR);

if (DxStatus != DX_OK)
{
    return ;
}

//图像垂直镜像翻转（输入和输出不能为同一个 buffer）
//pIn16BitBuf 为输入图像数据
```

```
DxStatus = DxImageMirror16B((BYTE*)pIn16BitBuf, (BYTE*)pOut16BitBuf,
                             i32Width, i32Height, VERTICAL_MIRROR);

if (DxStatus != DX_OK)
{
    return ;
}
```

8.4.27. DxGetLut

声明：

VxInt32 DHDECL DxGetLut (VxInt32 nContrastParam, double dGamma, VxInt32 nLightness, VxUInt8 *pLut, VxUInt16 *pLutLength);

意义：

该函数为计算图像处理 8 位查找表。

形参：

nContrastParam 对比度调节参数，范围：-50~100

nGammaGamma 调节参数，范围：0.1~10

nLightness 亮度调节参数，范围：-150~150

pLut 查找表，当对比度、Gamma 或者亮度参数改变时，需要重新计算

pLutLength 对查找表长度，单位是字节

如果用户输入的长度值不等于当前实际的长度值，返回错误，并且 *pLutLength* 返回实际的长度值

如果用户输入的 *pLut* 为 NULL，返回成功，*pLutLength* 返回实际的长度值

如果操作成功，*pLutLength* 返回实际的长度值

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 i32ContrastParam = 0;
VxUInt16 ui16LutLength = 0;
int32_t i32Lightness = 0;
double dGamma = 0.0;

//获取查找表的长度
VxInt32 DxStatus = DxGetLut (i32ContrastParam, dGamma, i32Lightness,
                             NULL, &ui16LutLength);

if (DxStatus != DX_OK)
{
    return;
}

//为查找表申请空间
VxUInt8* pLut = new VxUInt8[ui16LutLength];
if (pLut == NULL)
{
    return;
}
```

```
//计算查找表
DxStatus = DxGetLut(i32ContrastParam, dGamma, i32Lightness, pLut,
                   &ui16LutLength);
if (DxStatus != DX_OK)
{
    if (pLut!= NULL)
    {
        delete []pLut;
        pLut = NULL;
    }
    return;
}
//图像处理
//.....
if (pLut!= NULL)
{
    delete []pLut;
    pLut = NULL;
}
```

8.4.28. DxCalcCCParam

声明：

**VxInt32 DHDECL DxCalcCCParam (VxInt64 nColorCorrectionParam, VxInt16nSaturation,
VxInt16 *parrCC, VxUInt8nLength);**

意义：

该函数为计算图像处理色彩调节数组。

形参：

nColorCorrectionParam 颜色校正值，可以通过 GxI API 库中的

GX_INT_COLOR_CORRECTION_PARAM 码获取，也可以设置为 0

nSaturation 饱和度调节参数，范围：0~128

parrCC 数组地址

nLength 数组长度 (sizeof(VxInt16 * 9))

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//为图像色彩调节数组申请空间
VxInt64 nColorCorrectionParam = 0;
VxInt16 nSaturation = 0;
VxInt16* parrCC = new VxInt16[sizeof (VxInt16) * 9];
if (parrCC== NULL)
{
    return;
}
```



```
//获取颜色校正系数
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &nColorCorrectionParam);

//计算图像彩色调节数组
DxStatus = DxCalcCCParam(nColorCorrectionParam, nSaturation, parrCC,
                        sizeof (VxInt16) * 9);
if (DxStatus != DX_OK)
{
    if (parrCC!= NULL)
    {
        delete []parrCC;
        parrCC= NULL;
    }
    return;
}

//图像处理
//.....

if (parrCC!= NULL)
{
    delete []parrCC;
    parrCC= NULL;
}
```

8.4.29. DxRaw8ImgProcess

声明：

```
VxInt32 DHDECL DxRaw8ImgProcess (void *pRaw8Buf, void *pRgbBuf,
                                VxInt32 nWidth, VxInt32 nHeight,
                                COLOR_IMG_PROCESS *pstClrImageProc);
```

意义：

该函数为对 Raw8 图像进行处理。

形参：

pRaw8Buf 指向原始图像 8 位数据缓冲区

pRgbBuf 指向目标图像数据缓冲区（RGB 数据），开辟大小为图像宽 x 图像高 x3

nWidth 图像宽

nHeight 图像高。

注意：如果当前 COLOR_IMG_PROCESS::bAccelerate 设置为 true，要使能加速，此时必须要求 nHeight 是 4 的整倍数

*pstClrImageProc*彩色图像处理方法结构体指针，定义如下：

```
typedef struct COLOR_IMG_PROCESS
{
    bool        bDefectivePixelCorrect; ///坏点校正开关
```

bool	bDenoise;	/// 降噪开关
bool	bSharpness;	/// 锐化开关
bool	bAccelerate;	/// 加速开关
VxInt16	*parrCC;	/// 色彩处理参数数组地址
VxUInt8	nCCBufLength;	/// parrCC长度 (sizeof(VInt16)*9)
float	fSharpFactor;	/// 锐化强度因子
VxUInt8	*pProLut;	/// 查找表Buffer
VxUInt16	nLutLength;	/// 查找表长度
DX_BAYER_CONVERT_TYPE	cvType;	/// 插值方法
DX_PIXEL_COLOR_FILTER	emLayOut;	/// BAYER格式
bool	bFlip;	/// 翻转标志
VxUInt8	arrReserved[32];	/// 保留32字节

} COLOR_IMG_PROCESS;

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 i32ContrastParam = 0;
VxUInt16 ui16LutLength = 0;
int32_t i32Lightness = 0;
double dGamma = 0.0;
float fSharpen = 0.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int64_t i64ContrastParam = 0;
int64_t i64ColorCorrectionParam = 0;
BYTE* pContrastLut;
int32_t nPixelColorFilter = 0;
VxInt16 nSaturation = 0;
//彩色图像处理功能设置结构体参数初始化
COLOR_IMG_PROCESS stClrImageProc;

stClrImageProc.bAccelerate = false;
stClrImageProc.bDefectivePixelCorrect = false;
stClrImageProc.bDenoise = false;
stClrImageProc.bFlip = true;
stClrImageProc.bSharpness = false;
stClrImageProc.fSharpFactor = fSharpen;
stClrImageProc.cvType = RAW2RGB_NEIGHBOUR;
stClrImageProc.emLayOut = (DX_PIXEL_COLOR_FILTER)nPixelColorFilter;

//获取查找表长度
VxInt32 DxStatus = DxGetLut (i32ContrastParam,dGamma,i32Lightness,
                             NULL,&stClrImageProc.nLutLength);

if (DxStatus != DX_OK)
```

```

{
    return;
}

//为查找表申请空间
stClrImageProc.pProLut = new VxUInt8[stClrImageProc.nLutLength];
if (pContrastLut == NULL)
{
    return;
}

//计算查找表
DxStatus = DxGetLut(i32ContrastParam, dGamma, i32Lightness,
                    stClrImageProc.pProLut, &stClrImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    if (stClrImageProc.pProLut != NULL)
    {
        delete []stClrImageProc.pProLut;
        stClrImageProc.pProLut = NULL;
    }
    return;
}

//为图像色彩调节数组申请空间
stClrImageProc.nCCBufLength = sizeof (VxInt16)*9;
stClrImageProc.parrCC = new VxInt16[stClrImageProc.nCCBufLength];
if (stClrImageProc.parrCC == NULL)
{
    if (stClrImageProc.pProLut != NULL)
    {
        delete []stClrImageProc.pProLut;
        stClrImageProc.pProLut = NULL;
    }

    return;
}

//获取颜色校正系数
emStatus = GXGetInt (hDevice, GX_INT_COLOR_CORRECTION_PARAM,
                    &i64ColorCorrectionParam);

//计算图像彩色调节数组
DxStatus = DxCalcCCParam(i64ColorCorrectionParam, nSaturation,
                        stClrImageProc.parrCC, stClrImageProc.nCCBufLength);
if (DxStatus != DX_OK)
{
    if (stClrImageProc.pProLut != NULL)
    {
        delete []stClrImageProc.pProLut;
        stClrImageProc.pProLut = NULL;
    }
}

```

```

    }

    if (stClrImageProc.parrCC!= NULL)
    {
        delete []stClrImageProc.parrCC;
        stClrImageProc.parrCC= NULL;
    }
    return;
}

BYTE * pRgbBuf = new BYTE[i32Width * i32Height]; //输出图像 ARGB 数据
if (pRgbBuf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pRgbBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//对 Raw8 图像进行处理
// pRaw8Buf 为输入图像数据
emStatus = DxRaw8ImgProcess(pRaw8Buf,pRgbBuf, i32Width, i32Height,
                             &stClrImageProc);

if (stClrImageProc.pProLut!= NULL)
{
    delete []stClrImageProc.pProLut;
    stClrImageProc.pProLut = NULL;
}
if (stClrImageProc.parrCC!= NULL)
{
    delete []stClrImageProc.parrCC;
    stClrImageProc.parrCC= NULL;
}

```

8.4.30. DxMono8ImgProcess

声明：

```

VxInt32 DHDECL DxMono8ImgProcess(void *pInputBuf, void *pOutputBuf,
                                   VxUInt32 nWidth, VxUInt32 nHeight,
                                   MONO_IMG_PROCESS *pstGrayImgProc);

```

意义：

对 Mono8 图像进行处理。

形参：

pInputBuf 指向原始图像 8 位数据缓冲区

pOutputBuf 指向目标图像数据缓冲区，开辟大小为图像宽 x 图像高

nWidth 图像宽

nHeight 图像高。

注意：如果当前 MONO_IMG_PROCESS::bAccelerate 设置为 true，要能使加速，此时必须要求 *nHeight* 是 4 的整倍数。

pstGrayImgProc 灰度图像处理方法结构体指针，定义如下：

```
Typedef struct MONO_IMG_PROCESS
{
    bool        bDefectivePixelCorrect; ///<坏点校正开关
    bool        bSharpness;           ///<锐化开关
    bool        bAccelerate;          ///<加速开关
    float        fSharpFactor;        ///<锐化强度因子
    VxUInt8      *pProLut;            ///<查找表Buffer
    VxUInt16     nLutLength;          ///<查找表长度
    VxUInt8      arrReserved[32];     ///<保留32字节
} MONO_IMG_PROCESS;
```

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 i32ContrastParam = 0;
int32_t i32Lightness = 0;
double dGamma = 0.0;
float fSharpen = 0.0;
int32_t i32Width = 1600;
int32_t i32Height = 1234;
BYTE* pContrastLut;

//Mono8 图像处理功能设置结构体参数初始化
MONO_IMG_PROCESS stGrayImageProc;
stGrayImageProc.bAccelerate= false;
stGrayImageProc.bDefectivePixelCorrect= false;
stGrayImageProc.bSharpness          = false;
stGrayImageProc.fSharpFactor= fSharpen;

//获取查找表长度
VxInt32 DxStatus=DxGetLut (i32ContrastParam,dGamma,i32Lightness,
                           NULL,&stGrayImageProc.nLutLength);

if (DxStatus != DX_OK)
{
    return;
}

//为查找表申请空间
stGrayImageProc.pProLut = new VxUInt8[stGrayImageProc.nLutLength];
if (pContrastLut == NULL)
```

```

{
    return;
}

//计算查找表
DxStatus = DxGetLut(i32ContrastParam, dGamma, i32Lightness,
                    stGrayImageProc.pProLut, &stGrayImageProc.nLutLength);
if (DxStatus != DX_OK)
{
    if (stGrayImageProc.pProLut!= NULL)
    {
        delete []stGrayImageProc.pProLut;
        stGrayImageProc.pProLut = NULL;
    }
    return;
}

BYTE * pOutputBuf = new BYTE[i32Width * i32Height]; //输出图像 ARGB 数据
if (pOutputBuf == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOutputBuf, 0, i32Width * i32Height * sizeof(BYTE));
}

//对 Mono8 图像进行处理
//pInputBuf 为输入图像数据
emStatus = DxMono8ImgProcess(pInputBuf, pOutputBuf, i32Width, i32Height,
                             &stGrayImageProc);

if (stGrayImageProc.pProLut!= NULL)
{
    delete []stGrayImageProc.pProLut;
    stGrayImageProc.pProLut = NULL;
}

```

8.4.31. DxGetFFCCoefficients

声明：

```

VxInt32 DHDECL DxGetFFCCoefficients
(FLAT_FIELD_CORRECTION_PROCESS
 stFlatFieldCorrection, void *pFFCCoefficients, int *pnLength,
 int *pnTargetValue = NULL);

```

意义：

该函数为计算图像平场校正系数，仅支持 8~12 位 Raw 图。

形参：

stFlatFieldCorrection 平场校正数据结构体

pFFCCoefficients 平场校正系数
pnlength 平场校正系数长度，单位是字节
pnTargetValue 平场校正期望灰度值，默认为NULL

如果用户输入的长度值小于当前实际的长度值，返回错误，并且 *pnLength* 返回实际的长度值

如果用户输入的 *pFFCCoefficients* 为 NULL，返回成功，*pnLength* 返回实际的长度值

如果操作成功，*pnLength* 返回实际的长度值

平场校正数据结构体定义如下：

```
typedef struct FLAT_FIELD_CORRECTION_PROCESS
{
    void                *pBrightBuf;    ///<亮场图像 Buffer
    void                *pDarkBuf;      ///<暗场图像 Buffer
    VxUInt32            nImgWid;        ///<图像宽
    VxUInt32            nImgHei;        ///<图像高
    DX_ACTUAL_BITS      nActualBits;    ///<实际位深
    DX_PIXEL_COLOR_FILTER emBayerType;  ///

```

注意：当计算平场校正系数时，可以不采集暗场图像，此时应将指向暗场图像的指针置为 NULL。

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int32_t i32FFClength = 0;
//平场校正数据结构体参数初始化
//pImgBrightBuf 为亮场图像数据
//pImgDarkBuf 为暗场图像数据
FLAT_FIELD_CORRECTION_PROCESS stFlatFieldCorrection;
stFlatFieldCorrection.pBrightBuf = pImgBrightBuf;
stFlatFieldCorrection.pDarkBuf = pImgDarkBuf;    //若不采集暗场，将其置为 NULL
stFlatFieldCorrection.nImgWid = i32Width;
stFlatFieldCorrection.nImgHei = i32Height;
stFlatFieldCorrection.nActualBits = DX_ACTUAL_BITS_8;
stFlatFieldCorrection.emBayerType = BAYERRG;

//获取平场校正系数的长度
VxInt32 DxStatus= DxGetFFCCoefficients (stFlatFieldCorrection, NULL,
&i32FFClength);

if(DxStatus != DX_OK)
{
    return;
```

```

}

//为平场校正系数申请空间
BYTE* pFFCCoefficients = new BYTE[i32FFClength];
if (pFFCCoefficients == NULL)
{
    return;
}

//计算平场校正系数
DxStatus = DxGetFFCCoefficients (stFlatFieldCorrection, pFFCCoefficients,
                                &i32FFClength);

if (DxStatus != DX_OK)
{
    if (pFFCCoefficients!= NULL)
    {
        delete []pFFCCoefficients;
        pFFCCoefficients = NULL;
    }
    return;
}

//图像处理
//.....

if (pFFCCoefficients!= NULL)
{
    delete []pFFCCoefficients;
    pFFCCoefficients = NULL;
}

```

8.4.32. DxFlatFieldCorrection

声明：

```

VxInt32 DHDECL DxFlatFieldCorrection (void *pInputBuffer, void *pOutputBuffer,
                                      DX_ACTUAL_BITS nActualBits,
                                      VxUInt32 nImgWidth, VxUInt32 nImgHeight,
                                      void *pFFCCoefficients, int *pnLength);

```

意义：

该函数对图像进行平场校正操作。输入图像为 8~12 位的 Raw 图，且该函数支持 Bayer 格式的 Raw 图像，不支持 Packed 格式的 Raw 图。

形参：

<i>pInputBuffer</i>	指向输入图像数据缓冲区
<i>pOutputBuffer</i>	指向目标图像数据缓冲区
<i>nActualBits</i>	实际位深
<i>nImgWidth</i>	图像宽

nImgHeight 图像高
pFFCCoefficients 平场校正系数
pnLength 平场校正系数长度，单位是字节

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
int32_t i32Width = 1600;
int32_t i32Height = 1234;
int32_t i32FFCLength = 0;
DX_ACTUAL_BITS nActualBits = DX_ACTUAL_BITS_8;

//输出图像 ARGB 数据
BYTE * pOutputBuffer = new BYTE[i32Width * i32Height];
if (pOutputBuffer == NULL)
{
    return;
}
else
{
    //缓冲区初始化
    memset(pOutputBuffer, 0, i32Width * i32Height * sizeof(BYTE));
}

//对图像进行平场校正操作
//pInputBuffer 为输入图像数据
//pFFCCoefficients 为平场校正系数数据
VxInt32 DxStatus = DxFlatFieldCorrection (pInputBuffer, pOutputBuffer,
                                          nActualBits, i32Width,
                                          i32ImgHeight, pFFCCoefficients,
                                          &i32FFCLength);

if (DxStatus != DX_OK)
{
    if (pFFCCoefficients != NULL)
    {
        delete []pFFCCoefficients;
        pFFCCoefficients = NULL;
    }
    return;
}

//对 Raw 图像处理
//.....
```

8.4.33. DxCalcUserSetCCParam

声明：

VxInt32 DHDECL DxCalcUserSetCCParam(COLOR_TRANSFORM_FACTOR

***pstColorTransformFactor, VxInt16 nSaturation,
VxInt16 *parrCC, VxUInt8 nLength);**

意义：

该函数为根据用户设置计算图像处理色彩调节数组。

形参：

pstColorTransformFactor 指向颜色校正/颜色转换结构体指针，可设为 NULL（不进行颜色校正）
nSaturation 饱和度调节参数，范围：0~128
parrCC 数组地址
nLength 数组长度（sizeof(VxInt16 * 9)）

注意：颜色校正结构体参数建议范围-4~4，如果参数设置小于-4，校正效果与-4一致，如果参数设置大于4，校正效果与4一致。

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//为图像色彩调节数组申请空间
VxInt16* parrCC = new VxInt16[sizeof (VxInt16) * 9];
if (parrCC== NULL)
{
    return;
}

//设置颜色校正系数
COLOR_TRANSFORM_FACTOR stColorTransformFactor;

stColorTransformFactor.fGain00 = 1.463680;
stColorTransformFactor.fGain01 = -0.545809;
stColorTransformFactor.fGain02 = 0.082128;
stColorTransformFactor.fGain10 = -0.229388;
stColorTransformFactor.fGain11 = 1.354321;
stColorTransformFactor.fGain12 = -0.124932;
stColorTransformFactor.fGain20 = 0.097042;
stColorTransformFactor.fGain21 = -0.627857;
stColorTransformFactor.fGain22 = 1.530815;

//设置饱和度系数
VxInt16 nSaturation = 64;
VxInt32 DxStatus = DxCalcUserSetCCParam(&stColorTransformFactor,
                                         nSaturation, parrCC,
                                         sizeof (VxInt16 ) * 9);

if (DxStatus != DX_OK)
{
    if (parrCC!= NULL)
    {
```

```
        delete []parrCC;
        parrCC= NULL;
    }
    return;
}

//图像处理
//.....

if (parrCC!= NULL)
{
    delete []parrCC;
    parrCC= NULL;
}
```

8.4.34. DxImageFormatConvert

声明：

```
VxInt32 DHDECL DxImageFormatConvert (DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
                                     void *pInputBuffer, int nInBufferSize,
                                     void *pOutputBuffer, int nOutBufferSize,
                                     GX_PIXEL_FORMAT_ENTRY emInPixelFormat,
                                     VxUInt32 nImgWidth, VxUInt32  nImgHeight,
                                     bool bFlip);
```

意义：

该函数为根据用户设置的输入图像格式与输出图像格式进行图像格式转换。

形参：

<i>handle</i>	图像格式转换句柄，不可为 NULL
<i>pInputBuffer</i>	输入图像 buffer，建议为 unsigned char 类型
<i>nInBufferSize</i>	输入图像 buffer 内存大小（以 unsigned char 类型计算）
<i>pOutputBuffer</i>	输出图像buffer，建议为unsigned char类型
<i>nOutBufferSize</i>	输出图像 buffer 内存大小（以 unsigned char 类型计算）
<i>emInPixelFormat</i>	输入图像格式
<i>nImgWidth</i>	图像宽度
<i>nImgHeight</i>	图像高度
<i>bFlip</i>	输出图像垂直镜像标志位，false：不镜像，true：镜像

支持的输入输出：

	源图像格式	目的图像格式
Bayer 8bit	GX_PIXEL_FORMAT_BAYER_GR8	GX_PIXEL_FORMAT_MONO8
	GX_PIXEL_FORMAT_BAYER_RG8	GX_PIXEL_FORMAT_RGB8
	GX_PIXEL_FORMAT_BAYER_GB8	GX_PIXEL_FORMAT_BGR8
	GX_PIXEL_FORMAT_BAYER_RGBA8	GX_PIXEL_FORMAT_RGBA8
	GX_PIXEL_FORMAT_BAYER_BGRA8	GX_PIXEL_FORMAT_BGRA8
	GX_PIXEL_FORMAT_BAYER_ARGB8	GX_PIXEL_FORMAT_ARGB8

		GX_PIXEL_FORMAT_ABGR8 GX_PIXEL_FORMAT_RGB8_PLANAR
Bayer 16bit	GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16 GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16	GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_RGB16_PLANAR
RGB 8bit	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8	GX_PIXEL_FORMAT_YUV444_8 GX_PIXEL_FORMAT_YUV422_8 GX_PIXEL_FORMAT_YUV411_8 GX_PIXEL_FORMAT_YUV420_8_PLANAR GX_PIXEL_FORMAT_YCBCR444_8 GX_PIXEL_FORMAT_YCBCR422_8 GX_PIXEL_FORMAT_YCBCR411_8 GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_RGB8
RGB 16bit	GX_PIXEL_FORMAT_RGB16 GX_PIXEL_FORMAT_BGR16	GX_PIXEL_FORMAT_MONO16
COORD3D_C 16	GX_PIXEL_FORMAT_COORD3D_C16	GX_PIXEL_FORMAT_COORD3D_ABC32F GX_PIXEL_FORMAT_COORD3D_ABC32F_PLANAR
Mono Packed (GVSP)	GX_PIXEL_FORMAT_MONO10_PACKED GX_PIXEL_FORMAT_MONO12_PACKED GX_PIXEL_FORMAT_MONO10_P GX_PIXEL_FORMAT_MONO12_P GX_PIXEL_FORMAT_MONO14_P	GX_PIXEL_FORMAT_MONO8 GX_PIXEL_FORMAT_MONO10 GX_PIXEL_FORMAT_MONO12 GX_PIXEL_FORMAT_MONO14 GX_PIXEL_FORMAT_MONO16 GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8
Bayer Packed (GVSP)	GX_PIXEL_FORMAT_BAYER_GR10_PACKED GX_PIXEL_FORMAT_BAYER_RG10_PACKED GX_PIXEL_FORMAT_BAYER_GB10_PACKED GX_PIXEL_FORMAT_BAYER_BG10_PACKED GX_PIXEL_FORMAT_BAYER_GR12_PACKED GX_PIXEL_FORMAT_BAYER_RG12_PACKED GX_PIXEL_FORMAT_BAYER_GB12_PACKED GX_PIXEL_FORMAT_BAYER_BG12_PACKED GX_PIXEL_FORMAT_BAYER_GR10_P GX_PIXEL_FORMAT_BAYER_RG10_P GX_PIXEL_FORMAT_BAYER_GB10_P GX_PIXEL_FORMAT_BAYER_BG10_P GX_PIXEL_FORMAT_BAYER_GR12_P GX_PIXEL_FORMAT_BAYER_RG12_P GX_PIXEL_FORMAT_BAYER_GB12_P GX_PIXEL_FORMAT_BAYER_BG12_P GX_PIXEL_FORMAT_BAYER_GR14_P GX_PIXEL_FORMAT_BAYER_RG14_P GX_PIXEL_FORMAT_BAYER_GB14_P GX_PIXEL_FORMAT_BAYER_BG14_P	GX_PIXEL_FORMAT_RGB8 GX_PIXEL_FORMAT_BGR8 GX_PIXEL_FORMAT_BAYER_GR8 GX_PIXEL_FORMAT_BAYER_RG8 GX_PIXEL_FORMAT_BAYER_GB8 GX_PIXEL_FORMAT_BAYER_BG8 GX_PIXEL_FORMAT_BAYER_GR10 GX_PIXEL_FORMAT_BAYER_RG10 GX_PIXEL_FORMAT_BAYER_GB10 GX_PIXEL_FORMAT_BAYER_BG10 GX_PIXEL_FORMAT_BAYER_GR12 GX_PIXEL_FORMAT_BAYER_RG12 GX_PIXEL_FORMAT_BAYER_GB12 GX_PIXEL_FORMAT_BAYER_BG12 GX_PIXEL_FORMAT_BAYER_GR14 GX_PIXEL_FORMAT_BAYER_RG14 GX_PIXEL_FORMAT_BAYER_GB14 GX_PIXEL_FORMAT_BAYER_BG14 GX_PIXEL_FORMAT_BAYER_GR16 GX_PIXEL_FORMAT_BAYER_RG16

		GX_PIXEL_FORMAT_BAYER_GB16 GX_PIXEL_FORMAT_BAYER_BG16
--	--	--

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

GX_PIXEL_FORMAT_ENTRY emOutPixelFormat = GX_PIXEL_FORMAT_RGBA8;
GX_PIXEL_FORMAT_ENTRY emInPixelFormat = GX_PIXEL_FORMAT_BAYER_RG8;

VxUInt8 nAlphaValue = 128;
DX_BAYER_CONVERT_TYPE cvtype= RAW2RGB_NEIGHBOUR;
int nBufferSizeIn = 0;
int nBufferSizeOut = 0;
int nWidth = 720;
int nHeight = 540;
unsigned char* pszBufferIn = NULL;
unsigned char* pszBufferOut = NULL;
bool bFlip = false;

//设置图像输出格式
emDxStatus = (DX_STATUS)DxImageFormatConvertSetOutputPixelFormat(handle,
                                                                    emOutPixelFormat);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//设置 alpha 通道值
emDxStatus = (DX_STATUS)DxImageFormatConvertSetAlphaValue(handle,
                                                             nAlphaValue);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}
    
```

//设置图像插值方法

```
emDxStatus = (DX_STATUS) DxImageFormatConvertSetInterpolationType (
                                                                    handle, cvtype);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;
    return;
}
```

//获取输出图像内存大小

```
emDxStatus = (DX_STATUS) DxImageFormatConvertGetBufferSizeForConversion (
                                                                    handle, emOutPixelFormat,
                                                                    nWidth,nHeight, &nBufferSizeOut);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;
    return;
}
```

//获取输入图像内存大小

```
emDxStatus = (DX_STATUS) DxImageFormatConvertGetBufferSizeForConversion (
                                                                    handle, emInPixelFormat,
                                                                    nWidth,nHeight, &nBufferSizeIn);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;
    return;
}
```

//申请内存

```
pszBufferIn = new unsigned char [nBufferSizeIn];
if (pszBufferIn == NULL)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;
    return;
}

pszBufferOut = new unsigned char [nBufferSizeOut];
if (pszBufferOut == NULL)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;

    delete pszBufferIn;
    pszBufferIn = NULL;
    return;
}
```

```
//执行图像格式转换
```

```
emDxStatus = (DX_STATUS)DxImageFormatConvert(handle, pszBufferIn ,
                                              nBufferSizeIn , pszBufferOut,
                                              nBufferSizeOut, emInPixelFormat,
                                              nWidth,nHeight,bFlip);

DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.35. DxImageFormatConvertCreate

声明：

```
VxInt32 DHDECL DxImageFormatConvertCreate (DX_IMAGE_FORMAT_CONVERT_HANDLE
                                           *phandle);
```

意义：

该函数为用户创建图像格式转换句柄。

形参：

phandle 图像格式转换句柄

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
```

```
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;
```

```
//创建图像格式转换句柄
```

```
DxStatus = (DX_STATUS)DxImageFormatConvertCreate(&handle);
```

8.4.36. DxImageFormatConvertDestroy

声明：

```
VxInt32 DHDECL DxImageFormatConvertDestroy (DX_IMAGE_FORMAT_CONVERT_HANDLE
                                           handle);
```

意义：

该函数为用户销毁图像格式转换句柄。

形参：

handle 图像格式转换句柄

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
```

```
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;
```

```
//创建图像格式转换句柄
emDxStatus = (DX_STATUS) DxImageFormatConvertCreate (&handle);
if (handle == NULL)
{
    return;
}

//销毁图像格式转换句柄
emDxStatus = (DX_STATUS) DxImageFormatConvertDestroy (handle);
```

8.4.37. DxImageFormatConvertSetOutputPixelFormat

声明：

```
VxInt32 DHDECL DxImageFormatConvertSetOutputPixelFormat(
                                DX_IMAGE_FORMAT_CONVERT_HANDLE handle
                                GX_PIXEL_FORMAT_ENTRY emPixelFormat);
```

意义：

该函数为用户设置输出图像格式。

形参：

<i>handle</i>	图像格式转换句柄
<i>emPixelFormat</i>	输出图像格式，详见 GX_PIXEL_FORMAT_ENTRY 枚举变量

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate (&handle);
if (handle == NULL)
{
    return;
}

GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_MONO8;

//设置图像输出格式
emDxStatus = (DX_STATUS) DxImageFormatConvertSetOutputPixelFormat (handle,
                                                                    emPixelFormat);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy (handle);
    handle = NULL;
    return;
}
```



```
//其他操作
// .....

DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.38. DxImageFormatConvertSetAlphaValue

声明：

```
VxInt32 DHDECL DxImageFormatConvertSetAlphaValue (
                                DX_IMAGE_FORMAT_CONVERT_HANDLE handle
                                VxUInt8 nAlphaValue);
```

意义：

该函数为用户设置带有 alpha 通道图像的 alpha 值。

形参：

<i>handle</i>	图像格式转换句柄
<i>nAlphaValue</i>	输出图像带有 Alpha 通道格式（例如 RGBA 格式），可以设置 alpha 通道值，不设置的话，默认值为 255

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

VxUInt8 nAlphaValue = 128;

//设置图像 Alpha 值
emDxStatus = (DX_STATUS) DxImageFormatConvertSetAlphaValue(handle,
                                                            nAlphaValue);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//其他操作
```

```
//.....
```

```
DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.39. DxImageFormatConvertSetInterpolationType

声明：

```
VxInt32 DHDECL DxImageFormatConvertSetInterpolationType (
                                DX_IMAGE_FORMAT_CONVERT_HANDLE handle
                                DX_BAYER_CONVERT_TYPE emCvtType);
```

意义：

该函数为用户设置 Bayer 转 RGB 格式的插值方法。

形参：

<i>handle</i>	图像格式转换句柄
<i>emCvtType</i>	输入图像格式为 Bayer 格式，输出图像为 RGB 格式时，需要设置插值方法，如果不设置，默认为最近邻插值方法

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

//选择插值算法
DX_BAYER_CONVERT_TYPE cvtype= RAW2RGB_NEIGHBOUR;

//设置图像插值方法
emDxStatus = (DX_STATUS) DxImageFormatConvertSetInterpolationType (
                                                    handle, cvtype);
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//其他操作
```

```
// .....
DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.40. DxImageFormatConvertGetBufferSizeForConversion

声明：

```
VxInt32 DHDECL DxImageFormatConvertGetBufferSizeForConversion (
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
    GX_PIXEL_FORMAT_ENTRY emPixelFormat,
    VxUInt32 nImgWidth, VxUInt32 nImgHeight,
    int *pBufferSize);
```

意义：

该函数为用户获取已知图像宽高和像素格式的图像大小。

形参：

<i>handle</i>	图像格式转换句柄
<i>emPixelFormat</i>	图像格式
<i>nImgWidth</i>	图像宽度
<i>nImgHeight</i>	图像高度
<i>pBufferSize</i>	输出图像内存大小，以 unsigned char 类型计算

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

int nBufferSize = 0;
int nWidth      = 720;
int nHeight     = 540;
GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_MONO8;

//获取图像内存大小
emDxStatus =(DX_STATUS) DxImageFormatConvertGetBufferSizeForConversion (handle,
                                                                           emPixelFormat,
                                                                           nWidth,nHeight,
                                                                           &nBufferSize);

if (emDxStatus != DX_OK)
```

```

{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//其他操作
// .....
DxImageFormatConvertDestroy(handle);
handle = NULL;

```

8.4.41. DxImageFormatConvertGetOutputPixelFormat

声明：

```

VxInt32 DHDECL DxImageFormatConvertGetOutputPixelFormat (
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
    GX_PIXEL_FORMAT_ENTRY *pemPixelFormat);

```

意义：

该函数为用户获取输出图像格式。

形参：

<i>handle</i>	图像格式转换句柄
<i>pemPixelFormat</i>	输出图像格式

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

GX_PIXEL_FORMAT_ENTRY emPixelFormat = GX_PIXEL_FORMAT_UNDEFINED;

//获取图像输出格式
emDxStatus = (DX_STATUS)DxImageFormatConvertGetOutputPixelFormat (handle,
                                                                    &emPixelFormat);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

```

```

}

//其他操作
// .....

DxImageFormatConvertDestroy(handle);
handle = NULL;

```

8.4.42. DxImageFormatConvertSetValidBits

声明：

```

VxInt32 DHDECL DxImageFormatConvertSetValidBits(
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
    DX_VALID_BIT emValidBits);

```

意义：

该函数为用户设置 10、12、14、16 位深数据转 8 位数据时，设置有效位数。默认是低 8 位。

形参：

handle 图像格式转换句柄
emValidBits 有效位数，默认是低 8 位。

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;
DX_STATUS emDxStatus = DX_OK;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

DX_VALID_BIT emValidBits = DX_BIT_2_9;

//设置图像 Alpha 值
emDxStatus = (DX_STATUS) DxImageFormatConvertSetValidBits(handle,
    emValidBits);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//其他操作

```

```
//.....

DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.43. DxImageFormatConvertSet3DCalibParam

声明：

```
VxInt32 DHDECL DxImageFormatConvertSet3DCalibParam(
    DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
    const void* pCalibParamBuffer, const int nBufferSize);
```

意义：

该函数用于给转换句柄加载 3D 标定参数，以便于调用图像格式转换接口将高度图转换为点云图。

形参：

<i>handle</i>	图像格式转换句柄
<i>pCalibParamBuffer</i>	3D 标定参数内存
<i>nBufferSize</i>	3D 标定参数内存长度

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
#include <iostream>
#include <fstream>
//定义工具函数：从 bin 文件中读取 buffer
DX_STATUS LoadCalibFile(const char* strFileName,
    char*&pBuffer, int& nBufferLen)
{
    std::ifstream file(strFileName, std::ios::binary);
    if (!file)
    {
        return DX_FILE_OPEN_FAIL;
    }

    //获取文件长度
    file.seekg(0, std::ios::end);
    std::streampos fileSize = file.tellg();
    file.seekg(0, std::ios::beg);
    nBufferLen = fileSize;

    //创建缓冲区并读取文件内容
    pBuffer = new char[nBufferLen];
    file.read(pBuffer, nBufferLen);
    //关闭文件
    file.close();
    return DX_OK;
}
GX_PIXEL_FORMAT_ENTRY emOutPixelFormat =
    GX_PIXEL_FORMAT_COORD3D_ABC32F;
```

```

GX_PIXEL_FORMAT_ENTRY emInPixelFormat =
                                                                    GX_PIXEL_FORMAT_COORD3D_C16;

int nBufferSizeIn = 0;
int nBufferSizeOut = 0;
int nWidth        = 720;
int nHeight       = 540;
unsigned char* pszBufferIn = NULL;
unsigned char* pszBufferOut = NULL;
bool bFlip = false;
DX_STATUS emDxStatus  = DX_OK;
//声明图像格式转换句柄
DX_IMAGE_FORMAT_CONVERT_HANDLE handle;

//创建图像格式转换句柄
DxImageFormatConvertCreate(&handle);
if (handle == NULL)
{
    return;
}

DX_VALID_BIT emValidBits = DX_BIT_2_9;

//从标定参数文件中读取 buffer
char* pBuffer = NULL;
int nBufferLen = 0;
emDxStatus = LoadCalibFile("CalibParam.bin", pBuffer, nBufferLen);
if (emDxStatus != DX_OK)
{
    if (NULL!=pBuffer)
    {
        delete[] pBuffer;
        pBuffer=NULL;
    }
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

emDxStatus = (DX_STATUS) DxImageFormatConvertSet3DCalibParam(
                                                                    handle,
                                                                    pBuffer,
                                                                    nBufferLen);

if (NULL!=pBuffer)
{
    delete[] pBuffer;
    pBuffer=NULL;
}
if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

```

```

}

//设置 Y 方向步长
emDxStatus = (DX_STATUS) DxImageFormatConvertSetYStep(handle, 0.1);

//获取输出图像内存大小
emDxStatus = (DX_STATUS) DxImageFormatConvertGetBufferSizeForConversion(
    handle,
    emOutPixelFormat,
    nWidth, nHeight,
    &nBufferSizeOut);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//获取输入图像内存大小
emDxStatus=(DX_STATUS) DxImageFormatConvertGetBufferSizeForConversion(
    handle,
    emInPixelFormat,
    nWidth,
    nHeight,
    &nBufferSizeIn);

if (emDxStatus != DX_OK)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

//申请内存
pszBufferIn = new unsigned char[nBufferSizeIn];
if (pszBufferIn == NULL)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;
    return;
}

pszBufferOut = new unsigned char[nBufferSizeOut];
if (pszBufferOut == NULL)
{
    DxImageFormatConvertDestroy(handle);
    handle = NULL;

    delete pszBufferIn;
    pszBufferIn = NULL;
    return;
}

```


//执行图像格式转换

```
emDxStatus = (DX_STATUS) DxImageFormatConvert(handle, pszBufferIn,
                                                nBufferSizeIn,
                                                pszBufferOut,
                                                nBufferSizeOut,
                                                emInPixelFormat,
                                                nWidth,
                                                nHeight,
                                                bFlip);

DxImageFormatConvertDestroy(handle);
handle = NULL;
```

8.4.44. DxImageFormatConvertSetYStep

声明：

```
VxInt32 DHDECL DxImageFormatConvertSetYStep(
                                                DX_IMAGE_FORMAT_CONVERT_HANDLE handle,
                                                const double dY);
```

意义：

该函数实现设置高度图转点云的 Y 方向步长。

形参：

<i>handle</i>	图像格式转换句柄
<i>dY</i>	Y 方向步长

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：见 [DxImageFormatConvertSet3DCalibParam](#) 函数代码样例。

8.4.45. DxStaticDefectCorrection

声明：

```
VxInt32 DHDECL DxStaticDefectCorrection(void *pInputBuffer, void *pOutputBuffer,
                                         STATIC_DEFECT_CORRECTION stDefectCorrection,
                                         void* pDefectPosBuffer, VxUInt32 nDefectPosBufferSize);
```

意义：

该函数实现采集图像静态坏点校正功能。

形参：

<i>pInputBuffer</i>	指向原始图像数据缓冲区
<i>pOutputBuffer</i>	指向目标图像数据缓冲区
<i>stDefectCorrection</i>	坏点校正需要的图像参数，参见 8.3.5
<i>pDefectPosBuffer</i>	指向坏点位置数据缓冲区
<i>nDefectPosBufferSize</i>	坏点位置数据缓冲区大小

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
//坏点校正数据结构体参数初始化
STATIC_DEFECT_CORRECTION stDefectCorrection;
stDefectCorrection.nImgWidth = nImgWidth;
stDefectCorrection.nImgHeight = nImgHeight;
stDefectCorrection.nImgOffsetX = nImgOffsetX;
stDefectCorrection.nImgOffsetY = nImgOffsetY;
stDefectCorrection.nImgWidthMax = nImgWidthMax;
stDefectCorrection.nBayerType = BAYERRG;
stDefectCorrection.emActualBits= DX_ACTUAL_BITS_8;

//以二进制方式读取坏点文件

//获取参数 pDefectPosBuffer 和 nDefectPosBufferSize

//坏点文件需要通过坏点校正插件保存

//执行坏点校正
VxInt32 DxStatus= DxStaticDefectCorrection (pInputBuffer,
                                             pOutputBuffer, stDefectCorrection,
                                             pDefectPosBuffer,nDefectPosBufferSize);

if(DxStatus != DX_OK)
{
    return;
}
```

8.4.46. DxCalcCameraLutBuffer

声明：

```
VxInt32 DHDECL DxCalcCameraLutBuffer (VxInt32 nContrastParam, double dGamma,
                                       VxInt32 nLightness, void *pLut,
                                       VxUInt32 *pnLutLength);
```

意义：

该函数为根据对比度、gamma、亮度调整查找表数值。

形参：

<i>nContrastParam</i>	对比度调节参数，范围：-50~100
<i>nGamma</i>	Gamma 调节参数，范围：0.1~10
<i>nLightness</i>	亮度调节参数，范围：-150~150
<i>pLut</i>	查找表，根据对比度、Gamma 或者亮度参数计算
<i>pLutLength</i>	查找表长度，单位是字节

查找表长度值需要通过 GXGetBufferLength 接口获取，用户根据获取的长度自己申请内存。如果用户输入的查找表指针为空，返回错误。

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
VxInt32 DxStatus= DX_OK;
VxInt32 nContrastParam = -20;
double dGamma = 1.0;
VxInt32 nLightness = -20;
//读取查找表长度
size_t nLutLength = 0;
DxStatus = GXGetBufferLength(hDevice,
                             GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);

if(DxStatus != DX_OK)
{
    return;
}

//申请 Buffer
uint8_t *pSetLutBuffer = new uint8_t[nLutLength];
if(NULL == pSetLutBuffer)
{
    return;
}

//设置对比度值、gamma 值、亮度值调整查找表数值
DxStatus = DxCalcCameraLutBuffer(nContrastParam,
                                 dGamma,
                                 nLightness,
                                 pSetLutBuffer,
                                 (VxUInt32*)&nLutLength);

if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete []pSetLutBuffer;
        pSetLutBuffer = NULL;
    }
    return;
}

//设置查找表
DxStatus = GXSetBuffer(hDevice,
                       GX_BUFFER_LUT_VALUEALL,
                       pSetLutBuffer,
                       nLutLength);

if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete []pSetLutBuffer;
```

```

        pSetLutBuffer = NULL;
    }
    return;
}

```

8.4.47. DxReadLutFile

声明：

```

VxInt32 DHDECL DxReadLutFile( const char *pchLutFilePath,
                             void *pLut,
                             VxUInt32* pnLutLength);

```

意义：

该函数为读取查找表插件保存的.lut 文件，并根据相机查找表长度，将数据解析为可以设置到相应相机内的查找表数据。

形参：

<i>pchLutFilePath</i>	查找表文件路径。查找表文件（xxx.lut）可以通过查找表插件获得。查找表插件路径：GalaxyView->菜单栏->插件->查找表生成工具插件
<i>pLut</i>	查找表，从.lut 文件中解析后的数据。需要用户事先申请内存，申请的指针内存大小，也就是查找表长度（nLutLength），可以通过 GXGetBufferLength 接口获取：GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL, &nLutLength)
<i>pnLutLength</i>	查找表长度，单位是字节。可以通过 GXGetBufferLength 接口获取：GXGetBufferLength(hDevice, GX_BUFFER_LUT_VALUEALL, &nLutLength)

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

VxInt32 DxStatus= DX_OK;
const char* pszLutFilePath = "D:\\a.lut";
//读取查找表长度
size_t nLutLength = 0;
DxStatus = GXGetBufferLength(hDevice,
                             GX_BUFFER_LUT_VALUEALL,
                             &nLutLength);

if(DxStatus != DX_OK)
{
    return;
}

//申请 Buffer
uint8_t* pSetLutBuffer = new uint8_t[nLutLength];
if(NULL == pSetLutBuffer)
{
    return;
}

```

```

//设置对比度值、gamma 值、亮度值调整查找表数值
DxStatus = DxReadLutFile (pszLutFilePath,
                          pSetLutBuffer,
                          (VxUInt32*)&nLutLength);

if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete []pSetLutBuffer;
        pSetLutBuffer = NULL;
    }
    return;
}

//设置查找表
DxStatus = GXSetBuffer(hDevice,
                      GX_BUFFER_LUT_VALUEALL,
                      pSetLutBuffer,
                      nLutLength);

if(DxStatus!=DX_OK)
{
    if(NULL != pSetLutBuffer)
    {
        delete[] pSetLutBuffer;
        pSetLutBuffer = NULL;
    }
    return;
}

```

8.4.48. DxSetSystem

声明：

VxInt32 DHDECL DxSetSystem(DX_SYSTEM_PARAM emSystemParam, int nValue)

意义：

该函数用于设置系统参数，比如设置线程数、SSSE3 是否使能等参数。

形参：

<i>emSystemParam</i>	系统参数
<i>nValue</i>	系统参数取值

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```

DX_SYSTEM_PARAM emSystemParam = THREAD_NUM;
int nValue = 6;
// 设置线程数
VxInt32 emStatus = DxSetSystem(emSystemParam, nValue);
if (emStatus != DX_OK)
{
    return emStatus;
}

```

}

8.4.49. DxGetSystem

声明：

VxInt32 DHDECL DxGetSystem(DX_SYSTEM_PARAM emSystemParam, int *pnValue)

意义：

该函数用于获取系统参数，比如获取线程数、SSSE3 使能状态等参数。

形参：

<i>emSystemParam</i>	系统参数
<i>pnValue</i>	系统参数取值

返回值：

如果成功则返回 DX_OK，否则请参见 DX_STATUS 相关定义。

代码样例：

```
DX_SYSTEM_PARAM emSystemParam = THREAD_NUM;
int nValue = 0;
// 获取线程数
VxInt32 emStatus = DxGetSystem(emSystemParam, &nValue);
if (emStatus != DX_OK)
{
    return emStatus;
}
```

8.5. 功能

8.5.1. 图像质量提升功能

该功能可以实现颜色校正功能，对比度调节、Gamma 调节功能的任意组合。

8.5.1.1. 相关函数

- 设置对比度查找表函数
VxInt32 DHDECL [DxGetContrastLut](#) (int nContrastParam, void *pContrastLut, int *pLutLength);
- 设置 Gamma 查找表函数
VxInt32 DHDECL [DxSetGammatLut](#) (double dGammaParam, void *pGammaLut, int *pLutLength);
- 图像质量提升函数
VxInt32 DHDECL [DxImageImprovment](#) (void *pInputBuffer, void *pOutputBuffer, VxUInt32 nWidth, VxUInt32 nHeight, VxInt64 nColorCorrectionParam, void *pContrastLut, void *pGammaLut);

函数的使用，请参照接口部分。

8.5.1.2. 颜色校正（颜色转换）

- 名词解释

颜色校正（颜色转换）：提高相机色彩还原度，使图像更加接近人眼视觉感受。

颜色转换模式：设置要执行的颜色转换的模式。0：设置为默认模式，颜色校正系数使用相机出厂时提供的系数；1：设置为用户自定义模式。用户可以根据实际情况输入颜色校正系数。用户可以在采集期间切换此功能的值。

颜色转换矩阵值选择：在颜色转换矩阵中设置要输入的值，用于自定义颜色转换。注意：根据相机模型，颜色转换矩阵中的一些值可能是预先设置的，不能被更改。用户可以在采集期间切换此功能的值。

颜色转换矩阵值：获取在颜色转换矩阵中的值，用于自定义颜色转换的当前值。

我们期望相机能够输出“精确”的颜色，但是颜色的世界是一个“动态”的世界，每个人看到的颜色都有一些差别，同样对于 sensor 来说也一样，对颜色的解释也不一致，但是什么是“精确”的颜色？谁说了算，我们需要一个样板。这个样板包含 24 种颜色，每种颜色都有固定的 RGB 值，如图 8-17 所示。



图 8-17 样板

有了这个样板，我们就以这个样板为基准，用相机对这个色板进行拍摄，也许得到的每种颜色的 RGB 值和样板颜色的标准 RGB 不一样，厂商可以利用软件或者硬件将读到的 RGB 转换为标准的 RGB 值，因为颜色空间是连续的，所以所有读到的其他 RGB 颜色都可以用这 24 种颜色建立起来的映射表转换成标准的 RGB 值。

● 相关参数

GX_ENUM_COLOR_TRANSFORMATION_MODE：

颜色转换模式，参考：GX_COLOR_TRANSFORMATION_MODE_ENTRY

GX_BOOL_COLOR_TRANSFORMATION_ENABLE：颜色转换使能

X_ENUM_COLOR_TRANSFORMATION_VALUE_SELECTOR：

颜色转换矩阵元素选择，参考：GX_COLOR_TRANSFORMATION_VALUE_SELECTOR_ENTRY

GX_FLOAT_COLOR_TRANSFORMATION_VALUE：颜色转换矩阵元素

GX_INT_COLOR_CORRECTION_PARAM：颜色校正参数

● 效果图



图 8-18 颜色校正前



图 8-19 颜色校正后

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//使能颜色转换
emStatus = GXSetBool(hDevice, GX_BOOL_COLOR_TRANSFORMATION_ENABLE, true);

//设置颜色转换模式为用户自定义模式
GX_COLOR_TRANSFORMATION_MODE_ENTRY emValue;
emValue = GX_COLOR_TRANSFORMATION_SELECTOR_USER;
emStatus = GXSetEnum(hDevice, GX_ENUM_COLOR_TRANSFORMATION_MODE, emValue);

//获取颜色校正参数值
int64_t i64ColorParam = 0;
emStatus = GXGetInt(hDevice, GX_INT_COLOR_CORRECTION_PARAM, &i64ColorParam);
```

8.5.1.3. 对比度调节

- 名词解释

对比度：图像明亮部分与黑暗部分的亮度比称为对比度，又叫反差。对比度高或者反差大的图像，其中被摄景物的轮廓较清楚，图像也较清晰；反之，对比度低的图像轮廓不清，图像也不太清晰。

- 相关参数

GX_INT_CONTRAST_PARAM：对比度参数

- 效果图



图 8-20 对比度调节前



图 8-21 对比度调节后

8.5.1.4. Gamma 调节

- 名词解释

Gamma 调节：Gamma 调节是为了让显示器的输出尽量接近输入。

Gamma：Gamma 校正的值。对当前图像以幂函数的形式进行像素值的非线性变换。

Gamma 模式：手动调节方式 0：设置为默认模式，1：设置为用户自定义模式。

- 相关参数

GX_BOOL_GAMMA_ENABLE：Gamma 使能

GX_ENUM_GAMMA_MODE：Gamma 模式，参考 GX_GAMMA_MODE_ENTRY

GX_FLOAT_GAMMA：Gamma

GX_FLOAT_GAMMA_PARAM : Gamma 参数

● 效果图

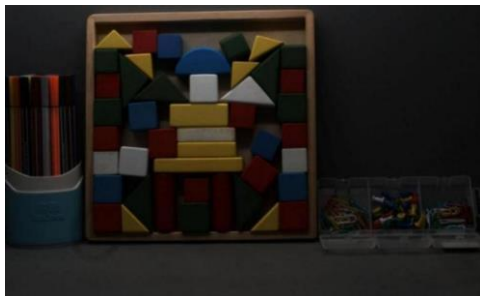


图 8-22 Gamma 调节前



图 8-23 Gamma 调节后

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//使能 Gamma
emStatus = GXSetBool(hDevice, GX_BOOL_GAMMA_ENABLE, true);
//设置 Gamma 模式为用户自定义模式
GX_GAMMA_MODE_ENTRY emValue;
emValue = GX_GAMMA_SELECTOR_USER;
emStatus = GXSetEnum(hDevice, GX_ENUM_GAMMA_MODE, emValue);

//获取 Gamma 参数值
double dColorParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_GAMMA_PARAM, &dColorParam);
```

8.5.1.5. 锐化

● 名词解释：

锐化：锐化是为了提高图像边缘的清晰度。清晰度越高，图像对应的轮廓就越清晰。

锐度：调节锐度值可调整相机对图像的锐度，调节范围为 0-3.0。数值越大，相机对图像的锐化程度越高。

锐化模式：决定是否开启锐化功能。ON 表示开启锐化功能；OFF 表示关闭锐化功能。

● 相关参数

GX_FLOAT_SHARPNESS :

锐度

GX_ENUM_SHARPNESS_MODE :

锐化模式，参考 GX_SHARPNESS_MODE_ENTRY

● 效果图



图 8-24 锐化后图像



图 8-25 锐化后图像

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//使能锐化
GX_SHARPNESS_MODE_ENTRY emValue;
emValue = GX_SHARPNESS_MODE_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_SHARPNESS_MODE, emValue);

//获取锐度的值
double dColorParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_SHARPNESS, &dColorParam);
```

8.5.1.6. 降噪

- 名词解释：

降噪：数字图像在数字化、传输过程中常受到成像设备与外部环境的噪声干扰，会产生含有噪声的图像，减少或抑制数字图像中噪声的过程称为图像降噪。

降噪：调节降噪值可调整相机对图像的降噪强度，调节范围为 0-4.0。数值越大，相机对图像的降噪程度越高。

降噪模式：决定是否开启降噪功能。ON 表示开启降噪功能；OFF 表示关闭降噪功能。

- 相关参数

GX_FLOAT_NOISE_REDUCTION： 降噪

GX_ENUM_NOISE_REDUCTION_MODE： 降噪模式，参考 GX_NOISE_REDUCTION_MODE_ENTRY

- 效果图

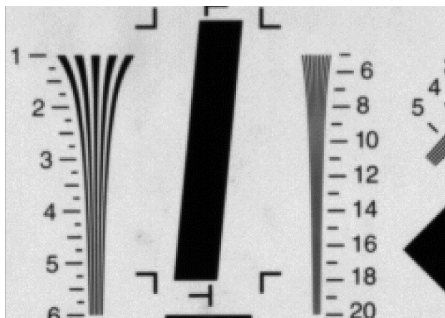


图 8-26 降噪前图像

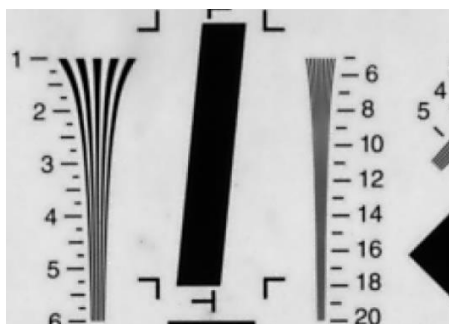


图 8-27 降噪后图像

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//使能降噪
GX_NOISE_REDUCTION_MODE_ENTRY emValue;
emValue = GX_NOISE_REDUCTION_MODE_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_NOISE_REDUCTION_MODE,
                    emValue);

//获取降噪的值
double dNoiseReductionParam = 0.0;
emStatus = GXGetFloat(hDevice, GX_FLOAT_NOISE_REDUCTION,
                    &dNoiseReductionParam);
```

8.5.1.7. 饱和度

● 名词解释：

饱和度指的是色彩纯度。是色彩的构成要素之一。纯度越高，表现越鲜明，纯度较低，表现则较黯淡。

饱和度：调节饱和度值可调整相机对图像的饱和度，调节范围为 0-128。其中，64：饱和度没有变化；大于 64：增加饱和度；小于 64：减小饱和度；128：饱和度为当前两倍；0：黑白图像。

饱和度模式：决定是否开启饱和度功能。ON 表示开启饱和度功能；OFF 表示关闭饱和度功能。

● 相关参数

GX_INT_SATURATION：	饱和度
GX_ENUM_SATURATION_MODE：	饱和度模式，参考 GX_ENUM_SATURATION_MODE_ENTRY

● 效果图



图 8-28 饱和度调节前图像



图 8-29 饱和度调节后图像

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//开启饱和度功能
GX_ENUM_SATURATION_MODE_ENTRY emValue;
emValue = GX_ENUM_SATURATION_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_SATURATION_MODE, emValue);
```

```
//获取饱和度值
int64_t i64Saturation = 0;
emStatus = GXGetInt(hDevice, GX_INT_SATURATION, &i64Saturation);
```

8.5.1.8. 静态坏点校正

- 名词解释：

静态坏点指的是不会随着时间、增益等改变，从 sensor 制造时因为工艺等产生的坏点。若感光元件出现坏点，会直接造成成像的瑕疵。

静态坏点校正模式：决定是否开启静态坏点校正功能。ON 表示开启静态坏点校正功能；OFF 表示关闭静态坏点校正功能。

- 相关参数

GX_ENUM_STATIC_DEFECT_CORRECTION：静态坏点校正模式，参考：
GX_ENUM_STATIC_DEFECT_CORRECTION_ENTRY

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;
//开启静态坏点校正功能
GX_ENUM_STATIC_DEFECT_CORRECTION_ENTRY emValue;
emValue = GX_ENUM_STATIC_DEFECT_CORRECTION_ON;
emStatus = GXSetEnum(hDevice, GX_ENUM_STATIC_DEFECT_CORRECTION,
emValue);
```

8.5.1.9. 2D/3D 降噪模式

- 名词解释：

降噪：数字图像在数字化、传输过程中常受到成像设备与外部环境的噪声干扰，会产生含有噪声的图像，减少或抑制数字图像中噪声的过程称为图像降噪。

降噪模式：分为低、中、高三种降噪模式。

- 相关参数

GX_ENUM_2D_NOISE_REDUCTION_MODE：2D 降噪模式，参考
GX_2D_NOISE_REDUCTION_MODE_ENTRY
GX_ENUM_3D_NOISE_REDUCTION_MODE：3D 降噪模式，参考
GX_3D_NOISE_REDUCTION_MODE_ENTRY

- 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//选择 2D 降噪模式：Low，具体模式选择根据需要自行设置
GX_2D_NOISE_REDUCTION_MODE_ENTRY emValue;
//emValue = GX_2D_NOISE_REDUCTION_MODE_OFF; // 关闭 2D 降噪模式
emValue = GX_2D_NOISE_REDUCTION_MODE_LOW; // 2D 降噪模式：Low
//emValue = GX_2D_NOISE_REDUCTION_MODE_MIDDLE; // 2D 降噪模式：Middle
//emValue = GX_2D_NOISE_REDUCTION_MODE_HIGH; // 2D 降噪模式：High
emStatus = GXSetEnum(hDevice, GX_ENUM_2D_NOISE_REDUCTION_MODE,
```

```
emValue);

//选择 3D 降噪模式：High，具体模式选择根据需要自行设置
GX_3D_NOISE_REDUCTION_MODE_ENTRY emValue;
//emValue = GX_3D_NOISE_REDUCTION_MODE_OFF;      // 关闭 3D 降噪模式
//emValue = GX_3D_NOISE_REDUCTION_MODE_LOW;      // 3D 降噪模式：Low
//emValue = GX_3D_NOISE_REDUCTION_MODE_MIDDLE;    // 3D 降噪模式：Middle
emValue = GX_3D_NOISE_REDUCTION_MODE_HIGH;      // 3D 降噪模式：High
emStatus = GXSetEnum(hDevice, GX_ENUM_3D_NOISE_REDUCTION_MODE,
                    emValue);
```

8.5.1.10. HDR 模式

● 名词解释

HDR 表示高动态范围，具体到工业相机中，是指相机直接生成高动态范围数字图像。相反，使用传统方法捕捉图像称为 LDR（低动态范围）。

图像的动态范围（即对比度）指最大亮度值与最小亮度值的比。换言之，如果图像同时包含较亮的区域和较暗的区域，则动态范围较高（例如，一张背对太阳的人的图像）。如果图像的动态范围极小，亮度也可以很高，因为没有较暗的区域（例如：正对太阳）。

高动态范围功能采用长短两帧曝光，获得高亮度和低亮度图像，并进行图像融合。融合后的图像既保留了高亮度图像中的暗部细节，又保留了低亮度图像中的亮部细节。

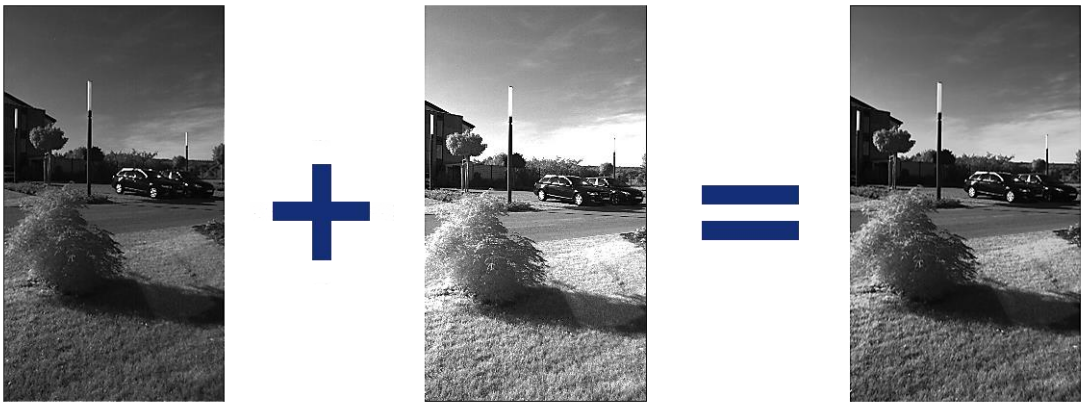


图 8-30 HDR 效果示意图

支持该功能的相机型号：

相机型号
MER-203-30GC-P-L

● 相关参数

GX_ENUM_HDR_MODE：	HDR 模式, 参考 GX_HDR_MODE_ENTRY
GX_INT_HDR_TARGET_LONG_VALUE：	亮场期望值
GX_INT_HDR_TARGET_SHORT_VALUE：	暗场期望值
GX_INT_HDR_TARGET_MAIN_VALUE：	融合期望值

● 代码样例

```
GX_STATUS emStatus = GX_STATUS_SUCCESS;

//开启连续 HDR 模式
GX_HDR_MODE_ENTRY emValue = GX_HDR_MODE_CONTINUOUS;
emStatus = GXSetEnum(hDevice, GX_ENUM_HDR_MODE, emValue);

//此处设置的值仅为样例代码，不同场景下不同，请根据需要自行设置
//设置亮场期望值
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_LONG_VALUE, 80);

//设置暗场期望值
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_SHORT_VALUE, 28);

//设置融合期望值
emStatus = GXSetInt(hDevice, GX_INT_HDR_TARGET_MAIN_VALUE, 255);
```


9. 版本说明

序号	修订版本号	API 版本	所做改动	发布日期
1	V1.0.0	1.0.1303.x 及以上版本	1) 初始发布	2013-03
2	V2.0.0	1.0.1901.x 及以上版本	1) Windows & Linux 合并版 C 软件开发说明书初始发布	2019-01-11
3	V2.0.1	1.0.1901.x 及以上版本	1) 修改了图像处理库部分示例代码	2019-01-23
4	V2.0.2	1.0.1902.x 及以上版本	1) 修改系统测试发现的问题	2019-02-12
5	V2.0.3	1.0.1902.x 及以上版本	1) 修改正文的页码	2019-02-14
6	V2.0.4	1.0.1902.x 及以上版本	1) 修改 2.2.9 章节代码样例对齐格式；修改 8.4.21 章节的部分格式	2019-02-21
7	V2.0.5	1.0.1902.x 及以上版本	1) 增加 3.5 章节，计数器计时器控制功能属性说明	2019-02-28
8	V2.0.6	1.0.1907.x 及以上版本	1) 增加设备复位重连接口说明	2019-07-18
9	V2.0.7	1.0.1908.x 及以上版本	1) 增加获取最优包长接口说明 2) 章节 2.2.9、3.3.2、7.4.3.1.12、7.4.5.8、0、7.4.5.10 中的“代码样例”中增加设置最优包长示例代码	2019-08-15
10	V2.0.8	1.0.1908.x 及以上版本	1) 增加 3.9 章节相关参数说明 2) 新增 3.13.6、3.13.7、3.13.8 章节	2019-08-29
11	V2.0.9	1.0.1910.x 及以上版本	1) 添加 8.3.3、8.3.4 章节结构体信息 2) 添加第 8.4.4、第 8.4.19、第 8.4.23、第 8.4.24、第 8.4.31、第 8.4.32、第 8.4.33 章节	2019-10-12
12	V2.0.10	1.0.1910.x 及以上版本	1) 增加 3.1.1 章节中节点信息	2019-10-22
13	V2.0.11	1.0.1912.x 及以上版本	1) 添加 8.2.7 章节句柄信息 2) 添加第 8.4.34~第 8.4.41 章节	2019-12-08
14	V2.1.0	1.0.1912.x 及以上版本	1) 在 5.2.1 节中添加 StreamBufferHandlingMode 相关参数信息	2019-12-25
15	V2.1.1	1.0.2002.x 及以上版本	1) 在 8.5.1 节中增加降噪功能介绍	2020-02-15
16	V2.1.2	1.0.2005.x 及以上版本	1) 在 3.3.3 节中添加 ExposureTimeMode 相关参数信息	2020-05-11

序号	修订版本号	API 版本	所做改动	发布日期
17	V2.1.3	1.0.2008.x 及以上版本	1) 在 3.5.2 节 中 增 加 GX_ENUM_COUNTER_TRIGGER_SOURCE 和 GX_INT_COUNTER_DURATION 功能码 2) 在 3.13.8 节中增加 GX_BUFFER_FFCLOAD 和 GX_BUFFER_ 3) FFCSAVE 功能码	2020-08-18
18	V2.1.4	1.0.2008.x 及以上版本	1) 增加 3.2.6 节 Sensor 曝光模式	2020-08-21
19	V2.1.5	1.0.2008.x 及以上版本	1) 在 2.2.7 节中增加通过演示程序获取属性读写示例代 码功能介绍	2020-08-28
20	V2.1.6	1.0.2010.x 及以上版本	1) 在 3.6.3 节中增加 LightSourcePreset 相关参数信息	2020-10-12
21	V2.1.7	1.0.2011.x 及以上版本	1) 在 7.4.5.12 节中增加 GXGetOptimalPacketSize 仅 Windows 支持	2020-11-17
22	V2.1.8	1.0.2012.x 及以上版本	1) 增加 8.3.5 和 8.4.42，添加静态坏点校正接口描述	2020-12-11
23	V2.1.9	1.0.2101.x 及以上版本	1) 增加 8.5.1.7 章节饱和度功能介绍 2) 增加 8.5.1.8 章节静态坏点校正功能介绍	2021-01-07
24	V2.1.1 0	1.0.2104.x 及以上版本	1) 增加 3.3.10 节多帧灰度控制模式功能介绍 2) 增加 8.5.1.9 节 2D/3D 降噪模式功能介绍 3) 增加 8.5.1.10 节 HDR 模式功能介绍	2021-04-01
25	V2.1.1 1	1.0.2110.x 及以上版本	1) 增加 3.1.1 节中，查询设备 ISP 固件版本功能码介绍 2) 增加 3.2.2 节中，抽样行数功能码介绍	2021-10-18
26	V2.1.1 2	1.0.2111.x 及以上版本	1) 增加 3.2.2 节中，Sensor 水平抽样和 Sensor 垂直抽 样功能码介绍	2021-11-23
27	V2.1.1 3	1.0.2201.x 及以上版本	1) 增加 8.4.46 节，添加计算相机查找表接口描述	2022-01-14
28	V2.1.1 4	1.0.2203.x 及以上版本	1) 修改 8.4.46，增加 8.4.47 节，添加查找表文件解析 接口描述	2022-03-14
29	V2.1.1 5	1.0.2209.x 及以上版本	1) 增加 3.3.5 节突发采集模式功能介绍 2) 修改 3.3.6 节清空帧存功能介绍	2022-09-16
30	V2.1.1 6	1.0.2209.x 及以上版本	1) 增 加 7.3.5 节 ， 增 加 GX_REGISTER_STACK_ENTRY 结构体 2) 增加 7.5.4.3、7.5.4.4、7.5.4.1、7.5.4.2 节，增加读 取、设置相机属性寄存器接口 3) 修改 3.3.5 节，修改突发采集模式注意事项	2022-09-21
31	V2.1.1 7	1.0.2301.x 及以上版本	1) 增加 2.1.3 节	2023-01-30

序号	修订版本号	API 版本	所做改动	发布日期
32	V3.0.0	2.0.2403.x 及以上版本	1) 新增 7.4.2 节初始化及销毁功能接口 a) 枚举设备接口，推荐使用新增接口 GXUpdateAllDeviceListEx，枚举接口 GXUpdateDeviceList 不推荐使用。 b) 获取设备信息接口，推荐使用新增接口 GXGetDeviceInfo，其他获取设备信息接口 GXGetAllDeviceBaseInfo、GXGetDeviceIPInfo 不推荐使用。 2) 7.1.2 节扩展新增以下接口类型： a) 新增 GX_IF_HANDLE 句柄类型 通过 GXGetInterfaceHandle 接口获取。用来对 Interface 层设备进行读写访问。 b) 新增 GX_LOCAL_DEV_HANDLE 句柄类型 通过 GXGetLocalDeviceHandleFromDev 接口获取。用来对本地设备层设备进行读写访问。 c) 新增 GX_DS_HANDLE 句柄类型 通过 GXGetDataStreamHandleFromDev 接口获取。用来对流层设备进行读写访问。 3) 修改 7.4.3 节设备参数设置 推荐使用新增的字符串进行读写访问的接口： GXGetIntValue、GXSetIntValue、GXGetEnumValue、GXSetEnumValue、GXSetEnumValueByString、GXGetFloatValue、GXSetFloatValue、GXGetBoolValue、GXSetBoolValue、GXGetStringValue、GXSetStringValue、GXGetRegisterLength、GXGetRegisterValue、GXSetRegisterValue、GXRegisterFeatureCallbackByString、GXUnregisterFeatureCallbackByString 不推荐使用功能码进行读写。	2024-03-20
33	V3.0.1	2.0.2405.x 及以上版本	1) 新增 7.2.10 节，GX_LOG_TYPE_LIST 结构体 2) 新增 7.4.2.3 节设置生成日志类型接口和 7.4.2.4 节获取生成日志类型接口 3) 修改 8.2.3 节，添加 RAW2RGB_WEIGHT 枚举常量 4) 新增 8.2.9 节 DX_SYSTEM_PARAM 常量 5) 新增 8.4.48 节设置系统参数接口和 8.4.49 节获取系统参数接口	2024-05-25
34	V3.0.2	2.0.2406.x 及以上版本	1) Windows 支持 DQBuf 接口	2024-06-19

序号	修订版本号	API 版本	所做改动	发布日期
35	V3.0.3	2.0.2412.x 及以上版本	1) 新增 7.3.12 节 GX_GIGE_ACTION_COMMAND_RESULT 2) 新增 GXGigEIssueActionCommand、 GXGigEIssueScheduledActionCommand 接口	2024-12-10
36	V3.0.4	2.0.2412.x 及以上版本	1) 新增 2.1.2.1.推荐配置 2) 新增 2.2.3.获取 Interface 信息、2.2.4.获取 Interface 对象 3) 新增 7.4.4.12 节，GXExportConfigFileW 接口、 7.4.4.13 节 GXImportConfigFileW 接口 4) 新增 7.4.5.14 节 GXRegisterBuffer 接口、7.4.5.15 节 GXUnRegisterBuffer 接口 5) 新增 7.5.4.35 节 GXGetFeatureNameList 接口、 7.5.4.36 节 GXGetFeatureName 接口、7.5.4.37 节 GXGetFeatureType 接口（不推荐使用） 6) 新增 8.2.7 节彩色图像 RGB 顺序 7) 新增 8.4.7 节 DxRotate90CW16B，8.4.8 节 DxRotate90CCW16B，8.4.12 节 DxSharpenMono8，8.4.26 节 DxImageMirror16B 8) 优化部分描述信息	2025-01-08
37	V3.0.5	2.0.2412.x 及以上版本	1) 修改 2.2.7 节，将黑 GalaxyView 截图替换为白 GalaxyView 截图 2) 优化部分描述信息	2025-03-05